

Systems Analysis and Modelling Lecture Notes

September 2026



"The Line of Business wants a new system ASAP. They are very busy dealing with issues so they can't assign any stakeholders to the project. When can you have the requirements document ready?"

(<https://www.modernanalyst.com>)

José Borbinha, André Vasconcelos, Luís Sousa

(for this text, the authors made heavy use of the tool ChatGPT, by OpenAI)

Contents

1	<cover> Introduction.....	2	2.19.3 Homeostasis.....	11
2	<cover> Systems Engineering.....	3	2.19.4 Organic Versus Mechanistic Views	11
2.1	<concept> Systems Engineering.....	4	2.19.5 Back to Engineering	11
2.2	<example> Historical Case Studies.....	5	2.20 Wrap-up Systems Engineering	12
2.3	<concept> Systems Thinking	6	3 <cover> Requirements Engineering	13
2.3.1	Core Concepts of General Systems Theory	6	3.1 <concept> Requirements Engineering: Purpose and Scope	14
2.3.2	Application of GST in Systems Engineering	6	3.1.1 Requirements Do Not Arrive Ready-Made	14
2.3.3	Conclusion	6	3.1.2 What We Call a Requirement	14
2.4	<concept> Systems versus Problems	7	3.1.3 Systems Are Not Necessarily Software	14
2.4.1	Systems and Problem Solving	7	3.1.4 Role in Systems Analysis.....	14
2.4.2	Nature of Systems	7	3.1.5 Scope of This Chapter	14
2.4.3	Problem Understanding and Solving.....	7	3.2 <concept> From the UoD to System Requirements	15
2.4.4	Conclusion	7	3.2.1 From concerns to a system of interest	15
2.5	<concept> The Universe of Discourse.....	8	3.2.2 Requirements have sources	15
2.5.1	Universe of Discourse.....	8	3.2.3 Formulating system requirements	15
2.5.2	Key Aspects of the Universe of Discourse	8	3.3 <technique> The Requirements Engineering Process	16
2.5.3	Traceability	8	3.3.1 Main activities.....	16
2.5.4	The System Context and the System Domain	8	3.3.2 Work products	16
2.5.5	Conclusion	8	3.4 <further> Pohl's Requirements Engineering Framework	17
2.6	<concept> System Context and System Domain	9	3.5 <technique> Elicitation Techniques	18
2.6.1	System Domain.....	9	3.5.1 Common techniques	18
2.6.2	System Context.....	9	3.5.2 Combining techniques.....	18
2.6.3	Key Differences.....	9	3.6 <technique> Analysing, Negotiating, and Prioritising	19
2.6.4	System of Systems.....	9	3.6.1 Analysing.....	19
2.6.5	Conclusion	9	3.6.2 Negotiating.....	19
2.7	<concept> System Boundary and Responsibility in Sociotechnical Systems.....	10	3.6.3 Prioritising	19
2.7.1	Sociotechnical systems	10	3.7 <concept> Goals, Needs, Concerns, Rules, and Requirements Sources.....	20
2.7.2	Boundary and responsibility	10	3.7.1 Concerns	20
2.7.3	Boundaries are consequential	10	3.7.2 Goals	20
2.8	<concept> Systems as Composed Structures	2	3.7.3 Needs.....	20
2.8.1	System Boundary and Black-box Perspective.....	2	3.7.4 Rules	20
2.8.2	Internal Structure and White-box Perspective.....	2	3.7.5 Requirements sources.....	20
2.8.3	Example: An Automated Parking System.....	2	3.8 <concept> What Does the Obligation Govern?	21
2.8.4	Modelling Guidance.....	2	3.8.1 System requirements	21
2.8.5	Conclusion	2	3.8.2 Project-delivery requirements.....	21
2.9	<example> UoD, Context and Domain	1	3.8.3 Engineering-process requirements.....	21
2.9.1	Example 1: Autonomous Vehicle Control System	1	3.8.4 Why the distinction matters	21
2.9.2	Example 2: Online Banking System	1	3.9 <concept> Requirements, Assumptions, and Design Decisions	22
2.9.3	Conclusion	1	3.9.1 Requirements.....	22
2.10	<imagine> ... a UoD.....	2	3.9.2 Assumptions.....	22
2.11	<concept> Life Cycle and Methods	3	3.9.3 Design decisions.....	22
2.11.1	The System's Life Cycle.....	3	3.9.4 Avoiding a false "what versus how" rule	22
2.11.2	Methods for Systems Engineering	3	3.10 <concept> The Kinds of Requirements	23
2.11.3	Conclusion	3	3.10.1 Functional requirements	23
2.12	<concept> Understanding versus Doing	4	3.10.2 Quality requirements	23
2.12.1	Plan-Do-Check-Act.....	4	3.10.3 Constraints	23
2.12.2	Understanding: The Analysis Activity.....	4	3.10.4 Classification depends on the obligation	23
2.12.3	Doing: The Design Activity	4	3.11 <further> "Business Requirements" and "Stakeholder Requirements"	24
2.12.4	Relationship Between Understanding and Doing.....	4	3.11.1 The risk of calling everything a requirement	24
2.12.5	MAPE-K: A Related Framework.....	4	3.11.2 Convention used in this course.....	24
2.12.6	Conclusion	4	3.12 <heuristic> Analysing a Candidate Requirement Statement	25
2.13	<technique> Conceptual Models and ISO/IEC/IEEE 42010	5	3.12.1 (1) Is the statement an obligation?.....	25
2.13.1	What is a "system"?.....	5	3.12.2 (2) Who or what must satisfy it?	25
2.13.2	Stakeholders, Concerns, and Domains	5	3.12.3 (3) Only one or several obligations?	25
2.13.3	Core Concepts	5	3.12.4 (4) What is the nature of the obligation?	25
2.13.4	Conclusion	5	3.12.5 (5) Is there enough context?	25
2.14	<technique> ISO/IEC/IEEE 42010 for SysEng and ReqEng	6	3.12.6 Final checks	25
2.14.1	SysEng and Architecture Description	6	3.13 <exercise> Analysing Ambiguous Candidate Statements	26
2.14.2	ReqEng and Stakeholder Concerns	6	3.13.1 Statements	26
2.14.3	Modelling Languages.....	6	3.13.2 Discussion guide.....	26
2.14.4	Conclusion	6	3.14 <concept> Quality of an Individual Requirement	27
2.15	<further> Strategy, Goals, Outcomes, and Principles	7	3.14.1 Necessary and justified	27
2.15.1	Goals: The Motivational Drivers.....	7	3.14.2 Appropriate	27
2.15.2	Outcomes: The Realised Benefits	7	3.14.3 Unambiguous	27
2.15.3	Principles: The Guiding Rules	7	3.14.4 Complete as an individual statement	27
2.15.4	Linking Goals, Outcomes, and Principles to the UoD	7	3.14.5 Singular	27
2.15.5	Conclusion	7	3.14.6 Feasible	27
2.16	<concept> ...Enterprise Architecture	8	3.14.7 Verifiable.....	27
2.16.1	Organisations as Systems.....	8	3.14.8 Correct and conforming	27
2.16.2	Enterprise Architecture as a Process.....	8	3.14.9 A warning about SMART	27
2.16.3	Enterprise Architecture Frameworks	8	3.15 <concept> Quality of a Requirements Set	28
2.16.4	Conclusion	8	3.15.1 Sufficient coverage.....	28
2.17	<example> The Portuguese Legal System.....	9	3.15.2 Consistency	28
2.17.1	...as a Social System	9	3.15.3 Feasibility as a set.....	28
2.17.2	...as a Socio-Technical System	9	3.15.4 Coherence and comprehensibility	28
2.17.3	Conclusion	9	3.15.5 Traceability and modifiability	28
2.18	<example> The Banking Industry as a System	10	3.15.6 Validatable and verifiable as a whole	28
2.18.1	Key Aspects of the Online Banking UoD.....	10	3.16 <technique> Writing Atomic Textual Requirements.....	29
2.18.2	Traceability in the Online Banking UoD.....	10	3.16.1 A basic sentence pattern	29
2.18.3	System Context and Domain.....	10	3.16.2 Writing procedure.....	29
2.18.4	Conclusion	10	3.16.3 Decomposing a compound statement.....	29
2.19	<further> ...More on "Systems".....	11	3.16.4 Avoid mechanical rules.....	29
2.19.1	Organic Systems	11	3.17 <concept> Requirements Specifications as Work Products.....	30
2.19.2	Living Systems	11	3.17.1 Typical contents	30

3.17.2	Text and models	30	3.35	<further> Popular Heuristics and Their Limits: Pareto and DIKW .	48
3.17.3	Specification is not “the how”	30	3.35.1	The Pareto principle	48
3.17.4	Baselines and evolution	30	3.35.2	The DIKW pyramid	48
3.18	<concept> Relationships Between Requirements	31	3.35.3	Responsible use	48
3.18.1	Refinement and decomposition	31	3.36	<heuristic> Consolidating the Analysis Heuristics	49
3.18.2	Dependency	31	3.36.1	(1) What is the purpose?	49
3.18.3	Qualification and restriction	31	3.36.2	(2) What evidence do we have?	49
3.18.4	Conflict	31	3.36.3	(3) What is the system of interest?	49
3.18.5	Alternatives, overlap, and duplication	31	3.36.4	(4) Who is responsible?	49
3.18.6	A wider traceability network	31	3.36.5	(5) What kind of statement is this?	49
3.19	<concept> Verification and Validation	32	3.36.6	(6) What kind of requirement is it?	49
3.19.1	Requirements verification	32	3.36.7	(7) Is each requirement well formed?	49
3.19.2	Requirements validation	32	3.36.8	(8) Does the set work as a whole?	49
3.19.3	System verification	32	3.36.9	(9) Can we verify and validate it?	49
3.19.4	System validation	32	3.36.10	(10) What happens when something changes?	49
3.19.5	V&V throughout the work	32	3.36.11	Use the questions iteratively	49
3.20	<example> From Statements to a Requirements Set	33	3.37	<further> Software Tools for Requirements Engineering	50
3.20.1	Investigating the statement	33	3.37.1	Useful capabilities	50
3.20.2	Framing the system	33	3.37.2	Tool categories	50
3.20.3	Illustrative requirements	33	3.37.3	Common failure modes	50
3.20.4	Traceability and remaining work	33	3.37.4	AI assistance	50
3.21	<example> Reviewing and Repairing a Requirements Set	34	3.37.5	Minimum discipline	50
3.21.1	Review findings	34	3.38	<core> Wrap-up Requirements Engineering	51
3.21.2	Additional analysis	34	3.38.2	Keep the essential distinctions clear	51
3.21.3	Repaired requirements set	34	3.38.3	Requirements must work as a set	51
3.22	<concept> Requirements and Conceptual Models	35	3.38.4	Requirements remain living artefacts	51
3.22.1	Models during requirements engineering	35	3.38.5	A compact working discipline	51
3.22.2	Descriptive, exploratory, or normative	35	3.39	<imagine> Digital Fingerprint: Viewpoint and Perception	52
3.22.3	Models reveal requirements problems	35	3.39.1	Imagine the same organisation	52
3.22.4	Combining models and text	35	3.39.2	From perception to viewpoint	52
3.23	<technique> User Stories	36	3.39.3	The analyst also leaves fingerprints	52
3.23.1	Card, conversation, and confirmation	36	3.39.4	The digital world also selects what can be seen	52
3.23.2	From a story to requirements	36	3.40	<imagine> Domain-Specific Requirements	53
3.23.3	Limitations	36	3.40.1	Haiku	53
3.24	<technique> Use Cases	37	3.40.2	Kenningar	53
3.24.1	Use-case model and specification	37	3.40.3	Why this is relevant	53
3.24.2	Example: Cancel Appointment	37	4	<cover> Engineering with Conceptual Models	54
3.24.3	Use cases and requirements	37	4.1.1	From Representation to Knowledge	54
3.24.4	Using use cases during analysis	37	4.1.2	Modelling as a Knowledge-Based Engineering Activity	54
3.25	<technique> Scenarios	38	4.1.3	Conceptual Modelling and Ontology Engineering	54
3.25.1	Scenario structure	38	4.1.4	Model-Based and Model-Driven Approaches	54
3.25.2	Example: an online cancellation	38	4.1.5	Engineering as an Iterative Learning Process	54
3.25.3	Scenario, use case, and test case	38	4.1.6	Concluding Perspective	54
3.25.4	Scenarios are not complete requirements	38	4.2	<concept> Models in Engineering	55
3.26	<technique> Acceptance Criteria	39	4.2.1	Analytical Models: Quantifying the World	55
3.26.1	Writing useful criteria	39	4.2.2	Conceptual Models: Structuring Understanding	55
3.26.2	Relationship with requirements	39	4.2.3	Complementarity	55
3.26.3	Criteria are not test cases	39	4.3	<concept> Why We Model - The Purpose of Modelling	56
3.26.4	Criteria are not a Definition of Done	39	4.4	<further> Why We Model Systems – The Science of Cycology	57
3.26.5	Acceptance is not full validation	39	4.4.1	The Experiment	57
3.27	<technique> Specification by Example	40	4.4.2	The Illusion of Understanding	57
3.27.1	From a rule to examples	40	4.4.3	Modelling as an Antidote	57
3.27.2	Given-When-Then	40	4.4.4	Reflection for Systems Engineers	57
3.27.3	Developing useful examples	40	4.4.5	The Analyst’s Motto	57
3.27.4	Examples and requirements	40	4.5	<further> Ontology and Ontology Engineering	58
3.27.5	Executable examples	40	4.5.1	Ontology as representation of knowledge	58
3.28	<technique> Prototypes and Wireframes	41	4.5.2	From symbolic AI to knowledge engineering	58
3.28.1	Match fidelity to the question	41	4.5.3	Example: reasoning with an ontology	58
3.28.2	Prototyping as requirements work	41	4.5.4	Ontology engineering	58
3.28.3	Prototypes are not requirements	41	4.5.5	Conclusion	58
3.28.4	Beyond the successful screen flow	41	4.6	<further> Models as Representation of Knowledge	59
3.29	<technique> Traceability and Change Management	42	4.6.1	Beyond Visual Rendering	59
3.29.1	Traceability relationships	42	4.6.2	Example: BPMN	59
3.29.2	Managing change	42	4.7	<concept> On Types of Systems and Types of Models	60
3.29.3	Example of impact	42	4.7.1	Physical, Logical, and Socio-technical Conceptions	60
3.29.4	Proportionate control	42	4.7.2	One Situation, Different Conceptions	60
3.30	<further> ISO/IEC/IEEE 29148 in Short	43	4.7.3	Conceptions Are Selective, but Not Arbitrary	60
3.30.1	What the standard addresses	43	4.7.4	Choosing Models	60
3.30.2	Relationship with this course	43	4.7.5	Conclusion	60
3.31	<heuristic> Goals, Scope, and Fit for Purpose	44	4.8	<further> From Concepts to Systems	61
3.31.1	Defining the scope	44	4.8.1	Model-Based Systems Engineering	61
3.31.2	Fit for purpose	44	4.8.2	Model-Driven Engineering	61
3.31.3	Example	44	4.8.3	Relationship Between MBSE and MDE	61
3.31.4	Enough detail, not maximum detail	44	4.8.4	Conclusion	61
3.32	<heuristic> When Is It Enough to Analyse?	45	4.9	<further> Model-Driven Architecture	62
3.32.1	Factors affecting the required depth	45	4.9.1	Transformation Tools	62
3.32.2	Indicators of sufficient analysis	45	4.9.2	Example Scenario:	62
3.32.3	Warning signs	45	4.10	<example> Enterprise Architect Tool	63
3.32.4	Deliberate deferral	45	4.10.1	Computation Independent Model in Enterprise Architect	63
3.32.5	Example	45	4.10.2	Platform Independent Model in Enterprise Architect	63
3.33	<further> Power, Ethics, and Cognitive Bias in Analysis	46	4.10.3	Platform Specific Model in Enterprise Architect	63
3.33.1	Power in analysis	46	4.10.4	Model Transformations in Enterprise Architect	63
3.33.2	Ethical concerns	46	4.10.5	Integration and Tool Support	63
3.33.3	Cognitive and organisational bias	46	4.10.6	Conclusion	63
3.33.4	Practical safeguards	46	4.11	What Therefore is a Modelling Language?	64
3.34	<further> Tacit and Explicit Knowledge	47	4.11.1	Components of a Language	64
3.34.1	Why interviews are insufficient	47	4.11.2	The Role of Metamodels	64
3.34.2	Discovering tacit knowledge	47	4.11.3	On General-Purpose and Domain-Specific Languages	64
3.34.3	Example	47	4.11.4	Reflection	64
3.34.4	Limits of externalisation	47			

4.12	<concept> Models and Metamodels	65	4.30.3	Conclusion	83
4.12.1	Models and Metamodels	65	4.31	<further> Model Debt	84
4.12.2	The Role of Meta-Object Facility	65	4.31.1	Causes of Model Debt	84
4.12.3	Conceptual Modelling with Visual Languages	65	4.31.2	Consequences of Model Debt	84
4.12.4	Conclusion	65	4.31.3	Managing and Reducing Model Debt	84
4.13	<technique> Modelling and Requirements Engineering	66	4.31.4	Conclusion	84
4.13.1	Languages for Model-Based Approaches	66	4.32	<concept> Wrap-up Engineering with Models	85
4.13.2	Conclusion	66	4.32.1	Models as a Bridge	85
4.14	<concept> Requirements Engineering with Models	67	4.32.2	Model-Based Systems Engineering and Model-Driven Engineering	85
4.14.1	Early exploration: mind maps and concept maps	67	4.32.3	Key Nowadays Modelling Languages	85
4.14.2	Structuring the problem: architectural and process viewpoints	67	4.32.4	Modelling "As-Is" and "To-Be"	85
4.14.3	Detailing structure and behaviour: system-level models	67	4.32.5	Conclusion	85
4.14.4	Conclusion	67	4.33	<further> Complementary Modelling Frameworks	86
4.15	<technique> Informal – Mind Maps	68	4.33.1	Rational Unified Process	86
4.15.1	Structure and Flexibility	68	4.33.2	ICONIX: Bridging Analysis and Design	86
4.15.2	Benefits of Mind Mapping	68	4.33.3	The C4 Model	86
4.15.3	How to Create a Mind Map	68	4.33.4	Goal-Oriented Modelling	86
4.15.4	Applications of Mind Maps	68	4.33.5	Integrating Perspectives	86
4.16	<technique> Informal – Concept Maps	69	4.34	<further> ... recall	87
4.16.1	Concept Maps vs. Conceptual Models	69	5	<cover> Enterprise Architecture and ArchiMate	88
4.16.2	Role in the ReqEng Process	69	5.1	<concept> Enterprise Architecture as Coherence and Change	89
4.16.3	Good Practices	69	5.1.1	Architecture and Architecture Description	89
4.16.4	Key Takeaway	69	5.1.2	Current State, Target State, and Change	89
4.17	<technique> UML, BPMN, and ArchiMate	70	5.2	<technique> ArchiMate as an Architecture Description Language	90
4.17.1	Origins and Governance	70	5.2.1	A Language, Not a Method	90
4.17.2	Concepts, Language Versions, and Tools	70	5.2.2	Describing Current and Future Architectures	90
4.18	<technique> Views and Viewpoints	71	5.2.3	Models, Elements, and Relationships	90
4.18.1	Views and Modelling Languages	71	5.2.4	Selective Representation	90
4.18.2	Viewpoints and Modelling Languages	71	5.3	<example> A Change Across an Enterprise	91
4.18.3	Integration Across Views	71	5.3.1	Current-State Facts	91
4.18.4	Why This Distinction Matters	71	5.3.2	Current-State Problems	91
4.18.5	In Short	71	5.3.3	Intended Change and Outcomes	91
4.19	<concept> Views and Viewpoints versus Stakeholders and Concerns	72	5.3.4	Decisions Not Yet Made	91
4.19.1	From Concerns to Views	72	5.3.5	Architectural Description	92
4.19.2	Viewpoints as Modelling Conventions	72	5.4	<technique> The ArchiMate Language Structure	93
4.19.3	Models Are Not the Same as Diagrams	72	5.4.1	The Domains of ArchiMate	93
4.19.4	Practical Modelling Sequence	72	5.4.2	Aspects: Structure, Behaviour, and Objects	93
4.20	<technique> ArchiMate in short	73	5.4.3	A Connected Language	93
4.20.1	Core Concepts of ArchiMate	73	5.5	<concept> The Common Domain	94
4.20.2	ArchiMate and Requirements Engineering	73	5.5.1	Active Structure: Who or What Acts?	94
4.20.3	Conclusion	73	5.5.2	Behaviour: What Happens?	94
4.21	<technique> BPMN in short	74	5.5.3	Passive Structure: What Is Used or Produced?	94
4.21.1	Core Concepts of BPMN	74	5.5.4	One Pattern, Several Domains <AQUL>	94
4.21.2	BPMN and Requirements Engineering	74	5.6	<technique> Services, Processes, and Functions	95
4.21.3	Conclusion	74	5.6.1	Service: Behaviour Seen from Outside	95
4.22	<technique> About Object-Oriented	75	5.6.2	Process: Behaviour Through Time	95
4.22.1	Key Principles of Object-Oriented Design	75	5.6.3	Function: Behaviour Organised by a Criterion	95
4.22.2	Risks of Introducing Polymorphism in Analysis	75	5.6.4	Connecting the Concepts	95
4.22.3	Risks of Modelling Methods for Classes in Analysis	75	5.7	<technique> The Business Domain	96
4.22.4	Benefits of Object-Oriented Design	75	5.7.1	Business Actor: The Organisational System	96
4.22.5	Modelling Languages: BPMN, ArchiMate, and UML	75	5.7.2	Process: Behaviour Through Time	96
4.22.6	Conclusion	75	5.7.3	Function: Behaviour Grouped by Purpose	96
4.23	<technique> UML in short	76	5.7.4	Service: Behaviour Offered to Others	96
4.23.1	Core Concepts of UML	76	5.7.5	Business Object: What the Work Concerns	96
4.23.2	UML and Requirements Engineering	76	5.7.6	Reading a Simple Business View	96
4.23.3	Conclusion	76	5.8	<technique> The Application Domain	97
4.24	<further> SysML in short	77	5.8.1	Application Component: Which Software System?	97
4.24.1	Historical Overview	77	5.8.2	Service: What Does the Application Offer?	97
4.24.2	SysML 2.0	77	5.8.3	Process and Function: What Does the Software Do?	97
4.24.3	An Example	77	5.8.4	Data Object: Which Data Is Processed?	97
4.25	<example> Automated Parking System	78	5.8.5	Application Interface: How Is the Service Accessed?	97
4.25.1	ArchiMate	78	5.8.6	Reading a Simple Application View	97
4.25.2	BPMN	78	5.9	<technique> The Technology Domain	98
4.25.3	UML	78	5.9.1	Node: A General Technological System	98
4.26	<example> Challenges in a Smart Parking	79	5.9.2	Device: The Hardware System	98
4.26.1	Stakeholders and Concerns	79	5.9.3	System Software: The Supporting Software Environment	98
4.26.2	Conflicting and Complementary Requirements	79	5.9.4	Communication Network: How Are Systems Connected?	98
4.26.3	Discussion Points	79	5.9.5	Service: What Does the Infrastructure Provide?	98
4.27	<concept> States of Affairs	80	5.9.6	Artifact: What Is Deployed or Stored?	98
4.27.1	Ontology and States of Affairs	80	5.9.7	Reading a Simple Technology View	98
4.27.2	Epistemology and States of Affairs	80	5.10	<further> Motivation and Strategy	99
4.27.3	Semiotics and States of Affairs	80	5.10.1	Stakeholder: Who Has an Interest?	99
4.27.4	Logic and States of Affairs	80	5.10.2	Driver: What Creates Pressure for Change?	99
4.27.5	Knowledge Representation and States of Affairs	80	5.10.3	Assessment: What Does the Driver Mean Here?	99
4.27.6	Value of States of Affairs in Requirements Engineering	80	5.10.4	Goal and Outcome: What Should Be Achieved?	99
4.27.7	Conclusion	80	5.10.5	Requirement: What Must the Architecture Provide?	99
4.28	<further> Semiotics	81	5.10.6	Capability: What Must the Enterprise Be Able to Do?	99
4.28.1	The Semiotic Triangle	81	5.10.7	Course of Action: Which Strategic Approach Is Chosen?	99
4.28.2	The Semiotic Square	81	5.10.8	Reading a Motivation and Strategy View	99
4.28.3	Semiotics and Conceptual Modelling Languages	81	5.11	<technique> Relationships as Architectural Statements	100
4.28.4	Conclusion	81	5.11.1	Composition: What Is Part of What?	100
4.29	<concept> "As-Is" versus "To-Be"	82	5.11.2	Assignment: Who or What Performs the Behaviour?	100
4.29.1	Purpose of Modelling an "As-Is" Scenario	82	5.11.3	Realisation: What Makes the More Abstract Element Concrete?	100
4.29.2	Purpose of Modelling a "To-Be" Scenario	82	5.11.4	Serving: Who Benefits from or Uses What Is Provided?	100
4.29.3	Integrating "As-Is" and "To-Be" Models	82	5.11.5	Access: Which Information or Object Is Used?	100
4.29.4	Conclusion	82	5.11.6	Triggering: What Causes Behaviour to Begin or Continue?	100
4.30	<example> From "As-Is" to "To-Be"	83	5.11.7	Flow: What Is Transferred?	100
4.30.1	As-Is Scenario (Current State of Affairs)	83	5.11.8	Connect Only What You Can Explain	100
4.30.2	To-Be Scenario (Future State of Affairs)	83	5.12	<example> Constructing an End-to-End Architecture View	101
			5.12.1	Begin with the Concern	101

5.12.2	Step 1: Identify the Business Behaviour	101	5.23.4	Overview	113
5.12.3	Step 2: Add the Applications That Support the Work	101	5.24	<imagine> ...black-box versus white-box... ..	114
5.12.4	Step 3: Represent the Information	101	5.25	<core> Wrap-up Enterprise Architecture and ArchiMate	115
5.12.5	Step 4: Make the Problem Visible	101	6	<cover> Business Processes and BPMN	116
5.12.6	Step 5: Check the Architectural Story	101	6.1	<concept> Business Processes	117
5.12.7	Stop at the Right Depth	101	6.1.1	What is a Business Process?	117
5.13	<heuristic> Choosing the Right Concept	102	6.1.2	The Concept of “Flow”	117
5.13.1	Is It Something That Acts?	102	6.1.3	Case and Case Management	117
5.13.2	Actor or Role?	102	6.1.4	Object-Centric Process Modelling	117
5.13.3	Is It Behaviour?	102	6.1.5	Conclusion	117
5.13.4	Service or Internal Behaviour?	102	6.2	<concept> Introduction to Business Processes and BPMN	118
5.13.5	Is It Something Used or Produced?	102	6.3	<concept> Processes in a Business System	119
5.13.6	Business, Application, or Technology?	102	6.3.1	The Value of Business Processes	119
5.13.7	Current Fact, Motivation, or Proposed Solution?	102	6.3.2	Challenges in Establishing Process Architecture	119
5.13.8	Prefer the Meaning Required by the View	102	6.3.3	Types of Business Processes	119
5.14	<heuristic> Choosing the Right Relationship	103	6.3.4	Compliance, Auditing, and Maturity	119
5.14.1	Begin with a Sentence	103	6.3.5	Conclusion	119
5.14.2	Assignment: Who or What Performs Behaviour?	103	6.4	<further> Reference Process and Process Framework	120
5.14.3	Realisation: What Provides or Implements This?	103	6.4.1	Reference Process	120
5.14.4	Serving: What Is Made Available to Whom?	103	6.4.2	Process Framework	120
5.14.5	Access: What Is Read or Changed?	103	6.4.3	Relationship Between Reference Process and Process Framework	120
5.14.6	Triggering: What Causes or Precedes What?	103	6.4.4	Conclusion	120
5.14.7	Flow: What Is Transferred?	103	6.5	<concept> Process Architecture	121
5.14.8	Composition: What Belongs Inside What?	104	6.5.1	Key Elements of Process Architecture	121
5.14.9	Association: When Only a General Connection Is Known	104	6.5.2	The Importance of Process Architecture	121
5.14.10	Support Is Not Always Assignment	104	6.5.3	Conclusion	121
5.14.11	Check Both Meaning and Validity	104	6.6	<further> Languages to Model Business Processes	122
5.14.12	Prefer the Most Precise Supported Statement	104	6.6.1	Business Process Model and Notation (BPMN)	122
5.14.13	Relationships and Junctions	104	6.6.2	Unified Modelling Language (UML)	122
5.15	<heuristic> Black-Box, White-Box, and Modelling Depth	105	6.6.3	Event-Driven Process Chain (EPC)	122
5.15.1	Black-Box: What Is Visible from Outside?	105	6.6.4	Integrated Definition for Function Modelling (IDEF)	122
5.15.2	White-Box: How Is It Internally Realised?	105	6.6.5	Business Process Execution Language (BPEL)	122
5.15.3	The Same Element Can Appear in Both Perspectives	105	6.6.6	Petri Nets	122
5.15.4	Modelling Depth Is Relative	105	6.6.7	Conclusion	122
5.15.5	Do Not Invent Internal Detail	105	6.7	<technique> BPMN	123
5.15.6	Do Not Hide the Relevant Problem	105	6.7.1	Types of Processes in BPMN	123
5.15.7	Avoid Mixing Incompatible Depths	105	6.7.2	Types of BPMN Diagrams	123
5.15.8	Use Viewpoint and Concern to Choose Depth	105	6.7.3	Conclusion	123
5.15.9	A Practical Rule	105	6.8	<technique> The Core BPMN Elements	124
5.16	<heuristic> Common Modelling Errors	106	6.8.1	Flow Objects	124
5.17	<example> Reviewing and Correcting an ArchiMate View	107	6.8.2	Connecting Objects	124
5.18	<exercise> From a Narrative to Consistent Architecture Views ..	108	6.8.3	Artifacts	124
5.18.1	The Narrative	108	6.8.4	Conclusion	124
5.18.2	Task 1: Establish the Universe of Discourse	108	6.9	<technique> BPMN Events in Short	125
5.18.3	Task 2: Create a Context and Services View	108	6.9.1	Start Events	125
5.18.4	Task 3: Create a Refund Handling View	108	6.9.2	Intermediate Events	125
5.18.5	Task 4: Create a Motivation for Change View	108	6.9.3	End Events	125
5.18.6	Task 5: Check Consistency Across Views	108	6.9.4	Special Event Types	125
5.18.7	Task 6: Explain the Modelling Decisions	108	6.10	<technique> BPMN Activities in Short	126
5.18.8	Review Questions	108	6.10.1	Task	126
5.18.9	Closing	108	6.10.2	Sub-Process	126
5.19	<further> How ArchiMate Evolved	109	6.10.3	Call Activity	126
5.19.1	Growth Through Successive Specifications	109	6.10.4	Loop and Multi-Instance Activities	126
5.19.2	Continuity Matters	109	6.10.5	Compensation and Ad-Hoc Activities	126
5.19.3	ArchiMate 4	109	6.11	<technique> BPMN Gateways in Short	127
5.19.4	Why Older Material Still Appears	109	6.12	<technique> Collaboration Diagrams	128
5.19.5	Evolution Is Not the Same as Architecture Change	109	6.12.1	The Concept of Collaboration	128
5.20	<further> Reading Older ArchiMate Models	110	6.12.2	Key Elements of Collaboration Processes	128
5.20.1	Identify the Language Version	110	6.12.3	Collaboration Process Scenarios	128
5.20.2	Begin with Meaning, Not Symbols	110	6.12.4	Benefits of Modelling Collaboration Processes	128
5.20.3	Separate Language Age from Factual Age	110	6.12.5	Conclusion	128
5.20.4	Expect Differences in Vocabulary	110	6.13	<heuristic> Common Issues in BPMN Models	129
5.20.5	Preserve the Author’s Intended Statement	110	6.13.1	Uncontrolled Flow	129
5.20.6	Do Not Translate the Diagram in Isolation	110	6.13.2	Gateway Divergences without Convergences	129
5.20.7	Reconstruct the Model, Not the Page Layout	110	6.13.3	Improper Use of Sequence Flows	129
5.20.8	Record Translation Decisions	110	6.13.4	Misuse of Task Types	129
5.20.9	A Practical Reading Method	110	6.13.5	Incorrect Use of Message Flows	129
5.21	<technique> Using ArchiMate 4 Concepts in Tools Supporting ArchiMate 3.2	111	6.13.6	Conclusion	129
5.21.1	Begin with a Conceptual Model	111	6.14	<technique> BPMN – Execution in Private Diagrams	130
5.21.2	Use Native Elements Only When Their Meaning Fits	111	6.14.1	Practical Implication	130
5.21.3	Make Compatibility Decisions Explicit	111	6.14.2	Common Mistakes to Avoid	130
5.21.4	Do Not Misrepresent the Repository	111	6.14.3	Simple Examples	130
5.21.5	Treat Validation as Version-Specific	111	6.14.4	Golden Rule	130
5.21.6	Prefer Simple and Reversible Workarounds	111	6.15	<further> BPMN and MBSE & MDE	131
5.21.7	Use the Tool as a Repository, Not as the Authority	111	6.15.1	BPMN in the Context of MBSE & MDE	131
5.22	<core> Wrap-up Enterprise Architecture with ArchiMate 4	112	6.15.2	How BPMN Models Drive Automation	131
5.22.1	Architecture Description Is Purposeful	112	6.15.3	Benefits of Using BPMN in MBSE & MDE	131
5.22.2	Views Select; Models Connect	112	6.15.4	Conclusion	131
5.22.3	Read Elements by Their Architectural Meaning	112	6.16	<further> Tools for Business Process	132
5.22.4	Relationships Are Architectural Statements	112	6.17	<core> Wrap-up Business Processes and BPMN	133
5.22.5	Select the Right Modelling Depth	112	6.17.1	BPMN	133
5.22.6	Model Evidence, Not Plausibility	112	6.17.2	Collaboration Processes in BPMN	133
5.22.7	Prefer Coherence Over Completeness	112	6.17.3	Common BPMN Pitfalls to Avoid	133
5.22.8	Tools Assist; Analysts Decide	112	6.17.4	Conclusion	133
5.22.9	Modelling Is an Accountable Interpretation	112	7	<cover> Software Systems and UML	134
5.22.10	What beginners Should Now Be Able to Do	112	7.1	<core> Introduction to Software Systems and UML	135
5.23	<technique> ArchiMate Reference Cards	113	7.2	<technique> The Unified Modelling Language	136
5.23.1	Core Elements	113	7.2.1	Structural Diagrams	136
5.23.2	Motivation, Strategy, Implementation and Migration Elements	113			
5.23.3	Relationships and Junctions	113			

7.2.2	Behavioural Diagrams.....	136	7.14.5	Scope of UML.....	148
7.2.3	The Role of UML in the Software Development Lifecycle.....	136	7.14.6	Conclusion.....	148
7.2.4	Conclusion.....	136	7.15	<technique> Components, Parts, Ports, Interfaces, and Connectors	149
7.3	<technique> UML in Problem Analysis.....	137	7.15.1	Components.....	149
7.3.1	Use Case Diagrams to Capture Functional Requirements.....	137	7.15.2	Interfaces.....	149
7.3.2	Class Diagrams for Domain Modelling.....	137	7.15.3	Ports.....	149
7.3.3	State Machine Diagrams to Model Dynamic Behaviour.....	137	7.15.4	Parts and Internal Structure.....	149
7.3.4	Sequence Diagrams for Detailed Interaction Analysis.....	137	7.15.5	Connectors.....	149
7.3.5	Conclusion.....	137	7.15.6	Component and Composite Structure Diagrams.....	149
7.4	<technique> Use Case Diagrams in UML.....	138	7.15.7	Modelling Guidance.....	149
7.4.1	Core Concepts.....	138	7.15.8	Conclusion.....	149
7.4.2	Purpose and Benefits.....	138	7.16	<technique> Deployment Diagrams.....	150
7.4.3	Best Practices.....	138	7.16.1	Nodes.....	150
7.4.4	Conclusion.....	138	7.16.2	Artifacts.....	150
7.5	<technique> Scenarios in Use Cases.....	139	7.16.3	Deployment and Communication Paths.....	150
7.6	<example> A Use Case with Scenarios.....	140	7.16.4	From Logical to Physical Decisions.....	150
7.6.1	Main Scenario: Enrol in Course.....	140	7.16.5	Modelling Guidance.....	150
7.6.2	Alternative Scenario 1: Course Full.....	140	7.16.6	Conclusion.....	150
7.6.3	Alternative Scenario 2: Student Does Not Meet Prerequisites.....	140	7.17	<example> Automated Parking System: From Physical Referents to	
7.6.4	Alternative Scenario 3: System Error During Enrolment.....	140	Logical Architecture.....		151
7.6.5	Conclusion.....	140	7.17.1	Physical Referents and Domain Concepts.....	151
7.7	<technique> Class Diagrams for Domain Models.....	141	7.17.2	Black-box Logical Architecture.....	151
7.7.1	Core Concepts.....	141	7.17.3	Internal Components.....	151
7.7.2	Purpose and Benefits.....	141	7.17.4	Ports, Interfaces, and Connectors.....	151
7.7.3	Best Practices.....	141	7.17.5	Behaviour Across the Structure.....	151
7.7.4	Conclusion.....	141	7.17.6	Deployment.....	151
7.8	<heuristic> Normalization Principles.....	142	7.17.7	What This Example Shows.....	151
7.8.1	Eliminating Redundancy.....	142	7.17.8	Conclusion.....	151
7.8.2	Ensuring Consistency.....	142	7.18	<technique> Object Constraint Language.....	152
7.8.3	Managing Dependencies.....	142	7.18.1	Primitive Types in OCL.....	152
7.8.4	Modular Design.....	142	7.18.2	Operators in OCL.....	152
7.8.5	Conclusion.....	142	7.18.3	Use of OCL.....	152
7.9	<example> A University Course Management System.....	143	7.18.4	Conclusion.....	152
7.9.1	Use Case Diagram.....	143	7.19	<example> Using OCL.....	153
7.9.2	Domain Model.....	143	7.19.1	UoD Description:.....	153
7.9.3	Sequence Diagram.....	143	7.19.2	Class Diagram (Domain Model):.....	153
7.9.4	Conclusion.....	143	7.19.3	Sequence Diagram:.....	153
7.10	<example> Normalization Principles in the UCMS.....	144	7.19.4	State Machine Diagram:.....	153
7.10.1	Modular Design.....	144	7.19.5	Conclusion:.....	153
7.10.2	Eliminating Redundancy.....	144	7.20	<further> UML and MBSE & MDE.....	154
7.10.3	Ensuring Consistency.....	144	7.20.1	Core Concepts of MBSE & MDE with UML.....	154
7.10.4	Managing Dependencies.....	144	7.20.2	Integration of UML in MBSE & MDE Processes.....	154
7.10.5	Conclusion.....	144	7.20.3	Conclusion.....	154
7.11	<technique> State Machine Diagrams.....	145	7.21	<core> Wrap-up Software Systems and UML.....	155
7.11.1	Core Concepts.....	145	7.21.1	UML Diagrams.....	155
7.11.2	Purpose and Benefits.....	145	7.21.2	UML for Problem Analysis and Requirements Engineering.....	155
7.11.3	A Very Simple Example.....	145	7.21.3	Object Constraint Language.....	155
7.11.4	Best Practices.....	145			
7.11.5	Conclusion.....	145			
7.12	<example> What is the Sate of the Student?.....	146	8	<core> Summary & Roadmap.....	156
7.12.1	Recaping.....	146	8.1	The Big Picture.....	157
7.12.2	State Machine Diagram for a Student.....	146	8.1.1	A disciplined modelling path.....	157
7.12.3	Conclusion.....	146	8.1.2	The modelling languages used in this course.....	157
7.13	<technique> Sequence Diagrams.....	147	8.1.3	What makes a model useful?.....	157
7.13.1	Core Concepts.....	147	8.1.4	Looking forward.....	157
7.13.2	Purpose and Benefits.....	147	Key takeaway.....		157
7.13.3	A Very Simple Example.....	147	8.2	<further> AI at the Front Edge of Systems Analysis and Modelling	
7.13.4	Best Practices.....	147			158
7.13.5	Conclusion.....	147	8.2.1	Analysis at the Leading Edge of Business Design.....	158
7.14	<concept> Logical Architectures and Physical Referents.....	148	8.2.2	Practices for Human–AI Co-Modelling.....	159
7.14.1	Physical Referents.....	148	8.2.3	Ambiguity and Silent Decisions.....	159
7.14.2	Logical Architecture.....	148	8.2.4	AI-Enabled Systems as Objects of Analysis.....	159
7.14.3	No Necessary One-to-one Correspondence.....	148			
7.14.4	Logical Models and Digital Twins.....	148	8.3	<core> Glossary.....	160
			8.4	<core> Key Terms in European Portuguese.....	173

1 <cover> Introduction

Engineering is fundamentally about solving problems, often through working with systems— integrated combinations of components that work together to perform a specific function. Whether designing a mechanical system like a car engine or an electrical system like a power grid, engineers focus on analysing and optimising these systems. A critical first step in engineering is problem analysis, where the system's requirements, limitations, and challenges are thoroughly understood. Only after this careful analysis can effective solutions be developed.

To address these challenges, engineers rely on several key approaches:

- **Methods:** Structured, systematic processes that provide a framework for problem-solving.
- **Techniques:** Specific ways of carrying out steps within a method to achieve precise results; what in a narrow perspective can be classified as a method, when in a broad perspective also can be referred as a technique.
- **Heuristics:** Experience-based shortcuts or rules of thumb that help solve problems more efficiently, often when time or data are limited.
- **Tools:** Instruments or resources, either physical or digital, that assist in applying methods, techniques, and heuristics.

In summary, engineering is centred around systems, with problem analysis being a crucial first step before acting. Engineers use methods for structure, techniques for precision, heuristics for efficiency, and tools to support the entire process in solving complex challenges.

This document outlines the fundamentals of system analysis, with a focus on techniques and heuristics. It presents the information in the form of cards of the following types:

- **<cover>** - mean cover pages, in some cases with introductory explanations
- **Core cards (*must reads*):**
 - **<concept>** - define core concepts that are foundational to the discussion.
 - **<technique>** - describe a specific technique or a set of related techniques.
 - **<heuristic>** - explain one or more heuristics and how they relate to specific techniques.
 - **<example>** - illustrate concepts, techniques, or heuristics through practical examples.
- **Extras (for research or to relax...):**
 - **<further>** - provide extra concepts or references for optional further exploration.
 - **<imagine>** - a tribute to John Lennon, Saint Exupéry, The Monty Python, and those that can dream...



Opposite thinking is a creative way to find an alternative solution to a problem.

SCOPE NOTE: The focus here is the analysis and modelling of problems as a foundation for requirements engineering. We introduce several modelling notations (ArchiMate, BPMN, and UML) only to illustrate how problems and requirements can be represented in a structured way. They are not intended to teach Model-Based Systems Engineering (MBSE) or Model-Driven Engineering (MDE) as development methodologies. In practice, the complete transformation from analysis to implementation depends strongly on the chosen method or lifecycle framework, which may follow diverse “process pipelines.” Understanding those pipelines lies beyond our scope here; our concern is to reason rigorously about the problem space before any solution is designed.

2 <cover> Systems Engineering

System derives from the Greek *sústēma*: a whole formed by elements that “stand together”. The word therefore names more than a collection of parts. It suggests an organised whole whose elements acquire meaning through their relationships, interactions and contribution to a common purpose.

Systems engineering begins with this shift of attention: from isolated components to the whole they jointly create. A system may include technologies, people, information, rules and physical resources; it exists within a wider environment; and its behaviour cannot always be understood by examining each element separately.

Systems engineering is concerned with creating new systems, but also with understanding systems that already exist. At the start, future systems are blank pages, but also existing systems are often only partially known: their documentation may be incomplete or outdated; knowledge may be fragmented among different people; and years of local changes may have made the system substantially different from what was originally designed. Getting a consensus about a to-be system or reconstructing an adequate understanding of the whole of an existing system is essential for deciding how more on.

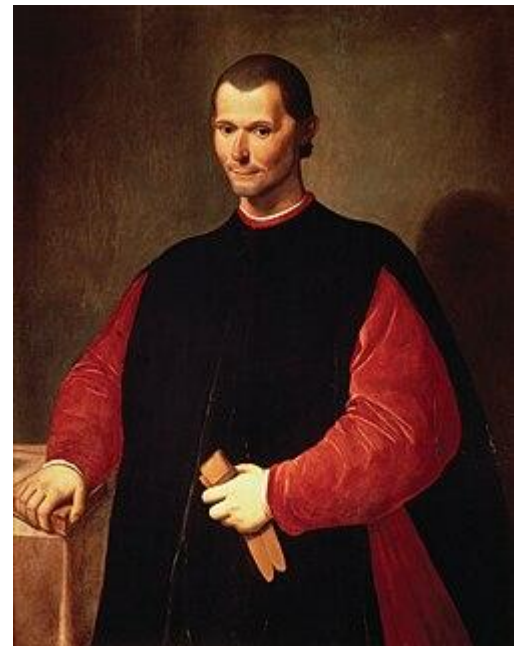
Whether investigating an existing system or designing its future, systems engineers must determine what the elements of the system are, how they interact, which purposes and stakeholders matter, how the system depends on its environment and how its behaviour may change under different conditions.

Any significant intervention also introduces a new order of things: it alters activities, responsibilities, possibilities and sometimes the distribution of costs and benefits. Machiavelli warned that nothing is more difficult, uncertain or dangerous than introducing such an order, because those who benefit from the old will resist it, while those who may benefit from the new often support it only cautiously.

Systems engineering therefore joins technical rigour with inquiry, judgement and an understanding of organisations and change. Its central challenge is to understand how things stand together today (and to decide how they should stand together tomorrow).

“Nothing is more difficult to undertake, more uncertain in its success, or more dangerous to manage than the introduction of a new order of things.”

(freely rendered from Niccolò Machiavelli, The Prince, Chapter VI)



2.1 <concept> Systems Engineering

In the increasingly interconnected world of modern engineering, traditional problem-solving approaches often prove insufficient when dealing with the complexities of today's systems. Enter systems thinking, a powerful analytical approach that allows engineers, designers, and decision-makers to examine problems holistically, recognising the interplay between a system's components and its broader environment. This method moves beyond focusing on individual elements in isolation, enabling a more comprehensive understanding of how systems behave as a whole.

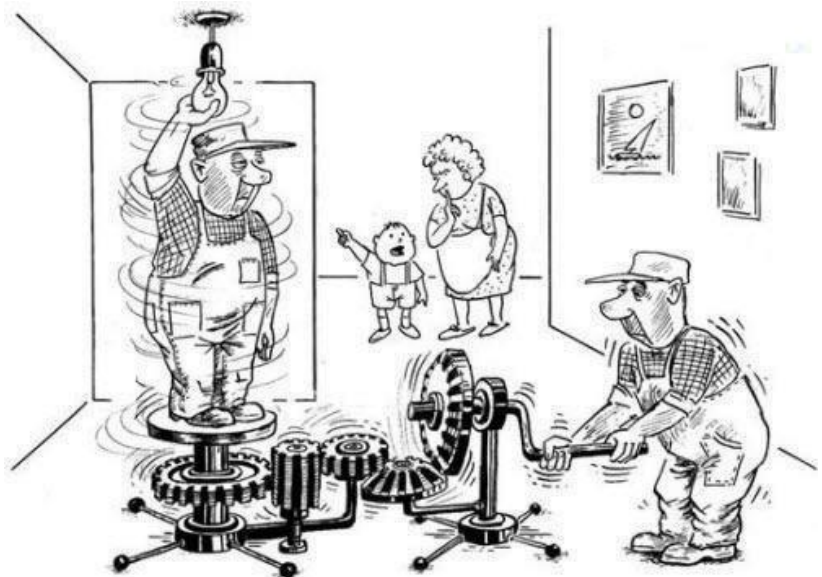
The text that follows delves into the core principles of systems thinking, focusing particularly on its application within the field of systems engineering. At its foundation is General Systems Theory (GST), introduced by Ludwig von Bertalanffy, which offers a multidisciplinary framework for understanding how systems function and interact. Systems thinking recognises that to fully grasp how a system operates, one must consider not just its internal elements but also the dynamic relationships, feedback loops, and external influences that govern its behaviour. For engineers tasked with addressing multifaceted problems, whether in technical systems, social systems, or socio-technical hybrids, systems thinking provides a vital toolkit for understanding complexity and driving effective solutions.

This text will explore the fundamental concepts of systems thinking, including systems and subsystems, the distinctions between open and closed systems, and the concept of holism, where the whole is seen as greater than the sum of its parts. These principles are not abstract; they find direct application in fields such as requirements engineering, systems design, and lifecycle management. For example, understanding a system as an "open" entity, interacting with its environment, is essential for identifying external factors that may influence requirements during the system's development. Similarly, recognising feedback loops—where a system's output may influence its future inputs—can help refine design processes and ensure the system meets stakeholder needs effectively.

The importance of systems thinking cannot be overstated, particularly in engineering contexts where complexity, interdependencies, and unforeseen consequences often complicate the design and implementation process. For instance, in socio-technical systems (where human interactions and technological components intertwine) the interplay between social behaviours and technical performance must be carefully managed to avoid system failures or inefficiencies. Systems thinking provides a means to address these challenges by fostering a holistic view of problems and their potential solutions.

As we engage with the text, we will see how these concepts translate into practical strategies for designing, managing, and optimising complex systems. The discussion will cover not only the theoretical underpinnings of systems thinking but also its application across various engineering disciplines, such as control systems, risk management, and lifecycle management. Moreover, the text illustrates how systems thinking supports a more integrated approach to requirements engineering, guiding the decomposition of high-level requirements into actionable tasks and aligning subsystems to function cohesively.

In conclusion, this introduction sets the stage for a detailed exploration of systems thinking. By understanding and applying the principles of General Systems Theory, engineers can develop more robust, adaptable systems capable of addressing modern challenges.



2.2 <example> Historical Case Studies

We can find many historical cases where the success resulted from approaches that we nowadays recognize as systems engineering best practices, as also failures due to ignoring that:

- The Wasa Ship (1628): The Wasa was a Swedish warship that sank minutes into its maiden voyage due to design flaws. Changes in requirements mid-construction, ordered by the king, made the ship top-heavy, and poor communication between the shipbuilders and the monarchy led to the disaster. This case highlights the importance of requirements management, proper testing, and stakeholder communication in systems engineering.
- The First War Navy (1316): The Portuguese Navy is credited as the first organized war navy in the world due to its innovative formalization through naval ordinances that regulated its operations, discipline, and logistics. This approach, which nowadays we'd classify as Enterprise Architecture, created a specific military branch with structured processes and governance, which enabled Portugal to dominate maritime exploration and warfare during the "Age of Discovery".
- Telephone Cooperatives in Finland (1900s-1950s): When the state monopoly failed to expand telephone service to rural Finland, local cooperatives stepped in, which in the long-term resulted in Finland (as also Sweden, which partially replicated the model) being a reference in related know-how and technology. This showcases how innovation can fill gaps in large systems, emphasizing the role of user-driven requirements and local adaptations in systems engineering and business process design.
- The Phoebus Cartel (1920s-1930s): The Phoebus Cartel, formed by leading lightbulb manufacturers, agreed to limit the lifespan of lightbulbs to maximize sales. This is a classic case of planned technological obsolescence, where companies deliberately engineered products to wear out sooner. It's a key example of how non-functional requirements (e.g., lifespan) are shaped by business strategy rather than user needs.
- Ford's Assembly Line (Early 20th Century): Henry Ford revolutionized manufacturing by introducing the assembly line. This is a foundational example of process engineering, where workflow optimization dramatically improves efficiency, demonstrating the importance of modelling and automating business processes.
- Fordlândia (1928-1945): Henry Ford's attempt to build an industrial city and rubber plantation in the Amazon failed due to a combination of wicked problems and emergent negative properties from the interaction of social, environmental, and technical systems. The project faced labour unrest, ecological collapse, and logistical failures, highlighting the dangers of ignoring local context and system interdependencies in complex systems. It exemplifies how a system of systems can produce unpredictable and destructive outcomes when not properly understood and integrated.
- Supply Chain of the US Armed Forces During WWII (1940s): The US Armed Forces' supply chain during WWII was a massive logistical challenge, driving innovations in logistics, operations research, and supply chain management. Standardization, modularization, and new project management techniques emerged from the need to manage the complex supply system across multiple theatres of war. These innovations laid the foundation for modern supply chain management practices and operations research.
- Apollo Program (1960s): NASA's Apollo missions are a landmark in systems engineering, where multiple systems (guidance, life support, propulsion) had to be integrated seamlessly. This is a classic example of managing functional and non-functional requirements, and it showcases the importance of thorough system verification and validation.
- Concorde (1960s-2000s): The Concorde supersonic jet was an engineering marvel. However, some features were discovered post-development without proper documentation, leading to confusion over their purpose. This case highlights the importance of rigorous documentation and requirements traceability in complex projects, especially when managing multinational teams and cutting-edge technology.
- London Ambulance Service Computer-Aided Dispatch Failure (1992): A poorly implemented computer-aided dispatch system led to severe delays in emergency response times, resulting in fatalities. This case demonstrates the dangers of inadequate requirements gathering and system testing, especially in critical systems where real-time functionality is essential.
- Airbus A380 Production Delays (2000s): Production delays of the Airbus A380 were caused by design inconsistencies due to different teams using incompatible versions of CAD software. This early case of Model Driven Engineering highlights the critical importance of model consistency and collaboration in large-scale systems engineering projects, where even small misalignments can have significant consequences.
- Boeing 787 Dreamliner Supply Chain Issues (2000s): Boeing outsourced the production of the Dreamliner to multiple suppliers worldwide, which led to problems in system integration and delays. This system of systems case emphasizes the need for requirements traceability and effective communication across global supply chains to ensure seamless system integration.
- Toyota Recall Crisis (2009-2010): Toyota faced a crisis due to unintended acceleration in its vehicles, linked to both mechanical and electronic failures. This case underscores the need for robust systems engineering in safety-critical systems, where failure modes must be thoroughly tested and redundancy built into the design.
- Boeing 737 Max Crisis (2018-2020): The MCAS system in the Boeing 737 Max, designed to prevent stalls, malfunctioned due to inadequate system documentation, single-sensor reliance, and lack of pilot training and awareness about a "black-box" feature). This is a key example of how ethics (governance greed), incomplete requirements and a lack of communication with end-users (pilots) can lead to catastrophic failures in safety-critical systems.

2.3 <concept> Systems Thinking

Systems thinking is an analytical approach that focuses on understanding how the components of a system interrelate and how systems work within the context of larger systems. It emphasizes viewing problems and solutions holistically, considering not just the individual parts, but also the complex interactions, feedback loops, and dynamics that shape the behaviour of the entire system. Systems thinking is particularly valuable in addressing complex (complicated), multifaceted issues, where understanding the broader context and interdependencies is essential for effective problem-solving and decision-making.

General Systems Theory (GST) offers a multidisciplinary framework for Systems Thinking. Introduced by Ludwig von Bertalanffy, GST provides principles for analysing how systems function, interact, and evolve.

2.3.1 Core Concepts of General Systems Theory

- **Systems and Subsystems:** GST suggests that the world is composed of a hierarchy of systems, each made up of interconnected subsystems. In Systems Engineering, this concept is useful in understanding how different components of a larger system interact. Similarly, in Requirements Engineering, recognising the system as a composition of subsystems helps in decomposing high-level requirements into detailed, actionable ones.
- **Open vs. Closed Systems:** A closed system is isolated from external influences; this concept is relevant in, for example, thermodynamics when analysing systems where external factors need to be controlled or minimised, so that internal processes can be studied interference. Open systems interact with their environment, exchanging energy, matter, or information. Engineering systems are typically open, engaging with users and other systems or factors. Understanding a system's openness guides identifying external factors influencing requirements.
- **Holism:** Holism in GST suggests the whole is greater than the sum of its parts. This is a principle that underscores viewing the system rather than focusing solely on individual components. This holistic view is crucial for overall system performance. In Requirements Engineering, holism supports comprehensive requirements capture, considering interdependencies among requirements.
- **Feedback Loops:** Feedback, where a system's output is fed back as input to the same system, influencing subsequent outputs, can be positive (amplifying changes) or negative (stabilising the system). This concept is critical in control systems. In the Requirements Engineering process, feedback loops between stakeholders and engineers are crucial in refining requirements, ensuring the system meets stakeholders' needs.
- **Homeostasis:** Homeostasis refers to a system's ability to maintain stability through internal regulatory mechanisms. This principle can guide engineers in designing systems that maintain functionality under varying conditions, such as safety-critical systems. In Requirements Engineering, recognising homeostasis can lead to requirements that ensure system robustness and reliability under stress or failure conditions.
- **Emergence:** Emergence is where complex behaviours arise from interactions of simpler components. In Requirements Engineering, understanding emergence supports identifying potential unintended consequences, leading to thorough requirements capture and validation processes.

2.3.2 Application of GST in Systems Engineering

- **System Design and Integration:** GST motivates for holistic system design and integration, to ensure subsystems are individually optimised and work together. In Requirements Engineering, GST motivates to confirm that requirements for different subsystems are aligned.
- **Lifecycle Management:** Systems Engineering often emphasises a system's lifecycle, which GST supports. In Requirements Engineering, this means lifecycle management can be enhanced by identifying evolving requirements, ensuring system relevance throughout its lifecycle.
- **Risk Management:** Managing risk is a key aspect of SysEng and ReqEng. Analysing system interactions with its environment enables effective risk mitigation. In Requirements Engineering, considering risks early leads to identifying critical requirements ensuring system safety and reliability.
- **Systems Thinking:** GST fosters systems thinking, a holistic approach to problem-solving considering the entire system. This mindset is useful for systems engineers in large, multidisciplinary projects. In Requirements Engineering, systems thinking helps in the integrated capture of requirements.

2.3.3 Conclusion

Systems thinking is an analytical approach that focuses on understanding how the components of a system interrelate and how systems work within the context of larger systems. GST provides a foundation for systems thinking in SysEng, enabling the design, management, and optimisation of complex systems. By applying GST principles (such as holism, feedback, emergence, and homeostasis) engineers can ensure systems are resilient, adaptable, and effective. This comprehensive approach is valuable for addressing modern engineering challenges.

2.4 <concept> Systems versus Problems

Engineering, at its core, is about solving problems, and in many cases, these problems revolve around systems. A system can be broadly defined as a thing that exists to achieve a specific goal that holds value for a stakeholder. That means a system only comes into being when a stakeholder expresses a concern, giving the system its purpose and relevance.

2.4.1 Systems and Problem Solving

Given that engineering often involves solving problems either caused by existing systems or for what the perceived solutions is a system, those challenges are also themselves often generically referred as simple, complicated, complex, or wicked.

For example, a technical system like a switch and a light bulb can be understood as simple, while a large-scale IT infrastructure is always assumed complicated. The understanding of a socio-technical system, such as a business information system involving multiple stakeholders with varying needs, often begins as complex due to its unpredictability, or even wicked if it involves deep conflicts or ambiguous goals.

2.4.2 Nature of Systems

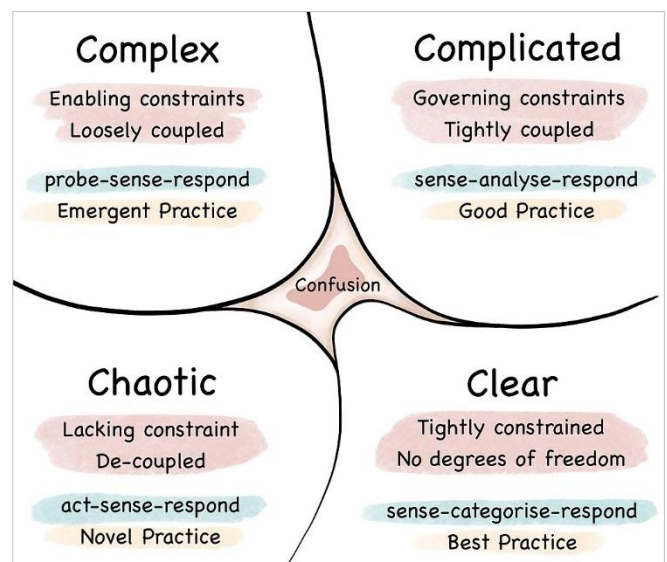
Systems can be categorised based on their primary concerns or technological nature, offering insights into their composition and behaviour:

- **Natural Systems:** Function governed by natural laws, independently of human (engineering) intervention, such as ecosystems, weather patterns, or the human body. Stakeholders typically seek to understand these systems, but if efforts are made to control them, they lose their “natural” status and transition into one of the following categories.
- **Social Systems:** A social system is composed of people and their relationships, such as in a family, community, or organisation. These systems are shaped by social interactions, cultural norms, and collective behaviours.
- **Technical Systems:** These are human-made systems designed to perform specific tasks, such as machines or software applications. Their behaviour is generally predictable, governed by engineering principles and technical specifications.
- **Socio-technical Systems:** These systems integrate both social and technical elements. For example, a modern farm involves not only technology (machines, computers) but also human processes and interactions. The interplay between social and technical components makes these systems more complex to understand, design, and manage.

2.4.3 Problem Understanding and Solving

The Cynefin Framework^{1,2} is a decision-making model to help navigate complex situations. It categorizes challenges into five domains:

- **Simple (or “Clear”):** Problems with straightforward, well-known solutions based on practices (as baking a cake using a recipe: steps are known, outcome is predictable and involves little uncertainty).
- **Complicated:** Issues that require expertise or analysis and may have multiple solutions. Involve multiple variables but are still solvable with the right approach (constructing a building demands coordination among various stakeholders sharing goals, but the process can be planned and executed with known methods).
- **Complex:** Situations where cause and effect are not immediately obvious. Understanding emerges through experimentation, and solutions evolve over time. Characterised by interdependent variables and unpredictable outcomes, having multiple solutions and minor changes can lead to different results (as in climate change or managing a large organisation).
- **Chaotic (or “Wicked”):** There is no clear cause-and-effect relationship, socially complex, with stakeholders conflicting in views and interests, thus lacking clear definitions and straightforward solutions. Require action to restore order before identifying patterns. Solutions to poverty or healthcare reform are typically not right or wrong but rather better or worse, as often lead to unintended consequences, which can show new problems.
- **Confusion (or “Disorder”, or “Apex”):** A state of confusion where it’s unclear which domain the situation belongs to, requiring careful discernment.



2.4.4 Conclusion

The concepts of simple, complicated, complex, and wicked problems are useful for effective systems analysis and design. Since engineering frequently involves navigating these categories, particularly in the context of systems, conceptual modelling emerges as an invaluable for managing socio-technical systems, especially information systems, and moving them towards a more structured, complicated problem space.

However, it is also important to note that the boundaries between these categories might be blurry. For example, the term "complex" stress the difficulty involved, even when "complicated" might be more accurate. Understanding these concepts while also recognising these nuances is essential for deciding on how the most appropriate strategies, methods, techniques, and tools to apply in problem-solving.

¹ <https://hbr.org/2007/11/a-leaders-framework-for-decision-making>

² https://en.wikipedia.org/wiki/Cynefin_framework

2.5 <concept> The Universe of Discourse

In systems engineering and requirements engineering, the **Universe of Discourse** is a fundamental concept that serves as the source from which all system requirements derive. It encompasses the entire scope of relevant information, concepts, entities, and relationships within a specific context or domain. It provides the foundational understanding necessary to ensure that the system under development aligns with the needs and constraints of its environment.

2.5.1 Universe of Discourse

*The **Universe of Discourse (UoD)** encompasses all relevant information, concepts, entities, and relationships that are pertinent to the system's operation. The UoD is not a product of any analysis process; rather, it is the focus of the analysis and the source from which requirements derive. As such, the UoD serves as the baseline for understanding what the system is or needs to achieve, and the environment in which it must function.*

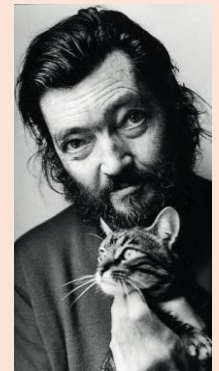
The UoD is dynamic and can evolve as the requirements process uncovers new insights or as external conditions change. However, its primary role remains to feed the requirements process with the necessary contextual information, ensuring that all relevant aspects of the system's environment are considered.

2.5.2 Key Aspects of the Universe of Discourse

- **Source of Requirements:** The UoD is the starting point for the requirements engineering process. It defines the context in which the system operates, outlining all pertinent elements that must be considered when gathering and specifying requirements. The UoD is not a product of the requirements process; rather, it informs and guides that process.
- **Contextual Understanding:** The UoD provides a comprehensive understanding of the system's environment, including stakeholders, their needs, operational constraints, and external factors that could influence the system. This contextual knowledge is essential for making informed decisions throughout the development process and ensuring the system meets real-world conditions.
- **Dynamic Nature:** While the UoD is the source for requirements, it is not static. As the requirements engineering process progresses, the understanding of the UoD may evolve. New information, stakeholder insights, or external changes can refine and expand the UoD. However, these changes serve to better inform the process, not to alter the fundamental role of the UoD as the origin of system requirements.
- **Common Language:** Establishing a UoD helps create a shared language and understanding among stakeholders. This common framework is crucial for avoiding ambiguity and ensuring that all parties involved in the project have a consistent view of the system's context and goals.

Preâmbulo às instruções para dar corda ao relógio

Pensa nisto: quando te oferecem um relógio oferecem-te um pequeno inferno florido, uma prisão de rosas, um calabouço de ar. Não te dão somente o relógio, muitos parabéns, que te dure muitos e bons, é uma ótima marca, suíço com não sei quantos rubis; não te oferecem somente esse pequeno pedreiro que prenderás ao pulso e passearás contigo. Oferecem-te – ignoram-no, é terrível ignorá-lo – um novo bocado frágil e precário de ti mesmo, algo que é teu mas não é o teu corpo, que tens de prender ao teu corpo com uma corrente como um bracito desesperado pendente do pulso. Oferecem-te a necessidade de lhez dar corda todos os dias, a obrigação de dar corda para que continue a ser um relógio; oferecem-te a obsessão de ver as horas certas nas montras das joalharias, o sinal horário na rádio, o serviço telefónico. Oferecem-te o medo de o perder, de seres roubado, de que caia no chão e se parta. Oferecem-te uma marca, a convicção que é uma marca superior às outras, oferecem-te a tentação de comparares o teu com os outros relógios. Não te oferecem um relógio, és tu o oferecido, a ti oferecem para o nascimento do relógio.



Julio Cortázar

2.5.3 Traceability

Traceability with the UoD is critical to maintaining the integrity of the requirements throughout the system's lifecycle. Traceability means that each requirement must be traced back to its origin within the UoD and followed through every stage of the development process. This traceability is crucial for verification, validation, change management, and accountability, ensuring that the final system meets all its requirements and that all aspects of the UoD have been addressed.

2.5.4 The System Context and the System Domain

From the UoD, one or more specific system domains are identified, representing distinct areas or problem spaces that the system must address. These domains are internally focused aspects of the system, and to effectively analyse them, domain models are created. These models provide a formal representation of the system domain, capturing key entities and their relationships. Additionally, the system context, which includes the external environment interacting with the system, is also derived from the UoD. The context model represents these interactions, ensuring that external influences are adequately considered in the system design.

2.5.5 Conclusion

The Universe of Discourse is the essential source from which all system requirements are derived. It provides the contextual foundation necessary for understanding the environment in which the system will operate. By defining the scope, facilitating a shared understanding, and dynamically informing the requirements engineering process, the UoD ensures that the system is developed in alignment with its intended environment and stakeholder needs.

2.6 <concept> System Context and System Domain

In systems engineering and requirements engineering, the System Context and System Domain are critical concepts that stem directly from the Universe of Discourse (UoD). These concepts help to delineate the internal and external aspects of the system, ensuring that all relevant factors are not left behind during the next steps of the systems engineering process.

2.6.1 System Domain

The system domain refers to the internal environment of the system, encompassing all relevant entities, objects, rules, and relationships that define the specific problem space the system is intended to address.

The system domain is derived from the UoD and focuses on what happens within the system's boundaries. It typically includes:

- Entities and Objects: The key components within the system, such as data structures and components.
- Relationships and Interactions: The connections between these entities, dictating how they collaborate to achieve the system's objectives.
- Business Rules and Policies: The specific rules and constraints that govern the system's operation within the domain.

The domain model is a formal representation of the system domain. This model provides a structured representation of the system domain, capturing the essential entities and their interactions. It is crucial for ensuring that the system's internal structure is well-defined and aligns with the requirements derived from the UoD.

2.6.2 System Context

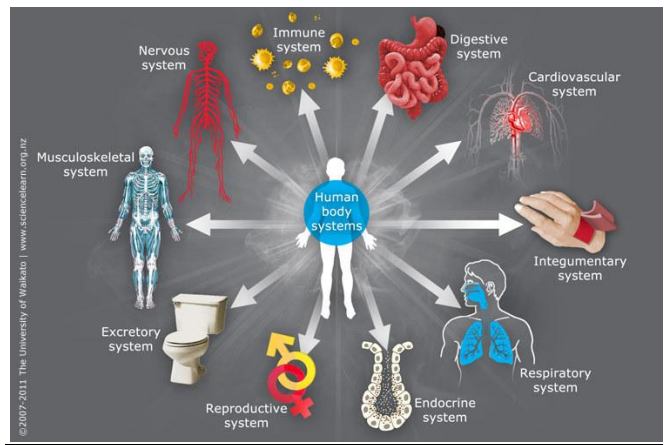
The system context refers to the external environment that interacts with or influences the system. This includes all external entities, processes, and factors that are outside the system's boundaries but have a significant impact on its operation.

The system context also derives from the UoD. Understanding it is essential for designing a system that can effectively interact with its environment.

In an architectural representation, the context model must make clear the boundaries of the system and the interactions of the system with the external actors (what can be human users or other systems, for example) and also stress any other relevant environmental factors (for example, those imposing the so called "contextual requirements", such as compliance).

2.6.3 Key Differences

- Internal vs. External Focus: The system domain is internally focused, dealing with the entities, relationships, and rules within the system. The system context, however, is externally focused, dealing with how the system interacts with external factors.
- Scope: The system domain is concerned with the detailed aspects of the system's internal operations, while the system context encompasses everything outside the system that could affect its performance or functionality.



2.6.4 System of Systems

The concept of **System of Systems (SoS)** refers to a collection of independent, self-contained systems that, when combined, form a larger, more complex system that offers capabilities beyond those of the individual systems. Each constituent system in an SoS operates within its own context and domain and interacts with others through shared goals or missions.

In an SoS, the concept of system domain is not limited to a single, isolated environment. Instead, multiple domains coexist, each representing the internal workings of the individual systems that make up the SoS. These domains are characterised by their own entities, relationships, and rules, which are often distinct. Each system in an SoS also has its own external environment, leading to the existence of multiple system contexts. These contexts must be understood both individually and collectively to ensure that the SoS operates cohesively.

The challenge lies in understanding and modelling how these diverse system domains and contexts interact with each other. This requires an extension of the traditional models to account for multiple sub-contexts and cross-domain interactions, dependencies, and potential conflicts. The result are more comprehensive domain models, that might not only represent individual systems but also their interactions within the SoS.

2.6.5 Conclusion

The context and the domain of the system are integral concepts in systems engineering. They serve different but complementary roles: **the system domain focuses on the internal workings of the system, while the system context addresses the external environment and its interactions with the system.** By understanding and modelling both the system domain and context, analysts can ensure that the system is robust, well-integrated, and capable of operating effectively within its intended environment.

The domain model and the context model must be part of a broader concept known as the system model. The system model must include these models along with other models, such as behavioural or interaction models, providing a comprehensive representation of how the system operates internally and interacts with its environment.

The SoS paradigm expands the traditional concepts of domain and context by introducing multiple domains and sub-contexts. In an SoS approach, each system maintains its own domain and context, yet these must be understood within the larger framework of the SoS to ensure cohesion. This holistic approach is essential for interdependent systems that function effectively in a dynamic and interconnected environment.

A **system boundary** distinguishes the system of interest (or **Eol - Entity of Interest**) from its environment. It establishes which entities and activities are treated as constituents of the system and which remain external but may interact with or affect it.

The boundary is conceptual rather than necessarily physical, organisational, or technological. It does not have to follow the limits of a software application, a department, a building, or an organisation. It is drawn according to the concerns being addressed and the responsibilities assigned to the system.

Different concerns may justify different system boundaries. However, the selected boundary must be explicit, useful for the intended analysis, and consistent with the available evidence.

2.7.1 Sociotechnical systems

An Eol may include more than technology. A **sociotechnical system** can comprise:

- people acting in organisational or operational roles;
- processes and coordinated activities;
- rules, procedures, and responsibilities;
- information and communication;
- software applications and services;
- devices, machines, and other physical resources.

For example, a hospital appointment system might be conceived narrowly as an appointment software application. Alternatively, it might be conceived as a broader service including patients, administrative staff, scheduling procedures, organisational rules, communication channels, and software.

Neither boundary is automatically correct. They represent different systems and support different analyses.

2.7.2 Boundary and responsibility

The boundary determines what the system can be required to do. A **system requirement** assigns an obligation to the system of interest or to one of its constituents. It should not impose an obligation on an external entity that the system does not control.

Consider a patient who must provide valid contact details. If the patient is external to the system, this statement is not a system requirement: “The patient shall provide a valid telephone number.”

Depending on the context, it may instead express an assumption, participation rule, service condition, or external responsibility.

A corresponding system requirement could be: “The appointment system shall request and validate the patient’s contact details before confirming the appointment.”

A requirement may therefore refer to an external entity while still assigning the obligation to the system.

When the system is sociotechnical, obligations may be allocated to different kinds of constituent: “The appointment procedure shall require urgent requests to be reviewed by a qualified healthcare professional.”

This is a system requirement if the appointment procedure and the professional role are within the system boundary. It is not a software requirement, although software may support its satisfaction.

2.7.3 Boundaries are consequential

Changing the boundary changes the responsibilities that can be assigned. To reduce missed appointments, a software-only system might be responsible for sending reminders. A broader sociotechnical system might combine automated reminders, telephone follow-up by staff, and rules for rescheduling appointments.

System boundaries should therefore be reviewed whenever requirements appear impossible to assign, depend excessively on external behaviour, or restrict the analysis prematurely to a technological solution.

The system boundary should also be distinguished from the **project boundary**: the project may change only part of an existing system, while requirements may still concern the behaviour and qualities of the broader system.

Key takeaway: A system requirement is meaningful only when the system boundary makes clear who or what is responsible for satisfying it.

2.8 <concept> Systems as Composed Structures

Once a system of interest has been defined, it may be examined from two complementary perspectives:

- A **black-box perspective** focuses on what the system does and how it interacts with its environment.
- A **white-box perspective** exposes an internal structure through which those responsibilities are performed.

Neither perspective reveals the system as it “naturally is.” The boundary, internal decomposition, and relevant interactions are selected according to the concerns and purpose of the analysis.

2.8.1 System Boundary and Black-box Perspective

A system boundary distinguishes what is considered part of the system from what is treated as its environment. This boundary is conceptual: it expresses an analytical decision and does not necessarily correspond to a physical enclosure, organisational division, or ownership boundary.

In a black-box perspective, the system is treated as a whole. The modeller describes:

- the responsibilities or behaviour attributed to the system;
- the external entities with which it interacts;
- what crosses the system boundary;
- the conditions governing those interactions.

This perspective is especially useful because it allows stakeholders to agree on the system’s responsibilities without committing to an internal solution.

2.8.2 Internal Structure and White-box Perspective

A white-box perspective describes the system as a composition of interacting **parts**. A part is an element considered to contribute to the whole within a particular system conception. It may represent a physical component, logical element, organisational unit, human role, or another relevant constituent. Composition is more than listing parts.

An internal structure also describes:

- the responsibilities assigned to each part;
- how the parts depend on or interact with one another;
- how external interactions are handled internally;
- how the collaboration of the parts produces the behaviour attributed to the whole.

Different concerns may produce different decompositions of the same system. A physical decomposition may identify devices and material connections; a logical decomposition may identify information-processing elements; a functional decomposition may identify responsibilities; an organisational decomposition may identify teams and roles. These structures may complement one another, but they must not be assumed to correspond one-to-one.

In this perspective, encapsulation is also an important concept. External elements interact through defined interfaces without needing to know the internal structure that provides them. Internal parts can therefore be reorganised or replaced, provided that the externally relevant interfaces and behaviour remain valid.

2.8.2.1 Interfaces

An **interface** specifies how a system or part may interact with its environment. It describes what may be provided,

requested, received, or exchanged and may include associated rules, conditions, or constraints. Depending on the system conception, an interface may concern information, services, signals, energy, or material.

2.8.2.2 Interaction Points

An **interaction point** identifies where or through which role an interface is made available at the boundary of a system or part. Some modelling languages call such an interaction point a **port**. A port is therefore not the interface itself: the port identifies an interaction point, while the interface specifies the interaction supported there.

2.8.2.3 Interaction Paths

An **interaction path** connects interaction points or parts and indicates that interaction can occur between them. Depending on the modelling language, such a path may be represented as a connector, communication link, channel, or flow.

2.8.3 Example: An Automated Parking System

From a black-box perspective, an Automated Parking System may admit vehicles, manages available spaces, collect payments, and authorise exits. It interacts with drivers, vehicles, operators, and external payment services.

A logical white-box perspective might identify an Access Controller, Occupancy Manager, Payment Manager, and Device Gateway. Their interfaces describe the information and commands they provide or require, while interaction paths show how they collaborate.

A physical white-box perspective might instead identify barriers, sensors, payment terminals, computing devices, and communication networks. The physical and logical decompositions address different concerns and need not contain corresponding parts.

2.8.4 Modelling Guidance

When describing a composed system:

1. Begin with the concern and purpose of the model.
2. Define the system boundary and its external interactions.
3. Describe the system first as a black box.
4. Introduce an internal structure only when it is relevant to the concern.
5. Assign a clear responsibility to every included part.
6. Distinguish interfaces from interaction points and interaction paths.
7. Check that every external interaction is supported by the internal structure.
8. Check that every internal part contributes to the responsibilities attributed to the whole.

2.8.5 Conclusion

Systems may be understood as composed structures, **but their decomposition is an analytical construction rather than a natural partition of reality**. Black-box models describe externally visible responsibilities and interactions; white-box models explain how selected internal parts collaborate to produce them. Interfaces, interaction points, and interaction paths connect these perspectives while supporting modularity, encapsulation, and reasoned system decomposition.

2.9 <example> UoD, Context and Domain

What follows are two generic examples to illustrate, for a given **Entity of Interest, or EOI**, the relations between UoD, context and domain.

2.9.1 Example 1: Autonomous Vehicle Control System

Universe of Discourse: The Autonomous Vehicle Control System (AVCS) UoD includes all the relevant information required to understand and design a system for autonomous driving. This involves the concepts and entities related to vehicle dynamics, sensor inputs, environmental interactions (such as detecting obstacles, road markings, and other vehicles), decision-making algorithms, and actuators. The UoD covers the complete set of elements that influence how the AVCS operates and interacts with its surroundings.

Analysis:

- In this case, the EOI is the AVCS, a subsystem of the vehicle that included several components (such as sensors, controllers, and actuators) as well as software systems responsible for decision-making and control. The system is designed to process sensor data, make real-time decisions, and control vehicle behaviour.
- The context of the AVCS involves its interaction with external elements like traffic laws, pedestrians, road conditions, and weather. These external factors provide constraints and requirements that the system must account for during its operation, such as detecting and avoiding obstacles or responding to traffic signals. Also, part of that context are the parts of the vehicle that must be controlled, such as breaks, engine, lights, etc.
- The domain of the system refers to the conceptual space that the AVCS models and operates within. This includes entities such as internal conceptualizations of the vehicle, road segments, traffic signals, pedestrians, and other vehicles, along with their relationships (e.g., vehicle following a road, pedestrian crossing a street). The domain model in this case defines how the AVCS perceives and interacts with these elements (whether through block or class diagrams representing vehicles, obstacles, and sensor data, or through state machines modelling the decision-making process). This domain defines the internal representation of the world that the system uses to make decisions. The domain model would capture all the necessary concepts, entities, and their relationships for autonomous driving.

In this example, the UoD must provide the necessary information about both the physical world and the AVCS's internal operations. The EOI is the AVCS itself, designed to operate within the constraints of its context (traffic, environment, vehicle) and based on the domain that therefore includes models of that (concepts, attributes and relations for roads, vehicles, traffic behaviour, etc.).

2.9.2 Example 2: Online Banking System

Universe of Discourse: The UoD for an Online Banking System (OBS) covers all the information related to conducting secure financial transactions over the internet, involving everything required to understand the system's functionality and constraints. This includes concepts like user authentication, transaction management, account balances, security protocols (e.g., encryption), and compliance with financial regulations.

Analysis:

- The EOI is the OBS, which must allow users to perform tasks like checking balances, transferring funds, and making payments. The OBS can be perceived as system of systems consisting of subsystems such as user interfaces, databases for storing account information, transaction processing systems, as also the need for security protocols ensuring safe financial exchanges.
- The context of the OBS includes external factors such as users, regulatory bodies (e.g., sources of anti-money laundering laws, privacy laws, etc.), external financial systems (e.g., interbank networks), and cyber threat agents. These external elements impose constraints on how the system should behave, ensuring compliance and security.
- The domain of the system refers to the conceptual model of banking that the system needs to represent internally. It includes entities like users, accounts, transactions, balances, and banks, along with relationships such as users owning accounts and accounts being linked to transactions. A domain model for the OBS would capture these concepts in diagrams like UML class diagrams.

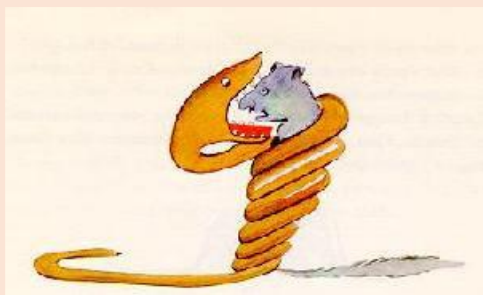
In this scenario, the UoD comprises the broader online banking activities, regulations, and security. The EOI is the OBS itself, structured around a domain model that represents the internal structure of users, accounts, transactions, etc. The system operates within the legal and cybersecurity constraints of its context, but its internal operations are driven by the rules of banking.

2.9.3 Conclusion

In both cases, the Universe of Discourse (UoD) encompasses all the information necessary for understanding the problem space and defining the system. The **EOI** is the system itself, whether it is the AVCS or OBS. The context refers to external factors that influence how the system must behave. Finally, the domain of the system refers to the internal conceptual model, where key entities and relationships relevant to the system's function are defined and modelled, helping to organise and guide the system's development. This domain forms the basis for how the system interacts with and represents the world it operates in.

(...)

Once when I was six years old I saw a magnificent picture in a book, called True Stories from Nature, about the primeval forest. It was a picture of a boa constrictor in the act of swallowing an animal. Here is a copy of the drawing.



In the book it said: "Boa constrictors swallow their prey whole, without chewing it. After that they are not able to move, and they sleep through the six months that they need for digestion."

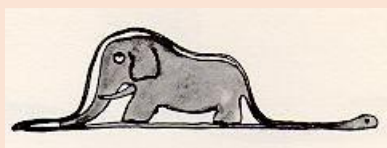
I pondered deeply, then, over the adventures of the jungle. And after some work with a colored pencil I succeeded in making my first drawing. My Drawing Number One. It looked something like this:



I showed my masterpiece to the grown-ups, and asked them whether the drawing frightened them.

But they answered: "Frighten? Why should any one be frightened by a hat?"

My drawing was not a picture of a hat. It was a picture of a boa constrictor digesting an elephant. But since the grown-ups were not able to understand it, I made another drawing: I drew the inside of a boa constrictor, so that the grown-ups could see it clearly. They always need to have things explained. My Drawing Number Two looked like this:



The grown-ups' response, this time, was to advise me to lay aside my drawings of boa constrictors, whether from the inside or the outside, and devote myself instead to geography, history, arithmetic, and grammar. That is why, at the age of six, I gave up what might have been a

magnificent career as a painter. I had been disheartened by the failure of my Drawing Number One and my Drawing Number Two. Grown-ups never understand anything by themselves, and it is tiresome for children to be always and forever explaining things to them.

So then I chose another profession, and learned to pilot airplanes. I have flown a little over all parts of the world; and it is true that geography has been very useful to me. At a glance I can distinguish China from Arizona. If one gets lost in the night, such knowledge is valuable.

In the course of this life I have had a great many encounters with a great many people who have been concerned with matters of consequence. I have lived a great deal among grown-ups. I have seen them intimately, close at hand. And that hasn't much improved my opinion of them.

Whenever I met one of them who seemed to me at all clear-sighted, I tried the experiment of showing him my Drawing Number One, which I have always kept. I would try to find out, so, if this was a person of true understanding. But, whoever it was, he, or she, would always say:

"That is a hat."

Then I would never talk to that person about boa constrictors, or primeval forests, or stars. I would bring myself down to his level. I would talk to him about bridge, and golf, and politics, and neckties. And the grown-up would be greatly pleased to have met such a sensible man.

(...)



2.11 <concept> Life Cycle and Methods

Systems engineering (SysEng) is an interdisciplinary field focused on designing, integrating, and managing complex systems throughout their life cycles. Unlike software engineering, which primarily deals with software development, systems engineering encompasses a broader scope, including hardware, software, processes, and even human factors. The goal is to ensure that all components of a system work together effectively to meet the needs of stakeholders.

2.11.1 The System's Life Cycle

In SysEng, the concept of system's life cycle refers to the stages a system goes through from conception to decommissioning. Understanding and managing this life cycle is crucial for delivering systems that meet stakeholder needs, perform reliably, and can be maintained over time:

- **Conceptual Phase:** This initial phase involves defining the problem, identifying stakeholders, and gathering high-level requirements. The focus is on exploring different solutions and selecting the best approach.
- **Requirements and Architecture Development:** Here, detailed requirements are gathered and documented. The system's architecture is designed, specifying the components and their interactions.
- **Design and Implementation:** During this phase, the system components are designed in detail and developed. Integration planning begins to ensure that all parts will work together as intended.
- **Integration and Testing:** The individual components are integrated into a complete system, and rigorous testing is conducted to verify that the system meets its requirements and performs reliably.
- **Deployment and Operation:** Once testing is complete, the system is deployed in its intended environment. Systems engineering continues to monitor performance, ensuring the system operates as expected.
- **Maintenance and Upgrades:** Systems require ongoing maintenance and updates to remain functional and meet evolving needs. This includes bug fixes, enhancements, and adapting the system to new requirements or technology.
- **Decommissioning:** Eventually, systems reach the end of their useful life. This phase involves safely decommissioning the system, ensuring for example that data is preserved or securely disposed of, that the system is retired without disrupting ongoing operations (e.g., replacing a software application by a new one), or that waste is properly handled (e.g., dismantling a power plant, a big ship, etc.).

2.11.2 Methods for Systems Engineering³

Several methods are commonly used in SysEng to manage the challenges of large-scale projects. Each has its strengths and is suited to different types of projects; in many cases, more than one method is used in relation to the same system. The main families methods for this purpose are:

- **Waterfall:** The Waterfall model is one of the earliest methods used in systems

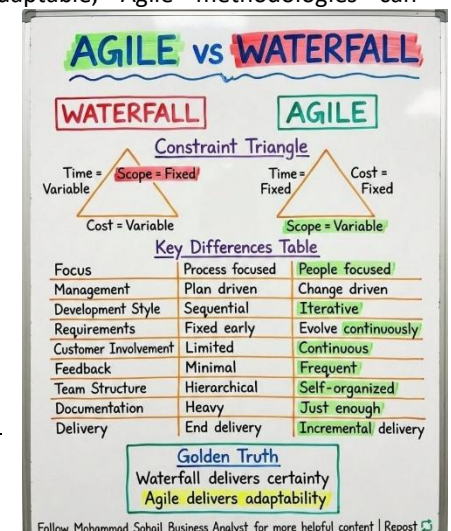
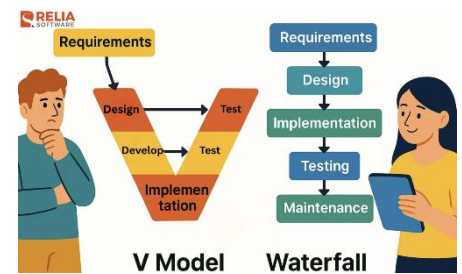
engineering. It follows a linear, sequential approach where each phase of the project must be completed before the next begins. This model is straightforward and easy to manage, making it suitable for projects with well-defined requirements and minimal changes. However, it is less flexible in adapting to changes once the project is underway.

- **V-Model:** The V-Model is an extension of the Waterfall model that emphasizes verification and validation at each stage of development. It's particularly useful in safety-critical systems, like in aerospace and medical devices, where ensuring that each component works correctly before moving on is crucial.
- **Iterative and Incremental Development:** This method involves breaking down the system into smaller parts, which are developed, tested, and integrated incrementally. This approach allows for continuous feedback and adaptation, making it more flexible than the previous models. It's particularly useful in systems where requirements might evolve over time.
- **Agile Systems Engineering:** Agile methods, originally developed for software engineering, have been adapted for systems engineering. Agile focuses on iterative development, flexibility, and stakeholder collaboration. It's particularly useful in dynamic environments where requirements change frequently, allowing for rapid adjustments and continuous delivery of value. However, while highly adaptable, Agile methodologies can sometimes lead to challenges in maintaining long-term strategic focus and ensuring comprehensive documentation, which are crucial for complex, large-scale projects.

2.11.3 Conclusion

SysEng deals with the complexity of large-

scale, multidisciplinary projects. By focusing on the integration of multiple system components and ensuring that they work together effectively, SysEng helps deliver reliable, efficient, and user-focused solutions. Understanding the core concepts and methods of SysEng is important to tackle those challenges.



³ Images respectively from:
<https://reliasoftware.com/blog/v-model>

https://www.linkedin.com/posts/mohammadsahilbusinessanalyst_the-debate-is-not-agile-vs-waterfall-the-activity-7457270391401533440-Q81G/

2.12 <concept> Understanding versus Doing

In systems engineering, the distinction between understanding and doing is crucial, especially in the context of analysis and design. These concepts align closely with the phases of the PDCA cycle, which is foundational to both management and engineering processes.

2.12.1 Plan-Do-Check-Act

The PDCA (Plan-Do-Check-Act) cycle is a widely used iterative process for continuous improvement in business and engineering. Developed by Walter A. Shewhart and popularised by W. Edwards Deming, it is rooted in quality management and aims to drive ongoing enhancement through systematic problem-solving. Its simplicity and flexibility have made it a key generic method in industries like manufacturing, healthcare, and project management. The PDCA cycle ensures that actions are based on thorough analysis and that solutions are constantly refined through feedback and iteration.

2.12.2 Understanding: The Analysis Activity

"Understanding" refers to the analysis phase, where the focus is on planning and gaining a comprehensive understanding of the problem. In the PDCA cycle, this is primarily associated with the Plan and Check phases. In this stage, systems engineers work to define the problem, study the existing system, and establish the necessary requirements.

During the Plan phase, engineers focus on identifying the system's structure, behaviour, and context. This involves gathering data, understanding stakeholder needs, and examining the system's current performance. For example, improving a transportation network might require an analysis of traffic patterns and bottlenecks to clarify the root causes of inefficiency. Additionally, in the Check phase, the analysis helps to validate whether the system's current performance aligns with the desired outcomes, identifying any gaps that need to be addressed. Raising accurate requirements, both functional and non-functional, is a key outcome of this understanding phase, forming the basis for subsequent actions.

No tangible solutions are created during this stage; instead, the goal is to ensure a full comprehension of the problem, laying the groundwork for effective design.

2.12.3 Doing: The Design Activity

"Doing" refers to the design and implementation activities, which are linked to the Do and Act phases of the PDCA cycle. This is where engineers take actionable steps based on the insights gained during the understanding phase.

In the Do phase, systems engineers begin designing solutions to meet the requirements identified in the analysis. For example, redesigning a transportation network might involve reconfiguring routes or introducing new traffic management technologies. Engineers create models, simulations, or prototypes to test the feasibility of these designs before full-scale implementation.

The Act phase involves implementing these changes and ensuring that the solution meets the established requirements. If the design does not perform as expected the cycle prompts engineers to revisit earlier phases, highlighting the iterative nature of systems engineering.

2.12.4 Relationship Between Understanding and Doing

Understanding and doing are interconnected within the PDCA cycle. Understanding focuses on accurate problem analysis, ensuring the foundation for the solution is sound. Doing translates this understanding into practical solutions through design and execution. The cycle highlights that these steps are not separate or one-off events but part of a continuous improvement loop. Agile methodologies, which favour shorter cycles, and even traditional waterfall methods with longer cycles, both rely on this balance between understanding and doing.

2.12.5 MAPE-K: A Related Framework

While PDCA is widely used for continuous improvement, it's important to note that there are other frameworks, such as **MAPE-K** (Monitor-Analyse-Plan-Execute over a shared Knowledge), which is commonly used in autonomic computing and self-managing systems. MAPE-K is particularly relevant in IT environments where systems need to autonomously adapt to changing conditions. It shares some structural similarities with PDCA in terms of its iterative approach but focuses more on automation and real-time responses, with minimal human intervention. MAPE-K is crucial for dynamic, self-adaptive systems like cloud computing, but it operates in a more automated context compared to the human-driven PDCA cycle.

2.12.6 Conclusion

The PDCA cycle provides an essential framework for balancing understanding (analysis) and doing (design). Effective systems engineering relies on thorough analysis to inform decision-making and design, while iterative cycles ensure that solutions are refined and improved over time. Understanding lays the groundwork for practical action, and doing brings those insights to life, creating a continuous loop of improvement. While other frameworks like MAPE-K serve different purposes in more automated settings, PDCA continues to be essential for human-guided, systematic improvement, allowing businesses to refine processes and systems, improve quality, and enhance efficiency over time.

2.13 <technique> Conceptual Models and ISO/IEC/IEEE 42010

A system architecture expresses the organisation of a system. The standard ISO/IEC/IEEE 42010:2022⁴ (informally referred to as only "ISO 42010") provides a framework for describing and understanding the architecture of systems, even if without using the concept of "system."

2.13.1 What is a "system"?

The concept of "system" can vary depending on the domain. What a software engineer considers a system might be quite different from how a medical doctor, from a legal person, or from other domain expert understands it. For example, ISO 9000:2015 defines system simply as "set of interrelated or interacting elements"⁵, while INCOSE defines that "a system is an arrangement of parts or elements that together exhibit behaviour or meaning that the individual constituents do not"⁶. In the legal domain the concept is born already complex: "A legal system is the framework of rules, procedures, and institutions that a community uses to interpret and enforce their laws"⁷. To allow ISO 42010 to be applied across different domains, it intentionally avoids defining "system" in a domain-specific way. Instead, the standard introduces the more flexible concept of an "**Entity of Interest**" to represent whatever is being architected.

2.13.2 Stakeholders, Concerns, and Domains

A key principle is that architecture descriptions must address the concerns of stakeholders, who might have different interpretations of what a "system" is, based on their roles, interests, and goals. For example, a legal person might be concerned with compliance with established laws, a business analyst might be concerned with workflows and information flows, while a technical architect might focus on software modules and infrastructure. By not defining "system," the standard allows stakeholders from any domain to apply the architecture framework to their particular **Entity of Interest**, addressing their unique concerns.

2.13.3 Core Concepts

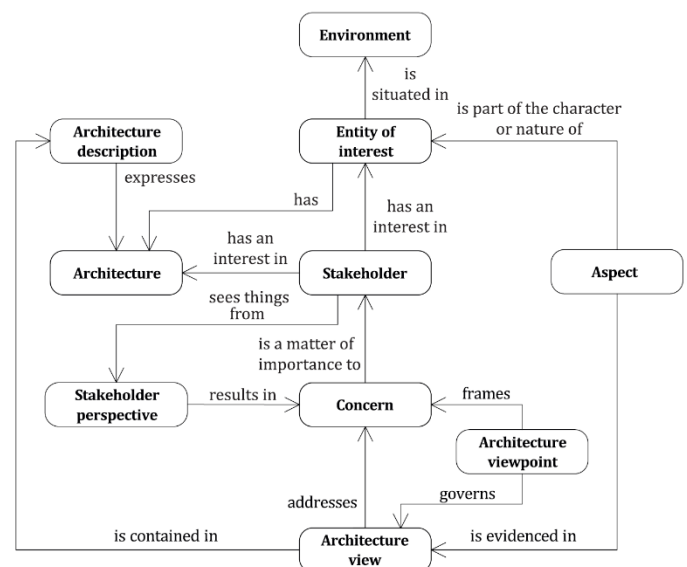
Rather than providing a rigid definition of "system," this standard introduces the abstract concept of "**Entity of Interest**", or **EoI**. The **EoI** could be a physical system, software system, socio-technical system, business process, or even an entire enterprise. The focus is on describing the architecture of whatever is relevant to stakeholders and their concerns, regardless of how it is labelled. This concept is broader than "system" because it includes not only traditional systems but also components, services, collections of systems, or even non-technical entities that might require an architectural description. While it avoids defining "system," ISO 42010 provides clear definitions for describing architectures. Key concepts for that include:

- **Entity of Interest:** The EoI is subject of the architecture description. It is whatever the architecture is concerned with, whether it's a unique system, a sub-system in a system of systems, or something else.
- **Architecture:** The fundamental organisation of the EoI, represented by its components, their relationships, and

the guiding principles behind its design and evolution. The standard allows this to apply to any type of entity.

- **Architecture Description:** An Architecture Description (**AD**) structured work product that expresses the architecture of an EoI. It includes various views and viewpoints designed to address the concerns of the stakeholders.
- **Stakeholders:** The individuals or organisations with an interest in the architecture of the **Entity of Interest**. Stakeholders can vary widely depending on the domain (e.g., end-users, system engineers, regulators, managers). Their concerns must be addressed by the architecture description. More recent thinking also proposes to refer to this concept as "Interested Party", instead of "stakeholder."⁸
- **Concerns:** Issues or aspects that are important to stakeholders regarding the architecture. Concerns can include performance, security, reliability, usability, cost, and more. The architecture description aims to address these concerns in a manner that satisfies the stakeholders' needs.
- **Viewpoint:** A specification that defines how a particular set of stakeholder concerns should be constructed.
- **View:** A representation of the architecture from the perspective of a specific viewpoint, showing the EoI in a way that is meaningful to certain stakeholders.

2.13.4 Conclusion



The ISO/IEC/IEEE 42010:2022 standard provides a framework for understanding and managing the architecture descriptions of complicated systems. It avoids defining the concept of "system" because it is intended to apply across a wide variety of domains, each of which may have different interpretations of what a "system" is. Instead, it defines the broader term "**Entity of Interest**" to refer to whatever is being described in an architectural framework. The focus on creating architecture descriptions that address the concerns of stakeholders, which are specific to each domain and context. This flexibility ensures it can be applied universally, without being constrained by a rigid definition of "system".

⁴ <http://www.iso-architecture.org/ieee-1471/> ISO/IEC 1471 (ISO/IEC/IEEE 42010 was first born as IEEE 1471)

⁵ <https://www.iso.org/obp/ui/#iso:std:iso:9000:ed-4:v1:en>

⁶ <https://www.incose.org/about-systems-engineering/system-and-se-definitions/general-system-definition>

⁷ https://www.law.cornell.edu/wex/legal_systems

⁸ <https://www.quality.org/knowledge/iso-90012015-%E2%80%93-who-are-interested-parties>



The Indian fable “Six blind man and the elephant” (or, how our sensory perceptions can lead to misinterpretations...)

The application of the concepts defined in ISO/IEC/IEEE 42010:2022 is essential for SysEng and ReqEng, as both disciplines heavily rely on clear, structured approaches to architecture description.

2.14.1 SysEng and Architecture Description

In systems engineering, the architecture is a central element that ensures the system's components, and their interactions are structured to meet the desired goals. The ISO/IEC/IEEE 42010:2022 standard provides a framework for this by enabling to describe the architecture in a way that addresses various stakeholders' concerns. The architecture description acts as a communication object between engineers, managers, regulators, and other stakeholders, facilitating the integration of hardware, software, processes, and people into a cohesive system.

2.14.2 ReqEng and Stakeholder Concerns

Requirements engineering focuses on identifying, documenting, and managing stakeholder needs and concerns to ensure that the system meets its intended objectives. ISO/IEC/IEEE 42010:2022 reinforces the importance of addressing these concerns within the architecture description, as failure to consider stakeholder needs early in the process can lead to costly rework and system failures.

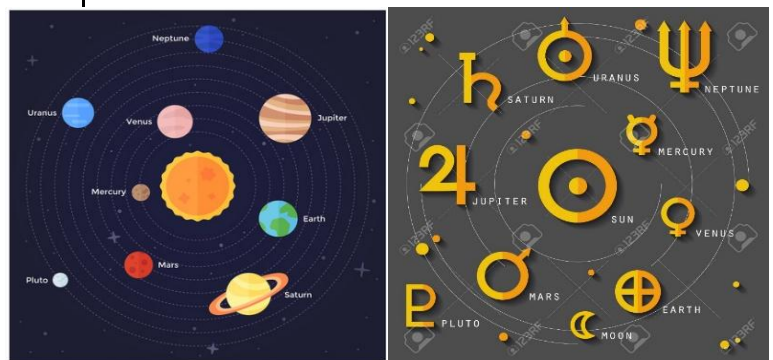
2.14.3 Modelling Languages

Language such as ArchiMate, BPMN, and UML provide different ways to structure and visualise systems depending on the stakeholders' concerns and the aspects of the system being modelled:

- ArchiMate is particularly useful in enterprise architecture, where different layers (business, application, and technology) are modelled. Each of these layers can be viewed from various perspectives, such as performance or security, ensuring that the architecture of the EoI addresses all stakeholder concerns across different domains.
- BPMN (Business Process Model and Notation) focuses on business process modelling. Stakeholders interested in workflow efficiency use it to represent processes in alignment with their concerns, such as optimising resource use or reducing bottlenecks. Here, the architecture of the process is a vital element of the broader EoI architecture.
- UML is used in software engineering and provides multiple diagram types (class diagrams, sequence diagrams, use case diagrams, etc.) to represent structural and behavioural aspects of a system. Each diagram offers a view that caters to specific stakeholder concerns, whether it's understanding system interactions or the overall functionality.

2.14.4 Conclusion

The ISO/IEC/IEEE 42010:2022 defines concepts such as concerns, stakeholders, systems, viewpoints, views, and models. Modelling languages like ArchiMate, BPMN, and UML implement these concepts, offering techniques to create models that support broader systems engineering efforts. By integrating these modelling languages with the architecture framework defined in ISO/IEC/IEEE 42010:2022, both systems engineering and requirements engineering can effectively align the system's architecture with stakeholder needs. The standard provides the structure to ensure that all relevant concerns are considered, and that the system architecture is robust, well-communicated, and thoroughly aligned with the system's goals and requirements.



2.15 <further> Strategy, Goals, Outcomes, and Principles

Make or change a system always implies turbulence, distraction and costs. Any external or internal condition that motivates an organisation to change is usual referred as a “driver.”

Understanding this high-level concept and the related concepts of “goal,” “outcome,” and “principle” is essential. These concepts are foundational as they guide the efforts to ensure that the systems of an organisation align with the strategic objectives of that organisation. This is detail is the purpose of the area of Enterprise Architecture, which goes beyond our scope here, but it is anyway important to brief it here-

2.15.1 Goals: The Motivational Drivers

A goal is a high-level statement of intent or a desired state that the system or organisation aims to achieve. Goals are strategic in nature, providing direction and ensuring that all subsequent design and development activities align with the overarching vision of the system.

For example, in a system designed to manage public transportation, a goal might be “Enhance the efficiency of public transport services.” This goal must influence every aspect of the system, from route planning algorithms to passenger information services, ensuring that the system's development consistently aims to improve efficiency.

Within the context of the UoD, goals help define the boundaries and focus of the system. They provide a clear understanding of what the system is expected to achieve within the specific domain being addressed. This alignment with the UoD ensures that the system remains relevant and focused on the core needs of its stakeholders.

2.15.2 Outcomes: The Realised Benefits

An outcome represents the tangible, measurable effects or benefits that occur when the system achieves its goals. Outcomes are critical in evaluating whether the system has successfully delivered on its intended purpose.

Returning to the public transportation system example, if the goal is to “Enhance the efficiency of public transport services,” an outcome might be “Reduced average wait times for passengers by 15%.” This outcome is specific and measurable, providing a clear indicator of the system's success in achieving its goal.

In relation to the UoD, outcomes serve as benchmarks that validate whether the system's implementation effectively addresses the domain's needs and challenges. They provide a way to measure the system's impact within its environment, ensuring that the benefits are both real and aligned with the original intent of the UoD.

2.15.3 Principles: The Guiding Rules

Principles are the fundamental norms, rules, or guidelines that influence how the system is designed and how decisions are made during its development. Principles ensure that the system adheres to best practices, regulatory requirements, and organisational standards.

For example, in the same public transportation system, a principle might be “Ensure data security and privacy in all user interactions.” This principle would guide design decisions related to data management, access controls, and user interface design, ensuring that the system is not only functional but also secure and compliant with relevant regulations.

Principles are crucial within the UoD because they maintain consistency and quality throughout the system's lifecycle. They act as constraints or guiding rules that shape the system's development, ensuring it remains aligned with both technical standards and ethical considerations specific to the domain.

2.15.4 Linking Goals, Outcomes, and Principles to the UoD

The concepts of goals, outcomes, and principles are deeply interconnected with the UoD, as they provide the strategic framework within which all requirements are defined and validated. By understanding these high-level concepts, we can ensure that the system remains focused on addressing the core needs of the UoD, avoiding the risk of scope creep or misalignment.

For instance, when defining requirements, it is crucial to map each requirement back to the relevant goals, expected outcomes, and guiding principles. This mapping ensures that all aspects of the system's design are purposeful and aligned with the domain's strategic objectives.

2.15.5 Conclusion

Understanding goals, outcomes, and principles is essential for effective systems engineering and conceptual modelling. These concepts provide the high-level strategic direction needed to ensure that the system design is both relevant and effective within its UoD. By grounding the design process in these foundational concepts, systems that are not only technically sound but also strategically aligned with the needs and goals of the stakeholders.

A domain of knowledge where these concepts are nowadays put in practice is **Enterprise Architecture**...



“Enough with all the strategic planning.
Get out there and kill something.”

2.16 <concept> ...Enterprise Architecture

Enterprise Architecture (EA) is a way for organisations to align their people, processes, and technology towards achieving desired outcomes. EA is a practice that relies on a set of techniques that treat the entire organisation as a complex socio-technical system. This perspective aligns with the idea that any business can be seen as a system of systems, where the interaction between its components is vital for achieving the organisation's goals.

2.16.1 Organisations as Systems

Organisations, when viewed as systems, are composed of people who execute processes using technology to produce goods, services, or other results. These systems do not operate in isolation; instead, they interact with other internal and external systems, including markets, regulatory environments, and technological ecosystems. Consequently, EA adopts systems thinking to address organisational challenges holistically. Systems engineering principles play a crucial role in this process, providing structured approaches to understand and solve complex problems by looking at both the system's individual components and their interactions.

In an EA context, people are often the most critical yet unpredictable elements. They operate in concert with technologies through defined processes to deliver business value. EA focuses on optimising these interactions, ensuring that the technology supports human activities efficiently and that processes are streamlined to minimise waste and maximise productivity. From a systems engineering perspective, this involves modelling and analysing the organisation's components and their relationships to ensure alignment with business objectives.

2.16.2 Enterprise Architecture as a Process

EA is fundamentally a process driven by requirements. At its core, EA involves understanding and documenting the needs of an organisation, aligning them with strategic goals, and designing solutions to meet those needs. Requirements serve as the foundation for EA, guiding decisions on technology, processes, and organisational structures and roles to ensure alignment with business objectives.

The EA process begins with gathering requirements from key stakeholders, including executives, IT professionals, and business leaders. These requirements reflect the desired outcomes, capabilities, and constraints of the organisation (including ethical and legal, among others). They are analysed, prioritised, and translated into architectural designs that balance current capabilities with future goals.

Enterprise Architecture supports continuous improvement by providing a structured framework to analyse the system's performance, identify inefficiencies, and propose targeted changes. By treating the organisation as a system of systems, EA enables decision-makers to assess how changes in one part of the system might impact others. For example, introducing a new software system might enhance process efficiency but could also require retraining staff, which affects the human system.

2.16.3 Enterprise Architecture Frameworks

Systems engineering approaches are invaluable in EA, especially when dealing with the inherent complexity of socio-technical systems (those that integrate people, processes, and technology). EA frameworks, methods and techniques, such as TOGAF or ArchiMate, provide structured methods and techniques to capture, document, and manage requirements throughout the architectural lifecycle. Ultimately, EA is a continuous process of refining and adjusting the architecture based on the requirements of the business to drive change where and when necessary. Moreover, EA uses methods and techniques like ArchiMate or BPMN for visualising and modelling these organisational components and processes. These help to map out the current state of the system ("as-is") and envision the future state ("to-be"), offering a blueprint for transformation (recall "2.12 <concept> Understanding versus Doing").



2.16.4 Conclusion

In summary, EA is deeply intertwined with systems thinking and engineering. It provides a comprehensive approach to managing the complexity of modern organisations by addressing the interdependencies between people, technology, and processes. Through this lens, organisations can be seen as systems of systems, where each subsystem must be optimally aligned to achieve overarching business goals. Systems engineering and requirement engineering methods are integral to this process, ensuring that the architecture is designed to meet both current and future organisational needs.

2.17 <example> The Portuguese Legal System...

The Portuguese legal system comprises interconnected institutions, processes, and stakeholders, as defined by Law n.º 62/2013. This law establishes the organisational framework for the judiciary, outlining the structure, roles, and hierarchies within the system, from local courts to the Supreme Court and the Constitutional Court. Above all, the Portuguese Constitution serves as the overarching legal framework that guides and limits the functioning of the entire legal system, ensuring that all laws and processes comply with constitutional principles. The system operates within a civil law framework, meaning that codified legal statutes are central to legal interpretation and application. The judiciary is responsible for interpreting these laws and applying them consistently across various legal domains, including civil, criminal, administrative, and constitutional law. The Portuguese Constitution, as the highest legal authority, ensures the protection of fundamental rights, the separation of powers, and the rule of law, framing the behaviour and structure of the entire legal system.

2.17.1 ...as a Social System

As a social system, the Portuguese judiciary is built around human interactions governed by codified laws and procedures. Its structure reflects a hierarchical order, with lower courts handling the majority of cases and higher courts serving as appellate bodies. The Constitutional Court has a unique role, ensuring that laws and judicial decisions align with the Constitution.

The processes that drive the system, such as case submissions, trials, hearings, and appeals, are framed by a set of codes known as *Códigos de Processo*. These include the *Código de Processo Civil*⁹ for civil matters and the *Código de Processo Penal*¹⁰ for criminal cases, among others. These procedural codes ensure uniformity in how cases are handled and provide clear guidelines for legal actors, such as judges, lawyers, and public prosecutors, to follow in the pursuit of justice.

The system's behaviour is defined by its commitment to upholding social values such as justice, equality, and public order. Courts are tasked with resolving disputes and ensuring the fair application of laws. The decisions made by

judges and the interactions between legal professionals shape the boundaries of acceptable behaviour in society and reinforce the rule of law. The Portuguese legal system, through its structure and processes, plays a crucial role in maintaining social cohesion and regulating conduct in accordance with national and constitutional principles.

2.17.2 ...as a Socio-Technical System

In recent years, the Portuguese legal system has transitioned into a socio-technical system, integrating technology to support its legal processes and structures. These technological tools optimise the management of judicial cases, enhance transparency, and improve access to legal services, complementing the social aspects of the system.

One of the most significant technological platforms is CITIUS¹¹, which facilitates the electronic submission and management of legal cases. This platform allows judges, lawyers, and prosecutors to access case files, submit documents, and track case progress in real time, streamlining many of the system's operations, reducing delays and improving procedural efficiency.

Recent developments are resulting in new platforms¹². MP Codex is a solution for the Public Prosecutor's Office, enabling effective case management, particularly in handling investigations, ensuring that cases are processed efficiently and securely. Magistratus is a platform exclusive for judges (that so far have been using CITIUS), with the goal to provide a centralised interface for case management, enabling judges to organise legal documentation, manage schedules, and reduce administrative workloads.

These platforms illustrate how the Portuguese legal system has been traveling from "as-is" to new "to-be" scenarios, integrating technical elements to enhance its operational capabilities. The system's behaviour is shaped not only by human interactions and legal norms but also by technological tools, which support the execution of legal processes defined by the codes and the broader legal framework of the Portuguese Constitution.

2.17.3 Conclusion

The Portuguese legal system, structured by Law n.º 62/2013 and framed by the Portuguese Constitution, operates as both a social and socio-technical system. The Constitution

ensures that the legal processes and structures comply with fundamental rights and principles, while the *Códigos* provide the procedural rules governing legal actions across various domains. Technological platforms such as CITIUS, MP-Codex, and Magistratus, enhance the system's efficiency and transparency, supporting its adaptation to modern challenges. Through this combination of social and technical elements, this system highlights in the European landscape¹³.

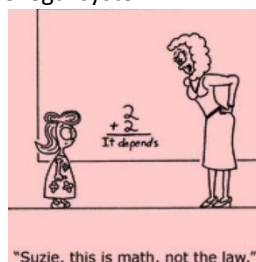
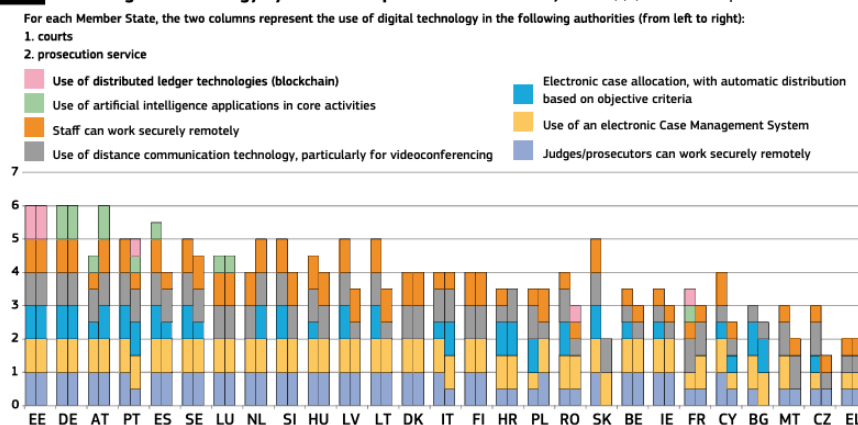


Figure 44 Use of digital technology by courts and prosecution services, 2023 (*) (source: European Commission ⁽⁹⁴⁾)



(*) Maximum possible: 7 points. For each criterion, one point was given if courts and prosecution services, respectively, use a given technology and 0.5 point was awarded when the technology is not always used by them.

⁹ <https://diariodarepublica.pt/dr/legislacao-consolidada/lei/2013-34580575>

¹⁰ <https://diariodarepublica.pt/dr/legislacao-consolidada/decreto-lei/1987-34570075-50509675>

¹¹ <https://www.citius.mj.pt>

¹² <https://justica.gov.pt/Noticias/Plataformas-tecnologicas-Magistratus-e-MP-Codex-avancam-nos-tribunais>

¹³ https://commission.europa.eu/strategy-and-policy/policies/justice-and-fundamental-rights/upholding-rule-law/eu-justice-scoreboard_en

2.18 <example> The Banking Industry as a System

A typical example of a Universe of Discourse (UoD) in the banking industry can be found in the development of an Online Banking System (OBS). The system allows customers to perform banking transactions, access account information, and manage financial services remotely. The UoD in this context encompasses all relevant entities, processes, and interactions between users, systems, and external regulatory frameworks, making it the foundation from which all system requirements are derived. In this scenario, the UoD typically includes:

- **Stakeholders:** The key stakeholders are bank customers (individuals and businesses), banking staff (such as customer service agents), system administrators, regulatory bodies (e.g., central banks and financial regulators), and third-party services (such as payment gateways and credit score providers).
- **Entities and Relationships:** The system must manage entities such as customer accounts, transactions, security protocols (authentication, encryption), payment services, and user interfaces (e.g., web portals, mobile apps). Relationships between these entities include, for example, the link between user authentication and access to account services or the relationship between financial transactions and security verification processes.
- **External Context:** The system interacts with external financial networks (e.g., interbank payment systems), government regulations (such as anti-money laundering (AML) laws and data privacy standards like GDPR), and external service providers (such as credit bureaus). The UoD must include these external dependencies and how they influence system functionality and requirements.
- **Constraints:** Legal constraints (e.g., compliance with financial regulations), technical limitations (e.g., system uptime, scalability), and security concerns (e.g., protecting against fraud and cyber threats) are crucial parts of the UoD that shape the requirements.

2.18.1 Key Aspects of the Online Banking UoD

- **Source of Requirements:** The UoD provides a rich context for gathering system requirements by defining how the OBS must function within the broader banking environment. Requirements such as the ability for customers to view account balances, initiate transfers, pay bills, or manage investments are all derived from the UoD. Regulatory constraints also impose non-functional requirements, such as the need for encryption and secure data storage.
- **Contextual Understanding:** The UoD ensures a thorough understanding of the system's operating environment, including how customers interact with the system, how it integrates with other financial services, and the compliance landscape. For example, the system must offer multi-factor authentication to comply with security regulations and provide audit trails for transactions to meet regulatory reporting standards.

- **Dynamic Nature:** The banking industry is subject to constant change, including evolving regulations, emerging cyber threats, and new customer expectations (e.g., the adoption of mobile payments and digital currencies), therefore the UoD must evolve to incorporate these changes. For instance, if new data privacy regulations are introduced, the data handling protocols must be updated, which requires a reassessment of the UoD to reflect these changes.
- **Common Language:** The UoD helps establish a shared understanding among stakeholders, ensuring that terminology related to banking functions (such as "account reconciliation," "transaction authentication," and "payment gateway integration") is consistently understood across all departments, development teams, compliance officers, etc.

2.18.2 Traceability in the Online Banking UoD

Traceability in the OBS UoD is critical. Every requirement (whether related to core banking functionality, security, or compliance) must be traced back to a particular aspect of the UoD. For example, the requirement for fraud detection mechanisms must link to the need to protect customer accounts, derived from both customer expectations and regulatory requirements. This ensures that the system stays aligned with both legal mandates and customer needs.

2.18.3 System Context and Domain

From the UoD, two primary perspectives emerge: the system domain and the system context.

System Domain: Internally, the system domain focuses on the banking services provided by the OBS, such as account management, funds transfer, and loan processing. Entities such as customer accounts, transaction histories, and customer support systems are part of this domain, and a domain model can represent how they interact, ensuring that the internal processes run smoothly.

System Context: The system context includes external factors such as regulatory compliance (AML, KYC—Know Your Customer), interaction with external payment processors, and compliance with national or international banking standards. The system must be designed to communicate securely with external services and meet these broader requirements.

2.18.4 Conclusion

In the case of an OBS, the UoD forms the foundation for understanding the operational environment and the interactions between various stakeholders, technical systems, and external regulations. The UoD ensures that requirements are derived from real-world banking contexts, considering both the internal needs of the bank and external legal and security constraints. As the banking environment evolves, so must the UoD, ensuring the system remains aligned with changing regulations, technological advancements, and customer expectations. This dynamic and comprehensive understanding is essential for the successful development and ongoing management of the online banking platform.

2.19 <further> ...More on “Systems”...

While engineering often involves dealing with rigid, mechanical systems, it is fascinating to explore concepts that can enrich our “system thinking” capabilities.

2.19.1 Organic Systems

The concept of an organic system is of one that evolves and adapts in a manner reminiscent of living organisms. Unlike rigidly defined mechanical systems, organic systems are flexible, interconnected, and responsive to their environments. An example is a forest ecosystem, where trees, plants, animals, microorganisms, etc. work together to create a balanced and resilient environment. Changes in one part of the system, such as a fluctuation in an animal population, can trigger a series of responses that ripple throughout the system, leading to new equilibriums. Engineers can learn from this interconnectedness, understanding that some systems may need similar adaptive capabilities, especially when developing technology that must function in unpredictable environments.

2.19.2 Living Systems

A living system, as the name suggests, draws from biology and ecology. It refers to systems that have qualities of life, such as growth, reproduction, and adaptation. For instance, a city could be described as a living system. It grows and adapts over time, with various subsystems (transportation, waste management, social infrastructure) all contributing to its overall vitality. A failure in one of these subsystems often requires the entire city to adapt, much like how the failure of an organ requires the body to compensate. Applying this concept helps us think about system resilience and redundancy in a more dynamic way, promoting designs that can evolve and compensate in the face of failure.

2.19.3 Homeostasis

Homeostasis is a biological concept that refers to the ability of a system to maintain internal stability despite external changes. For instance, the human body regulates its temperature regardless of weather conditions. This idea is useful when designing systems that need to maintain consistent performance under varying conditions.

2.19.3.1 Emergence

Emergence is when complex patterns arise from simple interactions. An ant colony, for example, is composed of simple individuals, but collectively, they create sophisticated behaviours like food foraging and nest building. Understanding emergence can aid in designing distributed systems, where local interactions lead to global outcomes, such as in swarm robotics or decentralized networks.

2.19.4 Organic Versus Mechanistic Views

In the world of engineering, systems are often viewed mechanistically: a set of parts working together in a predictable manner. However, adopting an organic view allows us to appreciate the subtleties of self-organisation, flexibility, and resilience. In an organic approach, the boundaries between components may blur, and feedback loops become crucial for ensuring the overall health of the

system. This perspective can be especially insightful for designing complex socio-technical systems, where human behaviours and technical components interact in unpredictable ways.

2.19.5 Back to Engineering

The previous concepts serve as reminders that systems as concepts are not always in relation to rigid or linear things; they can also refer to adaptable, responsive, and sometimes even self-healing things. By approaching system design with an awareness of concepts like organic and living, engineers can gain a richer understanding of how to engineer systems that are not just functional but also resilient, sustainable, and capable of evolving over time. For that, we are used to apply, for example, the concepts of control system and digital twin.

2.19.5.1 Control Systems

Control systems are systems designed to regulate the behaviour of other systems. They are fundamental in ensuring stability and desired performance, often using feedback loops to maintain control. Control systems are familiar challenges in engineering. An example is the thermostat in a heating system, which maintains a set temperature by adjusting the heat output based on feedback from a temperature sensor. By incorporating control principles, systems can be engineered to react predictably to changes, maintaining homeostasis and ensuring consistent operation.

2.19.5.2 Digital Twins

A digital representation is any model, dataset, or software element used to represent selected aspects of a real-world entity or process. A **digital twin** is a more specific concept. The [Digital Twin Consortium](#) defines it as a digital representation that is synchronised with its real-world referent at a specified frequency and fidelity.

Synchronisation may involve observations flowing from the real world to the digital representation, interventions flowing in the opposite direction, or both. It need not be continuous or occur in real time: the required frequency and fidelity depend on the purpose of the digital twin.

Digital twins may support monitoring, diagnosis, simulation, prediction, optimisation, decision-making, and sometimes control. However, a digital twin is not necessarily a control system. A control system regulates the behaviour of another system through feedback, while a digital twin maintains a synchronised representation. A control system may operate with or without a digital twin, and a digital twin may support analysis without directly controlling its referent.

For example, a digital twin of a wind turbine may combine sensor observations, maintenance records, and analytical models to estimate its current condition and predict failures. These results may inform maintenance decisions without automatically controlling the turbine.

UML can be used to model the logical and software architecture of a digital-twin solution, while SysML is particularly relevant when physical structure, quantities, constraints, and multidisciplinary engineering concerns must also be integrated. See the UML chapter to learn more about the relationship between logical models and operational digital twins.

Systems thinking is a holistic approach that serves as a cornerstone for effectively managing complex, multifaceted systems. The principles of holism, feedback loops, emergence, and the balance between open and closed systems, are crucial not only to make understandings of how systems operate but also for designing resilient, adaptive solutions in an increasingly complex world. Systems thinking equips engineers with the ability to step back from the details of individual components and consider the broader dynamics at play. In the scope of that, requirements engineering is where the success of a project often hinges on understanding the interdependencies and interactions between subsystems and the wider environment. By adopting a systems thinking mindset, we are better able to anticipate challenges, manage risks, and ensure that systems remain flexible enough to adapt to changing requirements or external conditions.

One of the key takeaways from this is the importance of emergence, where complex behaviours arise from the interactions of simpler elements within a system. Understanding how emergence works can help in identifying potential unintended consequences and ensuring that these are managed effectively throughout the development process.

Also, the exploration of systems thinking reinforces the idea that systems are not isolated. Typical challenges we face (whether technical, social, or a combination of both) are related to open systems, which interact with their environment in ways that may not always be predictable. Recognising this openness is crucial for identifying external factors that can influence system performance. For instance, a system designed to support large-scale IT infrastructure must account for evolving security threats, user demand, technological upgrades, and legal compliance, all of which are external influences that can impact the system's functionality.

Another critical aspect is lifecycle management, where systems thinking extends beyond the initial design and development phases. A system's lifecycle encompasses its entire existence, from conception to eventual decommissioning, and systems thinking ensures that engineers account for both current needs and future challenges. By considering how a system will need to evolve, adapt, or even be replaced over time, we provide a framework for sustainable, long-term engineering solutions.

As systems become more interconnected, and the problems they are designed to solve become more complex, the need for a systems thinking approach is growing. Whether we are managing a large-scale socio-technical system, overseeing risk management in an engineering project, or simply working to ensure that different subsystems function in harmony, systems thinking offers a structured, comprehensive method for addressing these challenges:

- **Systems Thinking and the Importance of Context:** Central to systems engineering is systems thinking, which, drawing from General Systems Theory (GST), stresses that a system is greater than the sum of its

individual parts. This principle encourages engineers to consider a system's emergent properties, feedback loops, and the interconnectedness of its components, leading to designs that are resilient, adaptable, and capable of evolving over time. This is particularly crucial in socio-technical systems, where the interplay of technology and human behaviour can create unexpected outcomes.



- **Understanding the Problem Space:** Before any design or implementation can occur, a thorough analysis of the problem space is crucial. This involves understanding the system's Universe of Discourse (UoD), which defines the problem's boundaries and scope. The UoD encompasses the system's domain, focusing on its internal entities and relationships, and its context, which addresses external influences and constraints. Defining the UoD ensures that both internal and external factors are considered in the design process.
- **Balancing Analysis and Implementation:** The text highlights the importance of differentiating between understanding (analysis) and doing (design and implementation). In the initial stages, thorough analysis is vital, particularly during the planning and checking phases of the Plan-Do-Check-Act (PDCA) cycle. This analysis should encompass understanding stakeholder needs, identifying system requirements, and defining the system's context and domain. Once a comprehensive understanding of the problem is established, the focus can shift to designing, implementing, and testing solutions.
- **Frameworks and Standards:** Several frameworks and standards support the systems engineering process. The ISO/IEC/IEEE 42010 standard provides a universal framework for describing system architecture. It introduces the concept of "**Entity of Interest**" instead of rigidly defining "system," offering flexibility in application across various domains. When we dive-in in domains, we also can find specialized frameworks following the systems engineering principles. For example, in the domain of Enterprise Architecture (EA), frameworks like TOGAF help ensuring that technical solutions integrate with business objectives, for the purpose, at an organisational level, align people, processes, and technologies with strategic goals.

In conclusion, effective systems engineering relies on a holistic understanding of the problem space, considering both internal and external factors, balancing analysis with implementation, and leveraging frameworks and standards to ensure consistency and clarity in design.

<further> Consider exploring this extra reading: [https://sebokwiki.org/wiki/Guide_to_the_Systems_Engineering_Body_of_Knowledge_\(SEBoK\)](https://sebokwiki.org/wiki/Guide_to_the_Systems_Engineering_Body_of_Knowledge_(SEBoK))

3 <cover> Requirements Engineering

Systems are created or changed because someone considers an existing situation unsatisfactory, identifies an opportunity, or expects a future condition that must be addressed. Requirements Engineering provides the disciplined reasoning needed to determine what the proposed system should be responsible for achieving (without prematurely reducing the problem to a preferred technical solution).

Requirements do not normally exist in the Universe of Discourse as complete, precise, and mutually consistent statements waiting to be collected. Analysts encounter goals, needs, concerns, rules, existing practices, problems, assumptions, proposed solutions, and sometimes conflicting accounts of the same situation. These are sources for requirements, but they are not all requirements. Moving from this heterogeneous evidence to justified obligations requires interpretation, negotiation, modelling, and explicit decisions about system boundaries and responsibilities.

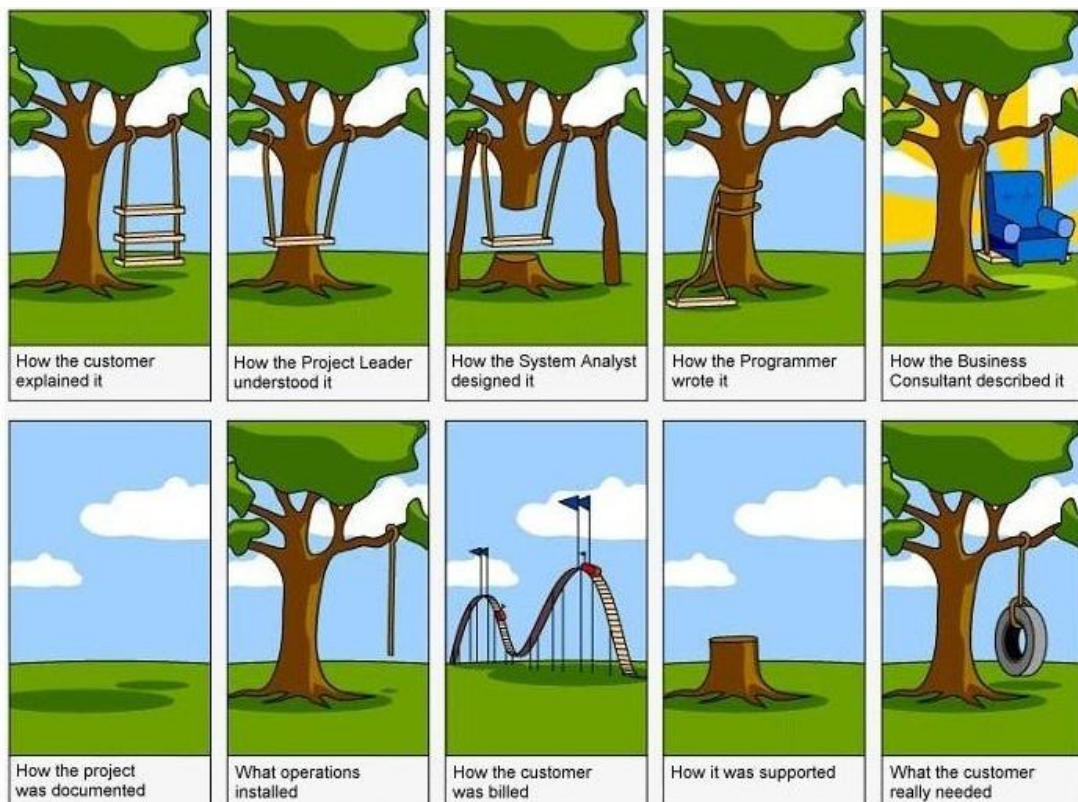
When the system of interest may be sociotechnical (its constituents can include people acting in defined roles, operational processes, rules, information, software, devices, and the coordination among them) a system requirement may therefore govern more than software behaviour. At the same time, requirements concerning the operational system must be distinguished from obligations governing project delivery or the engineering process used to create that system.

Once an obligation has been assigned to the system, its nature can be examined. Functional requirements prescribe required behaviour or outcomes; quality requirements establish required degrees of quality; constraints restrict the permitted solution space. Goals, stakeholder needs, concerns, assumptions, and design decisions remain essential to the analysis, but they should not be relabelled as system requirements merely because they are important.

The requirements lifecycle covers from elicitation and analysis to negotiation, specification, validation, verification, traceability, and controlled change. What follows examines the quality of individual requirements and of requirements sets, and introduces complementary techniques such as textual requirements, user stories, use cases, scenarios, acceptance criteria, specification by example, prototypes, and conceptual models. It also considers the effects of incomplete knowledge, power, ethics, cognitive bias, and the viewpoints through which analysts and stakeholders perceive a domain.

Requirements Engineering is not a one-way handover from “the business” to a development team. It is an iterative process in which understanding, requirements, models, and possible solutions inform and challenge one another. Its outcome is not merely a list of well-written sentences, but a coherent and manageable body of justified obligations connected to their sources, their consequences, and the evidence by which their satisfaction can eventually be assessed.

Chapter orientation: The central question is not simply “What does the stakeholder want?” but “Given the available evidence, concerns, boundaries, and responsibilities, what must the system be obliged to satisfy (and how will we know)?”



3.1 <concept> Requirements Engineering: Purpose and Scope

Requirements Engineering (RE) is the systematic and disciplined practice of analysing, specifying, validating, and managing the requirements for a system of interest. Its purpose is to establish a sufficiently clear and agreed understanding of what the system is expected to achieve, the behaviour and qualities it must provide, and the constraints it must respect.

3.1.1 Requirements Do Not Arrive Ready-Made

Requirements rarely appear in the Universe of Discourse (UoD) as complete, precise, and agreed statements. The UoD may contain:

- stakeholder goals, needs, and concerns;
- descriptions of existing practices and systems;
- domain and business rules;
- laws, standards, and contractual obligations;
- problems, conflicts, uncertainties, and assumptions;
- proposals for possible solutions.

These elements are **sources for analysis**, but they are not automatically well-formed system requirements.

Requirements Engineering is therefore not merely the transcription of what stakeholders say. Analysts interpret the available evidence, identify relevant obligations, resolve ambiguities and conflicts, and formulate requirements that can be discussed and validated. However, analysis is not permission to invent arbitrary requirements: every requirement must have an explicit justification and be traceable to relevant concerns, evidence, decisions, or authoritative sources.

3.1.2 What We Call a Requirement

In this course, the term **requirement** is used for an explicit obligation concerning a system of interest or one of its constituents. A requirement may specify behaviour, a required result, a quality to be achieved, or a limitation that must be respected.

A stakeholder statement such as “We need fewer errors when issuing invoices” expresses a need or goal. It may motivate several requirements, but it is not yet a precise statement of what the system is obliged to provide.

3.1.3 Systems Are Not Necessarily Software

Requirements Engineering applies to systems in general, not only to software applications. A system of interest may be technical, organisational, or **sociotechnical**, combining:

- people and organisational roles;
- operational or business processes;
- rules and responsibilities;
- information and documents;
- software applications and services;
- devices and other physical resources.

Consequently, a system requirement may concern the behaviour of the whole sociotechnical system or may later be allocated to people, processes, software, or devices.

System requirement must not be interpreted as a synonym for software requirement.

3.1.4 Role in Systems Analysis

Requirements help connect an understanding of a problem with the design, construction, operation, and evaluation of a system. They support:

- agreement about responsibilities and expected outcomes;
- conceptual modelling of structure and behaviour;
- evaluation of alternative designs;
- verification and validation;
- traceability and controlled change.

Requirements should be sufficiently precise for their purpose without prematurely fixing decisions that properly belong to design or implementation.

3.1.5 Scope of This Chapter

This chapter provides an introductory foundation for working with requirements as part of systems analysis and conceptual modelling. It does not attempt to cover the full professional discipline of Requirements Engineering. Its focus is learning how to move carefully from the UoD to explicit, justified, well-formed, and traceable system requirements.

Key takeaway: Requirements Engineering transforms relevant but potentially incomplete, conflicting, and informal knowledge into explicit obligations that can guide modelling, decision-making, and system development.

3.2 <concept> From the UoD to System Requirements

The **Universe of Discourse (UoD)** is the portion of the world about which we are concerned and communicate for the purposes of an analysis. It may include people, organisations, activities, information, technologies, physical objects, existing systems, rules, events, and relevant environmental conditions.

The UoD is not itself a system specification. It contains the circumstances and evidence from which a **system of interest** and its requirements may be conceived.

3.2.1 From concerns to a system of interest

Reality does not come pre-partitioned into systems. Analysts identify a system of interest because particular concerns make some entities, relationships, and behaviours relevant.

For example, when analysing problems with hospital appointments, one conception of the system might include only the appointment software. Another might also include patients, administrative staff, scheduling procedures, communication channels, and organisational rules. The appropriate boundary depends on the concerns being addressed and on the responsibilities that the system is expected to assume.

This dependence on concerns does not mean that every conception is equally useful or valid. A proposed system must remain consistent with the available evidence, relevant rules, observed conditions, and justified stakeholder concerns.

3.2.2 Requirements have sources

The UoD may contain many potential **requirements sources**, including:

- stakeholder goals, needs, concerns, and expectations;
- problems and opportunities;
- existing processes and organisational practices;
- laws, regulations, standards, policies, and contracts;
- existing systems and technological conditions;
- observed events, data, conflicts, and risks.

These sources are not automatically requirements.

For example, “*patients wait too long for an appointment*” describes a problem, while “*reduce waiting times*” expresses a goal. A regulation may impose rules that must be interpreted in the particular context. None of these statements or sources yet establishes, by itself, a sufficiently explicit obligation for the system. That is why a work of analysis is necessary.

3.2.3 Formulating system requirements

Moving from the UoD to system requirements involves analytical and design judgement.

Typically, analysts must:

1. understand the relevant situation and concerns;
2. identify the system of interest and establish its boundary;
3. decide what responsibilities should be assigned to the system;
4. resolve ambiguities, conflicts, assumptions, and missing information;
5. formulate explicit and verifiable obligations;
6. justify each requirement through its sources and validate it with the relevant stakeholders.

A resulting requirement might state:

The appointment-management system shall allow a patient to select an available appointment slot without assistance from administrative staff.

This obligation could be satisfied by a sociotechnical arrangement involving software, organisational procedures, people, and communication channels. It should not be silently interpreted as a software requirement unless the system boundary assigns that responsibility specifically to software.

There is therefore no purely mechanical transformation from stakeholder statements or domain observations into requirements. Different system boundaries may lead to different requirements, and each important analytical decision should be made explicit and justified.

Key takeaway: *The UoD provides the context and sources for analysis. System requirements are explicit, justified obligations formulated after the system of interest, its boundary, and its responsibilities have been established.*

3.3 <technique> The Requirements Engineering Process

Requirements engineering is not a single activity performed before design begins. It is an **iterative process** through which relevant concerns are investigated, system responsibilities are established, requirements are formulated and evaluated, and changes are controlled.

The process should not be understood as a rigid sequence. Activities overlap and may need to be repeated as knowledge improves. A newly discovered rule may change the system boundary; a conceptual model may reveal a missing requirement; validation may expose a conflict between stakeholders.

3.3.1 Main activities

A **requirements engineering process** commonly includes the following activities:

1. **Framing the analysis** : Clarify the purpose of the analysis, the concerns being addressed, the relevant stakeholders, the initial UoD, and the tentative system of interest. At this stage, the system boundary may still be revised.
2. **Eliciting information**: Investigate requirements sources through interviews, workshops, observation, document analysis, questionnaires, prototypes, and other techniques. Elicitation does not merely “collect requirements”: it produces evidence about goals, needs, problems, rules, existing practices, constraints, and possible responsibilities.
3. **Analysing and negotiating**: Interpret the collected information; identify ambiguities, omissions, assumptions, dependencies, and conflicts; examine feasibility; negotiate priorities; and decide which responsibilities should be assigned to the system. These decisions must be justified rather than silently introduced by the analyst.
4. **Specifying requirements**: Express the agreed obligations in suitable forms. These may include structured textual requirements, conceptual models, use cases, scenarios, acceptance criteria, and other complementary representations. The selected forms should fit the audience and the purpose of the specification.
5. **Verifying and validating**: **Verification** examines whether requirements are well formed, clear, consistent, feasible, and verifiable. **Validation** examines whether they express the right obligations for the intended purposes and justified stakeholder concerns. A requirement can be precisely written and still be the wrong requirement.
6. **Managing requirements and change**: Record sources, decisions, dependencies, and versions. When a requirement or its context changes, assess the consequences for other requirements, models, designs, tests, processes, and stakeholders. Requirements engineering continues while relevant change remains possible.

3.3.2 Work products

The process may produce more than a requirements specification. Its work products can include a glossary; records of sources and concerns; descriptions of the UoD; system context and boundary models; assumptions and unresolved issues; conceptual models; an agreed requirements set; traceability relationships; and records of validation and change.

The appropriate degree of formality depends on the scale, novelty, risk, regulation, and number of parties involved. A small exploratory project may use lightweight records, whereas a safety-critical or contractually complex system requires stronger documentation and control. The underlying analytical responsibilities nevertheless remain.

Key takeaway: *Requirements engineering is a continuing process of inquiry, judgement, agreement, specification, evaluation, and change management—not the transcription of stakeholder requests into a list.*

3.4 <further> Pohl's Requirements Engineering Framework

The academic and professional work on requirements engineering is vast, where Klaus Pohl is a major reference. This section is based on his work, especially on the most recent version of his monumental book (first published in 2010): "Pohl, K. (2025). Requirements engineering: Fundamentals, principles, and techniques (2nd ed.). Springer."

Pohl's framework presents Requirements Engineering as coordinated work within a system context, not as a simple sequence of phases. The 2010 framework distinguishes four context facets (subject, usage, IT system, and development); three core activities (elicitation, documentation, and negotiation); and three principal artefact types (goals, scenarios, and solution-oriented requirements). Validation and management are cross-sectional activities, applying to the context, the execution of RE, and the artefacts produced.

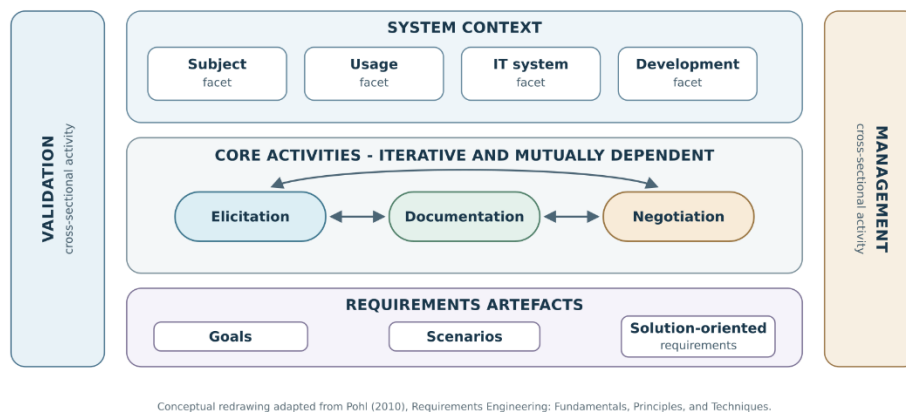


Figure 1 - Pohl's comprehensive RE framework.

Pohl uses requirements artefact as an umbrella term for three distinct but interrelated kinds of work product: goals, scenarios, and solution-oriented requirements. Goals express stakeholder intentions; scenarios describe exemplary system use or goal fulfilment or non-fulfilment; solution-oriented requirements specify required system properties at the necessary level of detail. A goal or scenario is therefore a requirements artefact, but should not be confused with a solution-oriented requirement.

These artefacts do not emerge complete, precisely represented, and agreed: they evolve iteratively as Requirements Engineering progresses along several interdependent dimensions. In Pohl's model an RE process moves along three axes (Pohl previously named "documentation" as "representation" and "content" as "specification", but the new terms better reflect the dynamics and fit for purpose, as the dimensions are distinct but coupled):

- **Content** - emphasising knowledge about requirements, from an opaque understanding towards sufficiently complete requirements;
- **Documentation** - from informal descriptions towards formal representations, recognising that appropriate forms depend on purpose and audience and need not always be maximally formal;

- **Agreement** - from differing personal views towards a common view.

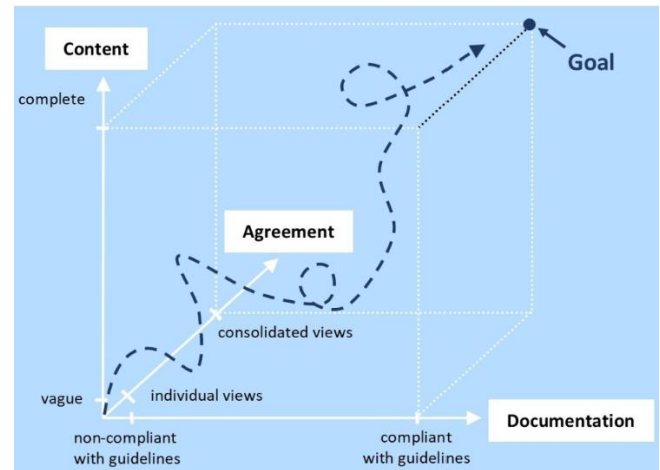


Figure 2 - A requirements trajectory over time: progress and setbacks across three dimensions.

Eliciting new knowledge may improve completeness while exposing disagreement; formalising a statement may reveal ambiguity or missing content. A requirements set therefore occupies a changing position in this space: an iteration may advance it on one axis and move it backwards on another. The aim is not an abstract "perfect corner", but sufficiently complete content,

appropriate documentation, and justified agreement for the project's purposes and risks (with traceability preserved as that position changes).

Pohl relates requirements and design through the questions why, what, and how. Goals primarily express why a system or change is needed; requirements state what properties or behaviour the system must provide to fulfil those goals; and design decisions describe how those requirements can be realised. These categories are nevertheless relative to the level of abstraction: a how chosen at one level may become a what to be realised at the next lower level, while each what should remain traceable to a higher-level why. Requirements Engineering and design therefore overlap and co-evolve rather than being separated by a fixed boundary.

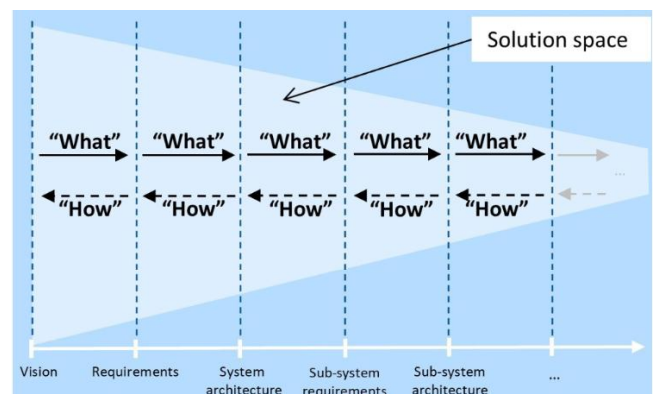


Figure 3 - Alignment between RE and design.

3.5 <technique> Elicitation Techniques

Elicitation is the systematic discovery of information relevant to understanding the UoD, defining the system of interest, and formulating its requirements. Its purpose is not simply to ask stakeholders what the system “shall” do. Stakeholders may express goals, problems, preferences, proposed solutions, assumptions, or conflicting interpretations rather than ready-made requirements.

Relevant information may be obtained from people, documents, existing systems, observed work, operational data, physical environments, and applicable rules.

3.5.1 Common techniques

Interviews allow detailed exploration of individual knowledge, concerns, and experiences. Questions should invite explanation and examples rather than merely confirm the analyst’s expectations. However, an interview reveals one participant’s account, not necessarily how work actually occurs.

Workshops bring several participants together to develop a shared understanding, compare perspectives, resolve terminology, identify conflicts, and negotiate priorities. They can be productive but require facilitation to prevent dominant participants from suppressing other relevant views.

Observation and contextual inquiry examine activities in their actual setting. They are particularly valuable for discovering informal practices, exceptions, physical dependencies, and tacit knowledge that participants may omit because they consider it obvious. Observed behaviour should not automatically be treated as desirable behaviour.

Document analysis examines laws, regulations, contracts, standards, policies, procedures, forms, manuals, reports, and previous specifications. Documents may provide authoritative constraints or useful historical evidence, but they may also be outdated, inconsistent, or disconnected from current practice.

Questionnaires can gather information from many participants efficiently and support quantitative comparison. They are less suitable for exploring unexpected issues, understanding reasoning, or resolving ambiguity.

Analysis of existing systems and operational data can reveal actual workflows, usage patterns, failures, volumes, timings, and dependencies. Nevertheless, the existing solution does not define what the future system ought to be.

Scenarios, prototypes, and sketches help participants react to concrete situations and expose missing information, exceptions, usability concerns, and differing expectations. They should be used carefully: an early proposed solution may anchor discussion and prematurely restrict the alternatives considered.

3.5.2 Combining techniques

No single technique provides a complete or neutral account of the UoD. Analysts should **triangulate** important findings by comparing multiple sources and techniques. A stated procedure may differ from observed practice; stakeholder testimony may conflict with operational data; and existing practice may violate a formal rule.

For each relevant finding, analysts should record its source, context, uncertainties, assumptions, and unresolved questions. They should then confirm their understanding with the appropriate participants without treating agreement as proof that the finding is correct.

Elicitation is therefore exploratory and iterative. New models, proposed requirements, and validation activities frequently reveal additional questions that require further investigation.

Key takeaway: *Elicitation discovers and challenges information from multiple sources. It does not extract a pre-existing, definitive set of requirements from stakeholders.*

3.6 <technique> Analysing, Negotiating, and Prioritising

Elicitation produces heterogeneous and sometimes unreliable material: descriptions of current practice, goals, complaints, proposed solutions, rules, assumptions, and candidate requirements. **Analysis** transforms this material into a coherent understanding from which justified system requirements can be formulated.

3.6.1 Analysing

Requirements analysis involves examining both individual statements and their relationships. Analysts typically:

- clarify terminology and resolve ambiguous expressions;
- distinguish goals, concerns, rules, assumptions, design proposals, and system requirements;
- examine the system boundary and the allocation of responsibilities;
- identify omissions, duplicates, conflicts, dependencies, and inconsistencies;
- consider feasibility, risk, cost, and possible unintended consequences;
- use conceptual models to explore structure, behaviour, information, and interactions;
- decompose compound statements and combine unnecessarily fragmented ones.

Analysis is not merely linguistic correction. A perfectly clear statement may assign responsibility to the wrong system, contradict an applicable rule, or solve a problem that has been misunderstood. Conversely, an important concern may require several related requirements rather than a single rewritten sentence.

3.6.2 Negotiating

Different stakeholders may pursue legitimate but incompatible interests. Users may prefer convenience, regulators may require stronger controls, operators may seek simpler procedures, and sponsors may limit cost or time. Conflicts should not be hidden through vague wording.

Negotiation should:

1. make the disagreement explicit;
2. identify the concerns and evidence behind each position;
3. explore alternatives and their consequences;
4. clarify who has authority to decide;
5. record the decision and its rationale.

Consensus is useful but not always possible. Where agreement cannot be reached, the issue must be escalated to an authorised decision-maker. The analyst facilitates understanding and exposes consequences but should not silently decide whose interests prevail.

3.6.3 Prioritising

Priority is not an intrinsic property of a requirement. It is assigned relative to a particular purpose, scope, release, or planning horizon. Saying that a requirement has “high priority” is meaningless unless the consequences and decision context are understood.

Useful prioritisation criteria include stakeholder value; urgency; legal, contractual, safety, or ethical significance; risk reduction; dependencies; implementation effort; operational impact; and the consequences of omission or delay.

Simple schemes such as **Must, Should, Could, and Won’t for now** can support discussion, but their labels require discipline. A **Must** is indispensable within the defined scope: without it, the considered solution or release is unacceptable. It must not mean merely “very desirable”. Applicable legal or contractual obligations cannot simply be given a lower priority, although alternative ways of satisfying them may exist.

Dependencies must also be considered. A lower-value capability may need to be implemented first because several higher-value capabilities depend on it.

The results of analysis, negotiation, and prioritisation should remain traceable to their sources, participants, decisions, and rationale. Otherwise, later changes will reopen old disputes without preserving what was previously learned.

Key takeaway: Analysis discovers what the statements mean and imply; negotiation resolves competing interests; prioritisation determines what matters most within an explicit decision context.

3.7 <concept> Goals, Needs, Concerns, Rules, and Requirements Sources

Requirements should be justified by what has been learned about the UoD. During analysis, however, many relevant statements are not yet requirements. Distinguishing **concerns, goals, needs, rules, and requirements sources** prevents analysts from treating every stakeholder statement as an obligation for the system.

These categories support analysis; they are not intended as a complete ontology.

3.7.1 Concerns

A **concern** is a matter of interest or importance to a stakeholder regarding the UoD or the system of interest. Concerns may relate to problems, risks, opportunities, qualities, costs, values, or possible consequences.

Examples include patient waiting times, privacy, working conditions, environmental impact, or operational resilience.

A concern identifies something that deserves attention but does not necessarily prescribe a particular outcome or solution.

3.7.2 Goals

A **goal** describes a desired outcome or state of affairs.

Examples include *reducing appointment waiting times*, *increasing access to public services*, or *preventing unauthorised disclosure of personal data*.

Goals provide direction and help justify requirements, but they usually do not establish a precise and verifiable obligation for a particular system. One goal may motivate several system requirements, organisational changes, or actions outside the system boundary.

3.7.3 Needs

A **need** expresses something that a stakeholder or group considers necessary to address a concern or pursue a goal.

For example, patients may need to know which appointment slots are available without repeatedly telephoning the hospital.

Needs are important sources for analysis, but they may be incomplete, subjective, conflicting, or based on an assumed solution. Analysts must investigate why the need exists, who experiences it, under which conditions it applies, and what system responsibilities might address it.

3.7.4 Rules

A **rule** prescribes or restricts behaviour within the UoD. Rules may originate in laws, regulations, contracts, standards, organisational policies, professional practices, or locally agreed operating procedures.

A rule is not automatically a system requirement. Its applicability and implications must first be analysed. A legal provision, for example, may motivate several requirements concerning human responsibilities, operational processes, information handling, and technology.

3.7.5 Requirements sources

A **requirements source** is an identifiable origin of information or justification used when formulating a requirement. Sources may include stakeholders, documents, observations, operational data, existing systems, conceptual models, incidents, decisions, goals, needs, concerns, and rules.

Sources can be incomplete, outdated, unreliable, or mutually inconsistent. Traceability to a source explains why a requirement exists; it does not prove that the requirement is correct.

Consider the following progression:

Concern: administrative staff might see confidential clinical information.

Goal: protect the privacy of patients.

Need: receptionists need scheduling information but not clinical notes.

Rule: access to clinical information is restricted to authorised healthcare professionals.

System requirement: the appointment-management system shall prevent receptionists from accessing clinical notes.

The resulting requirement is not obtained through simple transcription. It follows from interpreting the sources, defining the system boundary, allocating responsibility, and validating the proposed obligation.

Key takeaway: Concerns, goals, needs, and rules explain what matters and why. A system requirement states an explicit obligation assigned to the system or one of its constituents.

3.8 <concept> What Does the Obligation Govern?

WARNING: The word **requirement** is used broadly in projects. It may describe an obligation concerning the system being created, the delivery of that system, or the activities used to engineer it. These obligations should not be placed in the same category merely because they contain words such as *shall*, *must*, or *required*.

The first classification question is therefore:

What entity or activity is expected to satisfy the obligation?

3.8.1 System requirements

A **system requirement** establishes an obligation concerning the system of interest or one of its constituents. It may govern system behaviour, outcomes, qualities, interfaces, or permitted solutions.

Examples:

- *The appointment system shall notify patients when an appointment is cancelled.*
- *The appointment service shall remain available during normal hospital operating hours.*
- *The appointment procedure shall require urgent requests to be reviewed by a qualified healthcare professional.*

The third example is still a system requirement if the procedure and professional role are constituents of the sociotechnical system. A requirement concerning an **operational process** is therefore not the same as a requirement concerning the process used to engineer the system.

Unless otherwise qualified, this chapter uses requirement as a short form of system requirement.

3.8.2 Project-delivery requirements

A **project-delivery requirement** governs the execution or outputs of the project that creates or changes the system. It may concern delivery dates, budgets, contractual deliverables, training, migration, installation, or other transition activities.

Example:

- *The supplier shall deliver the accepted appointment system by 30 September.*

This does not describe a property or behaviour of the appointment system. It establishes an obligation concerning its delivery. A missed project deadline is not a failure of the operational system.

3.8.3 Engineering-process requirements

An **engineering-process requirement** governs how the system is to be analysed, designed, implemented, verified, validated, or managed.

Examples:

- *The engineering team shall conduct a security review before operational deployment.*
- *All source-code changes shall be reviewed by a second developer.*

These obligations may contribute to system quality, but they are not themselves quality requirements for the system. Following a prescribed process does not, by itself, prove that the resulting system has the desired qualities.

The term *engineering-process requirement* is used deliberately. It avoids confusion with requirements concerning operational or organisational processes that belong inside a sociotechnical system.

3.8.4 Why the distinction matters

Different obligations require different evidence:

- system requirements are evaluated against the resulting system;
- project-delivery requirements are evaluated against project commitments and deliverables;
- engineering-process requirements are evaluated through evidence that the prescribed activities were performed.

A specification may need to contain all three kinds, particularly in contractual settings, but they should be clearly separated. A compound statement that governs more than one object should normally be decomposed.

Only after identifying a **system requirement** should it be classified further as a functional requirement, quality requirement, or constraint.

Key takeaway: *Classify an obligation first by what must satisfy it. Requirements concerning the system, its delivery, and its engineering process serve different purposes and require different evidence.*

3.9 <concept> Requirements, Assumptions, and Design Decisions

Requirements analysis involves statements with different roles. In particular, **requirements**, **assumptions**, and **design decisions** must be distinguished because they express different kinds of commitment.

3.9.1 Requirements

A **requirement** establishes an obligation concerning the system of interest or one of its constituents.

Examples:

- *The appointment system shall send the patient a confirmation when an appointment is booked.*
- *The system is responsible for satisfying this statement. Its satisfaction must therefore be evaluated against the resulting system.*

3.9.2 Assumptions

An **assumption** is a proposition treated as true for the purposes of analysis, despite not being guaranteed by the system.

Examples:

- *The hospital identity service will be operational whenever staff members access the appointment system.*
- *The identity service is external to the appointment system in this example. Its availability affects the system but is not controlled by it.*

Assumptions may concern external services, stakeholder behaviour, operating conditions, available resources, expected volumes, or other aspects of the environment. They should be made explicit because a requirement or design may cease to be adequate if an assumption proves false.

An assumption is not a weaker form of requirement. If a condition must be guaranteed by the system, it should be expressed as a requirement. If it depends on an external party, it may need an agreement, contract, environmental constraint, monitoring mechanism, or contingency response.

For important assumptions, analysts should record:

- their source and justification;
- the requirements and decisions that depend on them;
- the consequences if they prove false;
- how and when they will be checked.

3.9.3 Design decisions

A **design decision** selects a particular way of satisfying one or more requirements.

Example:

- *Appointment confirmations will be dispatched through an asynchronous message broker.*

This is a solution choice. Other designs might satisfy the same requirement. A design decision should have a rationale and remain traceable to the requirements, risks, and alternatives that motivated it.

A proposed solution mentioned by a stakeholder is not automatically a requirement. For example, “we need a mobile app” may conceal a need for remote access, faster interaction, or better notifications. The underlying concern should be investigated before the proposed solution is accepted.

Sometimes a solution choice is externally imposed:

- *The appointment system shall use the hospital’s central notification service to dispatch messages.*

If the project has no authority to choose an alternative, the statement is a **constraint** on the system rather than an internal design decision. Its external source and justification should be recorded.

3.9.4 Avoiding a false “what versus how” rule

The common advice that requirements describe *what* and designs describe *how* is useful only as a reminder. It is not a reliable classification rule. Requirements may legitimately constrain implementation, and apparently solution-neutral statements may still contain hidden design decisions.

The decisive questions are:

- Is this an obligation that the system must satisfy?
- Is it a condition outside the system’s control?
- Is it a choice made by those designing the solution?
- Who has the authority to change it?

Key takeaway: Requirements state system obligations; assumptions state conditions relied upon but not guaranteed by the system; design decisions select ways to satisfy requirements. Confusing them hides risk and unnecessarily restricts solutions.

3.10 <concept> The Kinds of Requirements

After confirming that a statement is a **system requirement**, the obligation can be classified as a **functional requirement**, **quality requirement**, or **constraint**. These categories describe the nature of the obligation, not its importance.

3.10.1 Functional requirements

A **functional requirement (FR)** specifies behaviour or an outcome that the system must provide under stated conditions.

Examples:

- *The appointment system shall allow a patient to cancel a scheduled appointment.*
- *The appointment system shall notify the assigned healthcare professional when an urgent appointment is requested.*

A functional requirement may concern software, a person, an operational process, or their coordinated behaviour within a sociotechnical system. Functional does not mean “implemented by software”.

3.10.2 Quality requirements

A **quality requirement (QR)** specifies a required degree of a quality exhibited by the system, one of its constituents, or a function it performs.

Quality requirements may concern performance, availability, reliability, usability, accessibility, security, safety, maintainability, or other relevant qualities. How to classify quality requirements for a specific system is a pragmatic decision of the analysis team.

Examples:

- *For at least 95% of requests, the appointment system shall display the available appointment slots within two seconds.*
- *The appointment service shall be available for at least 99.9% of scheduled operating time in each calendar month.*

A useful quality requirement identifies the relevant subject, operating conditions, expected level, and means of evaluation. Statements such as “*the system shall be fast*” or “*the system shall be user-friendly*” express concerns but do not yet establish sufficiently precise quality requirements.

A quality requirement may apply to the entire system or only to a particular function. It is not necessarily global or less important than a functional requirement.

3.10.3 Constraints

A **constraint** restricts the permitted solution space beyond what is inherently necessary to satisfy the functional and quality requirements.

Examples:

- *The appointment system shall use the hospital’s existing identity provider for staff authentication.*
- *The appointment system shall exchange clinical scheduling data using the mandated interoperability format.*

Constraints may impose technologies, interfaces, protocols, platforms, physical characteristics, locations, architectural rules, or other externally justified limitations. They reduce design freedom and should therefore have an explicit source and rationale.

A technology selected freely by the design team is a **design decision**, not a requirement. It becomes a constraint when an authority outside that design decision imposes it as an obligation on the system.

3.10.4 Classification depends on the obligation

Topic labels do not determine requirement type. Security, time, data, interface, and usability statements can express different kinds of obligation:

- *The system shall authenticate a staff member before displaying patient data is functional.*
- *The system shall resist the specified unauthorised-access test is a quality requirement.*
- *Authentication shall use the hospital’s existing identity provider is a constraint.*
- *The system shall send a reminder 24 hours before an appointment is functional because the timing defines when the behaviour occurs.*
- *The system shall respond within two seconds is a quality requirement because it establishes a performance level.*
- *The supplier shall deliver the system by 30 September is not a system requirement at all; it is a project-delivery requirement.*

WARNING: The expression **non-functional requirement (NFR)** is widely used but ambiguous. It often refers to quality requirements or to constraints, despite their different purposes. This course therefore prefers the more precise terms **quality requirement** and **constraint**. When encountering an NFR elsewhere, determine which kind of obligation it actually expresses.

All three kinds of system requirement (functional, quality or constraint) should be necessary, clear, feasible, and verifiable. If one statement contains several obligations, it should normally be decomposed before classification. If its context or responsible subject is unclear, classification should be postponed rather than guessed.

Key takeaway: Functional requirements prescribe system behaviour or outcomes; quality requirements prescribe required levels of quality; constraints restrict the permitted solution space.

3.11 <further> “Business Requirements” and “Stakeholder Requirements”

Some requirements engineering frameworks use the word *requirement* for statements at several levels of abstraction. The **International Requirements Engineering Board (IREB)**, for example, defines a **business requirement** in terms of an organisation’s goal, objective, or need, and a **stakeholder requirement** as an expression of a stakeholder’s desire or need. These terms belong to the official [IREB CPRE glossary](#).

This terminology is widespread in professional practice. It can help organisations arrange work products in layers and trace strategic intentions through stakeholder expressions to more detailed system requirements. However, it also creates conceptual risks.

3.11.1 The risk of calling everything a requirement

Consider the following statements:

- The hospital wants to reduce missed appointments.
- Receptionists need a clearer overview of appointment availability.
- The appointment system shall display the available slots for each medical service.

The first would be commonly classified as a **business requirement**, but it expresses an organisational **goal**. The second could be classified as a **stakeholder requirement**, but it expresses a stakeholder **need**. Only the third assigns an explicit obligation to the system.

Calling all three *requirements* may obscure the analytical work needed to move between them. The system requirement is not simply a more detailed transcription of the preceding statements. Analysts must establish the system boundary, investigate causes, consider conflicting concerns, allocate responsibility, and evaluate alternative responses.

Further difficulties include:

- organisations and stakeholder groups rarely speak with a single coherent voice;
- stated desires may conflict with rules, evidence, or other stakeholders’ legitimate concerns;
- a goal may require organisational action beyond the system of interest;
- one stakeholder need may motivate several system requirements;
- one system requirement may address several goals and concerns;
- a proposed “stakeholder requirement” may already contain an unjustified design choice.

The labels can therefore suggest a clean hierarchy where the actual reasoning is iterative, contested, and dependent on analytical judgement.

3.11.2 Convention used in this course

To preserve distinctions that are useful for introductory conceptual modelling, this course normally uses:

- **business or organisational goal** for a desired organisational outcome;
- **stakeholder need** for something a stakeholder considers necessary;
- **stakeholder concern** for a matter of interest or importance;
- **requirements source** for the origin of relevant information or justification;
- **system requirement** for an explicit obligation assigned to the system of interest or one of its constituents.

This convention does not imply that the IREB terminology is illegitimate. It means that different terminologies draw their boundaries differently. When encountering terms such as **business requirement** or **stakeholder requirement**, one should examine what the statement actually expresses instead of classifying it by its label.

Awareness note: These external terms are presented so that students can interpret professional literature and certifications such as IREB CPRE. Their definitions are not part of the assessable taxonomy used in this course.

Key takeaway: A goal, need, or concern may be called a requirement in some frameworks. In this course, the terms remain distinct so that the transition from requirements sources to justified system obligations remains visible.

3.12 <heuristic> Analysing a Candidate Requirement Statement

The distinctions in the previous section can be combined into a practical sequence for analysing a candidate statement. The purpose is to determine what the statement expresses, what entity or activity it governs, and (only when it is a system requirement) which kind of system requirement it represents.

3.12.1 (1) Is the statement an obligation?

First determine whether the statement establishes something that an identifiable subject is expected to satisfy. If it merely expresses a goal, need, concern, problem, rule, fact, assumption, or proposed solution, do not force it into a requirement category.

This is a goal, not yet a system requirement:

- *The hospital wants to reduce missed appointments.*

The presence of *shall* does not settle the question. Conversely, an informally written statement without *shall* may still express an intended obligation.

3.12.2 (2) Who or what must satisfy it?

Identify the subject responsible for satisfying the obligation and compare it with the system boundary.

- If the obligation concerns the system of interest or one of its constituents, it is a **system requirement**.
- If it concerns delivering or transitioning the system, it is a **project-delivery requirement**.
- If it concerns how engineering work must be performed, it is an **engineering-process requirement**.
- If it concerns an external entity, it may instead be an external responsibility, rule, agreement, or assumption.

This is a project-delivery requirement, not a system requirement:

- *The supplier shall deliver the system by 30 September.*

3.12.3 (3) Only one or several obligations?

A statement may combine several obligations that require different classifications or evidence.

This combines a constraint, a functional requirement, and a quality requirement. It should be decomposed before classification:

- *The system shall use the hospital identity provider, authenticate staff members, and complete authentication within two seconds.*

3.12.4 (4) What is the nature of the obligation?

For a system requirement, ask:

- Does it prescribe **behaviour or an outcome**? Classify it as a **functional requirement**.

- Does it prescribe a **required degree of quality**? Classify it as a **quality requirement**.
- Does it impose an externally justified **limitation on the permitted solution space**? Classify it as a **constraint**.

Examples:

- *The system shall send a reminder 24 hours before the appointment.* This is a **functional requirement**: the timing defines when the required behaviour occurs.
- *The system shall display available slots within two seconds for at least 95% of requests.* This is a **quality requirement**: it establishes a required performance level.
- *The system shall use the hospital's existing identity provider.* This is a **constraint**: it restricts the permitted solution.

3.12.5 (5) Is there enough context?

Some statements cannot be classified reliably without knowing their source, rationale, authority, or system boundary.

- *The system shall use PostgreSQL.*

This may be an externally imposed constraint, an internal design decision incorrectly written as a requirement, or an unsupported preference. The correct response may be **insufficient information**.

Similarly:

- *The system shall comply with the GDPR.*

This refers to an important legal source but is too broad to function as one verifiable requirement. The applicable legal obligations must be analysed and translated into concrete system requirements, project obligations, or organisational rules.

3.12.6 Final checks

Do not classify a requirement according to whether it is important, difficult, measurable, technical, or testable. All requirements should be justified and verifiable. Classification and quality are separate questions: a statement may belong to a recognisable category and still be badly written.

Key takeaway: First determine whether the statement establishes an obligation and who or what must satisfy it. Decompose compound statements and clarify missing context. Only then decide whether a system requirement concerns function, quality, or a constraint.

3.13 <exercise> Analysing Ambiguous Candidate Statements

A hospital is analysing an appointment service. For this exercise, the **system of interest** includes administrative staff acting in defined roles, appointment procedures, and supporting software. Patients, the system supplier, the engineering team, and the hospital identity service are outside its boundary.

For each statement:

1. decide whether it is a requirement;
2. identify what entity or activity the obligation governs;
3. if it is a system requirement, classify it as functional, quality, or constraint;
4. identify any ambiguity or missing information.

3.13.1 Statements

1. *The hospital aims to reduce missed appointments by 25% within two years.*
2. *Patients need a way to cancel appointments without travelling to the hospital.*
3. *The appointment service shall accept cancellation requests through the web portal or by telephone.*
4. *For at least 95% of web cancellation requests, the service shall display confirmation within two seconds.*
5. *The appointment service shall send a reminder 24 hours before each scheduled appointment.*
6. *Staff authentication shall use the hospital's existing identity service.*
7. *The receptionist shall record the stated reason when cancelling an appointment on behalf of a patient.*
8. *Patients shall keep their contact details up to date.*
9. *The supplier shall train administrative staff before operational deployment.*
10. *All changes to the appointment software shall be reviewed by a second developer.*
11. *The appointment system shall comply with the GDPR.*
12. *The appointment system shall use PostgreSQL.*

3.13.2 Discussion guide

1. **Organisational goal**, not yet a system requirement. Achieving it may depend on factors beyond the system boundary.
2. **A stakeholder need**, not yet a system requirement. The underlying conditions and appropriate system responsibilities must be investigated.
3. **Functional system requirement**. It specifies behaviour of the sociotechnical service and permits two interaction channels.
4. **Quality system requirement**. It establishes a measurable performance level for a particular function and operating condition.
5. **Functional system requirement**. The reference to time defines when the required behaviour occurs; it does not express a performance quality.
6. **Constraint**. It imposes an externally selected service and restricts the permitted authentication solution. A separate functional requirement may require staff authentication.
7. **Functional system requirement**. The receptionist is a constituent of the defined sociotechnical system. With a software-only boundary, the classification would be different.
8. **External responsibility or participation rule**, not a system requirement under the stated boundary. The system might separately be required to request, validate, or update contact details.
9. **Project-delivery requirement**. It concerns a transition activity for which the supplier is responsible.
10. **Engineering-process requirement**. It governs how software changes are engineered, not the behaviour or quality of the operational system.
11. **Insufficient as one requirement**. The GDPR is a legal source whose applicable obligations must be analysed and translated into concrete, verifiable requirements and organisational rules.
12. **Insufficient information**. It is a constraint if PostgreSQL is imposed by an authorised external source, a design decision if selected by the design team, or merely an unsupported preference.

Other answers may be defensible if they identify different contextual assumptions and explain their consequences. Classification is not a guessing game: when the system boundary, authority, rationale, or intended obligation is unknown, the correct analytical response is to request clarification.

Key takeaway: The same topic or grammatical form can express different kinds of statement. A defensible classification must follow from the meaning, boundary, responsible subject, and source of authority.

3.14 <concept> Quality of an Individual Requirement

A requirement is not good merely because it uses *shall* or follows a textual template. Its quality depends on whether it communicates a justified obligation that can be understood, implemented, and evaluated.

The following criteria provide a practical review checklist.

3.14.1 Necessary and justified

The requirement should contribute to an identified goal, need, concern, rule, risk, or other legitimate source.

Ask: “What would be the consequence of removing this requirement?”

If no relevant consequence can be identified, the requirement may be unnecessary. A trace to a source might record its justification but does not eliminate the need to evaluate that source critically.

3.14.2 Appropriate

The obligation should concern the correct system or constituent and be expressed at an appropriate level of abstraction.

It should not include an arbitrary design choice, project activity, or external responsibility disguised as a system requirement.

3.14.3 Unambiguous

The requirement should support one intended interpretation among its relevant readers.

Undefined terms, vague adjectives, pronouns, hidden alternatives, and words such as *appropriate*, *efficient*, *normally*, or *user-friendly* frequently create ambiguity. Domain terms should be defined consistently in a glossary or conceptual model.

3.14.4 Complete as an individual statement

The requirement should contain the information needed to understand its obligation, including the responsible subject, required result or behaviour, and relevant conditions.

This does not mean that every requirement must repeat the entire context. It may refer to defined terms, models, interfaces, or shared conditions. Nor does it mean that the **requirements set** is complete; that is a separate question.

3.14.5 Singular

A requirement should normally express one principal obligation.

Statements joined by several occurrences of *and*, *or*, or *unless* should be inspected for hidden requirements.

Excessive fragmentation should also be avoided. Separating every grammatical detail into an isolated statement can destroy meaning and make the set difficult to understand.

3.14.6 Feasible

There should be a credible way to satisfy the requirement within relevant technical, operational, legal, economic, and temporal conditions.

Feasibility should be supported by evidence when uncertainty or risk is significant.

3.14.7 Verifiable

It must be possible to obtain objective evidence that the requirement has or has not been satisfied.

Verification may use testing, demonstration, inspection, analysis, or a justified combination of methods.

Verifiable does not always mean numerical. A required document can be inspected, an interaction can be demonstrated, and conformance to a precisely identified interface can be analysed. Nevertheless, vague quality claims usually require measurable criteria.

3.14.8 Correct and conforming

The requirement should accurately express the agreed obligation, respect the system boundary, and conform to the adopted writing conventions, terminology, units, and identification scheme.

Consider: “The appointment system should quickly and securely notify users of any relevant changes.”

This statement uses a weak modal verb, contains vague qualities, leaves *users* and *relevant changes* undefined, and combines several obligations. It should be replaced by separate requirements defining the notification behaviour, intended recipients, triggering events, delivery-time criterion, and required security properties.

3.14.9 A warning about SMART

SMART is a mnemonic used to establish criteria in businesses, meaning they must be **specific, measurable, assignable, realistic, and time-bound**. SMART can be useful when formulating goals or planning work, but it should not be applied mechanically to every system requirement. In particular, *time-bound* for example does not mean attaching a project deadline to each requirement. Time belongs in a requirement only when it is part of the required system behaviour or quality.

Key takeaway: A good requirement is justified, appropriately scoped, unambiguous, sufficiently complete, singular, feasible, and verifiable. Correct wording helps, but it cannot compensate for incorrect analysis. An individual requirement should also have a unique identifier, recorded source, rationale when needed, and defined verification method.

3.15 <concept> Quality of a Requirements Set

A collection of individually well-written requirements is not necessarily a good **requirements set**.

In fact, requirements interact: they may overlap, depend on one another, compete for resources, or impose incompatible obligations.

The set must therefore be evaluated as a whole.

3.15.1 Sufficient coverage

The set should cover the system responsibilities justified by the agreed goals, needs, concerns, rules, risks, and system boundary. Analysts should examine normal behaviour, exceptional situations, interfaces, relevant system states, quality concerns, and externally imposed constraints.

Completeness is always relative to an explicit scope and purpose. It does not mean that every imaginable future requirement has been discovered. A reasonable claim of completeness should state what boundary, stakeholders, concerns, scenarios, and lifecycle situations were considered.

Conceptual models, scenarios, and traceability can reveal omissions. For example, an activity with no supporting requirement, an unhandled state transition, or a stakeholder concern with no response may indicate a gap.

3.15.2 Consistency

Requirements should not impose mutually incompatible obligations. Terminology, quantities, units, conditions, priorities, and referenced models must also remain consistent.

Consider:

- R1: The appointment service shall be available from 08:00 to 18:00 on every working day.
- R2: Scheduled maintenance may make the appointment service unavailable from 17:00 to 19:00 on any working day.

Each statement is understandable, but the pair is inconsistent unless maintenance is excluded from the availability obligation or otherwise reconciled.

Consistency does not prove correctness. An entire set can be internally consistent while addressing the wrong problem.

3.15.3 Feasibility as a set

Requirements that are feasible individually may be infeasible collectively. Stronger security, lower cost, shorter response

time, higher availability, simpler operation, and earlier delivery can compete with one another.

Feasibility should therefore consider the combined technical, operational, economic, legal, and temporal consequences of the set. Trade-offs must be made explicitly rather than hidden in implementation.

3.15.4 Coherence and comprehensibility

The set should form a structured and understandable account of the system obligations. Requirements should be organised by meaningful concerns, functions, interfaces, or system constituents rather than presented as an arbitrary list.

Unnecessary duplication should be removed. Related requirements should remain connected, and levels of abstraction should not be mixed without explaining their relationships.

3.15.5 Traceability and modifiability

Each requirement should have a stable identifier and be traceable to relevant sources, decisions, models, assumptions, related requirements, and verification evidence. Traceability helps answer:

- Why does this requirement exist?
- What depends on it?
- What must be reconsidered if it changes?

Versioning and change control should allow agreed baselines to evolve without losing their history. A baseline is an accepted set for a stated purpose and time; it is not a claim that change has ended.

3.15.6 Validatable and verifiable as a whole

Relevant stakeholders should be able to evaluate whether the set expresses the right system responsibilities. A credible verification strategy should also exist for every requirement, without incompatible acceptance conditions or uncovered obligations.

Reviews should examine the requirements together with their models, assumptions, definitions, and system context. Reviewing isolated sentences is insufficient.

Key takeaway: *The quality of a requirements set depends on justified coverage, consistency, collective feasibility, coherent structure, traceability, and its ability to be validated and verified as a whole.*

3.16 <technique> Writing Atomic Textual Requirements

An **atomic** or **singular** requirement expresses one principal obligation. Atomicity makes requirements easier to classify, trace, change, prioritise, and verify independently.

Atomic does not mean extremely short, entirely self-contained, or unrelated to other requirements. A requirement may include conditions, quantities, defined terms, and references while still expressing one obligation.

3.16.1 A basic sentence pattern

A useful starting pattern is:

[Under stated conditions,] the [system or constituent] shall [required behaviour, outcome, quality, or restriction].

Examples:

- When a patient submits a valid cancellation request, the appointment system shall cancel the corresponding appointment.
- For at least 95% of valid cancellation requests, the appointment system shall display confirmation within two seconds.
- The appointment system shall use the hospital identity service for staff authentication.

The word **shall** conventionally identifies a binding obligation. *Should* expresses a recommendation, *may* expresses permission, and *will* usually describes an intention or expected fact. These words should not be used interchangeably.

3.16.2 Writing procedure

1. **Identify the responsible subject.** Name the system or constituent that must satisfy the obligation. **Avoid vague subjects such as *the solution*, *it*, or *users* unless they are precisely defined.**
2. **State the relevant condition or trigger.** Specify when, where, or under which circumstances the obligation applies. **Do not add a condition if it contributes no useful distinction.**
3. **Use an active and precise verb.** Prefer verbs that describe an observable outcome, such as *display*, *record*, *reject*, *calculate*, or *notify*. **Verbs such as *support*, *handle*, *manage*, or *optimise* often hide the required behaviour.**
4. **Define the object and necessary qualifiers.** Identify what is processed, produced, restricted, or evaluated. Use defined domain terms and explicit units.

5. **Separate distinct obligations.** Inspect statements containing several verbs or conjunctions. Separate requirements that may be implemented, changed, prioritised, or verified independently.

6. **Add identification and supporting information.** Assign a stable identifier and record its source, rationale when necessary, priority, status, and verification method.

3.16.3 Decomposing a compound statement

Consider:

- The system shall validate cancellation requests, record them securely, and quickly notify the patient.

This statement combines several behaviours with vague quality claims. A better set might include:

- R1: When a cancellation request is received, the appointment system shall validate the appointment identifier and cancellation authority.
- R2: After accepting a cancellation request, the appointment system shall record the cancellation.
- R3: After recording a cancellation, the appointment system shall notify the patient.

Separate quality requirements should define the required notification time and protection of the recorded information.

3.16.4 Avoid mechanical rules

The occurrence of *and* does not always prove that a statement contains several requirements. *"The system shall compare the supplied identifier and date with the appointment record"* may describe one indivisible validation operation. Conversely, a sentence without *and* may still hide several obligations.

Negative requirements are also legitimate when they express a necessary prohibition, particularly for safety or security. However, analysts should determine what behaviour must occur when the prohibited condition is attempted.

Excessive decomposition creates fragmented specifications that are difficult to understand. Atomic requirements should remain connected through structure, relationships, scenarios, and conceptual models.

Key takeaway: Write one justified obligation at a time, with an explicit subject, relevant conditions, precise terminology, and a defined means of verification.

A **requirements specification** is a work product that organises and communicates the requirements for a defined system, scope, purpose, and audience. It is more than a collection of isolated sentences and does not have to be a single document.

A specification may be maintained as a document, repository, model package, product backlog, contractual annex, or controlled combination of these forms. The medium is less important than the meaning, authority, relationships, and change history of its contents.

3.17.1 Typical contents

Depending on its purpose, a requirements specification may include:

- purpose, scope, and intended audience;
- system context and boundary;
- relevant definitions and a domain glossary;
- assumptions, dependencies, and external conditions;
- functional requirements, quality requirements, and constraints;
- conceptual models, interfaces, use cases, and scenarios;
- relationships and traceability to sources;
- priorities, status, rationale, and allocation information;
- planned verification methods and acceptance conditions;
- version, approval, and change information.

These items do not all have the same status. A textual requirement establishes an obligation; an assumption records a condition relied upon; a model may provide context or specify part of the required system behaviour. Their roles should be made explicit.

3.17.2 Text and models

Requirements may be expressed through natural language, structured text, tables, formal expressions, or models. Different representations can complement one another.

A conceptual model may clarify terminology, structure, behaviour, states, interactions, or system boundaries that would be difficult to communicate through isolated statements. However, not every model element is automatically a requirement. The specification should make clear which parts are normative, which are explanatory, and how models relate to textual requirements.

Duplicating the same requirement independently in several representations creates a risk of inconsistency. Where possible, maintain one authoritative expression and use traceable views for different audiences.

3.17.3 Specification is not “the how”

The distinction between requirements and design should not be reduced to the claim that a requirements specification describes *what* while a design describes *how*. A requirements specification may legitimately contain externally imposed interfaces, technologies, or architectural constraints. Conversely, a vague description of desired behaviour does not become a valid requirement merely because it avoids implementation detail.

The important distinction is the type and source of commitment:

- requirements establish obligations that the system must satisfy;
- design decisions select ways of satisfying those obligations;
- supporting information explains their context and rationale.

If design information is included, it should be identified as such rather than silently presented as a requirement.

3.17.4 Baselines and evolution

A **baseline** is an identified and agreed version of the specification used as a reference for subsequent work. It enables controlled evaluation of proposed changes and their consequences.

Baselining does not freeze understanding permanently. Requirements may change as the UoD changes, evidence improves, risks emerge, or stakeholders revise decisions. A good specification preserves the history and rationale of those changes.

A requirements specification should be reviewed as a whole for coverage, consistency, collective feasibility, comprehensibility, traceability, and verifiability. Good sentence structure alone is insufficient.

Key takeaway: *A requirements specification is a controlled and structured work product that combines system obligations with the context and relationships needed to understand, evaluate, and maintain them.*

3.18 <concept> Relationships Between Requirements

Requirements rarely stand alone. They form a network in which one requirement may elaborate, depend on, qualify, conflict with, or provide an alternative to another. Recording these relationships supports analysis, verification, change management, and system design.

A requirements set should therefore not be treated as an unordered list or assumed to form a simple hierarchy.

3.18.1 Refinement and decomposition

A requirement **refines** another when it adds precision or detail while preserving the original obligation.

A requirement may be **decomposed** into several more specific requirements whose combined satisfaction is intended to satisfy the broader requirement. The decomposition should cover the parent obligation without introducing contradictions or unjustified restrictions.

Derived requirements often emerge while allocating responsibilities or developing the architecture. They still require an explicit rationale: being introduced by an engineer does not make them arbitrary.

Satisfaction of lower-level requirements does not automatically prove satisfaction of the higher-level requirement. Interactions between constituents and emergent system qualities may also need to be evaluated.

3.18.2 Dependency

One requirement **depends on** another when its satisfaction, meaning, or implementation relies on the other.

For example:

- R1: The appointment system shall allow an authenticated patient to cancel an appointment.
- R2: The appointment system shall authenticate the patient before accepting a cancellation request.
- R1 depends on the capability required by R2. If R2 changes, R1 and its verification may need to be reconsidered.

Dependencies may be logical, operational, temporal, or implementation-related. The intended meaning of the relationship should be recorded rather than hidden behind a generic *related to* link.

3.18.3 Qualification and restriction

A quality requirement may **apply to** one or more functional requirements:

- R3: For at least 95% of valid cancellation requests, the system shall display confirmation within two seconds.

R3 establishes a performance level for the cancellation function. It should be linked to that function rather than left as an apparently global quality claim.

Similarly, a constraint may restrict how one or more requirements can be satisfied:

- R4: Patient authentication shall use the hospital identity service.

R4 constrains the solution permitted for R2.

3.18.4 Conflict

Requirements **conflict** when they cannot all be satisfied under the same relevant conditions. Conflicts may be direct, conditional, or caused by competition between qualities such as security, usability, availability, cost, and performance.

A conflict should not be concealed through vague wording. The affected concerns, sources, alternatives, decision authority, and resolution should be recorded.

3.18.5 Alternatives, overlap, and duplication

Requirements may express alternative ways of satisfying a broader obligation or may apply only to different product variants or operating conditions. The conditions under which each alternative applies must be explicit.

Overlapping requirements should be reviewed for accidental duplication. Duplication increases the risk that one copy changes while another remains outdated. Similar wording does not always imply duplication, however: two requirements may apply to different actors, states, or interfaces.

3.18.6 A wider traceability network

Requirements also relate to entities outside the requirements set:

- goals, needs, concerns, rules, and other sources explain their justification;
- assumptions identify conditions on which they depend;
- models clarify their context and interactions;
- system constituents receive allocated responsibilities;
- design decisions explain how they are satisfied;
- verification evidence demonstrates their satisfaction.

These relationships are generally many-to-many.

Key takeaway: Requirements form a traceable network. Explicit relationship types reveal coverage, dependencies, conflicts, consequences of change, and the reasoning that connects sources to system evidence.

3.19 <concept> Verification and Validation

Verification and **validation** evaluate different questions. Both can be applied to requirements and to the resulting system.

Object	Verification asks	Validation asks
Requirements	Are the requirements sufficiently well-formed and coherent?	Do they express the appropriate system obligations?
System	Does the system satisfy its specified requirements?	Is the system fit for its intended purposes and context?

3.19.1 Requirements verification

Requirements verification examines the quality of the requirements as work products:

- For an **individual requirement**, it considers whether the statement is necessary, appropriately scoped, unambiguous, sufficiently complete, singular, feasible, and verifiable.
- For a **requirements set**, it considers coverage, consistency, collective feasibility, structure, traceability, and whether a suitable means of verification exists for every obligation.

Reviews, inspections, terminology checks, model analysis, consistency analysis, and preliminary verification designs may all contribute evidence. **A requirement can pass verification and still be the wrong requirement.**

3.19.2 Requirements validation

Requirements validation examines whether the proposed requirements express responsibilities that are appropriate for the intended system, purposes, stakeholders, and operational context. It asks questions such as:

- Do these requirements address the justified goals, needs, concerns, and rules?
- Is the system boundary appropriate?
- Have responsibilities been allocated to the right constituents?
- Are relevant stakeholders and operating situations represented?
- Are the priorities and trade-offs acceptable?
- Would satisfying these requirements address the problem under analysis?

3.19.5 V&V throughout the work

Verification and validation should begin while requirements are being formulated. Defining expected evidence early exposes vague, infeasible, or misplaced obligations before they become expensive to change.

The common mnemonic “*building the product right*” versus “*building the right product*” may aid memory, but it is incomplete. The object being evaluated and the evidence required must always be stated explicitly.

Validation may use stakeholder reviews, scenarios, prototypes, simulations, walkthroughs, conceptual models, and comparison with evidence from the UoD. Stakeholder agreement is important but not sufficient: statements must also be checked against applicable rules, observed conditions, conflicts, and available evidence.

3.19.3 System verification

System verification obtains objective evidence that the implemented system satisfies its specified requirements.

Possible methods include:

- **Test:** exercise the system and compare observed with expected results;
- **Demonstration:** show required behaviour under controlled conditions;
- **Inspection:** examine a system constituent, document, configuration, or record;
- **Analysis:** use calculations, models, static analysis, or other reasoned evidence.

The appropriate method depends on the requirement. Functional requirements, quality requirements, and constraints must all be verifiable, although they may require different evidence.

3.19.4 System validation

System validation examines whether the resulting system is fit for its intended use in its intended operational context. A system may satisfy every stated requirement and still fail to address the original concerns because the requirements were incomplete, inappropriate, or based on false assumptions.

For example, an appointment application may correctly display confirmation within two seconds and still be unsuitable if relevant patients cannot use the available interaction channel or if the surrounding administrative procedure remains ineffective.

Validation of a sociotechnical system may therefore require operational trials, observation, exercises, user evaluation, or analysis of real outcomes (not only software testing).

Acceptance is related but distinct: it is a decision by an authorised party that contractual or organisational acceptance conditions have been met.

Key takeaway: *Verification evaluates conformity with specified obligations; validation evaluates fitness for intended purposes and context. Both the requirements and the resulting system require verification and validation.*

3.20 <example> From Statements to a Requirements Set

During an interview, a hospital manager states:

“Patients must be able to cancel appointments easily. We need a mobile app that immediately notifies everyone.”

This is not a ready-made requirement. It combines a stakeholder need, a quality concern, a proposed solution, and several ambiguous expressions.

3.20.1 Investigating the statement

The analyst asks:

- Why is cancellation currently difficult?
- Which patients and appointments are involved?
- What does *easily* mean in relevant situations?
- Is a mobile app necessary, or merely one possible solution?
- Who is included in *everyone*?
- What does *immediately* mean, and why is that timing needed?
- How is the requester authorised to cancel an appointment?
- What operational actions follow a cancellation?
- Which rules and existing systems apply?

Further elicitation reveals that cancellations are currently accepted only by telephone during office hours; some patients have difficulty reaching the service; not all patients use smartphones; receptionists may cancel appointments on behalf of patients; cancelled slots must be returned to availability; and staff authentication must use the hospital identity service.

3.20.2 Framing the system

The system of interest is defined as a sociotechnical **appointment service** comprising administrative staff, appointment procedures, and supporting software. Patients and the hospital identity service remain external.

The analysis identifies:

- **goal:** reduce missed appointments and make cancelled slots reusable;
- **stakeholder need:** cancel an appointment without travelling to the hospital;
- **concerns:** accessibility, authorisation, timely confirmation, and operational coordination;
- **constraint source:** mandated use of the hospital identity service;
- **rejected premature decision:** a mobile app is not assumed to be necessary.

3.20.3 Illustrative requirements

The following is a partial requirements set:

- **FR1:** The appointment service shall accept cancellation requests through the web portal and by telephone.

- **FR2:** Before accepting a cancellation request, the appointment service shall verify that the requester is the patient or an authorised representative.
- **FR3:** After accepting a cancellation request, the appointment service shall mark the appointment as cancelled and return its slot to the set of available slots.
- **FR4:** After cancelling an appointment, the appointment service shall notify the patient through the registered contact channel.
- **FR5:** For each accepted cancellation, the appointment service shall record the appointment identifier, request channel, time, and identified requester.
- **QR1:** The web cancellation function shall be available for at least 99.5% of each calendar month, excluding agreed maintenance periods.
- **QR2:** For at least 95% of web cancellation requests, confirmation shall be displayed within two seconds after the request is accepted.
- **C1:** The supporting software shall use the hospital identity service to authenticate administrative staff.

The quantitative criteria in QR1 and QR2 are not invented by the analyst. In this example, they result from additional operational evidence, feasibility analysis, and an explicit stakeholder decision.

3.20.4 Traceability and remaining work

FR1 traces to the stated need and the observed limitations of the current telephone-only process. FR2 and FR5 trace to the applicable authorisation and accountability rules. FR3 traces to the operational goal of making cancelled slots reusable. C1 traces to an externally mandated technology decision.

An important assumption is that the external identity service will provide sufficient availability. Its failure consequences and required fallback behaviour still need investigation.

This set is illustrative, not complete. Exceptions, accessibility conditions, notification failures, privacy, operating volumes, and relationships with other hospital services require further analysis and validation.

The original request for a mobile app remains recorded as a proposed solution. A responsive web portal, mobile app, or another design might satisfy the approved requirements; the choice should be made through design evaluation unless an authorised source imposes a constraint.

Key takeaway: One stakeholder statement can initiate requirements analysis, but it cannot by itself justify a requirements set. Additional evidence, boundary decisions, negotiation, formulation, and validation are required.

3.21 <example> Reviewing and Repairing a Requirements Set

Consider this draft requirements set for a hospital appointment service:

- **R1:** The system should allow users to cancel appointments easily.
- **R2:** The application shall authenticate users, immediately send SMS and email notifications, and use NotifyNow.
- **R3:** The service shall always be available during working hours, except during maintenance whenever necessary.
- **R4:** All appointment information shall be deleted immediately after cancellation.
- **R5:** Cancellation records shall be retained for five years.
- **R6:** Patients shall provide valid contact details.
- **R7:** The supplier shall deploy the solution by 30 September.

The statements appear reasonable, but the set is not ready for agreement or implementation.

3.21.1 Review findings

- **R1** uses *should*, leaves *system*, *users*, and *easily* undefined, and does not state the applicable conditions.
- **R2** combines several obligations. *Immediately* is vague; requiring both SMS and email may be unjustified; and NotifyNow may be a design decision rather than a constraint.
- **R3** combines an absolute claim with an undefined exception. *Always*, *working hours*, and *whenever necessary* do not establish a verifiable availability level.
- **R4** and **R5** appear inconsistent because *appointment information* and *cancellation records* are not distinguished.
- **R6** governs patients, who are external to the defined system. It may express a participation rule or assumption but is not a system requirement.
- **R7** is a project-delivery requirement mixed into the system requirements set.

The set omits important behaviour following a cancellation, including releasing the appointment slot.

The requirements are not traceable to sources, assumptions, concerns, or verification methods.

3.21.2 Additional analysis

Repairing the set requires more than editing sentences.

Further investigation establishes that:

- the system of interest includes administrative staff, appointment procedures, and supporting software;
- patients and the supplier are external;
- patients may cancel through the portal, while staff may cancel following a telephone request;
- notification through one registered contact channel is sufficient;

- NotifyNow is an internal design proposal, not an imposed constraint;
- an active appointment and a cancellation audit record are different information objects;
- an approved policy requires a minimal cancellation audit record to be retained for five years;
- agreed availability and response-time criteria have been established.

3.21.3 Repaired requirements set

- **FR1:** The appointment service shall allow an authenticated patient to submit a cancellation request for their own future appointment through the patient portal.
- **FR2:** The appointment service shall allow an authorised receptionist to submit a cancellation request received by telephone.
- **FR3:** After accepting a cancellation request, the appointment service shall mark the appointment as cancelled and return its slot to the set of available slots.
- **FR4:** After cancelling an appointment, the appointment service shall notify the patient through the registered contact channel.
- **FR5:** For each accepted cancellation, the appointment service shall retain an audit record containing the appointment identifier, requester identity, request channel, and cancellation time for five years.
- **QR1:** For at least 95% of valid portal cancellation requests, confirmation shall be displayed within two seconds after acceptance.
- **QR2:** The portal cancellation function shall be available for at least 99.5% of the defined service hours in each calendar month, excluding announced maintenance periods.

The participation rule concerning valid patient contact details is documented separately. The deployment date is moved to the project-delivery requirements. NotifyNow remains a candidate design decision and may be evaluated against the approved requirements.

The concern expressed by *easily* has not been silently discarded. It remains a usability concern requiring further investigation, appropriate criteria, and validation with representative patients.

This repaired set is still only a partial specification. Its quality depends on the evidence and decisions stated in the analysis, not merely on the improved wording.

Key takeaway: Reviewing requirements means checking meaning, responsibility, relationships, sources, and coverage. Rewriting sentences without resolving the underlying analytical problems only produces better-written defects.

3.22 <concept> Requirements and Conceptual Models

Conceptual models and requirements serve different but complementary purposes. Requirements establish obligations; conceptual models organise selected concepts and relationships to support understanding, communication, analysis, and specification.

A conceptual model is not a neutral copy of reality. It represents aspects considered relevant to particular concerns and purposes. Different concerns may justify different models of the same UoD, but each model must remain consistent with the available evidence and its stated viewpoint.

3.22.1 Models during requirements engineering

Conceptual models can support several activities:

- **Elicitation:** making implicit knowledge visible and generating questions;
- **Analysis:** clarifying terminology, boundaries, structures, behaviour, and responsibilities;
- **Negotiation:** exposing different interpretations and their consequences;
- **Specification:** expressing required structures or behaviour;
- **Verification:** detecting omissions, inconsistencies, and undefined cases;
- **Validation:** allowing stakeholders to examine proposed system responsibilities.

Different models address different questions. A domain model may define concepts and relationships; a context model may establish the system boundary and external interactions; a process model may represent coordinated activities; a state model may expose permitted transitions; and an interaction model may describe exchanges between participants or constituents.

3.22.2 Descriptive, exploratory, or normative

The role of each model must be explicit:

- A **descriptive model** represents an understanding of an existing situation. It may reveal problems but does not state how the future system must operate.
- An **exploratory model** investigates a possible interpretation or solution. It supports discussion but has not yet been agreed as an obligation.
- A **normative model** represents required structure or behaviour and forms part of the specification.

The modelling language does not determine the role. A BPMN process model, for example, might describe current practice, explore a proposed process, or specify required

operational behaviour. A UML model may similarly represent the UoD, the required logical system, or a design.

Not every element in a normative model is automatically an independent requirement. The specification should identify which model contents are binding and how they relate to textual requirements.

3.22.3 Models reveal requirements problems

Consider an appointment service. A conceptual information model distinguishes an **Appointment**, an **Appointment Slot**, and a **Cancellation Record**. A state model shows that an appointment may move from *scheduled* to *cancelled*, while its slot becomes *available*.

These models expose the ambiguity in statements such as:

- “All appointment information shall be deleted immediately after cancellation.”

They allow analysts to ask what information changes state, what must remain for audit purposes, and which entities have different retention rules.

Likewise, an event with no permitted response may reveal a missing functional requirement; an activity without an assigned responsible constituent may reveal a boundary problem; and conflicting cardinalities or states may expose inconsistent requirements.

3.22.4 Combining models and text

Models are powerful for structure, relationships, sequences, and alternatives. Text is often more suitable for rationale, quantitative quality criteria, exceptions, legal sources, and detailed constraints. Neither representation is universally superior.

Requirements should be traceable to the model elements that explain or specify them. However, duplicating the same obligation independently in text and diagrams creates a risk of divergence. The authoritative representation should be identified.

Models must also be understandable to their intended audience. Formal correctness does not guarantee successful validation if stakeholders cannot interpret the notation.

Key takeaway: Conceptual models help discover, analyse, communicate, and sometimes specify requirements. Their descriptive, exploratory, or normative status must always be explicit.

A **user story** is a short statement used to express a desired capability or outcome from the perspective of a relevant role.

User stories are commonly used to support conversation, prioritisation, and incremental planning.

A frequent template is:

As a [role], **I want** [capability or outcome], **so that** [benefit or rationale].

For example:

- “As a patient, I want to cancel an appointment online so that I do not have to telephone the hospital and the slot can be offered to someone else.”

The template identifies a perspective, a desired capability, and a reason. It does not by itself specify the complete system obligation.

3.23.1 Card, conversation, and confirmation

User stories are often explained through three complementary elements:

- **Card:** a brief record used to identify and plan the item;
- **Conversation:** the analysis through which its meaning, rules, alternatives, and exceptions are discovered;
- **Confirmation:** agreed evidence or acceptance criteria used to evaluate the result.

The written story is therefore a reminder of a conversation, not a substitute for that conversation.

For the cancellation story, analysts might ask:

- Which patients may cancel which appointments?
- Must the patient be authenticated?
- Are there time limits or exceptions?
- What happens to the released appointment slot?
- Which parties must be notified?
- What happens if notification fails?
- Which interaction channels must be accessible?
- Which response time is acceptable?

3.23.2 From a story to requirements

After analysis, the story may motivate several system requirements:

- “The appointment system shall allow an authenticated patient to submit a cancellation request for their own future appointment.”

After accepting a cancellation request, the appointment system shall return the corresponding slot to the set of available slots.

- For at least 95% of valid cancellation requests, the system shall display confirmation within two seconds after acceptance.

The first two are functional requirements; the third is a quality requirement. Additional constraints, rules, assumptions, and exception behaviour may also be needed.

One story can therefore motivate several requirements, while one requirement may contribute to several stories.

3.23.3 Limitations

A user story is not automatically a well-formed system requirement. In particular:

- the role is usually external to the system and is not the bearer of the system obligation;
- the desired capability may remain ambiguous;
- quality requirements and constraints fit the template poorly;
- sociotechnical responsibilities may be hidden behind a software-oriented interpretation;
- exceptions, dependencies, and conflicting concerns may remain unexplored;
- a proposed solution may be presented as though it were the underlying need.

A story such as “*As a manager, I want a dashboard*” should trigger questions about the decisions, information, and concerns involved before a dashboard is accepted as necessary.

User stories should be traceable to their sources, related requirements, models, acceptance criteria, and implementation work. They may form part of a requirements specification or product backlog, but a collection of stories does not guarantee adequate coverage or consistency.

The story template should also not be forced onto technical obligations that have no meaningful user perspective. A direct system requirement or constraint may be clearer.

Key takeaway: A user story captures a perspective and starts a structured conversation. Its value lies in the analysis and *confirmation that follow, not in filling in a template.*

A **use case** describes how actors interact with a defined subject (a sociotechnical system, a software system, or a subsystem) to achieve a goal.

An **actor** represents an external role relative to that subject, not necessarily an individual person. People, organisations, devices, or other systems may act as actors.

This relativity matters. A receptionist may be a constituent of a sociotechnical appointment service but an external actor of the appointment software. The same person can therefore appear differently when the use-case subject changes.

3.24.1 Use-case model and specification

A use-case diagram provides a compact map of actors and their goals. It does not normally contain enough information to specify the required interactions.

A textual use-case specification may include:

- name and scope;
- primary actor and goal;
- relevant supporting actors;
- trigger;
- preconditions;
- main success scenario;
- alternative and exception flows;
- success and failure postconditions;
- related rules, requirements, qualities, and constraints.

The appropriate degree of detail depends on the purpose of the analysis.

3.24.2 Example: Cancel Appointment

Scope: appointment service
Primary actor: patient
Goal: cancel a future appointment
Trigger: the patient requests cancellation
Precondition: the patient has an existing appointment that can be identified
Main success scenario: 1. The patient identifies the appointment to be cancelled. 2. The appointment service verifies the patient's authority to cancel it. 3. The service accepts the cancellation. 4. The service marks the appointment as cancelled. 5. The service returns the appointment slot to availability. 6. The service records the cancellation. 7. The service confirms the result to the patient.
Alternative situations: • the patient cannot be authenticated; • the appointment does not exist or is not cancellable; • the cancellation conflicts with an applicable rule; • the confirmation cannot be delivered.

Alternative and exception flows are essential. A use case containing only a successful path hides much of the behaviour that requirements engineering must address.

3.24.3 Use cases and requirements

A use case can organise and express required behaviour, but it is not automatically a complete requirements specification. One use case may support several functional requirements, and one requirement may apply across several use cases.

Quality requirements and constraints often need separate statements linked to the relevant use cases.

For example:

- For at least 95% of valid online cancellation requests, the appointment service shall display confirmation within two seconds after acceptance.

A precondition is also not necessarily a requirement. It states something assumed to be true before the use case begins. If the system is responsible for establishing that condition, the corresponding obligation must be specified elsewhere.

Use cases should describe externally meaningful interactions and outcomes without prematurely exposing internal components, databases, algorithms, or screen designs. Internal behaviour may be explored later through other models.

3.24.4 Using use cases during analysis

Use cases help analysts:

- discover missing actors, responsibilities, and interactions;
- explore normal, alternative, and failure situations;
- establish the consequences of events;
- organise related requirements and acceptance criteria;
- validate proposed behaviour with stakeholders;
- identify dependencies between the system and its environment.

Use-case steps should be written at a consistent level of abstraction and use defined domain terminology. Each step is not necessarily an independent requirement; requirements should be separated where independent traceability, change, or verification is needed.

Key takeaway: A use case structures goal-oriented interaction across an explicit system boundary. Its value lies in revealing required behaviour, alternatives, and responsibilities (not merely in drawing actors and ovals).

3.25 <technique> Scenarios

A **scenario** describes a concrete sequence of events and interactions occurring under particular conditions. It shows one possible path through a situation involving the system of interest and its environment.

Scenarios may represent:

- current practice or proposed future behaviour;
- normal, alternative, or exceptional situations;
- successful or unsuccessful outcomes;
- common cases or rare but high-impact cases;
- intended use or deliberate misuse.

A scenario for a sociotechnical system may involve people, procedures, information, software, devices, and external organisations. It should not be restricted automatically to interactions with a user interface.

3.25.1 Scenario structure

A useful scenario identifies:

- its purpose and status;
- the relevant system boundary;
- participating actors and system constituents;
- initial conditions;
- triggering event;
- ordered events and interactions;
- resulting state or outcome;
- uncertainties, assumptions, and unresolved questions.

Concrete examples, values, and circumstances make scenarios easier to examine. However, these details must not be generalised into requirements without justification.

3.25.2 Example: an online cancellation

Ana has a scheduled appointment for the following week. At 18:30, after the telephone service has closed, she accesses the patient portal. She identifies the appointment and requests its cancellation. The appointment service authenticates her, verifies that the appointment can be cancelled, records the cancellation, returns the slot to availability, and displays confirmation. A notification is then sent to her registered contact channel.

This scenario raises analytical questions:

- Must online cancellation be available outside telephone-service hours?
- What identifies an appointment unambiguously?
- Under which conditions may cancellation be refused?
- When does the released slot become available to others?

- What happens if notification fails after cancellation succeeds?
- Can an authorised representative perform the same action?
- Which response times are acceptable?
- What evidence must be retained?

The answers may motivate requirements, rules, assumptions, alternative scenarios, and conceptual models.

3.25.3 Scenario, use case, and test case

A **scenario** represents one particular path.

A **use case** usually organises a family of related scenarios that pursue the same actor goal, including a main success scenario and alternative or exception flows.

A **test case** defines controlled inputs, execution conditions, expected results, and a pass-or-fail evaluation. A scenario can inspire a test case but normally lacks the precision and control needed to function as one.

3.25.4 Scenarios are not complete requirements

A scenario demonstrates what happens in one selected situation. It does not establish what must happen in every relevant situation.

For example, observing that the service displays confirmation within one second in the scenario does not justify a general one-second performance requirement. A required threshold needs an explicit source, operating conditions, and agreed evaluation criterion.

Conversely, a scenario may expose obligations omitted from a textual requirements set. Analysts should compare scenarios with requirements and models to identify unhandled events, missing actors, contradictory outcomes, and hidden assumptions.

Useful scenario sets should include more than the expected successful path. Boundary conditions, failures, concurrent events, unauthorised actions, unavailable external services, and recovery situations often reveal the most important missing requirements.

Scenarios may be descriptive, exploratory, or normative. Their status should be stated so that an observed current practice is not mistaken for required future behaviour.

Key takeaway: A scenario makes one situation concrete enough to analyse and discuss. It reveals requirements and tests assumptions, but it does not define the complete behaviour of the system.

3.26 <technique> Acceptance Criteria

An **acceptance criterion** states a condition used by an authorised party to decide whether a defined work item, capability, increment, or deliverable is acceptable.

Acceptance criteria clarify the expected outcomes of user stories, use cases, features, or other planned changes. They support shared understanding and provide a basis for verification, but they do not automatically constitute a complete requirements specification.

3.26.1 Writing useful criteria

Acceptance criteria should be:

- relevant to the item being accepted;
- observable and sufficiently precise;
- stated in terms of outcomes rather than implementation tasks;
- consistent with the applicable requirements and system boundary;
- able to support an objective acceptance decision;
- inclusive of important alternative and failure situations.

They may be written as concise rules, checklists, examples, tables, or structured statements such as **Given–When–Then**.

Consider the user story:

As a patient, I want to cancel an appointment online so that the slot can be offered to someone else.

Possible acceptance criteria include:

1. Given an authenticated patient and their cancellable future appointment, when the patient confirms cancellation, the appointment becomes *cancelled* and its slot becomes *available*.
2. Given an appointment that is not cancellable, when cancellation is requested, the request is rejected and the appointment remains unchanged.
3. When a cancellation is accepted, the patient receives confirmation through the registered contact channel.
4. The cancellation interaction satisfies the response-time requirement **QR-CANCEL-01**.

The fourth criterion refers to an authoritative quality requirement instead of duplicating its threshold. This reduces the risk of inconsistent copies.

3.26.2 Relationship with requirements

Acceptance criteria may clarify how existing requirements apply to a particular item. If a criterion introduces a new system obligation, threshold, exception, or constraint, that

obligation must be analysed and managed as a requirement rather than hidden inside a backlog item.

For example:

- The confirmation must be displayed within two seconds.

If no approved requirement previously establishes this threshold, the criterion has introduced a quality requirement. Its source, rationale, feasibility, and wider consequences must be considered.

Acceptance criteria should therefore remain traceable to applicable requirements, scenarios, rules, and models.

3.26.3 Criteria are not test cases

An acceptance criterion states a condition for acceptance. A **test case** defines the specific inputs, initial state, execution procedure, expected result, and evidence used to evaluate that condition.

One acceptance criterion may require several test cases, including boundary values and exceptional conditions. Some criteria may be evaluated through demonstration, inspection, analysis, or operational observation rather than software testing.

3.26.4 Criteria are not a Definition of Done

A **Definition of Done** normally establishes general completion conditions that apply across many work items, such as required reviews, documentation, tests, or deployment activities.

Acceptance criteria are specific to the capability or outcome being accepted. The Definition of Done often governs engineering or delivery work; acceptance criteria usually evaluate the result relevant to a particular item.

3.26.5 Acceptance is not full validation

Passing all acceptance criteria supports an acceptance decision but does not prove that the complete system is fit for its intended purpose. The criteria themselves may be incomplete or based on incorrect assumptions.

Acceptance criteria should be agreed early enough to guide analysis and implementation, then changed deliberately when understanding evolves.

Key takeaway: Acceptance criteria make expected outcomes examinable. They should clarify and connect to requirements—not create an uncontrolled second specification.

3.27 <technique> Specification by Example

Specification by Example is a collaborative technique in which concrete examples are used to discover, clarify, and communicate required behaviour. Examples make abstract rules examinable and help expose ambiguity, missing conditions, and disagreements before implementation.

The technique is more than adding an example after a written requirement. Analysts, stakeholders, designers, and testers work together to select examples that clarify what the system must do in relevant situations.

3.27.1 From a rule to examples

Suppose an approved rule states:

- A patient may cancel an appointment until 24 hours before its scheduled start.

The expression *until 24 hours before* contains a boundary that may be interpreted differently. Consider an appointment scheduled for 20 June at 10:00:

Cancellation request	Expected result
19 June at 09:59	Accepted
19 June at 10:00	Accepted
19 June at 10:01	Rejected

The examples establish that a request made exactly 24 hours before the appointment is accepted. They also expose further questions: Which clock and time zone apply? What happens when the appointment has already been cancelled? Can staff override the rule? What happens during clock changes or service unavailability?

3.27.2 Given–When–Then

Examples are often expressed using a simple structure:

Given an appointment scheduled for 20 June at 10:00
And an authenticated patient authorised to cancel it
When the patient requests cancellation on 19 June at 10:00
Then the appointment is cancelled
And its slot becomes available.

Given establishes relevant initial conditions; **When** identifies the event or action; **Then** states the expected observable outcome.

The format is useful but not mandatory. Tables, annotated scenarios, calculations, sample messages, and other concrete representations may be more appropriate.

3.27.3 Developing useful examples

A practical process is to:

1. identify the requirement, rule, story, or uncertainty under discussion;
2. agree on the terminology and system boundary;
3. propose a normal successful example;
4. add counterexamples, failures, and boundary values;
5. compare expected outcomes and resolve disagreements;
6. state or revise the general requirement;
7. trace the examples to the requirement and verification evidence.

Examples should be selected deliberately. Ten similar examples may provide less insight than one boundary case or counterexample.

3.27.4 Examples and requirements

An example illustrates a particular instance; a requirement establishes a general obligation. Neither should silently replace the other.

A finite set of examples cannot demonstrate every permitted or prohibited situation. Analysts must identify the general rule that explains the examples and determine whether additional cases exist.

If an example introduces a new outcome, threshold, exception, or constraint, that information must be incorporated into the managed requirements rather than remaining hidden in a test script.

3.27.5 Executable examples

Some examples can be automated and executed against the system. These may serve simultaneously as specification, acceptance evidence, regression tests, and living documentation.

Automation is useful only when the examples remain understandable, authoritative, and synchronised with the requirements. Executable examples can still encode the wrong rule or provide incomplete coverage.

Not every system quality is adequately specified through examples. Usability, resilience, security, safety, maintainability, and emergent sociotechnical behaviour may require additional models, analyses, trials, or evaluation methods.

Key takeaway: Concrete examples expose the meaning and boundaries of general rules. Their purpose is to sharpen requirements—not to replace analysis with a collection of test cases.

3.28 <technique> Prototypes and Wireframes

A **prototype** is a partial or provisional representation of a possible system used to investigate questions, test assumptions, explore alternatives, or obtain feedback.

A **wireframe** represents the organisation of information, controls, and navigation in an interface, usually without detailed visual styling or complete behaviour. Wireframes are one form of prototype.

Prototypes can range from paper sketches and role-played service interactions to clickable interfaces, simulated devices, and technically functional implementations.

3.28.1 Match fidelity to the question

Low-fidelity prototypes are quick to create and change. They are useful for exploring concepts, terminology, information organisation, workflows, and alternative solutions without implying that major decisions have already been made.

High-fidelity prototypes resemble the intended system more closely. They can support detailed interaction and usability evaluation but require more effort and may cause participants to assume that the solution is nearly complete.

The appropriate fidelity depends on what needs to be learned. A polished interface is unnecessary when the question concerns system boundaries or responsibilities. A paper sketch is insufficient when the question concerns interaction timing or technical feasibility.

3.28.2 Prototyping as requirements work

A disciplined prototyping activity should:

1. state the question, uncertainty, or hypothesis being investigated;
2. identify the relevant participants and scenarios;
3. select only the fidelity needed to obtain useful evidence;
4. make clear which parts are functional, simulated, or absent;
5. observe behaviour and reasoning rather than merely asking whether participants “like” the prototype;
6. record findings, disagreements, assumptions, and limitations;
7. update requirements, models, or design decisions through the normal analysis and change process.

For example, a wireframe for appointment cancellation may show a list of future appointments and a cancellation control. Evaluation may reveal that patients do not recognise

internal appointment identifiers, cannot distinguish similar appointments, or expect to see cancellation conditions before confirming.

These observations motivate further analysis. They do not automatically establish that the exact layout, labels, colours, or controls in the wireframe are required.

3.28.3 Prototypes are not requirements

A prototype embodies many decisions, including accidental ones. Participants may accept a design detail simply because it is the only alternative shown. This **anchoring** can prematurely restrict the solution space.

The normative status of prototype features must therefore be explicit:

- some features may illustrate existing requirements;
- some may be exploratory design decisions;
- some may be placeholders with no intended meaning;
- some may reveal new candidate requirements.

If a prototype feature becomes obligatory, its source, rationale, and verification conditions should be recorded in the requirements specification.

3.28.4 Beyond the successful screen flow

Prototypes often emphasise the normal path and visible interface. Evaluation should also consider errors, interrupted interactions, inaccessible channels, unavailable external services, unusual data, concurrency, recovery, and relevant quality concerns.

A prototype can support usability validation but cannot by itself demonstrate production-level availability, security, performance, safety, or maintainability.

Sociotechnical systems may require **service prototypes**, process walkthroughs, simulations, or role-playing involving people and organisational procedures. Prototyping should not silently reduce the system to software screens.

Exploratory prototypes are often discarded. If prototype code or artefacts are intended to evolve into the operational system, they must be subjected to the appropriate engineering, quality, security, and change controls.

Key takeaway: A prototype is an instrument for learning. Its value lies in the evidence and questions it produces, not in making an early solution appear real.

3.29 <technique> Traceability and Change Management

Traceability records meaningful relationships between requirements and the information needed to justify, interpret, realise, and evaluate them.

It helps answer three basic questions:

- Why does this requirement exist?
- What contributes to satisfying it?
- What may be affected if it changes?

3.29.1 Traceability relationships

Useful traceability may connect a requirement to:

- goals, needs, concerns, rules, and other sources;
- assumptions and decisions on which it depends;
- parent, refined, dependent, conflicting, or related requirements;
- conceptual models and system constituents;
- design decisions and implementation elements;
- acceptance criteria, verification methods, test cases, and evidence;
- change requests, approvals, and superseded versions.

Backward traceability connects a requirement to its justification and origin. **Forward traceability** connects it to downstream design, implementation, verification, and operational evidence. In practice, navigation should be possible in both directions.

Links should have explicit meanings. A generic *related to* relationship provides little support for analysis. *Derived from*, *refines*, *depends on*, *conflicts with*, *allocated to*, and *verified by* express different consequences.

Traceability does not prove that a requirement is correct. It preserves the reasoning and evidence required to examine it.

3.29.2 Managing change

Requirements change because the UoD changes, new evidence appears, assumptions fail, rules are revised, risks emerge, or stakeholders reconsider priorities. Change is normal; unexamined change is the problem.

For a baselined requirement, a disciplined change process should:

1. identify the proposed change and its source;
2. clarify the underlying concern or reason;
3. examine affected requirements, models, assumptions, interfaces, designs, processes, tests, and stakeholders;
4. evaluate benefits, risks, feasibility, conflicts, cost, and urgency;
5. obtain a decision from the appropriate authority;

6. update the requirement, relationships, rationale, version, and status;
7. communicate the approved consequences.

Impact analysis should occur before approval, not after implementation has begun.

3.29.3 Example of impact

Suppose a rule changes the permitted patient cancellation period from at least 24 hours before an appointment to at least 12 hours.

The change may affect:

- the requirement defining when cancellation is permitted;
- examples covering the exact boundary;
- portal and telephone procedures;
- information displayed to patients;
- released-slot scheduling behaviour;
- conceptual state or process models;
- test cases and acceptance criteria;
- operational staffing assumptions;
- performance and missed-appointment measures.

Changing one sentence without examining these relationships would leave an inconsistent specification and system.

3.29.4 Proportionate control

Traceability should be proportionate to project scale, risk, regulation, contractual complexity, and expected rate of change. A small project may use identifiers, source fields, and a lightweight relationship table. A complex or safety-critical system may require a controlled repository and formal approval records.

More links are not always better. Unnecessary or outdated links create false confidence and maintenance cost. Trace the relationships needed to support justification, coverage, impact analysis, and evidence.

Each managed requirement should normally have a stable identifier, current version or baseline, status, source, and relevant relationships. Tools can assist, but they cannot determine whether a link is meaningful or current.

Change management should preserve previous decisions and their rationale. It should not pretend that requirements can be frozen permanently.

Key takeaway: *Traceability preserves the chain from concerns and sources to system evidence. Change management uses that chain to revise commitments without losing coherence or reasoning.*

3.30 <further> ISO/IEC/IEEE 29148 in Short

ISO/IEC/IEEE 29148 is the principal international standard specifically concerned with requirements engineering for systems, software products, and services throughout their life cycles.

It was developed jointly within the ISO, IEC, and IEEE standardisation context and complements the broader life-cycle process standards **ISO/IEC/IEEE 15288** for systems and **ISO/IEC/IEEE 12207** for software.

3.30.1 What the standard addresses

The standard covers:

- requirements engineering processes and activities;
- characteristics and attributes of individual requirements;
- characteristics of requirements specifications;
- requirements analysis, verification, validation, and management;
- requirements-related information items and their expected contents;
- guidance on organising and presenting those information items.

It treats requirements engineering as iterative and applicable throughout the life cycle. It may support projects using the processes defined by ISO/IEC/IEEE 15288 or ISO/IEC/IEEE 12207, but its requirements engineering guidance may also be used independently.

The standard is not merely a collection of sentence templates. Nor does following a template establish conformance. Conformance concerns the applicable processes, work products, contents, and required outcomes defined by the standard.

3.30.2 Relationship with this course

Several principles used in this chapter are aligned with the standard, including:

- distinguishing requirements from their sources and supporting information;
- making system boundaries and responsibilities explicit;
- evaluating both individual requirements and requirements sets;
- requiring requirements to be necessary, unambiguous, feasible, and verifiable;
- preserving traceability and controlling change;
- treating verification and validation as distinct activities.

However, this chapter deliberately adopts a simpler terminology suitable for an introductory conceptual-modelling course. It does not attempt to reproduce the complete process model, terminology, or set of information items in ISO/IEC/IEEE 29148.

Using the guidance in these lecture notes therefore does **not** imply formal conformance with the standard.

Awareness note: *Students are expected to recognise the purpose of ISO/IEC/IEEE 29148 and know that it provides a rigorous reference for requirements engineering. Its detailed clauses and conformance rules are not assessable material in this course.*

Key takeaway: *ISO/IEC/IEEE 29148 provides a lifecycle-oriented reference for rigorous requirements engineering. This chapter adopts selected principles while intentionally using a smaller and more accessible conceptual framework.*

3.31 <heuristic> Goals, Scope, and Fit for Purpose

Analysis and modelling should begin with an explicit purpose. Without one, analysts tend to collect information and add detail without knowing what decisions the resulting work must support.

Several different ideas must be distinguished:

- a **goal in the UoD** describes a desired outcome, such as reducing missed appointments;
- the **purpose of the analysis** states why the analysis is being performed, such as deciding which responsibilities should be assigned to a new appointment service;
- the **project scope** identifies what the project is authorised or expected to change or deliver;
- the **analysis scope** identifies what the analysts will investigate and represent;
- the **system boundary** distinguishes the system of interest from its environment.

These concepts influence one another but are not interchangeable.

3.31.1 Defining the scope

An analysis scope may specify:

- relevant concerns and questions;
- included parts of the UoD;
- stakeholders and viewpoints to be considered;
- organisational, geographical, and temporal limits;
- current and future situations to be examined;
- required levels of abstraction and detail;
- expected work products and decisions;
- exclusions, assumptions, and deferred issues.

Something described as *out of scope* does not cease to exist or become unimportant. It may remain an external dependency, source of risk, or subject for later analysis.

The system boundary should not be chosen merely to match the project scope. A project may change only one software component while the relevant system of interest includes people, processes, rules, information, and other technologies.

3.31.2 Fit for purpose

An analysis or model is **fit for purpose** when it provides adequate and reliable support for its stated use and audience.

Useful questions include:

1. What decision, agreement, or action must this work support?

2. Which concerns and stakeholders are relevant to that purpose?
3. What system boundary makes the relevant responsibilities visible?
4. What evidence and viewpoints are needed?
5. What level of detail is sufficient?
6. What has deliberately been excluded, and with what consequences?
7. How will we know that the work is adequate for its intended use?

Fit for purpose does not mean that any convenient model is acceptable. The result must still use its concepts correctly, remain consistent with available evidence, expose relevant assumptions, avoid internal contradictions, and represent the concerns it claims to address.

3.31.3 Example

Suppose the purpose is to determine how to reduce missed hospital appointments. Restricting the system of interest to the appointment software may hide relevant causes involving telephone access, administrative procedures, patient communication, scheduling rules, and staff responsibilities. The analysis would then be poorly fitted to its purpose.

If the purpose is instead to specify a replacement for one portal while existing organisational responsibilities are fixed, a software-focused system boundary may be appropriate. External processes and services must still be represented where they constrain or interact with that software.

Neither model is universally better. Each must be evaluated against its stated purpose.

3.31.4 Enough detail, not maximum detail

More detail does not necessarily produce a better analysis. Unnecessary detail increases effort, obscures important relationships, introduces accidental commitments, and creates more information to keep consistent.

The appropriate stopping point depends on the intended decisions, uncertainty, risk, novelty, and consequences of error. It should be justified rather than determined by exhaustion or deadline alone.

Key takeaway: State why the analysis exists, distinguish its scope from the system and project boundaries, and judge every work product by whether it rigorously supports its intended purpose.

3.32 <heuristic> When Is It Enough to Analyse?

Analysis can always uncover more detail, alternatives, and uncertainty. Waiting for complete knowledge leads to paralysis; stopping merely because time has expired transfers hidden problems to later work.

The practical question is therefore:

Is the analysis sufficient to support the next decision at an acceptable level of risk?

The answer depends on the purpose of the analysis, the consequences of error, and the reversibility of the decision.

3.32.1 Factors affecting the required depth

More analysis is usually justified when:

- failure may cause serious harm or legal consequences;
- the decision is expensive or difficult to reverse;
- the UoD or proposed technology is unfamiliar;
- responsibilities cross organisational boundaries;
- important sources disagree;
- assumptions are uncertain;
- system behaviour is highly interconnected;
- verification will be difficult or expensive;
- later change would have major consequences.

A reversible experiment or low-cost prototype may proceed with substantial uncertainty. Operational deployment of a safety-critical or contractually binding system requires much stronger evidence.

3.32.2 Indicators of sufficient analysis

For a particular decision, analysis may be considered sufficient when:

- the purpose, scope, and decision authority are explicit;
- the UoD and system boundary are understood well enough for that purpose;
- important responsibilities have been allocated;
- relevant stakeholders, sources, rules, and viewpoints have been considered;
- critical normal, alternative, failure, and misuse situations have been explored;
- key functional, quality, and constraint concerns are covered;
- the requirements set is coherent, feasible, traceable, and verifiable;
- important assumptions and dependencies are explicit;
- proposed verification and validation evidence is credible;
- unresolved issues and residual risks are known and accepted by the appropriate authority.

These conditions do not prove that nothing has been missed. They provide a reasoned basis for proceeding.

3.32.3 Warning signs

Analysis is probably insufficient when:

- the system boundary changes whenever a difficult question appears;
- important terms have incompatible interpretations;
- responsibilities are assigned to entities outside the system's control;
- conflicting requirements remain hidden in vague language;
- major requirements depend on unexamined assumptions;
- relevant stakeholders or operating conditions have been excluded without justification;
- critical qualities are represented only by adjectives;
- no credible verification method can be identified;
- a prototype or existing solution is being mistaken for evidence of the required system;
- the team cannot explain why the current requirements address the original concerns.

3.32.4 Deliberate deferral

Not every question must be answered immediately. A decision may be deferred when resolving it now would provide little value and later evidence will be better.

A deliberate deferral should record:

- the unresolved question;
- why it can be postponed safely;
- what decisions depend on it;
- who is responsible for revisiting it;
- the event or date that triggers reconsideration;
- the consequences if it remains unresolved.

An undocumented omission is not a deferral.

3.32.5 Example

An appointment-service team may have analysed enough to prototype online cancellation while the precise notification technology remains undecided. That choice is reversible and may benefit from experimentation.

The same team is not ready for operational deployment if failure of the external identity service has not been analysed and no responsibility or fallback behaviour has been agreed.

Fixed percentages and slogans cannot determine sufficiency. Timeboxing can limit an investigation, but reaching the end of a timebox does not make the resulting analysis adequate.

Requirements engineering also continues after a decision. New evidence and change may invalidate earlier conclusions.

Key takeaway: Analysis is sufficient when it supports a stated decision with explicit evidence, assumptions, unresolved issues, and an accepted level of residual risk—not when every possible question has been answered.

Systems analysis is not a neutral transcription of reality. Decisions about system boundaries, stakeholders, goals, data, responsibilities, and priorities influence who benefits, who bears costs, and whose behaviour becomes visible or controlled.

Technical rigour therefore requires attention to power and ethics as well as notation and consistency.

3.33.1 Power in analysis

Stakeholders do not have equal authority, resources, confidence, availability, or access to decision-makers. Some people affected by a system may not be represented at all.

Analysts should ask:

- Who requested and funds the analysis?
- Who is authorised to decide?
- Who benefits from the proposed change?
- Who performs additional work or bears additional risk?
- Whose knowledge is treated as credible?
- Who is affected but absent from the discussion?
- Which disagreements or exclusions have not been recorded?

Power is not inherently illegitimate. Projects need decision authority. The problem arises when authority, interests, and exclusions remain hidden behind apparently objective requirements.

3.33.2 Ethical concerns

Relevant ethical concerns may involve safety, privacy, accessibility, autonomy, discrimination, working conditions, accountability, environmental impact, or unequal distribution of benefits and harms.

Legal compliance does not exhaust ethical analysis. A system may satisfy applicable law while still producing avoidable exclusion, manipulation, or unfair burdens.

For example, an appointment service might automatically cancel appointments when patients fail to confirm through a digital channel. This could reduce unused capacity while disproportionately affecting people with limited connectivity, accessibility difficulties, unstable living conditions, or low digital literacy.

The analyst should not convert personal moral preferences directly into requirements. Ethical concerns should be made explicit, supported by evidence and relevant principles, discussed with affected parties, and submitted to legitimate decision processes.

3.33.3 Cognitive and organisational bias

Human judgement is selective and influenced by context. Common risks include:

- **Confirmation:** seeking evidence that supports a preferred explanation or solution;
- **Anchoring:** allowing the first proposal, estimate, or prototype to constrain later thinking;
- **Availability:** overemphasising recent or memorable events;
- **Authority and conformity effects:** accepting a senior or dominant participant's view too readily;
- **Status-quo preference:** treating existing practice as justified merely because it exists;
- **Sunk-cost reasoning:** preserving a weak solution because substantial effort has already been invested.

These labels are prompts for reflection, not clinical diagnoses or universal laws. Simply naming a bias does not prove that it occurred, and accusing individuals of bias rarely improves analysis.

Bias may also be embedded in organisational routines, available data, performance measures, procurement rules, or the composition of the analysis team.

3.33.4 Practical safeguards

Useful safeguards include:

- using multiple sources and elicitation techniques;
- involving people with different roles and experiences;
- separating observations, interpretations, assumptions, and decisions;
- examining alternative system boundaries and explanations;
- actively seeking counterexamples and failure scenarios;
- asking what evidence would disconfirm a preferred proposal;
- recording dissent, exclusions, rationale, and decision authority;
- reviewing actual outcomes after deployment.

No checklist eliminates bias or guarantees an ethical result. These practices make reasoning more visible and open to challenge.

Awareness note: Students are not expected to memorise named biases. They should recognise that system analysis distributes responsibilities, benefits, and burdens, and that analytical judgement must remain open to evidence and challenge.

Key takeaway: Rigorous analysis asks not only whether a proposed system can work, but for whom it works, under whose authority, and with what consequences.

3.34 <further> Tacit and Explicit Knowledge

Explicit knowledge can be expressed and communicated through language, documents, data, procedures, rules, diagrams, or other representations.

Tacit knowledge is embedded in experience, skilled action, perception, habits, and local context. People may act effectively without being able to explain every judgement they make or every cue they use.

The distinction is better understood as a continuum than as a strict division. Knowledge may be partly articulated, meaning that attempts to make it explicit may capture only selected aspects.

3.34.1 Why interviews are insufficient

When asked how work is performed, people often describe:

- the official procedure;
- what they remember doing;
- what they believe they normally do;
- what they think the interviewer expects;
- a simplified account that omits obvious details.

Experienced practitioners may omit information precisely because it has become automatic. They may recognise exceptions, incomplete information, or warning signs without having a formal rule that explains the judgement.

This does not mean that people are dishonest or that interviews are useless. It means that stated accounts should be compared with other forms of evidence.

3.34.2 Discovering tacit knowledge

Useful techniques include:

- observing work in its actual context;
- asking participants to explain decisions while performing tasks;
- discussing unusual incidents and difficult cases;
- comparing official procedures with actual practice;
- reconstructing decisions from records and artefacts;
- using scenarios, examples, prototypes, and process models;
- asking what distinguishes an easy case from a difficult one;
- inviting practitioners to review and correct the analyst's interpretation.

The objective is not to extract everything from someone's mind. It is to make enough relevant knowledge examinable for the purpose of the analysis.

3.34.3 Example

An official appointment procedure may state that requests are handled in order of arrival. Observation reveals that

experienced receptionists sometimes redirect particular requests for faster clinical review based on phrases used by patients, their knowledge of hospital services, and previous incidents.

This practice may reveal:

- an important but undocumented triage need;
- a legitimate professional judgement;
- inconsistent treatment between receptionists;
- an unsafe responsibility assigned to unqualified staff;
- a possible source of discrimination;
- a gap between formal and operational processes.

The analyst should not automatically encode the observed practice as a requirement. Its purpose, authority, risks, variability, and ethical consequences must first be investigated.

3.34.4 Limits of externalisation

Conceptual models, decision tables, examples, and written procedures can make parts of tacit knowledge explicit. They may improve training, consistency, analysis, and system support.

However, formalisation may remove contextual judgement or create a false impression that every relevant situation is covered by a rule. Some expertise remains dependent on experience and interaction with the environment.

Explicit knowledge is not automatically correct either. Documents may be outdated, inconsistent, or written to describe an idealised process. Tacit practice may compensate for those deficiencies, but it may also conceal unsafe or unauthorised workarounds.

Observation and knowledge capture also raise ethical questions. Analysts should avoid turning legitimate investigation into unnecessary surveillance or exposing individuals who compensate informally for organisational failures.

Awareness note: *Students are not expected to adopt a detailed theory of knowledge. They should recognise that relevant knowledge may not be available through documents or direct questioning alone.*

Key takeaway: *People often know aspects of their work that they cannot readily state. Rigorous analysis combines accounts, observation, artefacts, models, and critical validation.*

3.35 <further> Popular Heuristics and Their Limits: Pareto and DIKW

Heuristics are simplified ways of directing attention. They may suggest useful questions, but they do not replace evidence, modelling, or engineering judgement. Two particularly popular heuristics are the **Pareto principle** and the **DIKW pyramid**.

3.35.1 The Pareto principle

The **Pareto Principle** (commonly known as the **80/20 Rule**) states that roughly **80% of consequences come from 20% of causes**.

This is not a universal mathematical law. The numbers need not be 80 and 20, and many situations do not exhibit this type of concentration at all. But in systems analysis, the principle may motivate useful questions:

- Do a small number of failure causes explain most incidents?
- Do a few scenarios account for most operational volume?
- Which requirements contribute most to a stated goal?
- Which stakeholders or interfaces generate most changes?

The distribution must be investigated rather than assumed. Misuse occurs when analysts claim, without evidence, that **20% of the requirements provide 80% of the value** or that analysing only common cases is sufficient. Rare cases may have severe safety, legal, ethical, or operational consequences. A case representing 0.1% of transactions may deserve more attention than the most frequent case.

Measured concentration also depends on how causes and effects are defined. Existing data may reflect previous system limitations, unavailable channels, under-reporting, or excluded populations. Changing the system may change the distribution.

Use Pareto as the question **“Is the impact concentrated?”**, followed by measurement and consequence analysis (not as an automatic 80/20 answer).

3.35.2 The DIKW pyramid

DIKW arranges four concepts in a familiar hierarchy:
Data → Information → Knowledge → Wisdom

A typical interpretation suggests that data acquire context to become information, information is understood to become knowledge, and knowledge supports wise decisions.

The model is intuitive but conceptually weak when treated as a theory of how understanding is produced.

Its limitations include:

- the four terms have no single agreed definitions;
- the transitions between levels are not automatic or precisely explained;
- data are already selected, classified, and recorded for particular purposes;

- information can be incorrect, incomplete, or misleading;
- knowledge includes experience, judgement, and social practice that cannot be obtained merely by processing more data;
- *wisdom* is an ethical and practical judgement, not a predictable output of an information pipeline;
- feedback, disagreement, power, and changing context do not fit a simple upward pyramid.

In requirements engineering, operational data may reveal a pattern, but the pattern does not determine what the system ought to do. Analysts still need to interpret causes, concerns, rules, consequences, and alternative responses.

DIKW can nevertheless prompt useful questions:

- What observations or records support this claim?
- How were they selected and interpreted?
- What contextual information is missing?
- What knowledge is required to act?
- Who decides what constitutes a good decision?
- What values and consequences are involved?

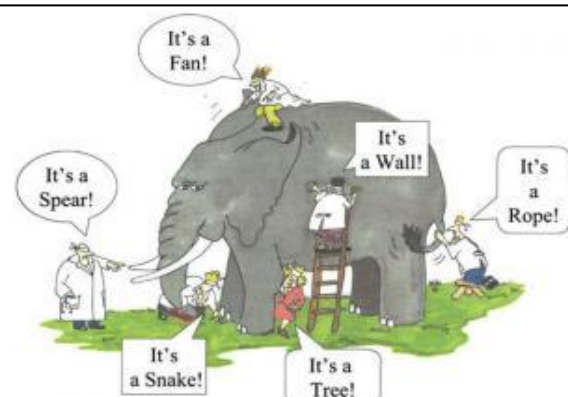
3.35.3 Responsible use

A useful heuristic makes reasoning easier to examine. A harmful heuristic closes discussion by making an unsupported conclusion appear obvious. Neither Pareto nor DIKW determines priorities, proves completeness, or transforms stakeholder statements into requirements. Both should remain subordinate to evidence and explicit reasoning.

Awareness note: Students are not expected to reproduce these heuristics as scientific laws. They should be able to recognise their popular use and question unsupported applications.

Key takeaway: Heuristics are prompts for investigation. When a memorable diagram or numerical rule starts supplying the answer, it has stopped being a heuristic and become a substitute for analysis.

The Indian parable of the Blind Men and the Elephant illustrates how wise individuals can miss the full truth. In the story, each blind man touches a different part of an elephant: one feels the trunk and thinks it's like a snake, another touches the leg and imagines a tree, while another, feeling the ear, describes it as a fan. Each has a valid but incomplete view, missing the whole. This teaches that personal perspectives are limited, and broader understanding is needed to grasp the full picture of complex situations.



3.36 <heuristic> Consolidating the Analysis Heuristics

Rigorous analysis depends less on applying one perfect technique than on asking important questions in a disciplined order. A useful habit is to distinguish four kinds of statement:

- **observed or reported:** what a source says or what was found in the UoD;
- **interpreted:** what the analysts believe the evidence means;
- **decided:** what an authorised party chooses about scope, boundary, priority, or solution;
- **required:** what the system or another identified subject is obliged to satisfy.

Confusing these categories allows assumptions and design preferences to enter the specification without scrutiny.

These questions are valuable for reviewing an analysis

3.36.1 (1) What is the purpose?

What decision, agreement, or action must the analysis support? Which concerns make the analysis necessary, and for whom? If the purpose is unclear, there is no principled way to select the relevant information, models, or detail.

3.36.2 (2) What evidence do we have?

Which stakeholders, documents, observations, rules, data, incidents, and existing systems have been examined? Where do sources disagree, and what remains uncertain? A source explains where information came from; it does not guarantee that the information is correct.

3.36.3 (3) What is the system of interest?

What system emerges from the concerns being addressed? What is inside its boundary, what remains in its environment, and why? Check whether the boundary includes relevant people, processes, rules, information, software, and physical resources rather than defaulting to a software application.

3.36.4 (4) Who is responsible?

For each proposed obligation, identify the entity expected to satisfy it. Is it the system, a system constituent, an external actor, the supplier, or the engineering team? An obligation assigned to the wrong subject is not repaired by better wording.

3.36.5 (5) What kind of statement is this?

Distinguish goals, needs, concerns, rules, assumptions, design proposals, decisions, and requirements. Do not classify every important statement as a requirement.

3.36.6 (6) What kind of requirement is it?

First separate system requirements from project-delivery and engineering-process requirements. Then classify system requirements as:

- functional requirements;
- quality requirements;
- constraints.

If the context is insufficient or the statement contains several obligations, clarify or decompose it before classification.

3.36.7 (7) Is each requirement well formed?

Check whether each requirement is justified, appropriately scoped, unambiguous, sufficiently complete, singular, feasible, and verifiable. Identify vague terms, hidden conditions, unjustified solution choices, and undefined verification evidence.

3.36.8 (8) Does the set work as a whole?

Examine coverage, consistency, dependencies, conflicts, collective feasibility, terminology, and levels of abstraction. Use conceptual models, scenarios, and examples to reveal omissions and incompatible interpretations. A list of good sentences may still be a poor requirements set.

3.36.9 (9) Can we verify and validate it?

For each requirement, identify credible evidence of satisfaction. For the set, ask whether the resulting system responsibilities address the justified concerns and intended context. Passing verification does not establish that the right system has been specified.

3.36.10 (10) What happens when something changes?

Trace requirements to their sources, assumptions, relationships, models, decisions, and verification evidence. Record unresolved issues, residual risks, decision authority, and triggers for reconsideration. Analysis is sufficient for a decision when these uncertainties and risks are explicit and acceptable (not when every possible question has disappeared.)

3.36.11 Use the questions iteratively

This sequence is not a waterfall. Difficulty at a later question often reveals a problem earlier in the reasoning. An apparently unclassifiable requirement may indicate a compound statement, an unclear responsibility, an unstable system boundary, or insufficient evidence.

A checklist cannot replace judgement. Its purpose is to expose where judgement is required and make that judgement open to review.

Key takeaway: Move explicitly from evidence and concerns to boundary, responsibility, requirements, evidence of satisfaction, and controlled change. When a conclusion cannot be justified, return to the question that remains unanswered.

Requirements engineering can be supported by ordinary office applications, collaborative platforms, issue trackers, modelling repositories, or specialised requirements-management environments.

The appropriate tool depends on project scale, risk, contractual obligations, number of participants, expected rate of change, and required evidence. A small project may be managed responsibly with a controlled document and relationship table. A large regulated programme may require baselines, formal approvals, detailed access control, and extensive traceability.

3.37.1 Useful capabilities

A requirements tool may support:

- stable identifiers and structured attributes;
- distinction between requirements, sources, assumptions, decisions, and other items;
- versioning, baselines, status, and change history;
- typed relationships and impact analysis;
- collaborative review, comments, and approval;
- links to conceptual models, architecture, implementation, and tests;
- reusable views for different audiences;
- coverage, consistency, and traceability reports;
- controlled import, export, backup, and long-term access;
- permissions, audit records, and information protection.

No single tool needs to provide every capability. Several tools may be integrated, provided that authority and synchronisation are clear.

3.37.2 Tool categories

Documents and spreadsheets are accessible and flexible. They can work well for small, stable specifications but become fragile when many relationships, versions, and concurrent edits must be managed.

Issue trackers and product backlogs support workflow, prioritisation, and incremental delivery. However, their generic items may mix requirements, user stories, tasks, defects, and design work unless the project defines clear semantics.

Modelling tools and repositories support conceptual consistency, multiple viewpoints, and relationships between model elements. Their models still need explicit normative status and links to textual requirements where appropriate.

Dedicated requirements or lifecycle-management platforms provide structured data, baselines, traceability, review, and reporting. Their benefits may be outweighed by configuration effort and workflow complexity in smaller projects.

3.37.3 Common failure modes

A tool does not determine whether a statement is a goal, assumption, requirement, or design decision. It can enforce a field called *requirement type* while users continue to classify items incorrectly.

Other risks include:

- recording every relationship as an uninformative generic link;
- creating large numbers of links that rapidly become outdated;
- duplicating authoritative requirements across documents and repositories;
- designing the process around tool limitations;
- adding mandatory fields whose information is never used;
- confusing workflow completion with requirements quality;
- losing access to project knowledge through proprietary formats or discontinued services.

Tool selection should consider usability, interoperability, information ownership, exportability, security, administration effort, and long-term maintainability (not only the number of advertised features).

3.37.4 AI assistance

AI-enabled tools may help suggest clearer wording, detect possible duplicates, identify undefined terms, summarise discussions, or propose scenarios and test cases.

Their output remains a proposal. Generated requirements may be plausible but unsupported, inconsistent with the UoD, or based on information that should not have been disclosed. Source verification, confidentiality, decision authority, and human accountability remain necessary.

3.37.5 Minimum discipline

Whatever the tool, each managed requirement should have a stable identity, explicit type, source, status, relevant relationships, and planned verification evidence. The project should also identify where the authoritative version resides.

Awareness note: *Students are not expected to master a specialised requirements-management product. They should understand what tool support can and cannot contribute.*

Key takeaway: *Tools preserve structure, relationships, history, and evidence. They can support disciplined requirements engineering, but they cannot perform the underlying analysis or judgement.*

3.38 <core> Wrap-up Requirements Engineering

Requirements Engineering is the disciplined process of determining, expressing, assessing and managing what must be achieved or respected within a proposed change. It is not merely the collection of stakeholder wishes, nor is it simply the writing of a specification. Requirements are engineered artefacts: they are constructed from evidence, interpretation, negotiation and explicit decisions.

3.38.1.1 From the Universe of Discourse to requirements

Requirements Engineering begins with the Universe of Discourse, not with a blank requirements template. Analysts examine the existing situation, relevant events, work practices, rules, information, systems and technical conditions. They identify stakeholders and seek to understand their purposes, concerns, needs and expectations.

These findings provide evidence and rationale, but they are not yet necessarily requirements. A complaint describes a perceived problem; a need expresses something considered necessary; an objective identifies a desired result; a requirement establishes an obligation upon an identified target. The analyst must therefore reason through a progression such as:

**evidence → concern or need → objective → requirement
→ design and implementation**

This progression is rarely linear. New evidence may challenge an assumed need, a proposed requirement may reveal a conflict, and a design alternative may show that a requirement is infeasible or unnecessarily restrictive.

3.38.2 Keep the essential distinctions clear

Several dimensions must not be confused:

- **Origin or rationale:** who needs something, and why?
- **Obligation target:** what entity must comply with the requirement?
- **Required property:** what capability, behaviour, quality or restriction is imposed?
- **Abstraction level:** how far is the statement from an implementable specification?
- **Priority:** how important or urgent is the requirement?
- **Status:** is it proposed, agreed, rejected, superseded or implemented?

A requirement may apply to an organisation, human role, business process, rule, information structure, software component, device or complete sociotechnical system. Requirements Engineering is therefore broader than software requirements engineering.

Terms such as *business requirement* and *stakeholder requirement* occur in recognised methods and standards, but their meanings depend on the adopted classification scheme. They may suggest an origin, a viewpoint, an abstraction level or an obligation target. They should consequently be used only when their intended meaning is explicitly defined. A label does not remove the need to identify what is actually required and which entity is responsible for satisfying it.

A **functional requirement** specifies something the target must do or provide. A **quality requirement** constrains how well it must do so or specifies another measurable quality. A **constraint** restricts the permissible solution, process or environment. These categories are useful, but they do not replace the analysis of rationale, target and scope.

3.38.3 Requirements must work as a set

A well-written individual requirement should be necessary, clear, feasible, appropriately precise, traceable and verifiable. *Verifiable* does not mean that every requirement must be tested through software execution: verification may also use inspection, analysis, demonstration or other forms of evidence.

Good individual statements do not guarantee a good specification. The complete set must also be sufficiently complete, consistent, non-redundant, prioritised and compatible with the defined system boundary. Interactions between requirements frequently expose problems that are invisible when each statement is inspected separately.

Models, scenarios, prototypes and structured textual forms can reveal omissions, ambiguities and contradictions. They complement natural-language requirements; they do not automatically replace them.

3.38.4 Requirements remain living artefacts

Requirements evolve because understanding improves, stakeholder priorities change, regulations are revised and design work reveals new possibilities or limitations. Change is therefore normal, but it must be controlled.

Traceability connects each requirement backwards to its evidence, stakeholders, concerns and objectives, and forwards to models, design decisions, implementation elements and verification evidence. Versioning, impact analysis and explicit approval make it possible to change requirements without losing accountability or coherence.

Validation and verification provide different checks. Requirements are **validated** against stakeholder purposes, justified needs and the relevant context. Requirement statements and sets are **verified** against the agreed quality rules. Later, the implemented system is verified against its requirements and validated in relation to its intended use.

3.38.5 A compact working discipline

A defensible Requirements Engineering process repeatedly asks:

1. What evidence do we have about the Universe of Discourse?
2. Whose purposes, concerns and needs are involved?
3. What boundary and allocation of responsibility are being assumed?
4. Is each statement a fact, assumption, need, objective, requirement or design decision?
5. What entity is obliged to satisfy each requirement?
6. Is the required behaviour, quality or constraint expressed clearly enough to be verified?
7. Does the complete set remain feasible, consistent and sufficiently complete?
8. Can every requirement be traced to its rationale and to the evidence of its eventual satisfaction?
9. Are changes recorded, assessed and agreed?

Key takeaway: *Requirements Engineering transforms incomplete and sometimes conflicting knowledge about a situation into an explicit, justified and manageable set of obligations. Its central challenge is not producing more statements. It is preserving the reasoning that connects real concerns to clear responsibilities, defensible requirements and verifiable change.*

3.39 <imagine> Digital Fingerprint: Viewpoint and Perception

IMPRESSÃO DIGITAL	Digital Fingerprint
Os meus olhos são uns olhos, E é com esses olhos uns que eu vejo no mundo escolhos onde outros, com outros olhos, não vêem escolhos nenhuns.	<i>My eyes are just my eyes, and with these eyes alone I see in the world shoals where others, with other eyes, see no shoals at all.</i>
Quem diz escolhos diz flores. De tudo o mesmo se diz. Onde uns vêem lutos e dores uns outros descobrem cores do mais formoso matiz. Nas ruas ou nas estradas onde passa tanta gente, uns vêem pedras pisadas, mas outros, gnomos e fadas num halo resplandecente.	<i>Who says shoals says flowers. The same may be said of everything. Where some see mourning and pain, others discover colours of the loveliest hue. In streets or on the roads where so many people pass, some see stones worn down, but others, gnomes and fairies in a shining halo.</i>
Inútil seguir vizinhos, querer ser depois ou ser antes. Cada um é seus caminhos. Onde Sancho vê moinhos D. Quixote vê gigantes.	<i>No use to follow neighbours, to want to be after, or to be before. Each person is their own paths. Where Sancho sees windmills, Don Quixote sees giants.</i>
Vê moinhos? São moinhos. Vê gigantes? São gigantes.	<i>Sees windmills? They are windmills. Sees giants? They are giants.</i>
António Gedeão In <i>Movimento Perpétuo</i> (1956)	(...free translation...)

The world is not perceived in the same way by everyone. Faced with apparently similar things, different observers notice different forms, meanings and possibilities. Where one person sees something ordinary, another may see beauty, danger, sadness or fantasy. The external world does not change each time the observer changes. What changes is the encounter between that world and a particular person: their experience, knowledge, interests, expectations, fears and imagination. Perception is not a neutral camera. Every observer leaves a kind of fingerprint on what they perceive.

3.39.1 Imagine the same organisation

Imagine several people observing the same service:

- a customer notices an unexplained delay;
- an employee notices an awkward but necessary workaround;
- a manager notices a missed performance target;
- a security specialist notices an unauthorised information flow;
- a developer notices a failing interface;
- an auditor notices that no evidence was retained.

They are observing the same organisation, but they are not seeing the same situation. Each observer attends to different phenomena and interprets them through different concerns. Their accounts may complement one another, but they may also conflict. A workaround seen by an employee as essential may be seen by a manager as non-compliance. A control seen by a security specialist as necessary may be experienced by a customer as pointless bureaucracy. No single account provides the complete picture.

3.39.2 From perception to viewpoint

In modelling, a **viewpoint** is a deliberately specified way of constructing a view for particular stakeholders and concerns.

It determines what should be represented, what may be omitted and how the selected phenomena should be expressed. A **view** is the concrete representation produced according to that viewpoint.

Human perception is not literally a modelling viewpoint. The comparison is nevertheless useful. Both involve selection: some things become visible, while others remain outside the field of attention.

The crucial difference is that a modelling viewpoint can be made explicit and examined. We can ask:

- For whom was this view created?
- Which concerns does it address?
- What does it deliberately include or exclude?
- Which concepts and representations does it use?
- What conclusions does it support?
- What might remain invisible?

An explicit viewpoint does not eliminate subjectivity, but it makes part of the selection process inspectable.

3.39.3 The analyst also leaves fingerprints

Requirements elicitation is shaped by the questions asked, the people invited, the vocabulary used, the documents consulted and the situations observed. It is also shaped by the models, scenarios and prototypes selected by the analyst. Consequently, a requirements specification is not a transparent transcription of reality. It inevitably bears the fingerprints of its production process.

This does not imply that every interpretation is equally valid. Interpretations remain constrained by evidence, logical consistency and the Universe of Discourse. Some claims are false, some views omit critical phenomena, and some requirements cannot be justified. The proper response is therefore neither to deny interpretation nor to surrender to relativism. It is to make viewpoints explicit, compare perspectives, seek disconfirming evidence and trace important claims to their sources.

3.39.4 The digital world also selects what can be seen

Digital systems appear precise, but they only record what they were designed to capture. A dashboard shows selected indicators; a database contains selected attributes; a log records selected events; an algorithm processes selected representations. What is visible through a digital system is already the result of earlier choices about relevance. The absence of a phenomenon from the available data does not prove that it does not exist. It may simply mean that nobody decided to observe or record it.

Key takeaway: We never observe a system from nowhere. Every view carries a fingerprint: the concerns, concepts and choices that made some parts of reality visible and left others in the background. Good analysis begins by recognising that fingerprint (and then looking for what it may have concealed).

3.40 <imagine> Domain-Specific Requirements

In RecEng, one of the common challenges faced by analysts is the use of domain-specific language by stakeholders. When stakeholders express themselves using highly specialized terms, it can be difficult for analysts, and even stakeholders from other domains, to fully grasp the requirements, which might sound metaphorical and unclear. To illustrate this challenge, we'll make a little of fun with Haiku¹⁴ and Kenningar¹⁵ (both metaphorical and condensed poetic forms) to represent requirements using obscure or domain-heavy language. This will mimic the struggle that external analysts often experience when dealing with strong domain concepts.

3.40.1 Haiku...

Haiku originated in Japan during the 17th century, evolving from earlier linked-verse forms like *renga*, and became a distinct poetic style known for capturing moments of nature or emotion in just three lines. For example:

An old silent pond,
A frog jumps into the pond -
Splash! Silence again.

This haiku captures a fleeting moment in nature, a hallmark of the form, evoking both sound and stillness in just three lines. A haiku, with its 5-7-5 syllable structure, distills complex ideas into very few words, much like how domain experts might use technical jargon. The ambiguity of the haiku form mirrors the confusion that arises when stakeholders from specialized fields use language that outsiders struggle to interpret. For example:

Green pulse on the wire,
PID loops hold firm control,
Process stays in flow.

Interpretation: This haiku can be expressing a requirement related to industrial process control. The phrase "green pulse on the wire" refers to the signal transmission in automation systems, "PID loops" is a control engineering term for Proportional-Integral-Derivative controllers used in industrial processes, and "process stays in flow" refers to maintaining continuous operations without disruption. Without knowledge of control systems, this would be difficult for an external analyst or a stakeholder from another domain to decode.

How much can I fit
Into a haiku format
Oh no I'm out of



3.40.2 Kenningar...

Kenningar emerged from Old Norse and Icelandic poetry, especially in the Eddas and skaldic verses of the Viking Age, using metaphorical, often cryptic, compound expressions to describe common objects or concepts, deeply tied to Norse mythology and culture. Here's a classic kenning example woven into a few lines of verse:

On the whale-road sails,
Under the wolf's sky of steel,
Gold-bearers ride forth.

Interpretation: This evokes an image of warriors sailing on a stormy sea, a vivid portrayal using metaphorical language typical of kenning (whale-road is the sea; wolf's sky of steel refers to a stormy or dangerous sky, and gold-bearers to warriors or wealthy travellers).

Kenningar, with their cryptic and metaphorical nature, are well-suited to mimic the use of domain-specific terms that are inaccessible to outsiders. When used in requirements, kenningar can represent complex concepts that are challenging for someone unfamiliar with the domain to decipher, just as strong domain language often does. For example:



Serpent of copper
Guides the whispers of the spark,
Thunder greets the forge.

Interpretation: This kenning can be describing a requirement in electrical engineering. "Serpent of copper" refers to copper wiring, "whispers of the spark" suggests electrical signals or currents, and "thunder greets the forge" could symbolize the power surge or voltage applied to heavy industrial machinery. To an analyst unfamiliar with electrical or industrial terminology, this kenning presents a significant interpretation challenge, just as domain-specific terms do.

3.40.3 Why this is relevant...

These relaxing examples of Haiku and Kenningar are designed to simulate the difficulties analysts face when stakeholders use strong domain language. By expressing requirements in these metaphorical styles, we capture the essence of the challenge: trying to understand and extract meaningful information from abstract, specialized, or obscure terms. It's a reminder of the importance of clear communication and the need for analysts to bridge the gap between technical jargon and broader understanding.

¹⁴ <https://www.tofugu.com/japan/haiku/>

¹⁵ <https://norse-mythology.org/what-is-a-kenning/>

4 <cover> Engineering with Conceptual Models

Engineering is increasingly model-centric. Systems, organisations, and infrastructures are now so complex that no artefact can be designed or managed effectively without models that describe not only their structure and behaviour but also their *meaning*. In this context, conceptual models occupy a distinctive position: they serve as **bridges between human understanding and engineered systems**.

4.1.1 From Representation to Knowledge

A conceptual model does not merely document what already exists. It represents *how we understand* a domain: its entities, relationships, and rules (and, equally important, *what remains uncertain or ambiguous*). By making such assumptions explicit, conceptual models transform tacit knowledge into a form that can be discussed, verified, and evolved collaboratively. This ability to *externalise and refine shared understanding* is what distinguishes conceptual modelling from mere documentation or data modelling.

4.1.2 Modelling as a Knowledge-Based Engineering Activity

In systems analysis, a conceptual model is a cognitive artefact that captures knowledge about a system's purpose, environment, and constraints. In systems engineering, it becomes a technical artefact that informs design, verification, and operation.

Engineering with conceptual models therefore means treating modelling itself as an **engineering discipline grounded in knowledge management**, concerned with consistency, traceability, and justification of what is represented. The quality of an engineered system is tightly coupled to the quality of its models. Accuracy and completeness are important, but so are *fidelity* (does the model correspond to reality?), *precision* (does it avoid vagueness?), *recall* (does it capture all relevant aspects?), and *soundness* (are its internal relations coherent?). These criteria define the epistemic rigour of the modelling process.

4.1.3 Conceptual Modelling and Ontology Engineering

Modern engineering increasingly relies on ontologies to ensure semantic alignment. Ontology engineering formalises the vocabulary and logic of a domain, while conceptual modelling provides the methods and notations to express and validate such structures. The two fields converge in their aim to achieve **semantic interoperability** (that is, to ensure that different systems “mean” the same things when they exchange information). Conceptual models, whether expressed informally or through formal ontologies, are therefore not only design artefacts but also **knowledge artefacts**. They enable reasoning, communication, and reuse across disciplinary and organisational boundaries.

4.1.4 Model-Based and Model-Driven Approaches

In model-based engineering (MBE), models are primary artefacts that drive the whole lifecycle of a system, from requirements to verification. Model-driven engineering (MDE) takes this further, enabling the automatic transformation of conceptual models into platform-specific implementations.

In both cases, conceptual modelling defines the *semantic foundation* upon which more specialised models (e.g., structural, behavioural, physical) can be derived.

Such approaches demand a clear separation between **purpose** and **goal**: the purpose of a conceptual model is to inform reasoning for a specific analysis or decision; the goal of the engineered system is what that system must achieve in operation. The ability to distinguish these perspectives is essential for rigorous systems analysis.

4.1.5 Engineering as an Iterative Learning Process

To engineer with conceptual models is to engage in *learning cycles*. Each model expresses a hypothesis about how a system works or should work. Through validation (whether analytical, empirical, or stakeholder-based) these hypotheses are tested, refined, or rejected.

This iterative nature aligns conceptual modelling with scientific inquiry: both seek to reduce uncertainty through systematic representation and reasoning.

4.1.6 Concluding Perspective

Conceptual modelling thus stands at the intersection of knowledge, reasoning, and engineering. It enables collaboration among stakeholders with different expertise, supports traceable and auditable decision-making, and establishes a shared foundation for subsequent technical modelling. In short, **engineering with conceptual models** means designing not only systems but also making **explicit the knowledge** from which the systems emerge.

<further> Consider exploring the “Software Engineering Body of Knowledge (SWEBOK)”, especially when it refers to “requirements”:

- Overview: https://en.wikipedia.org/wiki/Software_Engineering_Body_of_Knowledge
- Book min PDF: <https://ieeecs-media.computer.org/media/education/swebok/swebok-v4.pdf>

In engineering, models are indispensable for representing reality in ways that allow us to understand, predict, and design systems.

Yet not all models serve the same purpose or operate at the same level of abstraction. Two broad categories, **analytical models** and **conceptual models**, capture this distinction and clarify how engineers move from understanding to action.

4.2.1 Analytical Models: Quantifying the World

Analytical models express relationships in a **formal and mathematical** way.

They are used to compute, predict, or optimise measurable aspects of a system, such as energy, flow, stress, temperature, or probability. Typical examples include equations describing heat transfer, control system feedback, or queuing theory. Their purpose is **precision through calculation**: given parameters, the model yields specific numerical outcomes.

Because of this, analytical models are **closed-world abstractions**. They simplify reality into quantifiable variables and relationships, often ignoring unmeasurable or contextual factors. They are crucial for engineering design, performance evaluation, and optimisation, where the goal is to guarantee that a system behaves as intended under defined conditions.

4.2.2 Conceptual Models: Structuring Understanding

Conceptual models, on the other hand, serve to **clarify meaning and structure knowledge**.

They capture entities, relationships, behaviours, and constraints that define the *conceptual architecture* of a system, regardless of implementation details or measurable parameters.

Languages are used to represent such models. Their goal is **fidelity of understanding**, not numerical precision. A conceptual model can reveal inconsistencies, ambiguities, or missing information in the problem space, aspects that cannot be detected by analytical approaches alone.

It is therefore a primary instrument in **systems analysis and requirements engineering**, where the challenge is to make knowledge explicit before quantification or design occur.

4.2.3 Complementarity

Analytical and conceptual models complement rather than replace each other.

Conceptual models establish the *semantic foundation* (the “what” and “why” of the system), while analytical models address the *quantitative behaviour* (the “how much” and “under what conditions”).

In Model-Based Systems Engineering (MBSE), both coexist:

- conceptual models define the architecture and constraints;
- analytical models refine and test the system’s physical and operational performance.

Pede-se a uma criança. Desenhe uma flor! Dá-se-lhe papel e lápis. A criança vai sentar-se no outro canto da sala onde não há mais ninguém.

Passado algum tempo o papel está cheio de linhas. Umas numa direção, outras noutras; umas mais carregadas, outras mais leves; umas mais fáceis, outras mais custosas. A criança quis tanta força em certas linhas que o papel quase não resistiu.

Outras eram tão delicadas que apenas o peso do lápis já era demais.

Depois a criança vem mostrar estas linhas às pessoas: Uma flor !

As pessoas não acham parecidas estas linhas com as de uma flor.

Contudo, a palavra flor andou por dentro da criança, da cabeça para o coração e do coração para a cabeça, à procura das linhas com que se faz uma flor, e a criança pôs no papel algumas dessas linhas, ou todas. Talvez as tivesse posto fora dos seus lugares, mas são aquelas as linhas com que Deus faz uma flor!



Almada Negreiros

4.3 <concept> Why We Model - The Purpose of Modelling

Every model begins with a purpose.

A model, either analytical or conceptual, is never neutral: it is a *simplified and purposeful representation* of some reality, created to serve specific needs of understanding, analysis, communication, or action.

Without a clear purpose, creating a model is an effort that risks becoming an empty exercise (detailed perhaps, but meaningless).

Modelling is always a fundamental cognitive and engineering activity.

When it comes to conceptual models, it allows humans to reason about complexities in systems by making explicit what is known, assuming what is plausible, and exploring what is unknown.

The purpose of modelling defines what is represented, what is omitted, and what level of precision is appropriate.

The purposes of modelling vary across disciplines, but they can be grouped into several complementary categories:

- **Understanding and explanation**
 - Models help us *grasp* what is a system (how it works, what it requires, what it can deliver, ...).
 - They provide simplified views that expose mechanisms, dependencies, and causal relations.
 - For instance, a conceptual model of a business process reveals how activities, resources, and decisions interact to produce outcomes.
- **Design and construction**
 - Engineering models guide the creation of artefacts and systems.
 - They act as *blueprints*, meaning structured descriptions that inform realization and implementation while keeping design decisions traceable and coherent.
- **Communication and coordination**
 - Models serve as *shared languages* between stakeholders.
 - They reduce ambiguity and enable discussion among people with different roles (analysts, engineers, managers, or clients).
 - In this sense, a model is a social artefact as much as a technical one.
- **Decision support and evaluation**
 - Models provide a basis for *comparing alternatives, estimating risks, or testing scenarios*.
 - By making assumptions explicit, they allow reasoning about consequences before real-world action is taken.
- **Documentation and knowledge preservation**
 - A model captures knowledge in a structured form that outlives individual expertise.
 - It records how a system is understood at a certain moment, supporting maintenance, training, and evolution.
- **Prediction and simulation**
 - Models also allow us to *anticipate behaviour* under given conditions.
 - In physics, mathematical models predict motion or heat transfer; in software systems, performance models estimate load or response times; in information systems, structure models allow us to understand dependencies between subsystems and components.
 - The predictive purpose requires quantitative precision or logical inference.

4.4 <further> Why We Model Systems – The Science of Cycology

Source: Lawson, R. (2006). *The Science of Cycology: Failures to understand how everyday objects work. Memory & Cognition*, 34(8), 1667–1675.

4.4.1 The Experiment

When asked to draw a **bicycle from memory**, most people (even regular cyclists and engineers) failed completely. Chains did not connect to pedals, frames were impossible to steer, and many designs could not move at all. Participants were confident until they saw their drawings. Then came surprise and disbelief. Using something daily does not mean knowing how it works.

Lesson: familiarity does not equal understanding.

4.4.2 The Illusion of Understanding

Lawson's study shows a deep human bias: we **believe we understand systems we interact with**, when in fact we hold only vague fragments of their structure and logic. This "illusion of explanatory depth" affects systems analysis just as much as bicycles:

- users *believe* they know their processes;
- analysts *believe* they have captured reality after a few interviews;
- modellers *believe* their diagrams are correct because they "look right."

Insight: The first duty of an analyst is to doubt apparent clarity.

First, please can you rate your knowledge of how bicycles work on a scale from 1 to 7 where 1 means "I know little or nothing about how bicycles work" and 7 means "I have a thorough knowledge of how bicycles work". You do not need to be an expert to give yourself a "7" rating—an expert would be rated as 7+++. Also, you should rate how much you know, not how much you think you know compared to other people. Rating: _____

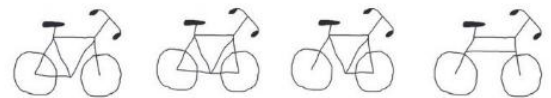


Frame ———
Pedals ———
Chain - - - - -

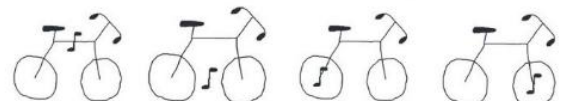
Second, as best you can, please fill in on the above schematic bicycle drawing the main bits of the frame of the bicycle that you think are missing, the pedals and the bicycle chain. Use the symbols given on the right of the drawing to show the frame, pedals and chain.

Figure 1. Response sheet for bicycle drawing task.

3. CIRCLE which one of these four bicycles best shows the usual position of the frame:



4. CIRCLE which one of these four bicycles best shows the usual position of the pedals:



5. CIRCLE which one of these four bicycles best shows the usual position of the chain:



Figure 2. Questions for forced-choice task. Correct responses: frame = first bicycle; pedals = second bicycle; chain = fourth bicycle.

4.4.3 Modelling as an Antidote

Formal models are **instruments of doubt**. They force us to articulate every connection, dependency, and constraint that our intuition hides.

Purpose: Modelling is not about decoration; it is about exposing what we do not know.

4.4.4 Reflection for Systems Engineers

Cognitive psychology teaches that our **mental models** are simplified and often wrong. Drawing (even clumsy sketches) externalises thought, making gaps visible. Thus, *to model is to think better*: diagrams are not reports; they are cognitive tools. Try this exercise before starting a formal model:

1. Draw your system as you think it works.
2. Compare it with reality.
3. Identify what you missed or misunderstood.

The distance between both drawings measures your learning.

4.4.5 The Analyst's Motto

*"The danger is not ignorance, but the illusion of knowledge."
(adapted from Lawson, 2006)*

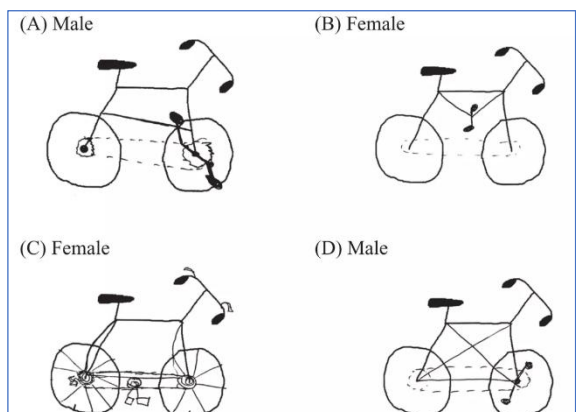


Figure 3. Four drawings from the bicycle drawing task (see Figure 1). Drawings show pedal and chain errors in (A) and (B), frame and chain errors in (C), and the rare triple combination of frame, pedal, and chain errors in (D). The four participants stated that they (A) cycled at least once a month, (B) never cycled, (C) cycled most days, and (D) rarely cycled.

Every time you believe you "already know" a system, imagine drawing its bicycle. That image (elegant or absurd) reminds us **why we model: to reveal, not to confirm**.

4.5 <further> Ontology and Ontology Engineering

At the foundation of conceptual modelling is ontology.

In philosophy, *ontology* is the domain that studies what exists: the categories of entities and the relations among them.

In computer science and artificial intelligence, the term *ontology* designates any artifact which intends to be a **formal, explicit specification of a shared conceptualisation of a domain**¹⁶.

We can state that ontology, first as a domain of philosophy and now as we understand it here as ontology engineering, are on the foundations of conceptual modelling.

4.5.1 Ontology as representation of knowledge

In practical terms, an ontology is a **structured representation of meaning**: it defines what kinds of things exist in a given domain, how they relate, and what rules or constraints govern those relations. It is therefore not a description of data but a representation of *knowledge*.

Ontologies make explicit the conceptual structures that people, organisations, or intelligent agents implicitly use to reason about their world. They are a cornerstone of **symbolic Artificial Intelligence**, where reasoning is performed on explicit knowledge rather than on statistical correlations.

4.5.2 From symbolic AI to knowledge engineering

Classical artificial intelligence aimed to reproduce reasoning itself. Instead of learning from examples, it relied on symbolic descriptions of facts and rules to infer new conclusions. Early **expert systems** such as *MYCIN* and *DENDRAL* used manually constructed knowledge bases (the first large ontologies) to represent the semantics of a domain in an interpretable and reusable form. Ontologies became central to that tradition because they made knowledge both **human-understandable** and **machine-interpretable**, providing a foundation for inference, explanation, and communication.

4.5.3 Example: reasoning with an ontology

Consider an ontology describing a transport and logistics domain, consisting in the following statements:

1. Every **truck** is a kind of **vehicle**.
2. Every **vehicle** has an **owner**.
3. Every **truck** that carries **perishable goods** must have **refrigeration equipment**.
4. Every **fruit** is a **perishable good**.
5. A **strawberry** is a **fruit**.
6. *Truck 14* carries *strawberries*.

From these statements, a reasoning engine can **infer**:

- *Truck 14* carries **perishable goods** (from 4 + 5 + 6).
- Therefore, *Truck 14* **must have refrigeration equipment** (from 3).

This chain of inference shows how an ontology transforms scattered facts into a **closed and computable representation of knowledge**. It defines not only categories and relations, but also the logic by which new knowledge can be derived automatically.

4.5.4 Ontology engineering

The discipline of **ontology engineering** provides methods for constructing and maintaining such knowledge structures.

It combines conceptual analysis, logic, and domain expertise, typically following these stages:

1. **Purpose and scope definition** — clarify why the ontology is needed and what reasoning it should support.
2. **Knowledge acquisition** — gather expertise from standards, documents, and specialists.
3. **Conceptualisation** — identify the essential entities, properties, and relations.
4. **Formalisation** — encode the ontology in a logical structure that computers can process.
5. **Validation** — check logical consistency and confirm semantic adequacy with experts.
6. **Maintenance** — revise the ontology as the domain and its understanding evolve.

Some solutions reuse special artifacts named “*upper ontologies*” (such as **BFO**, **DOLCE**, or **SUMO**, for example) as “root ontologies” which provide generic categories (*object, process, role, quality*) to ensure conceptual coherence across domains.

4.5.5 Conclusion

Ontologies are the **core technique of knowledge representation** in classical AI and remain fundamental today. The purpose is to turn implicit understanding into explicit, computable structures, allowing intelligent systems to reason transparently and share meaning. Through ontology engineering, knowledge becomes organised, explainable, and durable, therefore, the true foundation of meaningful intelligence.

The techniques of conceptual modelling that we will address here have foundations on that...

¹⁶ <https://tomgruber.org/writing/ontologia-kaj-1993/>

4.6 <further> Models as Representation of Knowledge

Languages such as ArchiMate, BPMN, and UML are specialised techniques that emerge from the principles of ontology engineering. These conceptual modelling languages are used to represent knowledge within specific domains, such as enterprise architecture (ArchiMate), business processes (BPMN), or logical system design (UML). While they may not be formal ontologies in the strict sense, they share many characteristics with ontology engineering, particularly in their structured, domain-specific representations of entities and relationships.

In ArchiMate, for example, concepts such as business actors, processes, and technology components are represented alongside their relationships, providing a shared language for architects. Similarly, UML models classes, objects, and interactions, offering a formalised structure of systems and their behaviours. BPMN focuses on the flow of business processes, mapping the activities and decisions that drive an organisation.

4.6.1 Beyond Visual Rendering

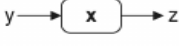
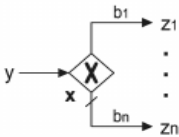
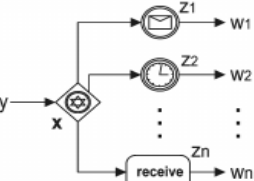
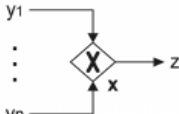
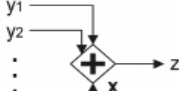
While these languages are often used to generate visual models, their value extends far beyond simple diagram rendering.

The models created in languages like for example ArchiMate, BPMN, and UML are forms of knowledge formalisation, as structured representations of domain-specific knowledge that can be processed by machines.

When encoded in machine-readable formats, such as XML, these models become techniques for automated reasoning and decision-making.

4.6.2 Example: BPMN

For example, a BPMN model can be converted to the Business Process Execution Language (BPEL) and that code expressed in XML can be executed to automate business processes, or simply used to check for process inefficiencies. Similarly, a UML model can be used to validate system designs, ensuring consistency across software development stages. These models allow for some degree of reasoning, verifying constraints or simulating scenarios based on the relationships and structures defined within the models.

BPMN Object		BPEL Event Handler
Task		<pre><onEvent msgReceipt (m_(y,x)) > <sequence> Mapping(x) <invoke m_(x,z) /> </sequence> </onEvent></pre>
Event		
Parallel Fork Gateway		<pre><onEvent msgReceipt (m_(y,x)) > <flow> <invoke m_(x,z1) /> . . <invoke m_(x,zn) /> </flow> </onEvent></pre>
Data-based XOR Decision Gateway		<pre><onEvent msgReceipt (m_(y,x)) > <if> <condition>b1</condition> <invoke m_(x,z1) /> . . <!-- condition:bn --> <invoke m_(x,zn) /> </if> </onEvent></pre>
Event-based XOR Decision Gateway		<pre><onEvent msgReceipt (m_(y,x)) > <pick> <onMessage z1> <invoke m_(z1,w1) /> </onMessage/> <onAlarm z2> <invoke m_(z2,w2) /> </onAlarm/> . . <onMessage zn> <invoke m_(zn,wn) /> </onMessage/> </pick> </onEvent></pre>
XOR Merge Gateway		<pre><onEvent msgReceipt (m_(y1,x)) > <invoke m_(x,z) /> </onEvent> . . <onEvent m_(yn,x) > <invoke m_(x,z) /> </onEvent></pre>
Parallel Join Gateway		<pre><onEvent msgReceipt (m_(y1,x)) > <sequence> <flow> <receive m_(y2,x) /> . <receive m_(yn,x) /> </flow> <invoke m_(x,z) /> </sequence> </onEvent></pre>

Chun Ouyang, Marlon Dumas, Wil M. P. Van Der Aalst, Arthur H. M. Ter Hofstede, and Jan Mendling. 2009. From business process models to process-oriented software systems. ACM Trans. Softw. Eng. Methodol. 19, 1, Article 2 (August 2009), 37 pages. <https://doi.org/10.1145/1555392.1555395>

In this sense, conceptual modelling languages embody the principles of ontology engineering by creating formal representations of knowledge that are valuable not only for communication and visualisation but also for automated reasoning and inference in specific domains.

4.7 <concept>On Types of Systems and Types of Models

Analysts often describe systems as **physical**, **logical**, **socio-technical**, or **conceptual**. These labels are useful, but they must be interpreted carefully. They do not identify categories into which reality is already divided. Instead, they describe diverse ways of conceiving a system of interest according to particular purposes and stakeholder concerns.

The same situation may therefore support several legitimate system conceptions. Each conception selects a boundary, identifies relevant elements, and emphasises particular relationships. It also makes other aspects temporarily less visible.

4.7.1 Physical, Logical, and Socio-technical Conceptions

A **physical conception** focuses on material entities and physically measurable phenomena. Relevant concerns may include geometry, movement, mass, energy, temperature, capacity, or material flow. Suitable models include mechanical representations, electrical circuits, spatial models, and mathematical simulations.

A **logical conception** focuses on information, states, rules, functions, decisions, and control.

Relevant concerns may include **what information is maintained, how states change, which rules apply, or how logical elements interact**. Suitable models include UML class, state machine, sequence, component, and composite structure diagrams, as well as formal specifications.

A **socio-technical conception** focuses on the interaction of people, organisations, procedures, information, and technology. Relevant concerns may include **responsibilities, coordination, authority, communication, regulation, or the effects of technology on human work**. Suitable models include BPMN process models, ArchiMate views, responsibility models, and organisational models.

A **conceptual or theoretical conception** organises selected phenomena to support explanation, prediction, or reasoning. Examples include an economic system, an ecological system, or the solar system as conceived for a particular scientific purpose. Suitable models include conceptual models, ontologies, causal models, mathematical theories, and simulations.

4.7.2 One Situation, Different Conceptions

Consider an automated parking situation. A physical conception may include vehicles, barriers, sensors, parking spaces, movement, and electrical power. A logical conception may include space-occupancy states, access rules, payment records, commands, and software components. A socio-technical conception may include drivers, operators, responsibilities, regulations, and exception-handling procedures.

These are not necessarily three separate systems. They are different conceptions constructed from the same broader Universe of Discourse. Each may be appropriate for a different concern, and a sufficiently complex analysis may require several complementary views.

4.7.3 Conceptions Are Selective, but Not Arbitrary

Dependence on concerns does not mean that every conception is equally valid.

A proposed system conception must be supported by evidence from the Universe of Discourse. Its boundary must be defensible, its concepts must be coherent, and its relationships must be consistent with relevant observations, documents, measurements, rules, and stakeholder knowledge. A model may be useful for one purpose and inadequate for another. For example, a logical model of a parking system may deliberately omit the physical dimensions of the parking spaces if the purpose is analyse payment rules, but not when assessing whether particular vehicles can use those spaces.

4.7.4 Choosing Models

The analyst should therefore select models according to the concern being addressed, rather than according to an preconceived objective type of system. Physical concerns often require quantitative or spatial models; logical concerns require models of information, states, rules, and interactions; socio-technical concerns require models of actors, responsibilities, and processes.

Hybrid and cyber-physical situations usually require multiple complementary models, as no single model must attempt to represent every relevant aspect.

4.7.5 Conclusion

Physical, logical, socio-technical, and conceptual must be understood as classifications of **system conceptions and modelling viewpoints**, not as natural divisions of reality. The conception determines what is treated as part of the system, while the purpose and concerns determine which models are appropriate. These choices are made by analysts, but they remain constrained by evidence from the Universe of Discourse.

Model-Based Systems Engineering (MBSE) and Model-Driven Engineering (MDE) are related concepts that share a focus on the use of conceptual models in engineering processes. They differ in their scope, application, and objectives. Understanding their relationship involves exploring both the commonalities and distinctions between them.

4.8.1 Model-Based Systems Engineering

MBSE is a concept common where the main entity of interest is a system where the main challenges are physical (e.g., a car, an airplane, an industrial plant, etc.). It is a methodology within systems engineering that emphasises the use of models to support the specification, design, analysis, verification, and validation of systems. In MBSE, models are central to the engineering process, serving as the primary artefacts around which all engineering activities revolve. This approach seeks to enhance communication, improve consistency, and reduce errors by providing a single, coherent view of the system that can be used across all stages of development. Key aspects of MBSE include:

- **System-Level Focus:** MBSE is concerned with the entire lifecycle of a system, from requirements capture through to design, implementation, testing, maintenance, and even eventually retirement (think in the challenge decommissioning and dismantling of a nuclear power plant). It addresses the complexity of heterogeneous systems that may involve software, hardware, people, processes, and data.
- **Integration Across Disciplines:** MBSE focus on the integration of different engineering disciplines (e.g., mechanical, electrical, software) by promoting unified modelling frameworks that can represent the various components and interactions within a system.
- **Use of Standards:** MBSE often relies on standardised modelling languages, to ensure that models are consistent, interoperable, and reusable across different projects and teams.

4.8.2 Model-Driven Engineering

MDE is a concept common in logical systems. It refers to a broader software engineering paradigm that focuses on the use of models to automate and streamline the software development process. In MDE, models are not just used for documentation or communication but also serve as the basis for generating executable code, configuration files, and other artefacts. The goal of MDE is to increase productivity, improve software quality, and facilitate the management of complex software systems by raising the level of abstraction in the development process. Key aspects of MDE include:

- **Software-Centric Approach:** MDE primarily focuses on software development, using models to represent the architecture, design, and behaviour of software systems.
- **Automation:** A central tenet of MDE is the automation of software production through model transformations. These transformations convert high-level models into lower-level models, and eventually into code, reducing the manual effort required in software development.
- **Use of Model Transformations:** MDE often involves the use of transformation languages to define how models at one level of abstraction are converted into models at another level, or into executable code.

4.8.3 Relationship Between MBSE and MDE

While MBSE and MDE share the use of models as central artefacts in their respective domains, they are related:

- **Scope and Focus:** MBSE is broader in scope, encompassing the entire system, including both hardware and software components, as well as non-technical aspects such as processes and personnel. MDE, on the other hand, is more focused on software engineering and the automation of the software development process.
- **Application Domain:** MBSE is a concept typically applied to systems engineering projects that require the integration of multiple disciplines, whereas MDE is more often used in software-intensive projects where the automation of software production is a primary concern.
- **Interoperability:** MBSE can incorporate MDE practices when dealing with the software components of a system. For example, within an MBSE framework, the software portion of the system might be developed using MDE techniques, where models are used to generate software code. Thus, MDE can be seen as a subset of the overall MBSE approach when applied to systems that include significant software elements.
- **Methodology vs. Paradigm:** MBSE is more of a methodology tailored to systems engineering, focusing on the application of models across the entire system lifecycle. MDE, however, is a broader paradigm that applies to the use of models in software engineering, emphasising model transformations and automation.

4.8.4 Conclusion

MBSE and MDE are complementary but distinct approaches. MBSE is concerned with the comprehensive modelling of systems across all engineering disciplines, while MDE focuses on using models to drive the software development process. In systems that include both hardware and software, MBSE might incorporate MDE techniques to handle the software aspects efficiently, demonstrating how the two approaches can be integrated to achieve the broader goals of systems engineering.

4.9 <further> Model-Driven Architecture

One other concept in the scope of MDE is Model-Driven Architecture (MDA), a specific approach to that, defined by the Object Management Group (OMG)¹⁷.

The main goal of MDA is to achieve platform independence and model-based code generation, allowing for more flexible adaptation to various platforms. It focuses on using models to describe the architecture in a platform-independent way. MDA emphasizes the transformation of models at different levels of abstraction, assuming those different levels of abstraction can be used to separate concerns (business logic, platform specifics) and facilitate system design and implementation in a structured manner.

It primarily defines three types of models:

- **Computation Independent Model (CIM):** This model describes the system from a high-level, business-oriented perspective without focusing on any technical details. It captures the requirements and overall functionality without any implementation concerns. Examples of languages for this are ArchiMate, BPMN, or UML.
- **Platform Independent Model (PIM):** The PIM represents the system's functionality and behaviour without considering the underlying platform. This model is technology-agnostic and allows the design to be adaptable to various technical platforms. UML is a good example of language for also PIM, as it provides a variety of diagrams to describe both structural (e.g., class, component, and composite structure diagrams) and behavioural (e.g., state machine, sequence diagrams) aspects of the system. Also Object Constraint Language (OCL), a formal language that can be used alongside UML to specify detailed constraints on models. Or Domain-Specific Languages (DSLs), a generic concept for custom languages created for specific domains to model aspects of the system without concern for the underlying technology.
- **Platform Specific Model (PSM):** The PSM translates the PIM into a model that is specific to a particular platform or technological paradigm (e.g., Java, .NET, C#, SQL, specific web frameworks like Angular or React for HTML, CSS, JavaScript, etc.). It defines how the system will be implemented on a particular platform.

4.9.1 Transformation Tools

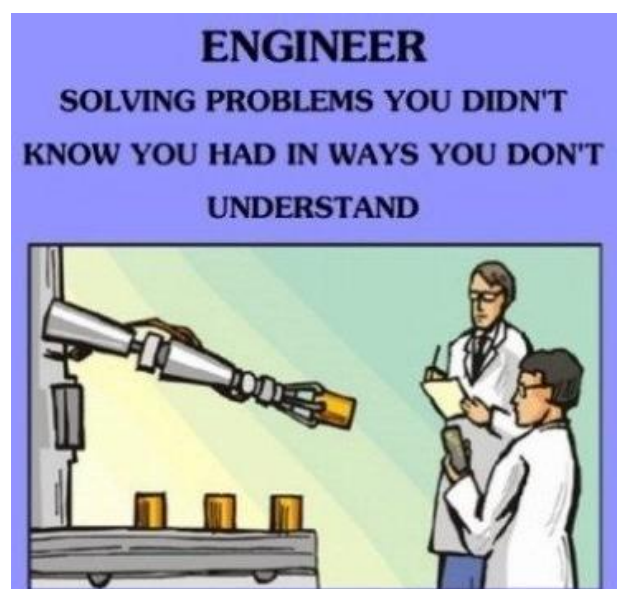
The transformation of a PIM into a PSM or a CIM into a PIM can be done using tools that support model transformation:

- **Eclipse Modelling Framework (EMF):** A modelling framework and code generation facility that helps transform PIM to PSM in Java.
- **Acceleo:** A code generator based on model-to-text (M2T) transformations, widely used for generating Java, Python, or other code from models.
- **ATL (ATLAS Transformation Language):** A model-to-model transformation language used to transform PIMs into PSMs or other models.
- **QVT (Query/View/Transformation):** A standard language for model transformations defined by the OMG, allowing transformations between CIM, PIM, and PSM.

4.9.2 Example Scenario:

- **CIM:** A business analyst models the customer onboarding process using BPMN or UML Use Case Diagrams to capture the required business functions.
- **PIM:** A software architect uses UML Class Diagrams to design the system's core functionality (e.g., customer management, account processing) without specifying the underlying platform.
- **PSM:** A developer transforms the PIM into platform-specific code using Java or C#, and frameworks like Spring or .NET. The UML classes are translated into Java classes, and databases are defined with SQL.

In this workflow, the transformations between the levels of abstraction (from CIM to PIM to PSM) help ensure that business requirements are maintained, and that the system can be adapted to different platforms.



¹⁷ <https://www.omg.org/>

4.10 <example> Enterprise Architect Tool

Enterprise Architect (EA) by Sparx Systems is a comprehensive modelling tool that fully supports Model-Driven Architecture (MDA) and broader Model-Driven Engineering (MDE) workflows.

4.10.1 Computation Independent Model in Enterprise Architect

At the CIM level, EA allows to model business processes, requirements, and goals without technical implementation details. Key features and tools within EA that support CIM include:

- BPMN: EA has built-in support for BPMN diagrams, which we can use to model business workflows and processes. This makes it easy to capture the high-level behaviour and processes of a system from a business perspective.
- Use Case modelling (UML): EA supports UML Use Case Diagrams to capture the system's functional requirements in terms of user interactions with the system. This is useful for defining system requirements at a high level.
- Requirements Management: EA has a dedicated module for managing and tracking requirements. We can link these requirements to models and trace them through to later stages of development.
- ArchiMate: EA also supports ArchiMate, a language used for enterprise architecture modelling, making it suitable for high-level organisational and business models.

4.10.2 Platform Independent Model in Enterprise Architect

EA's support for UML and other modelling languages makes it good for defining platform-independent models. EA provides support for OCL to add constraints and conditions, making the PIMs more precise and ensuring they meet specified requirements. It also includes tools for validating a PIM model against UML and domain-specific rules, ensuring that the model is logically consistent and ready for transformation into a PSM.

4.10.3 Platform Specific Model in Enterprise Architect

At the PSM level, EA allows to take the platform-independent models (PIM) and transform them into platform-specific representations. It supports code generation, reverse engineering, and detailed modelling for specific technologies.

- Code Generation: EA can automatically generate code from models for a variety of languages such as Java, C#, C++, PHP, Python, and more. We can use UML Class Diagrams to define the system's architecture and then generate platform-specific code directly from those models.
- Database modelling and SQL Generation: EA includes tools for Database modelling (ER Diagrams) and can generate SQL scripts for specific database platforms (e.g., Oracle, SQL Server, MySQL).
- XSD and WSDL Generation: If we are working with web services, EA can generate WSDL files from the PIM and supports XSD (XML Schema Definition) generation.
- Framework Support: EA allows us to customize and model specific frameworks like Spring for Java or .NET frameworks, which can be part of our PSM. This includes defining classes, methods, and dependencies specific to those frameworks.

4.10.4 Model Transformations in Enterprise Architect

One of the most powerful features of Enterprise Architect is its support for model transformations, which are key to MDA. These transformations allow us to automatically convert a PIM into a PSM or even a CIM into a PIM.

- Model Transformation: EA has built-in model-to-model transformation capabilities that allow us to transform a PIM into a PSM. For instance, a platform-independent UML class model can be transformed into a Java-specific or C#-specific model.
- Traceability and Model Synchronization: EA supports traceability across all model levels (CIM, PIM, PSM). We can track how business requirements (CIM) map to the architecture (PIM) and how those architecture components map to code and implementation (PSM). Synchronization ensures that changes in one model are reflected in related models.
- MDA Transformations: EA includes predefined MDA transformation templates, allowing automatic transformations between models. These transformations generate specific platform code or configuration artifacts from the higher-level models, automating a significant portion of the development process.

4.10.5 Integration and Tool Support

EA integrates with other development tools and technologies to ensure a seamless MDA process, such as version control systems (e.g., Git, Subversion), or Eclipse and Visual Studio for a smooth transition from model design to code implementation. It also offers an ecosystem of add-ins and extensions that further expand its capabilities for specific platforms or frameworks (e.g., add-ins for generating cloud infrastructure models).

4.10.6 Conclusion

Enterprise Architect by Sparx Systems fits into the MDA approach as a powerful tool that supports the full range of models (CIM, PIM, PSM) and the automated transformation between them. It provides capabilities for modelling, traceability, transformation, code generation, and synchronization, making it an excellent tool for implementing MDA and ensuring that business requirements are faithfully reflected in platform-specific implementations.

4.11 What Therefore is a Modelling Language?

A modelling language is a *system of symbols* used to describe or specify aspects of a system. Like natural languages, it has rules about what can be said and how it can be interpreted. In engineering, those rules are made explicit, enabling communication, automation, and analysis.

4.11.1 Components of a Language

- **Alphabet:** The basic graphical or textual symbols (e.g., rectangles, arrows, keywords).
- **Syntax:** Rules determining *how* those symbols can be combined (e.g., a “Process” may contain “Tasks” connected by “Flows”).
- **Grammar:** A structured definition of syntax, often described formally using notations such as Backus–Naur form (BNF¹⁸) or Extended Backus–Naur form or (EBNF¹⁹).
- **Semantics:** The *meaning* of each construct and of valid combinations (e.g., in BPMN, a “token” passing through a “gateway” defines process flow semantics).
- **Pragmatics:** The intended *use* of the language in practice — who uses it, for what, and how interpretation depends on context.

4.11.2 The Role of Metamodels

Every modelling language has an **underlying metamodel** that defines its abstract syntax and semantics independently of notation. The metamodel:

- Lists the *concepts* the language can express (e.g., Actor, Activity, Event).
- Defines *relationships* between concepts (e.g., “an Activity has Inputs and Outputs”).
- Provides a foundation for *tool support* (e.g., model validation, transformation, code generation).

Metamodels are themselves expressed in **meta-languages**, such as the Meta Object Facility (MOF), which define how to build consistent modelling languages.

4.11.3 On General-Purpose and Domain-Specific Languages

A **General-Purpose Modelling Language (GPML)** has a broad vocabulary usable across many contexts (e.g., UML). A **Domain-Specific Language (DSL)** deliberately narrows its vocabulary to a particular field or layer (e.g., BPMN for business processes, ArchiMate for enterprise architectures, SQL for relational data). DSLs are therefore *specialisations built on metamodels*, sometimes by extending a GPML or by defining an entirely new metamodel.

Benefits of DSLs:

- Clearer communication using domain vocabulary.
- Easier validation and automation, since semantics are tightly constrained.
- Improved alignment between models and domain concepts.

Challenges of DSLs:

- Limited reuse outside the target domain.
- Need for tool support and metamodel maintenance.
- Possible fragmentation if multiple DSLs overlap.

Within the scope of conceptual modelling for engineering, **BPMN** can be considered a DSL for processes, and **ArchiMate** a DSL for architecture, while **UML** can be considered a general purpose language (it has a wide range, covering structure and behaviour; in principle any model made in BPMN or in ArchiMate also could have one or more equivalent models in UML, but quite for sure it’d result less accurate and more challenging as a communication artifact).

These languages share common meta-principles but focus on distinct viewpoints. Understanding their *metamodels* helps you recognise their limits (and design new extensions or micro-languages if needed).

4.11.4 Reflection

Every modelling language is a compromise between **precision, expressiveness, and usability**. The concept of DSLs is a natural evolution of this balance. By learning to see languages as systems defined by syntax, semantics, and metamodels, modellers gain the power not just to use them, but to design or adapt them for new domains.

¹⁸ https://en.wikipedia.org/wiki/Backus%E2%80%93Naur_form

¹⁹ https://en.wikipedia.org/wiki/Extended_Backus%E2%80%93Naur_form

Model-based approaches rely on conceptual modelling. Conceptual modelling involves creating conceptual models as abstract representations of a system, focusing on the key elements and their relationships, without getting bogged down in unnecessary detail. The goal of conceptual modelling is to simplify reality in a way that helps us understand, communicate, and reason about the system we're dealing with.

4.12.1 Models and Metamodels

In the context of conceptual modelling, it's crucial to understand the concepts of models and metamodels, and relationship the between.

A model is a simplified representation of a system, focusing on specific aspects relevant to the problem at hand. For instance, a UML class diagram is a model that depicts concepts of the system as classes, and the relationships between those classes. A metamodel is a model of a model, but more specifically, it defines the rules, structure, and elements that a model must follow within a particular modelling language. Essentially, a metamodel provides the blueprint for how models are constructed in that language. For example, the UML metamodel defines the components (like classes, objects, and relationships) that can appear in a UML diagram and the rules for how they can be used.

4.12.2 The Role of Meta-Object Facility

The Meta-Object Facility (MOF) is a standard developed by the OMG that provides a framework for defining metamodels. MOF essentially standardises how metamodels are created and ensures that they are consistent across different modelling languages. In practical terms, MOF operates at four layers of abstraction:

1. M0 (Data): Represents real-world objects and their interactions.
2. M1 (Model): The model represents these real-world objects, such as a UML diagram that describes the structure of a software system.
3. M2 (Metamodel): The metamodel defines the structure and rules of the modelling language itself. For example, the UML metamodel specifies what elements (like classes, associations, and methods) are available in UML and how they can be used to create a UML model.
4. M3 (Meta-metamodel): The highest level, where MOF itself resides. It defines the language and framework for creating metamodels like UML's.

In this hierarchy, the metamodel (M2) defines the grammar of a modelling language, ensuring that models (M1) created in that language are valid and meaningful. MOF, at the M3 level, provides the foundation for defining these metamodels.

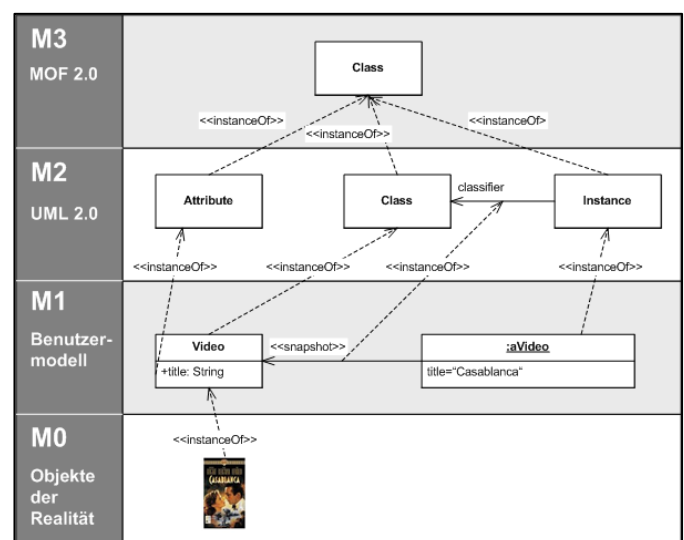
4.12.3 Conceptual Modelling with Visual Languages

Visual languages play a crucial role in conceptual modelling because they provide a standardised way to represent these abstract ideas. By using shapes, symbols, and connectors, we can create diagrams that convey the structure and behaviour of systems in a clear and concise manner. This is particularly useful when dealing with complex systems that involve many interacting components.

Languages like ArchiMate, BPMN (Business Process Model and Notation), and UML (Unified Modelling Language) offer a variety of diagram types to model distinct aspects of a system, such as its structure, processes, and interactions. These visual languages help bridge the gap between the real world and the abstract world of models, making it easier to align the system's design with its intended requirements.

4.12.4 Conclusion

Conceptual modelling, especially when done with visual languages, is essential in both software and systems engineering. It allows us to create simplified representations of complex systems, making them easier to understand, communicate, and design. The relationship between models and metamodels, structured by frameworks like MOF, ensures that models are constructed consistently and meaningfully within their respective languages. For students in computer science, grasping these concepts is crucial for effectively analysing and designing systems that meet real-world requirements.



The four-level meta-object facility (MOF) architecture for the full life cycle natural disaster event

<https://en.wikipedia.org/wiki/Metamodelling#/media/File:M0-m3.png>

4.13 <technique> Modelling and Requirements Engineering

Traditionally, requirements have been expressed in textual form. While straightforward and accessible to all stakeholders, textual requirements can be costly to maintain, especially in complex or long-term projects. As the system evolves, keeping a large set of textual requirements up to date can become cumbersome and prone to errors. Ambiguities or inconsistencies in the text can lead to misunderstandings, which might only surface later in the development process, often at a high cost in time and resources.

Moreover, in dynamic environments where agile methods are in use, the overhead of maintaining a detailed, textual requirements document can slow down the process. Agile methods thrive on flexibility and iterative development, where requirements often change rapidly. In such scenarios, traditional textual documentation might not keep pace with the need for quick adjustments and real-time feedback, which might be better supported with model-based approaches.

4.13.1 Languages for Model-Based Approaches

In agile environments, where requirements change rapidly, visual models allow for quick updates and real-time collaboration, keeping everyone aligned without the overhead of constantly rewriting documents. For example, during an agile sprint, a team might use UML diagrams to model the most recent requirements, allowing them to quickly see how changes impact the system. These models can be easily adjusted as the project evolves, making it easier to respond to new insights or changes in stakeholder needs.

To create effective conceptual models, we use standardized modelling languages. These languages provide a set of symbols, rules, and semantics that ensure models are consistent, understandable, and can be used across various tools and platforms.

Main languages for this purpose are, nowadays:

- ArchiMate: Used for modelling enterprise architectures, ArchiMate allows to express high-level requirements within the context of business processes, applications, and technology, and to see how those align with broader organisational goals, facilitating communication and alignment across teams. Is a key technique in aligning IT systems with business goals.
- BPMN (Business Process Model and Notation): BPMN provides a graphical notation that is easily understandable for all business stakeholders, which helps in documenting and improving business processes. By modelling workflows visually, it becomes easier to understand and adjust functional requirements, which is particularly valuable in agile environments where processes are refined iteratively.
- UML (Unified Modelling Language): UML is a general-purpose modelling language for software engineering. It provides a rich set of diagrams to model distinct aspects of a system, such as structure, behaviour, and interactions. UML is versatile, making it applicable not only to software but also to other types of systems.

4.13.2 Conclusion

While textual requirements have their place, especially for legal or regulatory documentation, they can be costly and inflexible in dynamic projects. Model-based approaches using languages like ArchiMate, BPMN, and UML offer a more agile, visual way to manage requirements, reducing the burden of maintenance and improving clarity and alignment across teams. Understanding when and how to use these techniques is essential.

However, it is important to recall that a conceptual model is not an end in itself but a reasoning tool: Its value lies in how well it supports understanding, decision-making, and validation in line with a given purpose.

4.14 <concept> Requirements Engineering with Models

Requirements Engineering is the discipline of discovering, analysing, documenting, and validating what a system should do and under what conditions it should operate. Modelling plays a central role in this process: it provides a *shared visual language* for reasoning about systems that do not yet exist. Through models, analysts capture knowledge, expose inconsistencies, and align the understanding of diverse stakeholders.

Natural language is flexible but ambiguous. Models reduce ambiguity by structuring information visually or formally. A good model helps identify *what is known*, *what is assumed*, and *what is still uncertain*. Different modelling techniques correspond to different moments in the requirements process, from creative exploration to formal specification.

4.14.1 Early exploration: mind maps and concept maps

Before formal modelling begins, analysts need to **make sense of the problem space**. At this stage, the focus is not precision but *understanding* and *shared meaning*. Lightweight visual techniques can be particularly useful for this purpose, as for example:

- **Mind Maps** – diagrams that radiate from a central idea, capturing thoughts, associations, and categories. They are effective for brainstorming sessions, interviews, and workshops. Mind maps help externalise tacit knowledge, reveal implicit structure, and create a collective picture of the problem.
- **Concept Maps** – more structured diagrams showing concepts connected by labelled relationships. While a mind map captures ideas, a concept map expresses *meaning*: it answers to “what is related to what, and how?”. Concept maps are often used as *precursors* to formal models, bridging human cognition and engineering structure.

Both techniques promote creativity, participation, and alignment among stakeholders. They do not replace formal modelling; rather, they prepare the ground for it by turning informal discourse into a first, explicit *representation of knowledge*.

4.14.2 Structuring the problem: architectural and process viewpoints

Once ideas have been explored, formal modelling begins. Different languages address different aspects of a system:

- **ArchiMate** – an *enterprise architecture language* for representing organisations at multiple layers: business, application, and technology. It supports traceability from strategic goals to system components and is especially valuable for large-scale contexts.
- **BPMN (Business Process Model and Notation)** – focuses on *process logic*: activities, events, decisions, and flows. BPMN diagrams clarify how work is done and how responsibilities are distributed, bridging business and system perspectives.

These languages help analysts move from *what is required* to *how it can be realised*, preserving semantic continuity between requirements and design.

4.14.3 Detailing structure and behaviour: system-level models

As the scope narrows to software or embedded systems, modelling becomes more technical. For that, **UML (Unified Modelling Language)** provides multiple diagram types (use cases, class diagrams, sequences, states, etc.) for describing system structure and behaviour. It is the de facto standard for object-oriented analysis and design.

4.14.4 Conclusion

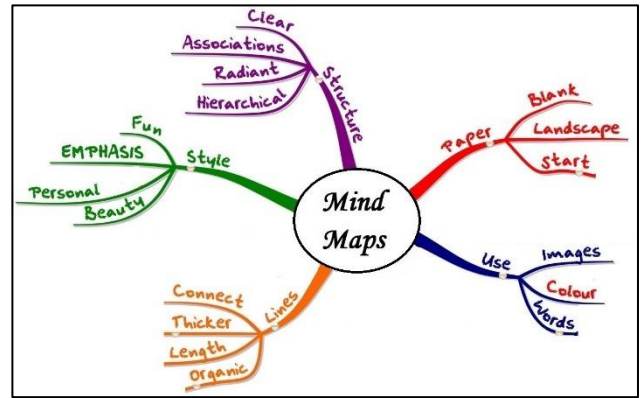
Requirements Engineering is, at its heart, a *modelling activity*.

By using the right modelling technique at the right time, analysts ensure that requirements are not only collected but understood, communicated, and ready to guide design.

4.15 <technique> Informal – Mind Maps

A mind map is a visual tool used to capture, organise, and structure ideas around a central concept. It was popularised by British author and educator Tony Buzan in the 1970s as a technique for enhancing learning, memory, and creativity. Mind maps are widely used in both personal and professional settings for brainstorming, planning, and problem-solving.

At the heart of a mind map is a central concept or theme, from which related ideas radiate outward in the form of branches. Each branch represents a sub-topic or related idea, and further sub-branches extend to capture additional levels of detail. This radial, tree-like structure makes mind maps intuitive and visually appealing, as they closely mimic how the brain organises information. Key concepts are often written as keywords or short phrases, and colours, images, or symbols can be added to make the map more engaging and memorable.



4.15.1 Structure and Flexibility

Mind maps are typically structured around a single central node, which represents the main idea or subject. From there, branches radiate outward, with each major branch representing a key sub-topic. For instance, in a mind map about "Digital Transformation," branches might include "Technology," "People," "Processes," and "Culture." These major topics can be further broken down into more specific sub-branches, creating a hierarchy of ideas.

While mind maps follow a tree-like structure, they remain highly flexible and can be adapted to different needs. Branches can be rearranged, merged, or extended as new ideas emerge. This makes mind maps a particularly useful tool for brainstorming, as they allow for non-linear thinking and the free flow of ideas.

4.15.2 Benefits of Mind Mapping

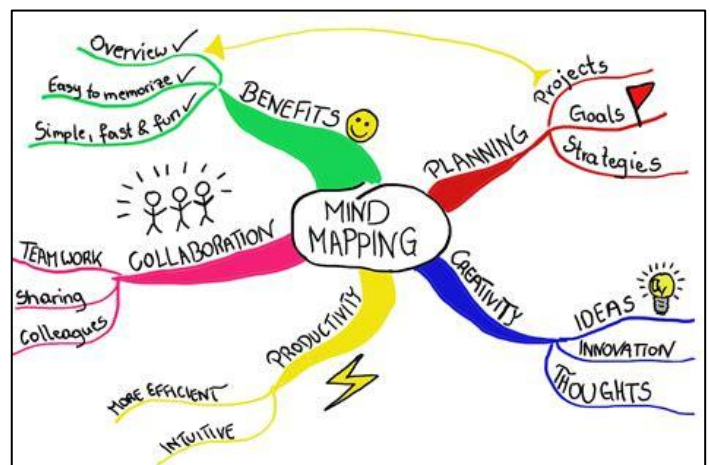
Mind maps offer several advantages, particularly for tasks that require creativity, planning, or problem-solving:

- **Visual Organisation:** Mind maps provide a simple, visual way to structure and organise complex information. By arranging ideas visually, they make it easier to see connections, identify gaps, and organise thoughts coherently.
- **Encouraging Creativity:** The radial structure of mind maps encourages non-linear thinking, allowing expressing associations between ideas that may not be obvious. This can lead to more creative and innovative solutions.
- **Memory and Recall:** Research suggests that using colours, images, and keywords in mind maps can help improve memory and recall, making them an effective tool for studying or presenting information.
- **Clarity and Focus:** By keeping the central concept at the heart of the map, mind maps help maintain focus on the main topic, while still allowing for exploration of sub-topics. This can be particularly useful in planning or decision-making, as it ensures that all relevant factors are considered.
- **Collaborative Tool:** Mind maps are excellent for brainstorming sessions, as they provide a framework that encourages the flow of ideas. They help teams visualise connections and organise thoughts in a clear, collaborative manner.

4.15.3 How to Create a Mind Map

Creating a mind map is a straightforward process:

1. **Start with a Central Concept:** Begin by placing the main idea or topic in the centre of the page.
2. **Add Main Branches:** Identify key themes or sub-topics related to the central concept and create branches radiating outward. These are the main categories or areas of focus.
3. **Add Sub-Branches:** For each main branch, add sub-branches that represent more detailed ideas or specific aspects of the sub-topic.
4. **Use Keywords and Images:** Keep each branch label concise, using keywords or short phrases. Add images, symbols, or colours to make the mind map more engaging and memorable.
5. **Review and Revise:** Mind maps are flexible tools, so we can continue to expand and modify them as new ideas emerge.



4.15.4 Applications of Mind Maps

Mind maps have a wide range of applications, from personal goal setting to project planning and team brainstorming. They can help students organise study material, brainstorm essay topics, or summarise complex concepts, or help in for strategic planning, problem-solving, and process improvement.

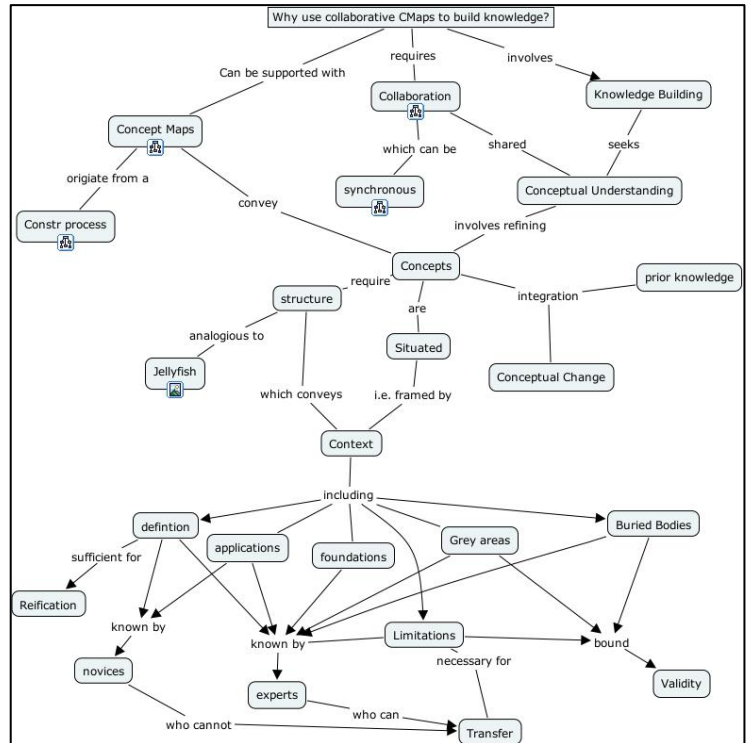
In summary, mind maps are a simple yet powerful tool for organising and structuring ideas. They encourage creativity, enhance memory, and provide a clear framework for exploring complex topics, making them invaluable in both personal and professional contexts.

4.16 <technique> Informal – Concept Maps

Building on the associative logic of *Mind maps*, Concept maps (often called *Cmaps*) introduce explicit labelled relationships ('propositions') between concepts. They consist of nodes (concepts) connected by labelled links (relationships). Cmaps have no strict syntax (unlike formal modelling languages): they are free-form, adaptable, and designed to help people externalise what they know or think.

In the early stages of a project, stakeholders often express their views in unstructured language, mixing goals, expectations, processes, and even opinions. Like *Mind maps*, Cmaps can help transform these expressions into a **shared visual artefact** that can be discussed, reorganised, and refined. They are therefore especially useful for:

- **Brainstorming:** capturing ideas quickly without worrying about formalism.
- **Elicitation:** surfacing hidden assumptions or overlooked concerns.
- **Negotiation:** providing a neutral diagram that stakeholders can comment on.



4.16.1 Concept Maps vs. Conceptual Models

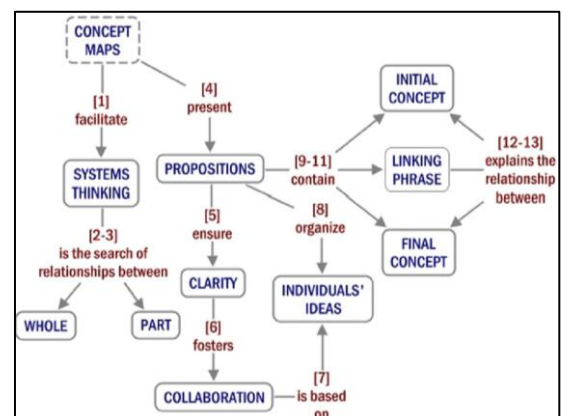
Cmaps are not a replacement for formal models. Instead, they prepare the ground for them.

It is essential to distinguish between **informal maps** and **formal models**:

Concept Maps	Conceptual Models (ArchiMate, BPMN, UML)
Informal, subjective	Formal, objective
Free structure	Defined syntax and semantics
Good for eliciting and sense-making	Good for specification and analysis
Brainstorming tool	Engineering tool

4.16.2 Role in the ReqEng Process

1. **Elicitation:** During interviews or workshops, a Concept Map can capture stakeholder input in real time (see "CONCEPT MAPS" in the figure²⁰)
2. **Sense-Making:** Teams can refine the map together, clustering related concepts and identifying contradictions.
3. **Transition:** Selected parts of the map are translated into formal models:
 - Stakeholder goals → e.g., ArchiMate Motivation View.
 - Business activities → e.g., BPMN processes.
 - Entities and roles → e.g., UML classes or actors.



Example: Suppose stakeholders of a hospital system mention "patient admission," "medical records," "billing," "doctors," and "emergency." A Concept Map can quickly show these items and their rough relations. Later, the analyst refines this into a BPMN collaboration diagram (admission process) and a UML class diagram (medical record structure).

4.16.3 Good Practices

- Keep Cmaps simple: no need for complete detail.
- Use them collaboratively (e.g., project a live map during workshops).
- Treat them as **temporary artefacts**: the goal is to evolve towards formal models.
- Record decisions: note which concepts are "moved forward" into, for example, ArchiMate, BPMN, or UML.

4.16.4 Key Takeaway

Concept Maps are **supporting tools for thinking**. They help stakeholders and analysts organise ideas at the start of requirements engineering, but they are **not final deliverables**. Their value lies in bridging the gap between informal discussion and formal conceptual modelling.

²⁰ https://www.researchgate.net/figure/Cmap-to-answer-the-following-focal-question-Why-can-Cmaps-improve-collaboration-among_fig1_221778155

4.17 <technique> UML, BPMN, and ArchiMate

ArchiMate, BPMN, and UML are standardised modelling languages developed to address different but complementary concerns. ArchiMate focuses on enterprise architecture; BPMN on business processes; and UML primarily on software-intensive systems.

4.17.1 Origins and Governance

UML emerged during the 1990s from the unification of several object-oriented modelling methods, particularly the work of Grady Booch, James Rumbaugh, and Ivar Jacobson. It was adopted by the Object Management Group in 1997 and has since evolved into a general-purpose modelling language, used especially in software and systems development.

BPMN was initially developed by the Business Process Management Initiative (BPMI) and was later transferred to the Object Management Group. BPMN 1.0 was released in 2004. BPMN 2.0 introduced a more precise metamodel and execution semantics for executable process models.

ArchiMate was initially developed in the Netherlands by a collaboration of government, industry and academia entities, lead by the Telematica Instituut, to provide a language for enterprise architecture modelling. It was adopted by The Open Group in 2008 and has evolved through several versions.

4.17.2 Concepts, Language Versions, and Tools

This introductory course follows a **concepts-first approach**. Students should first understand what must be expressed in a model (for example, structure, behaviour, services, processes, interactions, requirements, constraints, or system composition) before selecting version-specific elements and notation.

This does not mean that different language versions are semantically interchangeable. Concepts, relationships, and modelling rules may change between versions. Version differences are therefore discussed when they affect the meaning of a model, the available notation, or what can be represented in the tools used by students. Students are not expected to memorise version histories or detailed syntax comparisons that do not affect their modelling decisions.

The reference versions used in these notes are:

- ArchiMate 4;
- BPMN 2.0.2;
- UML 2.5.1;

BPMN 2.0.2 and UML 2.5.1 are taught directly using their respective concepts and notation.

ArchiMate 4 provides the reference language for enterprise architecture. However, the tools used in practical work may support only ArchiMate 3.2. When a relevant difference exists, these notes explain how the intended ArchiMate 4 meaning can be represented using an explicit ArchiMate 3.2 compatibility convention.

A model stored in an ArchiMate 3.2 repository must not be presented as a native ArchiMate 4 model. The intended meaning, the element actually stored by the tool, and any known semantic limitation should be documented.

4.18 <technique> Views and Viewpoints

Each language relies on a defined **metamodel** that constrains its semantics (e.g., *Activity* in BPMN, *Class* in UML, *Actor* in UML versus Business *Actor* in ArchiMate, etc.), ensuring consistency and traceability across views and viewpoints.

4.18.1 Views and Modelling Languages

A **view** is a representation of a system from a particular perspective, focusing on a coherent subset of its concerns. Each view presents part of the system's structure or behaviour, and together these views provide a comprehensive architectural description. Rather than being arranged as hierarchical *levels*, the system can be understood through complementary **views**, each emphasising a distinct domain of analysis.

In enterprise architecture the scope of the problem is usually a system that comprises these major views:

- **System view** – captures the Entity of Interest (Eol), its external boundaries, interactions, and goals
- **Process view** – focuses on internal workflows and transformations, describing how the system operates
- **Application view** – details logical or software components that implement or support those processes
- **Device view** – represents the physical infrastructure and technological components that enable applications

4.18.2 Viewpoints and Modelling Languages

A **viewpoint** defines the conventions used to construct a view: the language, notation, and rules that specify what may be modelled and how.

Each viewpoint is linked to specific stakeholder concerns and analytical purposes.

Examples include:

- A **business-process viewpoint**, typically expressed in **BPMN**, focuses on flows of activities and organisational roles;
- An **application-architecture viewpoint**, often using **ArchiMate** or **UML component diagrams**, describes how logical components and services interact;
- A **technology viewpoint**, supported by **ArchiMate**, represents hardware, networks, and deployment contexts;

4.18.3 Integration Across Views

Although created from distinct viewpoints, views are not independent.

Consistency between views ensures that, for example, a process model in BPMN is aligned with a context represented in ArchiMate, which is supported by corresponding logical application components in UML.

Integration across views is achieved when they are grounded in a coherent **metamodel framework**, enabling transformations and traceability between conceptual, logical, and physical representations.

This principle underlies **Model-Based Systems Engineering (MBSE)** and **Model-Driven Engineering (MDE)**, where models from different viewpoints collectively define the system architecture.

4.18.4 Why This Distinction Matters

*Replacing the usual notion of “levels” with **views and viewpoints** reflects that systems engineering is not a hierarchy of detail but a **multi-perspective representation**.*

Each modelling language provides tools to construct a view consistent with its viewpoint, addressing stakeholder concerns from business to technology. This approach reinforces key principles:

- **Clarity of purpose** – every model expresses a specific concern through a defined viewpoint;
- **Traceability** – views can be related across languages through model transformation or alignment;
- **Coherence** – the ensemble of views forms an integrated architectural description of the system.

4.18.5 In Short

In short:

- Models of **systems** show *why* things must exist.
- In highly complicated systems, such as **enterprise architectures**, it is common to have models of processes, applications and devices:
 - Models of **processes** show *how* work is coordinated.
 - Models of **applications** show *what* functions are needed.
 - Models of **devices** show *with what* physical means the functions are realised.

These perspectives together can form a coherent framework for understanding and communicating complicated systems through multiple, formally defined viewpoints.

4.19 <concept> Views and Viewpoints versus Stakeholders and Concerns

An **architecture description** is useful only when it helps someone understand or decide something. For that reason, architectural modelling must begin with **stakeholders** and their **concerns**, not with the available symbols in the modelling language.

*A **stakeholder** is an individual, group, or organisation with an interest in the enterprise or in a proposed change. Stakeholders may include customers, managers, employees, regulators, suppliers, architects, developers, operators, and security specialists.*

Different stakeholders often see the same enterprise in very different ways because they are responsible for different outcomes and exposed to different risks.

*A **concern** is an issue that matters to one or more stakeholders.*

Examples of concerns include cost, service quality, legal compliance, security, usability, resilience, maintainability, operational responsibility, or time to market. Concerns guide what must be represented and what may be omitted from an architecture description.

4.19.1 From Concerns to Views

*A **view** is a representation of selected parts of an architecture, created to address particular concerns. It does not represent the entire enterprise. Instead, it focuses attention on the elements and relationships needed for a specific purpose.*

For example, a regulator may need a view showing responsibilities, controls, information flows, and compliance obligations. An operations manager may need a view of processes, roles, applications, and dependencies. A technology architect may require a view showing applications, infrastructure, deployment, and communication networks.

These views may refer to the same underlying architectural elements. A business process shown in an operational view may also appear in a change-impact view or in a view explaining how a service is realised. Reusing the same element preserves consistency and avoids creating several disconnected descriptions of what is intended to be the same thing.

4.19.2 Viewpoints as Modelling Conventions

*A **viewpoint** defines the conventions used to construct a particular kind of view. It may specify:*

- *the concerns to be addressed;*
- *the intended stakeholders;*
- *the concepts and relationships that may be included;*

- *the recommended level of detail;*
- *the form in which the result should be presented.*

A **viewpoint** is therefore comparable to a reusable modelling template, while a view is a concrete representation produced by applying that template to a particular enterprise or problem.

For example, a Context and Services Viewpoint may specify that a view should show the entity under analysis, external actors or systems, and the services exchanged with its environment. A Service Realisation Viewpoint may instead require the service, the behaviour that realises it, the active structures that perform that behaviour, and the information or applications involved.

The form in which a viewpoint should be presented can comprise textual descriptions, models in the form of diagrams, or others (mock-ups, simulators, etc.).

Here, we focus on textual descriptions and diagrams!

4.19.3 Models Are Not the Same as Diagrams

An ArchiMate model is a structured body of architectural knowledge composed of reusable elements and relationships. A diagram is a visual arrangement of selected model content. A view is the meaningful selection and presentation of model content for a declared purpose.

These concepts are related but not interchangeable. A diagram may be visually correct yet fail as a view if its purpose, audience, and concerns are unclear. Conversely, the same model may support several valid views without duplicating its underlying elements.

4.19.4 Practical Modelling Sequence

There are methods for modelling, some generic, some for application in specific contexts

In a simplified approach, a disciplined modelling activity should follow this generic sequence:

1. identify the stakeholders;
2. identify their concerns;
3. define the purpose of the view;
4. select an appropriate viewpoint;
5. include only the relevant elements and relationships;
6. verify that the resulting view answers the intended questions.

This sequence helps prevent a common error: producing a large diagram first and only afterwards trying to explain what it is for.

The value of a view lies not in the number of elements it contains, but in how effectively it addresses a well-defined concern for a specific stakeholder.

ArchiMate is a modelling language specifically designed for enterprise architecture, providing a standardised way to describe, analyse, and visualise the relationships among business domains in an organisation. ArchiMate is used to create comprehensive high-level models that capture concerns about the structure, processes, information flows, and technology of an organisation.

4.20.1 Core Concepts of ArchiMate

ArchiMate is structured to reflect the different layers of an enterprise, allowing architects to model everything from high-level business processes to technical infrastructure. The language is designed to be consistent and comprehensive, making it easier to align business goals with IT strategies and systems.

ArchiMate is organised into three core layers:

- **Business Layer:** This layer focuses on the organisation's business processes, functions, roles, and services. It models the high-level activities that deliver value to customers, such as sales processes or customer support operations. For example, in a retail enterprise, the business layer might include processes like order processing, inventory management, and customer relationship management.
- **Application Layer:** The application layer models the software applications that support the business processes. It includes the applications themselves, the services they provide, and how they interact with each other. For instance, in the same retail enterprise, this layer might model the integration between an e-commerce platform, a payment gateway, and an inventory management system.
- **Technology Layer:** This layer captures the physical and infrastructure aspects of the organisation, such as servers, networks, and databases. It models how these technological components support the applications and, ultimately, the business processes. In our retail example, the technology layer might include a cloud infrastructure that hosts the e-commerce platform, the network configurations, and the security protocols in place.

In addition to these core layers, ArchiMate also supports cross-cutting aspects like motivation (capturing stakeholder goals and drivers), implementation and migration (showing how changes are realised over time), and strategy (representing the logic of strategy realization,).

4.20.2 ArchiMate and Requirements Engineering

In the context of conceptual modelling and requirements engineering, ArchiMate provides a powerful technique for bridging the gap between business needs and IT solutions. Its layered approach allows stakeholders to see how high-level business goals translate into specific technological implementations, ensuring that all aspects of the system are aligned with organisational objectives.

At the conceptual level, ArchiMate helps in defining and visualising the key components of the enterprise architecture, such as business functions and their relationships with supporting applications and technologies. For example, a conceptual model might show how a new business service will interact with existing IT systems, highlighting potential areas of integration or conflict.

ArchiMate can thus play a crucial role in requirements engineering by providing a clear framework to capture and analyse requirements across different layers of the enterprise. This ensures that the requirements for new systems or changes are fully aligned with business goals. For instance, if a business requires faster processing of customer orders, the ArchiMate model can help identify which applications and technologies need to be enhanced or integrated to meet this business goal.

4.20.3 Conclusion

ArchiMate's value lies in its ability to provide a holistic view of an enterprise, making it easier to manage complex systems and ensure that all components (from business processes to IT infrastructure) work together effectively. This is particularly important in large-scale systems engineering, where multiple stakeholders and diverse components must be coordinated. By using ArchiMate, systems engineers can create clear, consistent models that align technical solutions with business strategies, making it an essential technique for both conceptual modelling and requirements engineering. Understanding ArchiMate is therefore crucial for tackling the complexities of modern enterprise systems.

Business Process Model and Notation (BPMN) is a modelling language designed to provide a standard way to represent business processes in a graphical format. BPMN is used to create detailed models that capture the sequence of activities, decision points, and flows within a business process. While it is particularly valuable in the context of software engineering, BPMN is also a powerful technique in systems engineering, where understanding and optimising processes is crucial.

4.21.1 Core Concepts of BPMN

BPMN is designed to be intuitive and easy to understand, even for those without a technical background. Its graphical notation uses a set of standardised symbols to represent different elements of a business process, making it accessible to both business stakeholders and technical teams.

The primary elements of BPMN include:

- **Flow Objects:** These are the core elements of a BPMN diagram and include:
 - **Events:** Represent occurrences that trigger or happen during a process, such as the start, intermediate, or end of a process. For example, a customer submitting an order might be a start event, while the completion of order processing could be an end event.
 - **Activities:** Tasks or actions performed within the process. These could be manual or automated tasks, like processing a payment or updating a database.
 - **Gateways:** Decision points in the process where the flow can diverge or converge. Gateways control the branching and merging of paths, such as deciding whether a customer order meets certain criteria for approval.
- **Connecting Objects:** These show the flow of the process:
 - **Sequence Flows:** Arrows that indicate the order in which activities are performed.
 - **Message Flows:** Dashed arrows representing the communication between different process participants, such as between a customer and a service provider.
 - **Associations:** Dotted lines used to associate artefacts like text annotations or data objects with flow objects.
- **Swimlanes:** BPMN uses swimlanes to organise activities by the roles or departments responsible for them. This can be done through:
 - **Pools:** Represent major participants in the process, like an organisation or a department.
 - **Lanes:** Subdivisions within pools that represent specific roles or functions, such as customer service or sales.
- **Artefacts:** These provide additional information about the process:
 - **Data Objects:** Represent data that is required or produced by activities.
 - **Groups:** Used to categorise activities without affecting the flow.
 - **Annotations:** Provide additional textual information to clarify elements in the diagram.

4.21.2 BPMN and Requirements Engineering

BPMN is a powerful technique in both conceptual modelling and requirements engineering because it bridges the gap between business needs and technical implementation.

At the conceptual level, BPMN helps in visualising and understanding the high-level processes that are essential to an organisation. For instance, in a manufacturing company, BPMN can be used to model the entire order-to-cash process, showing how customer orders flow through different departments, from sales to shipping. This allows stakeholders to identify bottlenecks, inefficiencies, or areas for improvement before diving into the technical details.

In the context of requirements engineering, BPMN is used to capture the detailed workflows that need to be supported by a system. For example, if a new software system is being developed to manage customer orders, BPMN diagrams can help specify exactly how the order should be processed, from validation to fulfilment. These diagrams serve as a blueprint for developers, ensuring that the system meets business requirements.

4.21.3 Conclusion

The value of BPMN lies in its ability to provide a clear, standardised way to model complex processes. This is particularly important in systems engineering, where understanding and optimising processes across different domains is crucial. BPMN enables effective communication between business and technical teams, ensuring that both sides have a shared understanding of how processes should function.

Mastering BPMN is essential for analysing and designing systems that meet both business and technical objectives. Whether used for high-level conceptual modelling or detailed requirements engineering, BPMN provides techniques with value to align systems with organisational goals, as also to address efficiency in operations.

4.22 <technique> About Object-Oriented

The concept of Object-Oriented (OO) is a cornerstone of modern software development. Unlike procedural programming, which focuses on sequences of commands, OO is centred around encapsulating data and behaviours within objects (instances of classes) and defining interactions between these objects. This paradigm offers significant benefits for creating scalable, maintainable, and reusable systems.

4.22.1 Key Principles of Object-Oriented Design

Object-oriented design revolves around four fundamental principles: encapsulation, abstraction, inheritance, and polymorphism. These principles underpin the strength of OO systems:

- **Encapsulation:** This principle involves bundling data (attributes) and methods (functions) that act on the data within a class. By controlling access through a public interface and hiding internal details, encapsulation enhances modularity and reduces complexity by limiting interdependencies within the system.
- **Abstraction:** Abstraction allows for the simplification of complex systems by modelling classes that capture only essential features, disregarding unnecessary details. This makes the design clearer and easier to manage.
- **Inheritance:** This enables a new class (subclass) to inherit the attributes and behaviours of an existing class (superclass), extending or modifying the behaviour, allowing for hierarchical modelling and reuse of code.
- **Polymorphism:** Polymorphism allows objects from different classes to be treated as instances of a common superclass, supporting dynamic method invocation. This principle allows a single interface to represent various underlying forms, enhancing system flexibility and extension.

4.22.2 Risks of Introducing Polymorphism in Analysis

Despite its utility in design, polymorphism is best avoided during the analysis phase. Analysis focuses on understanding the problem domain and specifying system requirements, not on implementation-specific strategies. Polymorphism, being a design mechanism, is more appropriately applied in later phases when the system's structure and behaviour are fully understood.

4.22.3 Risks of Modelling Methods for Classes in Analysis

Incorporating methods in class diagrams during the analysis phase carries risks, especially if it is uncertain whether OO technology will be used for implementation. Analysis aims to model the problem domain by focusing on what the system must do, not how it will do it. Adding methods to class diagrams adds complexity and may obscure the main goals of analysis. For this reason, methods and polymorphism should generally be reserved for later design stages, keeping early models simple and focused on the domain's essential aspects.

4.22.4 Benefits of Object-Oriented Design

Object-oriented design offers several important advantages that have contributed to its widespread adoption:

- **Reusability:** OO design promotes code reuse through inheritance and the creation of generic classes. This allows developers to build libraries of reusable components that can be applied across different projects, saving development time.
- **Maintainability:** OO systems, composed of self-contained objects, are easier to test, debug, and modify. Thanks to encapsulation, changes can often be made to one part of the system without affecting other components.
- **Scalability:** The modular nature of OO design supports scalability. As systems grow, new functionality can be added through new classes and objects without disrupting existing code, making the system more adaptable to future changes.
- **Real-world Modelling:** OO models closely align with how we think about and interact with the world, which makes them an intuitive way to model complex systems, particularly those involving tangible entities.

4.22.5 Modelling Languages: BPMN, ArchiMate, and UML

When discussing object-oriented concepts, it is crucial to differentiate between languages that follow those principles and those that do not. BPMN is not object-oriented, as it does not use the core OO principles like inheritance or polymorphism, thus focus on representing behaviour and structural relationships with objects, not using generalizations. ArchiMate uses the concepts of specialization, composition and aggregation, but has no abstract concepts such as "class".

UML, however, is object-oriented. UML, commonly used for software design, incorporates class diagrams that encapsulate data (and methods), making it a natural fit for OO design.

4.22.6 Conclusion

Object-oriented concepts form the backbone of modern software and systems design. By encapsulating data and behaviour and leveraging inheritance and abstraction, OO design provides a powerful framework for creating scalable, maintainable, and reusable systems. The natural alignment between OO models and real-world concepts further enhances their effectiveness. Understanding the differences between OO and non-OO modelling languages is crucial for applying the right techniques in analysis and design phases.

The Unified Modelling Language (UML) is a powerful modelling language that plays a crucial role in both software engineering and systems engineering. UML provides a standardised way to visualise, specify, construct, and document the artifacts of a system, whether that system is purely software-based or a complex integration of software, hardware, and human processes. It is particularly valuable in the context of conceptual modelling and requirements engineering, where clear communication and shared understanding among diverse stakeholders are essential.

4.23.1 Core Concepts of UML

UML is not just one diagram or notation but a comprehensive language that includes a variety of diagrams, each serving different purposes. The language is designed to be flexible and adaptable, making it suitable for capturing everything from high-level requirement to detailed designs that can be directly implemented.

Some of the core UML diagram types include:

- **Class Diagrams:** These are perhaps the most well-known UML diagrams. Class diagrams model the static structure of a system by showing its concepts as classes, attributes, methods, and the relationships between classes. They are used to represent both conceptual models (defining key concepts and their relationships) and realization models (detailing the implementation in code).
- **Use Case Diagrams:** These diagrams capture functional requirements of a system by illustrating the interactions between actors (users or other systems) and the system itself. They are particularly useful during the requirements engineering phase, helping stakeholders understand how the system will behave from a user's perspective.
- **Sequence Diagrams:** Sequence diagrams model the dynamic behaviour of a system by showing how objects interact in a sequence of messages over time. These are essential for detailing the flow of control and data within a system, particularly in scenarios involving complex interactions.
- **State Diagrams:** Also known as state machine diagrams, these capture the states of an object in a system and how that object transitions from one state to another based on events. They are crucial for modelling systems where the behaviour of an entity changes over time, such as in embedded systems or user interfaces.

Other relevant UML diagram types include:

- **Activity Diagrams:** These diagrams represent the flow of activities or tasks within a system. They are like flowcharts and are useful for modelling workflows, or complex operations that need to be performed within the system. They are useful in scenarios of systems with complex domains, multiple components, or systems of systems. Before the language BPMN, these diagrams were also often used to model business processes.
- **Component Diagrams:** Component diagrams are used to model the architectural aspects of a system, such as the software components and their dependencies or how various components interact with one another.
- **Deployment Diagrams:** These diagrams show the physical deployment of artifacts on nodes, such as servers or devices. Deployment diagrams are particularly relevant in systems engineering, where understanding how software will be distributed across hardware is crucial.

4.23.2 UML and Requirements Engineering

While UML originated in the field of software engineering, its versatility makes it equally valuable in broader systems engineering contexts. For example, UML can be used to model not only the software components of a system but also the interactions between software, hardware, and human operators. This makes UML a powerful technique for ensuring that all aspects of a complex system are considered and integrated effectively.

In the context of conceptual modelling, UML diagrams like class diagrams and use case diagrams help stakeholders understand the high-level structure and requirements of a system without getting bogged down in technical details. These diagrams provide a shared language that both technical and non-technical stakeholders can understand, facilitating better communication and collaboration.

As the project moves from conceptual modelling to requirements engineering and design, UML continues to play a crucial role. Detailed UML diagrams such as sequence diagrams, activity diagrams, and state diagrams help translate high-level requirements into specific designs that developers can implement. This transition from conceptual to realization models ensures that the final system remains aligned with its original goals.

4.23.3 Conclusion

UML is a versatile and comprehensive modelling language that serves as a bridge between conceptual ideas and technical implementation. Whether used in software engineering or systems engineering at large, UML provides the techniques needed to model, design, and communicate the structure and behaviour of complex systems. For students of computer science, mastering UML is essential for understanding and applying best practices in both conceptual modelling and requirements engineering, ensuring that systems are built to meet both technical specifications and stakeholder needs effectively.

The **Systems Modeling Language (SysML)** is a general-purpose modelling language maintained by the **Object Management Group (OMG)** and intended primarily for **Model-Based Systems Engineering (MBSE)**.

SysML helps engineers represent and relate different aspects of a system, including requirements, structure, behaviour, interfaces, constraints, analysis, and verification. It is particularly relevant to multidisciplinary systems that combine software, hardware, people, information, and physical processes, such as aircraft, vehicles, medical devices, industrial equipment, and critical infrastructures.

SysML is a modelling language, not a systems-engineering method. It does not decide what the system of interest is, where its boundary should be placed, or which engineering process should be followed. These remain analytical and methodological decisions.

4.24.1 Historical Overview

The origins of SysML can be traced to a joint initiative by OMG and the International Council on Systems Engineering (**INCOSSE**). In March 2003, OMG issued the *UML for Systems Engineering Request for Proposals*, seeking an extension of UML that could address the broader needs of systems engineering.

OMG announced the adoption of SysML in 2006, and **SysML 1.0** became formally available in September 2007. SysML 1.x was defined as a profile of UML 2: it reused part of UML and added concepts for requirements, system structure, quantitative constraints, and multidisciplinary engineering. The language subsequently evolved through several maintenance revisions. **SysML 1.7**, adopted in June 2024, was the final formal version of this first generation.

Work on a new generation began with the SysML 2.0 request for proposals issued in December 2017. The objective was to improve the precision, expressiveness, usability, interoperability, and extensibility of the language. After several beta versions, **SysML 2.0** reached final adoption in 2025 and became a formal OMG specification in September 2025.

4.24.2 SysML 2.0

SysML 2.0 is a major redesign rather than a minor update to SysML 1.x. It has a new foundation based on the **Kernel Modeling Language (KerML)** and is independent of UML. It provides complementary normative textual and graphical notations over the same underlying model.

SysML 2.0 also introduces a more systematic distinction between reusable definitions and their usages in particular contexts. A standard API and associated services support model access, queries, interchange, and integration with other engineering tools.

SysML 2.0 is the current formal generation of the language, while SysML 1.7 remains the last formal version of SysML 1.x. Multiple commercial and open-source tools now provide some level of SysML 2.0 support. However, organisations and tool vendors are still progressing at different rates, and existing SysML 1.x repositories remain widely relevant.

SysML 1.7 and SysML 2.0 models are not automatically interchangeable. OMG provides a dedicated transformation specification, but migration may still require modelling decisions, validation, and tool-specific work.

SysML notation is not taught or assessed in this course. Students should nevertheless recognise SysML as an important language for MBSE and understand when it may be more appropriate than enterprise-, process-, or software-oriented modelling languages.

4.24.3 An Example

See here how engineers at Airbus are exercising: <https://github.com/airbus/apollo-11-sysml-v2>
In their own words:

This model represents a comprehensive, multi-layered SysML v2 implementation of the Apollo 11 mission. It demonstrates the application of the CoSMA (Complex System Modeling and Architecting) framework to a complex, real-world SoS. The result is a concrete example of how SysML v2 can be used to create detailed, traceable, and integrated system models. Both the Apollo 11 mission model and the CoSMA framework are provided as a dedicated research artifact developed by Airbus Central R&T. The CoSMA framework is presented here as representative "scaffolding" required to test the language's expressive power and is not intended to serve as a fully elaborated or universal industrial standard.

4.25 example> Automated Parking System

Let us explore a simple example that demonstrates the use of ArchiMate, BPMN, and UML on developing an Automated Parking System for a car.

4.25.1 ArchiMate

ArchiMate is used to model the structure and behaviour of the enterprise architecture, focusing on the relationships between business processes, applications, and technology. In the case of the Automated Parking System, we can model the architecture across its three core layers: Business, Application, and Technology.

At the Business Layer, we model the business roles, actors, and processes. For instance, the Customer actor would engage in the Parking Request Process, which represents their need to park a vehicle autonomously. This is a business process and does not include any technological details, such as the mobile app or car sensors.

At the Application Layer, we represent software components that support the business process. For instance, the Parking Control System and Vehicle Navigation System are part of this layer, responsible for handling requests and navigating the car based on the commands issued.

Finally, the Technology Layer focuses on the infrastructure that supports the application components. This includes the Sensors installed on the car (e.g., ultrasonic sensors) and the Vehicle's Embedded Systems, which execute commands for autonomous parking. These layers collectively offer a comprehensive view of the system from an enterprise perspective.

4.25.2 BPMN

Business Process Model and Notation (BPMN) is employed to detail the business processes involved in the system. In the Automated Parking System, BPMN is used to visualise how the system supports a parking process.

For example, the process starts when the Customer actor initiates the Request Parking action. The system then engages in a Scan for Available Parking task, followed by an interaction where the customer selects a parking spot. After this, the system autonomously controls the vehicle to execute the Autonomous Parking task, involving a sequence of activities, such as navigating, monitoring sensors, and adjusting the vehicle's position. BPMN is ideal for representing workflows and interactions without focusing on system-level details like software and hardware.

4.25.3 UML

Unified Modelling Language (UML) is primarily used for software design, focusing on the structural and behavioural aspects of the system. For the Automated Parking System, UML provides detailed software-level models.

A Class Diagram would represent the main software entities involved in the system, such as ParkingController, NavigationSystem, SensorModule, and VehicleControl. These classes interact to process inputs (e.g., sensor data), execute algorithms for pathfinding, and control the car's movements.

A Sequence Diagram is useful to show the interaction between components. For example, when a customer initiates a parking request, the ParkingController sends commands to the VehicleControl class, which interfaces with the SensorModule to guide the car safely into the spot. UML's focus here is on the internal architecture of the software, covering object interactions, data flow, and states within the system.

4.26 <example> Challenges in a Smart Parking

The city is developing a Smart Parking System to address parking issues. The system aims to help drivers find available spaces easily, reduce congestion, and optimise parking utilisation. Several stakeholders are involved, each with different concerns.

4.26.1 Stakeholders and Concerns

- City Council
 - Reduce Congestion: The council aims to reduce traffic by providing real-time information on available parking spots, thus improving air quality.
 - Revenue Generation: The council also wants to maximise revenue through dynamic pricing, adjusting rates based on demand.
- Drivers
 - Affordable and Convenient Parking: Drivers seek a system that provides easy and affordable parking.
 - Privacy: Drivers are also concerned about how their data, such as location, is collected and used, as they wish to maintain their privacy.
- Local Businesses
 - Increased Customer Traffic: Businesses want nearby parking available for customers. They are interested in short-term parking, encouraging frequent visits.
 - Fair Pricing: Business owners are concerned that high parking fees may deter customers.
- Environmental Groups
 - Reduce Emissions: Environmental groups want the system to cut emissions by reducing the time vehicles spend looking for parking. They also advocate incentives for electric vehicles (e.g., discounted parking).
- Law Enforcement
 - Efficient Enforcement: Law enforcement is interested in using technology like licence plate recognition to monitor and ensure parking compliance.

4.26.2 Conflicting and Complementary Requirements

- Dynamic Pricing vs Fair Pricing: The City Council wants dynamic pricing to manage demand and generate revenue, whereas Local Businesses and Drivers are concerned about affordability. To resolve this, a compromise could involve a tiered pricing system, with discounts for short-term parking sponsored by local businesses.
- Privacy Concerns vs Real-Time Tracking: The City Council wishes to track vehicles in real-time to optimise parking space usage, but Drivers worry about privacy. A possible resolution is implementing anonymisation or allowing drivers to opt in for tracking, ensuring transparency about data usage.
- Short-Term Parking vs Reducing Congestion: Local Businesses prefer prioritising short-term parking near shops, while the City Council wants fewer vehicle movements to reduce congestion. The solution might involve designating some spaces as short-term while using others for longer stays, balancing both needs.
- Incentives for Electric Vehicles vs Revenue Maximisation: Environmental Groups support incentives for electric vehicles, while the City Council wants to maximise revenue. These requirements can complement each other by providing electric vehicle discounts during off-peak hours, encouraging sustainable transport without major revenue losses.
- Licence Plate Recognition for Enforcement vs Privacy: Law Enforcement supports using licence plate recognition for efficient monitoring, but Drivers are worried about potential privacy breaches. A balanced solution is setting clear rules on data retention and use, limiting the information collected to what is necessary for enforcement.

4.26.3 Discussion Points

This example shows how different stakeholders' requirements can complement or contradict each other, highlighting the importance of balancing these perspectives in requirements engineering. Recall here "2.15 <further> Strategy, Goals, Outcomes, and Principles." For example:

- Stakeholder Priorities: How each stakeholder's concerns influence requirement prioritisation. For instance, why might the Drivers value privacy more than the Local Businesses?
- Conflict Resolution: Should we consider techniques like negotiation or re-prioritisation to balance different stakeholder needs?
- Visualising Perspectives: What is the role of tools like goal models²¹ or UML diagrams to illustrate stakeholder interactions, highlighting both conflicts and synergies.

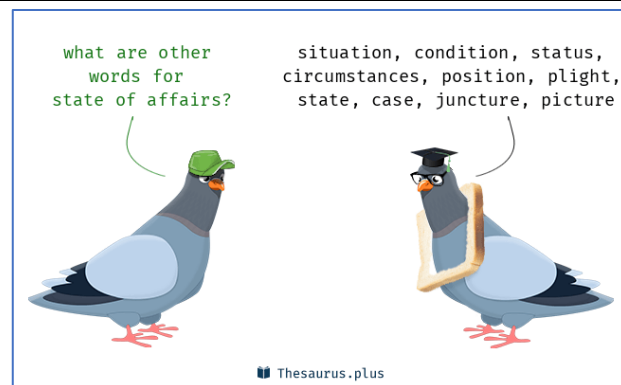
²¹ <further> "Goals" and "requirements engineering" have a longstanding, close relationship, with dedicated languages developed to capture goals and their role in requirements, such as the i* (iSTAR) framework:

- <https://www.cs.toronto.edu/km/istar/>
- <http://istar.rwth-aachen.de/>
- <https://sites.google.com/site/istarlanguage/home>

4.27 <concept> States of Affairs

The concept of "state of affairs" (also referred to as Sachverhalt in philosophical discussions, especially in the works of Ludwig Wittgenstein and Bertrand Russell) intersects with many of the concepts discussed in ontology, epistemology, semiotics, and knowledge representation.

It serves as a foundation for capturing and representing real-world conditions and their relationships, which is particularly in conceptual modelling and requirements engineering. A state of affairs describes a specific arrangement of entities, objects, or events, making it central to understanding how systems and requirements reflect reality.



4.27.1 Ontology and States of Affairs

In ontology, states of affairs provide the foundation for modelling the existence and relationships between objects, properties, and events within a system. Conceptual models, like entity-relationship diagrams, capture the state of affairs by describing how entities (such as users, products, or processes) relate to each other. In requirements engineering, this helps define the system's structure, ensuring that all necessary entities and their relationships are clearly represented. For example, the system must handle a state of affairs where an "order" is linked to a "customer," which is crucial for defining business rules.

4.27.2 Epistemology and States of Affairs

From an epistemological perspective, a state of affairs represents the knowledge that a system or stakeholder needs about the world. In requirements engineering, knowing what a state of affairs looks like is key to defining functional requirements. A requirement like "The system must send a notification when the stock falls below a threshold" involves understanding the state of affairs where stock levels are monitored. Requirements must therefore clearly describe how the system perceives, interprets, and acts upon certain states of affairs to ensure accuracy and reliability in real-world conditions.

4.27.3 Semiotics and States of Affairs

In semiotics, the representation of states of affairs through signs (such as language, symbols, or diagrams) is fundamental for communicating requirements. Models and requirements documents use these signs to represent different states of affairs. For instance, a use case diagram communicates how various actors (e.g., users) interact with the system to achieve specific outcomes, each representing a state of affairs. The signified in semiotics, which is the concept that a sign refers to, can be understood as a specific state of affairs that the system is designed to handle or produce.

4.27.4 Logic and States of Affairs

In logic, propositions describe the conditions or states of affairs that must hold true in the system. Requirements often define logical transitions between states, such as "If a user submits an application, the system must update the status to 'submitted'." The truth or falsehood of propositions in logical models corresponds to whether the specified state of affairs is realised. In dynamic systems, state transitions are used to model how the system changes from one state of affairs to another, guided by rules and triggers defined in the requirements.

4.27.5 Knowledge Representation and States of Affairs

In knowledge representation, states of affairs are expressed as sets of facts or conditions within the system. Ontologies and data models in requirements engineering capture these states of affairs by defining entities, attributes, and relationships. For example, a data model might describe a state of affairs in which a customer has placed an order, specifying the conditions (entities and relationships) under which this state holds. Effective requirements capture and represent these states of affairs to ensure that the system has the necessary information to make decisions and operate correctly.

4.27.6 Value of States of Affairs in Requirements Engineering

The concept of states of affairs also plays a significant role in evaluating non-functional requirements, such as performance, security, or usability. Requirements like "The system must remain responsive under heavy load" describe a desirable state of affairs, where the system behaves as expected even under specific conditions. Non-functional requirements often focus on ensuring that the system achieves or maintains certain states of affairs that ensure high-quality user experiences or meet operational standards.

4.27.7 Conclusion

The concept of a state of affairs provides a framework for understanding static and dynamic system states, representing knowledge, and ensuring that system behaviour aligns with user expectations and real-world needs. By clearly defining and modelling states of affairs, we can ensure that systems meet both functional and non-functional requirements. Here's how the concept of "state of affairs" relates to important domains for requirements engineering.

4.28 <further> Semiotics



ArchiMate, BPMN, and UML are visual languages.

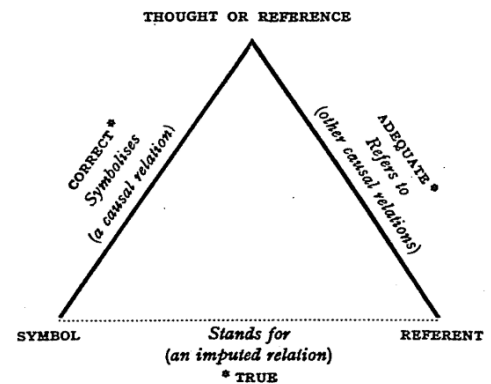
Semiotics is the study of signs and symbols as a means of communication. It plays a crucial role in understanding how meaning is constructed and conveyed. In the domain of conceptual modelling, where languages such as ArchiMate, BPMN, and UML are used to represent complex systems, awareness of the principles of semiotics helps ensuring clarity and accuracy in communication. Central to semiotics are two key models: the semiotic triangle and the semiotic square, both of which provide a framework for understanding how signs (symbols) relate to real-world objects and concepts.

4.28.1 The Semiotic Triangle

The semiotic triangle, developed by Ogden and Richards in 1923, illustrates the relationship between a referent (the object or concept in the real world), a symbol (the sign used to represent the object), and the interpretant (the meaning derived from the symbol). This triangular relationship highlights the fact that a symbol does not have a direct link to the referent; rather, it gains meaning through the interpretant, which depends on context, knowledge, and shared understanding.

In conceptual modelling, this is particularly important. For example, in ArchiMate, the symbol for "Business Actor" (a rectangle) may represent an individual, a department, etc. The meaning (interpretant) derived from this symbol depends on the model's context. A modeller must ensure that stakeholders share a common understanding of what the symbol represents within that particular system.

This concept also applies to languages like BPMN, where a process diagram might use different shapes to denote tasks, events, or gateways. The symbols used do not inherently carry meaning but require interpretation within the specific process being modelled. For instance, a diamond shape in BPMN represents a decision point, but the exact nature of the decision (referent) and its meaning (interpretant) depend on the model's context.

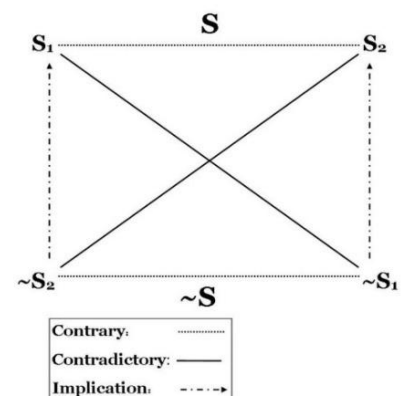


4.28.2 The Semiotic Square

The semiotic square, developed by A. J. Greimas, extends the triangle by introducing additional complexity in how meanings are related through opposition and contradiction. It represents four terms: two in direct opposition, and two that represent their contradiction or complement. This model is useful for showing relationships between concepts that are not simply binary, helping to clarify ambiguities and uncover hidden structures.

In conceptual modelling, the semiotic square can help manage complexity by showing how different elements in a system are interrelated. For example, in UML, which is used to model system behaviours and structures, the awareness of the semiotic square might motivate to clarify relationships between contradictory system states, such as "on" and "off," and intermediate states like "standby." Understanding these nuanced relationships allows for more accurate modelling of real-world systems, where not everything is strictly binary.

Greimas Semiotic Square



4.28.3 Semiotics and Conceptual Modelling Languages

Conceptual modelling languages like ArchiMate, BPMN, and UML rely heavily on the clear representation of abstract concepts. Without a shared understanding of how symbols relate to their referents, models can easily be misunderstood, leading to design flaws and miscommunication. By considering the principles of the semiotic triangle and square, modellers can be aware that it is necessary to confirm ahead if the different stakeholders understand and agree the symbols in use to represent elements in the system carry the intended meaning and are interpreted consistently.

For example, UML is often used to model software systems, and symbols such as rectangles (representing classes) and lines (representing relationships) must be interpreted correctly to understand the system's structure. If a modeller uses a generalisation symbol (a hollow arrow) without ensuring the audience understands its meaning, confusion can arise. The semiotic triangle's emphasis on linking the symbol, referent, and interpretant helps avoid such issues.

4.28.4 Conclusion

As conceptual models often rely on abstract representations, the risk of ambiguity increases when stakeholders interpret signs differently based on cultural, linguistic, or personal contexts. Without a shared understanding of these symbols, the model's utility diminishes, potentially compromising outcomes. Thus, careful consideration of semiotic risks is crucial in designing clear, universally understood conceptual models. Understanding semiotics, especially the semiotic triangle and square, is vital in conceptual modelling when it involves producing visual models.

4.29 <concept> "As-Is" versus "To-Be"

Recall "2.12 <concept> Understanding versus Doing."

In enterprise architecture and systems engineering, we can model an "as-is" scenario, representing the current state of a system or organisation, or a "to-be" scenario, illustrating the desired future state. These models are not only crucial for visualizing current and future conditions but also for reflecting the broader "state of affairs" of an organisation. The concept of "state of affairs" encompasses both the present situation and the envisioned future, offering a comprehensive view of the context within which decisions are made. While both "as-is" and "to-be" models are essential, they serve different purposes, require distinct approaches, and together, provide a more holistic understanding of organisational dynamics. Understanding these differences is crucial for effectively leveraging modelling to navigate and improve organisational states of affairs.

4.29.1 Purpose of Modelling an "As-Is" Scenario

The primary purpose of an "as-is" model is to provide an accurate and detailed representation of the current state of a system, process, or organisation. This model serves as a baseline for understanding the current "state of affairs" — how things operate, identifying inefficiencies, and diagnosing problems. It is instrumental in documenting existing processes and systems, ensuring that all stakeholders have a shared understanding of the present situation.

Key characteristics of "As-Is" modelling might be:

- **Detail-Oriented:** "As-is" models often require capturing the detailed nuances of existing processes, systems, and interactions, which are necessary to identify bottlenecks, redundancies, and other areas for improvement.
- **Accuracy:** The accuracy of an "as-is" model is critical, as it forms the foundation for any future changes. If the current state of affairs is not correctly understood, any plans for improvement may be flawed.
- **Documentation:** These models serve as documentation of the current state of affairs, which can be used for training, compliance, and as a reference point for future changes.

Example: Consider a manufacturing company that wants to optimize its production process. The "as-is" model would map out the entire current production process, including the sequence of tasks, the resources used, and the time taken at each stage. This model would help the company identify inefficiencies, such as bottlenecks where production slows down, or unnecessary steps that could be streamlined.

4.29.2 Purpose of Modelling a "To-Be" Scenario

The "to-be" model, in contrast, focuses on envisioning the future state of affairs for a system, process, or organisation. It is used for planning and designing changes aimed at improving performance, aligning with strategic goals, or adapting to new requirements. This model is aspirational, illustrating how things should work once the changes have been implemented.

Key characteristics of "To-Be" modelling might be:

- **Goal-Oriented:** "To-be" models are driven by the goals and objectives of the organisation, leading to the design of solutions that will meet these goals and define the future state of affairs.
- **Innovation:** These models encourage innovation by allowing organisations to explore different possibilities and scenarios for the future. They are not constrained by current limitations and can include entirely new processes, technologies, or organisational structures.
- **Validation:** "To-be" models are often used to validate the feasibility and impact of proposed changes before they are designed in detail and implemented. This ensures that the changes will achieve the desired outcomes without unintended negative consequences.

Example: Returning to the manufacturing company example, after identifying inefficiencies in the "as-is" model, the organisation might create a "to-be" model that includes streamlined processes, new machinery, or reconfigured workflows to eliminate bottlenecks and reduce production time. This model would depict the optimized process and serve as a guide for implementing the necessary changes, laying the foundation for the future state of affairs.

4.29.3 Integrating "As-Is" and "To-Be" Models

In practice, "as-is" and "to-be" models are often used in conjunction to fully understand and shape the state of affairs. The "as-is" model provides the foundation for understanding current operations, while the "to-be" model is used to design improvements. The transition from "as-is" to "to-be" is typically managed through change management methods, ensuring that the proposed changes are implemented effectively and that the benefits are realized in the overall state of affairs.

4.29.4 Conclusion

Modelling "as-is" and "to-be" scenarios serves distinct but complementary purposes in enterprise architecture and systems design. While the "as-is" model provides a detailed understanding of the current state of affairs, the "to-be" model offers a vision for the future. By employing appropriate modelling techniques and involving relevant stakeholders, projects and organisations can effectively plan and execute changes that drive improvement, align with strategic goals, and shape the desired state of affairs.

4.30 <example> From "As-Is" to "To-Be"

Let's consider a small retail business, "FreshGrocer," which sells organic food products.

FreshGrocer has been operating for several years, but recently, the owner has noticed some inefficiencies and has begun to explore how the business could improve. To achieve this, the owner develops two models: the "as-is" scenario, representing the current state of affairs, and the "to-be" scenario, illustrating the desired future state of the business.

4.30.1 As-Is Scenario (Current State of Affairs)

In the as-is scenario, FreshGrocer operates from a single physical location. Customers primarily visit the store in person to purchase products. While the business is doing well, it faces several challenges:

- **Manual Inventory Management:** FreshGrocer's inventory is tracked manually using spreadsheets. Staff must regularly check stock levels and update the inventory sheet, which can lead to errors, such as overstocking or running out of popular items.
- **Limited Customer Base:** The business relies entirely on foot traffic from the local area. Although the store has a loyal customer base, growth is limited to those who live or work nearby.
- **Time-Consuming Processes:** Daily tasks such as tracking sales, managing orders, and responding to customer queries are handled manually. This takes up a lot of time and prevents staff from focusing on higher-value activities, such as customer service and exploring new product lines.

In summary, the as-is model shows a business that operates traditionally, without digital tools, and with manual processes that are prone to inefficiencies and human error.

4.30.2 To-Be Scenario (Future State of Affairs)

In the to-be scenario, the owner envisions a more efficient and scalable operation by addressing the inefficiencies in the current state. The key changes include:

- **Automated Inventory Management System:** FreshGrocer implements an automated inventory management system that tracks stock levels in real-time. This system sends alerts when stock is low, reducing the chances of running out of popular products or over-ordering items that aren't selling as well.
- **E-Commerce Platform:** The business launches an online store, allowing customers to browse and purchase products online. This not only opens up FreshGrocer to a wider audience beyond the local community but also provides greater convenience to existing customers who can now order online for delivery or click-and-collect services.
- **Enhanced Digital Presence:** To support the online store, FreshGrocer invests in a digital marketing process, including social media, search engine optimisation (SEO), and email campaigns. This helps to attract new customers and retain existing ones through promotional offers and regular updates.
- **Streamlined Processes:** FreshGrocer introduces new software tools to manage sales, orders, and customer interactions. By automating these processes, staff have more time to focus on customer service, improving the in-store experience, and expanding the product range.
- **Sustainability Initiatives:** As part of its future vision, FreshGrocer introduces a loyalty programme that rewards customers for purchasing sustainably sourced products and reducing packaging waste, aligning with the growing demand for eco-friendly business practices.

4.30.3 Conclusion

In the to-be scenario, FreshGrocer has transitioned from a small, locally focused shop to a modern, digitally integrated business. By adopting new technology and expanding its market reach, the business is better positioned to grow, improve customer satisfaction, and operate more efficiently. The transition from the current state of affairs (as-is) to the desired future state (to-be) involves strategic investments in technology and processes, paving the way for an expected sustainable growth and success.

Recall “**Error! Reference source not found. Error! Reference source not found.**”.

In conceptual modelling and enterprise architecture, the concept of "model debt" or "architecture debt" refers to the accumulated suboptimal design decisions, compromises, and deficiencies within an organisation's models or architecture that make the system more complex, costly, and difficult to maintain or evolve over time.

This term is analogous to technical debt in software engineering, where shortcuts taken during design or development lead to future issues. In the context of enterprise architecture and conceptual modelling, model debt can emerge from various sources.

4.31.1 Causes of Model Debt

- **Incomplete or Inaccurate Models:** During the development of conceptual models or architectural frameworks, certain aspects of the system might be oversimplified or overlooked to meet tight deadlines or reduce initial complexity.
- **Outdated Architecture:** As an organisation's business processes or technology evolves, earlier architectural decisions may no longer align with current needs, creating inefficiencies.
- **Lack of Standardization:** When different parts of an organisation or project use inconsistent modelling standards or methodologies, it can lead to fragmented and disjointed architectural frameworks.
- **Overly Complex Models:** While trying to cover every aspect of a system, models may become too detailed and complicated, making them difficult to understand and implement, which can deter teams from using or updating them.
- **Organisational Changes:** Mergers, acquisitions, or internal reorganisations may render previous models obsolete or misaligned with new structures or objectives.

4.31.2 Consequences of Model Debt

- **Increased Maintenance Costs:** As models deviate from ideal designs or best practices, more resources are required to manage, update, and work with the existing architecture.
- **Reduced Agility:** A bloated or poorly structured architecture makes it harder for an organisation to adapt to new business requirements or technological changes.
- **Higher Risk of Errors:** Inaccuracies or inefficiencies in models increase the likelihood of misinterpretations, design flaws, and operational issues.
- **Slower Time to Market:** The need to constantly address architectural shortcomings can slow down the delivery of new features, systems, or services.

4.31.3 Managing and Reducing Model Debt

- **Regular Reviews and Refactoring:** Continuously assess and update models to keep them in line with the current state of the business and technology landscape.
- **Establishing Clear Modelling Standards:** Ensure consistency in modelling methodologies across different teams and departments.
- **Modular and Scalable Design:** Create architectures that are flexible and can evolve with minimal disruption as new requirements arise.
- **Investing in Training and Governance:** Make sure stakeholders have a deep understanding of architectural best practices and implement governance mechanisms to oversee architecture quality.

4.31.4 Conclusion

In essence, model debt represents the long-term cost of short-term decisions that trade off architectural quality for immediate goals. Managing this debt is crucial for sustaining a flexible, scalable, and efficient enterprise architecture over time.

4.32 <concept> Wrap-up Engineering with Models

This section shifts the focus from the conceptual aspects of systems engineering and requirements engineering to the practical application of models in representing and analysing complex systems. The section emphasises how models serve as powerful tools throughout the systems engineering lifecycle, aiding in communication, analysis, design, and validation.

At the heart of MBSE and MDE is the use of models and metamodels, which define not only the system being built but also the rules and structures governing how models are created. These models serve as simplified representations of reality, allowing engineers to manage complexity by focusing on key elements and their relationships.

The concept of "state of affairs" represents the conditions or configurations a system must handle. Whether expressed in a requirements diagram or captured through business process models, defining and managing these states is critical for ensuring that systems operate as intended in both the current and future contexts. By clearly modelling these states and their transitions, engineers ensure that systems remain responsive to real-world conditions and meet both functional and non-functional requirements.

4.32.1 Models as a Bridge

Models are essential bridges between abstract concepts and concrete implementations. Models provide visual and structured representations of systems, facilitating communication among stakeholders, enabling rigorous analysis, and guiding the design process.

4.32.2 Model-Based Systems Engineering and Model-Driven Engineering

Two key paradigms related to modelling in systems engineering are:

- **MBSE (Model-Based Systems Engineering):** This approach emphasises using models as the central artefacts throughout the entire systems engineering lifecycle. This means that models are not merely documentation; they are actively used and refined from the initial requirements gathering through to design, analysis, and validation.
- **MDE (Model-Driven Engineering):** This broader software engineering paradigm focuses on using models to automate and streamline the software development process. In MDE, models are not just descriptive; they are executable, meaning they can generate code or other artefacts, increase efficiency and reduce errors.

4.32.3 Key Nowadays Modelling Languages

Nowadays key modelling languages are:

- **ArchiMate:** A standardised language specifically designed for modelling enterprise architecture. ArchiMate excels at visualising the relationships between business processes, applications, data, and

technology infrastructure, providing a holistic view of the enterprise.

- **BPMN (Business Process Model and Notation):** As its name suggests, BPMN is tailored for modelling business processes. BPMN provides a rich set of symbols and constructs for representing the flow of activities, decisions, events, and data within a process.
- **UML (Unified Modelling Language):** A widely used language for modelling software systems. UML offers a comprehensive set of diagrams for capturing different aspects of a software system, including its structure, behaviour, interactions, and deployment.

The metamodels governing those languages define the rules by which high-level models can be converted, for example in the case of BPMN, and UML, into software or even physical artefacts, bridging the gap between conceptual design and implementation.

4.32.4 Modelling "As-Is" and "To-Be"

It is important to distinguish between "as-is" and "to-be" modelling in understanding and improving systems:

- **"As-Is" Models:** These models represent the current state of a system or organisation, capturing its existing processes, structures, and behaviours. "As-is" models are essential baselines for identifying areas for improvement and measuring the impact of changes.
- **"To-Be" Models:** These models illustrate the desired future state of the system after implementing changes. "To-Be" models provide a target vision and guide the design and implementation of solutions.
- The sources provide examples of how "as-is" and "to-be" models can be used in scenarios like improving a retail store's operations or designing a smart home lighting system.

4.32.5 Conclusion

Shifting from conceptual understanding to practical application means embracing models as essential tools in the systems engineering toolkit. By understanding different modelling languages, their specific applications, and the value of capturing both current and desired states through "as-is" and "to-be" modelling, practitioners can effectively leverage models to tackle complexity, improve communication, and drive successful system design and implementation.

In conclusion, the true strength of MBSE and MDE lies not just in their ability to model complex systems but in their capacity to guide the transformation from current "as-is" states to future "to-be" scenarios. By leveraging the power of models and metamodels, and carefully managing state transitions, engineers can build systems that are both adaptable and aligned with broader strategic goals. As systems continue to grow in complexity, mastering these methodologies and their underlying concepts will be essential for delivering successful engineering projects.

4.33 <further> Complementary Modelling Frameworks

There exist a few influential frameworks that complement the languages here addressed by offering different perspectives on **process**, **traceability**, **communication**, and **motivation**. These frameworks help analysts understand how modelling fits within broader development lifecycles and communication needs.

4.33.1 Rational Unified Process

The **Rational Unified Process (RUP)** is not a language but a **process framework** for managing the lifecycle of software development. It defines **how modelling methods are applied** within an iterative and risk-driven lifecycle.

Key ideas:

- Divides work into **four phases**: *Inception*, *Elaboration*, *Construction*, and *Transition*.
- Promotes **iterative development** and **continuous validation**.
- Organises work through **disciplines** (Business Modelling, Requirements, Analysis & Design, Implementation, Test, Deployment, etc.).
- Uses **UML as its main modelling notation**, with an emphasis on maintaining traceability across artefacts.

RUP provides a conceptual “home” for the models produced in analysis and that go until the design.

It shows how different diagrams relate to project phases and deliverables, preparing students to see modelling as part of a **development process**, not an isolated activity.

4.33.2 ICONIX: Bridging Analysis and Design

The **ICONIX process** simplifies RUP’s structure, focusing on **use case–driven traceability** from requirement to code. It uses a minimal set of UML artefacts: use cases, domain model, robustness diagrams, sequence diagrams, and class diagrams. It helps visualise the transition from textual requirements to structured design, reinforcing the continuity between analysis and architecture. The ICONIX process typically follows four iterative steps:

1. Domain Modelling – identification of key domain concepts and relationships (usually via UML class diagrams).
2. Use Case Modelling – definition of system interactions and user goals.
3. Robustness Analysis – creation of “robustness diagrams” that connect boundary objects (interfaces), entity objects (data), and control objects (logic), clarifying responsibilities before design.
4. Sequence and Class Design – specification of detailed interactions and class behaviour for implementation.

This process enforces semantic continuity between requirements and implementation, reducing ambiguity in early stages. Its robustness diagrams act as a bridge between conceptual and detailed design models, much like ArchiMate’s link between business processes and applications.

ICONIX’s strength lies in its discipline without excess formalism, ensuring that every diagram serves a clear purpose within the engineering workflow.

4.33.3 The C4 Model

The **C4 Model** offers a simple, hierarchical way to visualise software architecture: **Context → Container → Component → Code**. It is a **communication framework**, not a formal language, complementing UML and ArchiMate by simplifying the big picture for mixed audiences. It reinforces the concept of **abstraction levels and viewpoints**, helping present architecture clearly.

C4 Level	Purpose	Comparable ArchiMate Layer	Comparable UML Diagram
Context (C1)	Define boundaries and external interactions	Business & Application Layers	Use Case or Context Diagram
Container (C2)	Show applications, data stores, and APIs	Application Layer	Component Diagram
Component (C3)	Detail modules within containers	Application Function / Service	Class or Component Diagram
Code (C4)	Map to implementation structures	Technology Realisation	Class or Package Diagram

4.33.4 Goal-Oriented Modelling

Methods such as **KAOS** (Knowledge Acquisition in autOMated Specification) and **i*** model **intentions and responsibilities, focusing on the “why”**. They reveal *why* the system exists by linking stakeholder goals to functional and non-functional requirements. They connect requirements to motivation, enriching early analysis and supporting rationale documentation:

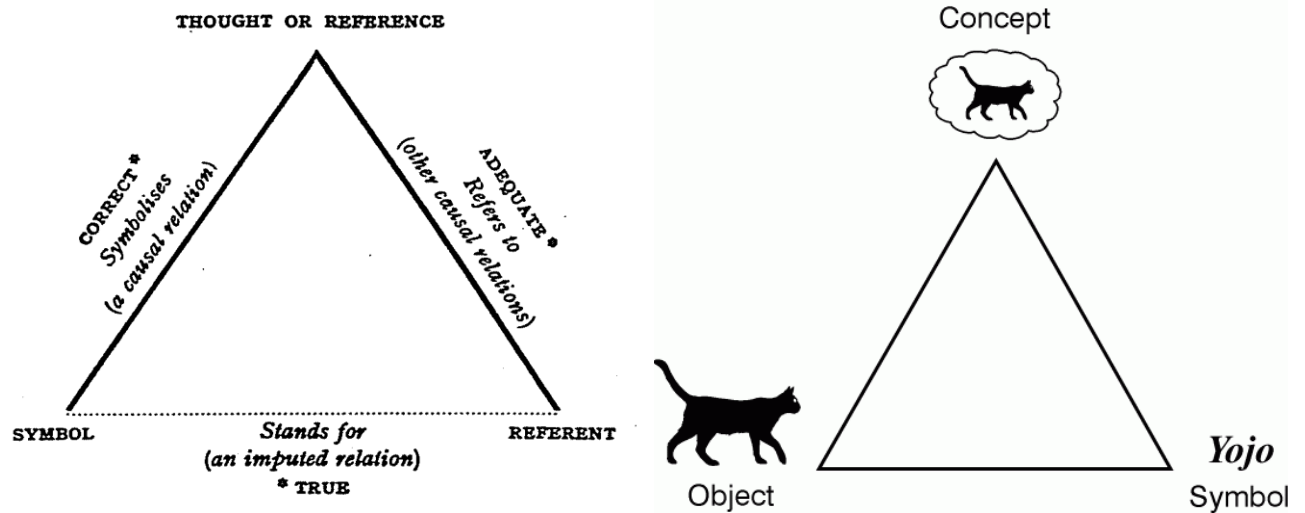
- KAOS uses a formal framework to refine high-level goals into operational requirements, ensuring traceability from stakeholder objectives to verifiable specifications. It supports reasoning about obstacles, alternatives, and responsibilities.
- i* focuses on actor dependencies and strategic rationale: it models social and organisational contexts, showing how agents depend on each other for goals, tasks, and resources.

	Focus	Typical Artefacts	Compares to...
KAOS	Goal refinement and formal requirement specification	Goal and Obstacle Models	Requirements and Business Motivation Models
i*	Actor dependencies and intentional relationships	Strategic Dependency and Rationale Models	ArchiMate Motivation & Business Layers

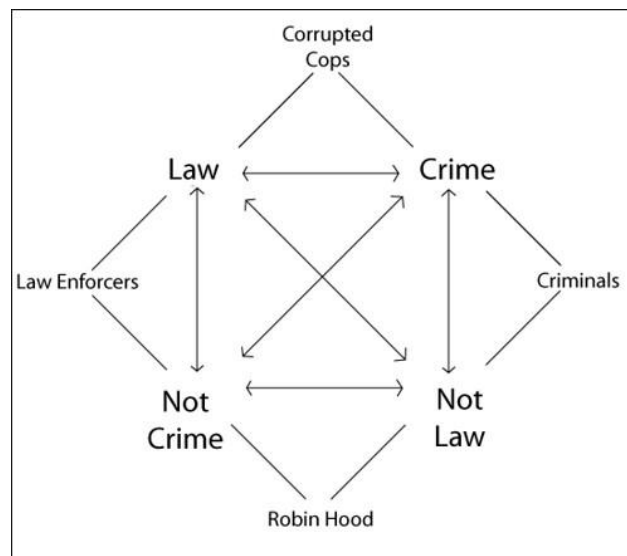
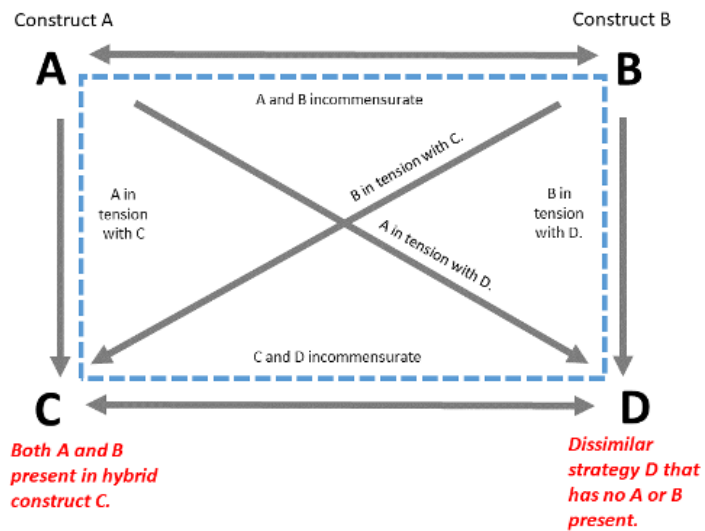
4.33.5 Integrating Perspectives

Analysts work **across methods and frameworks**, choosing the right tool for the problem, not for ideology. Each framework adds a unique value. RUP gives the **process context**, ICONIX shows **traceability**, C4 enhances **communication**, and goal-oriented models explain **motivation**. They show how modelling is a **multifaceted practice** that integrates reasoning, structure, and communication.

Focus	Framework	Adds Value by...
Process	RUP	Structuring modelling activities in a lifecycle
Traceability	ICONIX	Linking requirements to design artefacts
Communication	C4	Clarifying architecture across abstraction levels
Intentionality	KAOS / i*	Making stakeholder goals explicit



Explaining the semiotic square: a heuristic device for designing multiple futures



5 <cover> Enterprise Architecture and ArchiMate

To complement this reading, see the detailed definitions and specifications of ArchiMate here:

- ArchiMate 4 Specification: <https://pubs.opengroup.org/architecture/archimate4-doc/>
 - Focus on sections 1.2, 3.2, 3.3, 3.4, 4, 5, 8, 9, 10, and 11.
 - Also, the Annexes A1 and A3

Also, see:

- Reference Cards: <https://publications.opengroup.org/archimate-library>

5.1 <concept> Enterprise Architecture as Coherence and Change

Enterprise Architecture (EA) is the practice of understanding and shaping an enterprise as a coherent whole. In this context, an **enterprise** is not necessarily an entire company. It may be a public institution, a business unit, a partnership, a service ecosystem, or any other organisational scope that must be analysed and changed in a coordinated way.

Modern enterprises are socio-technical systems. Their outcomes depend on the interaction of people, responsibilities, processes, information, applications, physical resources, and technology. These elements are usually managed by different groups of peoples and evolve at different speeds. A local improvement may therefore create wider problems: a new application may overlap existing applications and duplicate existing capabilities, a process change may conflict with established responsibilities, or a strategic objective may require information and infrastructure that do not yet exist.

Enterprise Architecture addresses this problem of coherence. It helps stakeholders understand how different parts of the enterprise depend on, support, or constrain one another. It does not assume that everything must be centrally designed or standardised. Rather, it provides enough shared structure to make dependencies visible, decisions explainable, and change manageable.

5.1.1 Architecture and Architecture Description

The architecture of an enterprise is the organisation of its relevant elements and relationships, together with the principles that guide their evolution. An architecture description is a representation of that architecture, created for a particular purpose. The two should not be confused: the enterprise has an architecture whether or not it has been documented, while models and diagrams are artefacts used to describe selected parts of it.

The objective of an architecture description is to support stakeholders understanding and taking decision, by either help to understand the **current state** (the “**as-is**”) and to visualize a future **target state** (the “**to-be**”). That can be very complex, meaning that in a specific moment no architecture description can show everything. Different stakeholders have different concerns. Senior management may be concerned with strategic outcomes and investment; operational managers with responsibilities and processes; information-system teams with applications and data; and infrastructure teams with deployment, security, and resilience.

EA therefore relies on multiple **views**, each selecting the information required to address a specific concern.

5.1.2 Current State, Target State, and Change

*EA commonly distinguishes between a **current state** and a **target state**. The current state describes relevant aspects of how the enterprise operates now. The target state describes a desired future arrangement.*

Neither should be treated as a complete photograph or prediction. They are selective descriptions, shaped by the concerns being analysed.

The difference between the two states defines a **change** problem. Moving towards the target may require new capabilities, revised responsibilities, redesigned processes, replacement or integration of applications, migration of information, and changes to technology. These changes are interdependent. EA helps identify those dependencies and supports the planning of coherent transitions rather than isolated interventions.

5.1.2.1 Enterprise Architecture as an Ongoing Practice

EA is not merely the production of a large diagram or a one-off blueprint. Enterprises continue to change because of new goals, regulation, technology, risks, costs, and stakeholder expectations. Architectural descriptions must therefore be reviewed and updated as understanding and circumstances evolve.

The value of EA lies less in documenting every detail than in supporting better reasoning. A useful architecture description should help stakeholders answer questions such as:

- What outcomes are we trying to achieve?
- Which capabilities, services, processes, information, applications, and technologies are involved?
- How do they depend on one another?
- What will be affected by a proposed change?
- Which decisions, constraints, and assumptions must be made explicit?

ArchiMate supports this practice by providing a shared language for expressing selected elements of an enterprise and the relationships between them. It enables different views of the same enterprise to remain connected within one coherent model.

5.2 <technique> ArchiMate as an Architecture Description Language

ArchiMate is a visual modelling language for describing enterprise architectures. It provides a shared set of concepts, relationships, and graphical conventions for representing how organisational, informational, technological, and strategic elements fit together.

The language is designed to support communication between stakeholders who may understand the enterprise from different perspectives. A manager may think in terms of goals, responsibilities, services, and capabilities, while a software architect may focus on applications, interfaces, and data. An infrastructure architect may instead be concerned with devices, networks, platforms, and deployment. ArchiMate enables these different concerns to be represented within a single coherent model.

5.2.1 A Language, Not a Method

ArchiMate defines how architectural knowledge can be expressed, but it does not prescribe how an Enterprise Architecture initiative must be conducted. It does not prescribe any method, does not define project phases, governance structures, decision rights, or a complete process for transforming an enterprise.

This distinction is important:

- EA is a practice concerned with organisational coherence and change.
- An EA framework, such as TOGAF, may provide methods, processes, and governance guidance.
- ArchiMate provides a language for describing the resulting architectural knowledge.

ArchiMate can therefore be used within different methods and organisational contexts. It may support a formal architecture programme, a focused systems analysis, the evaluation of a proposed change, or the communication of dependencies within a project.

5.2.2 Describing Current and Future Architectures

ArchiMate can be used to describe both the **current state** of an enterprise and possible **future states**, as well as to support transformations, as change between states:

- A **current-state model** represents relevant elements of the enterprise as they are understood to exist now. It may show, for example, which actors perform a process, which applications support it, which information is used, and which technologies host those applications.
- A **target-state model** represents a desired future arrangement. It may introduce new services,

responsibilities, capabilities, processes, applications, or technologies, while also showing which existing elements should be retained, changed, or removed.

- An architecture **transformation** may require one or more intermediate states between the current and target states. These transition states are useful when the target cannot be reached through a single change. For example, an old application may need to remain operational while data is migrated, and a replacement service is gradually introduced.

ArchiMate does not automatically determine whether a proposed target state is desirable or feasible. Its value lies in making architectural claims and dependencies explicit so that stakeholders can examine, question, and improve them.

5.2.3 Models, Elements, and Relationships

An ArchiMate model is composed of elements and relationships:

- Elements represent relevant architectural concepts, such as a stakeholder, goal, actor, service, process, application component, data object, or device.
- Relationships express how those concepts are connected. They may indicate, for example, that an actor performs a process, that a process realises a service, that an application supports behaviour, or that a process accesses information.

The relationships are not merely connecting lines. Each one expresses a semantic statement that should be understandable and defensible. A diagram containing many elements but poorly chosen relationships may look detailed while conveying little reliable architectural knowledge.

5.2.4 Selective Representation

ArchiMate is not intended to reproduce every detail of an enterprise. Architecture descriptions are necessarily selective. They include only the concepts and relationships relevant to the purpose of the analysis and the concerns being addressed.

A single ArchiMate model may support several views. One **view** may present the external services of a system, another may explain how those services are realised, and another may connect a proposed change to its strategic motivation. ArchiMate is therefore most useful when it is treated not as a drawing notation, but as a language for constructing consistent and purposeful architecture descriptions.

5.3 <example> A Change Across an Enterprise

Northbridge Mobility is a regional public transport operator responsible for bus and light-rail services. It provides passengers with journey information, sells and validates tickets, manages passenger accounts, communicates service disruptions, and handles enquiries and refund requests.

The following description distinguishes facts about the current architecture from identified problems, intended outcomes, and decisions that have not yet been made. This distinction is essential: an architecture model should represent stated or validated knowledge, not merely plausible assumptions introduced by the analyst.

5.3.1 Current-State Facts

Passengers can plan journeys using the Journey Planner application. This application obtains timetable and route information from the Transport Information System.

Tickets can be purchased through ticket machines, authorised retailers, and the Ticketing Platform. The Ticketing Platform records ticket purchases and communicates with an external payment provider when an electronic payment is required.

Passenger accounts are stored in the Customer Management System. These accounts contain passenger identification and contact information. The description does not establish that every ticket purchase is associated with a passenger account, since some tickets may be purchased anonymously.

Ticket validation is performed through validation equipment located on vehicles or at stations. The detailed architecture of that equipment is outside the scope of this example.

Northbridge Mobility publishes service disruption information through its existing passenger-information channels. Transport operations staff are responsible for recording the disruption information that is communicated to passengers. The specific application used internally by those staff is not identified in the available information.

Passengers submit refund requests through a form on the Public Website. Customer-service agents review the submitted information and manually create a corresponding case in the Finance Case System. The Finance Case System is then used to process and record the decision concerning the refund.

Customer-service agents also respond to passenger enquiries. They may need to consult information held in the Customer Management System, the Ticketing Platform, and the Finance Case System.

Some applications operate in Northbridge Mobility's data centre, while others are supplied or operated by external providers. The available information does not identify the hosting arrangement of each individual application.

5.3.2 Current-State Problems

Passenger-facing digital services are fragmented across several channels and applications. Journey planning, ticket purchasing, account management, service information, and

refund requests do not form a single coherent digital experience.

Passenger information is represented in more than one application. The current description does not establish a common passenger identifier or an authoritative source for all passenger-related information.

Refund requests require manual re-entry from the Public Website into the Finance Case System. This creates additional work and introduces a risk of delay, omission, or transcription error.

Customer-service agents may need to consult several applications to understand the history and current status of a passenger's request.

Several integrations were developed for specific exchanges between individual applications. The available information does not provide a complete catalogue of those interfaces or establish that they support the real-time exchanges required by future services.

5.3.3 Intended Change and Outcomes

Northbridge Mobility intends to provide passengers with a more coherent digital travel experience. The proposed Digital Travel Portal should provide a common point of access for:

- journey planning;
- ticket purchasing;
- passenger-account services;
- service-disruption information;
- refund requests;
- customer enquiries.

The intended outcomes are fewer abandoned ticket purchases, faster refund handling, less duplication of passenger information, more timely disruption communication, and improved passenger satisfaction.

5.3.4 Decisions Not Yet Made

The Digital Travel Portal is currently a proposal, not an implemented application. It has not yet been decided:

- which existing applications will be retained, integrated, replaced, or retired;
- which application will be authoritative for passenger information;
- whether existing point-to-point interfaces will be reused or replaced;
- where the portal and its supporting components will be hosted;
- how the change will be introduced without interrupting existing services.

These unresolved questions belong to the analysis and design of the target architecture. They must not be represented as current-state facts.

This Universe of Discourse provides a controlled basis for constructing an evidence-constrained current-state ArchiMate view and, later, alternative target-state views.

5.3.5 Architectural Description

This exercise introduces Architecture Description through a small ArchiMate case. The same enterprise is represented through different views, each addressing a specific stakeholder concern and including only the elements needed for that purpose.

5.3.5.1 Current-State Context and Services View

This first diagram is a Current-State Context and Services View that presents Northbridge Mobility from a black-box perspective.

It shows the enterprise as the provider of a set of externally visible services (journey planning, ticket purchasing, passenger account management, disruption information, refund request submission, and customer enquiry handling) and identifies the external actors that consume them.

The Passenger is served by all six services, while the Authorised Retailer is served only by the Ticket Purchasing Service. Internal processes, applications, and technology are deliberately omitted because they are not required to answer the concern addressed by this view: who interacts with Northbridge Mobility, and which services are visible to them?

The diagram therefore introduces Business Actors, Common Domain Services, and the Serving relationship while demonstrating that an architecture view should include only the elements needed for its declared purpose.

5.3.5.2 Refund Request Submission Service

These two diagrams are alternative **views of the same current-state architecture**, but they address different concerns and use different levels of abstraction.

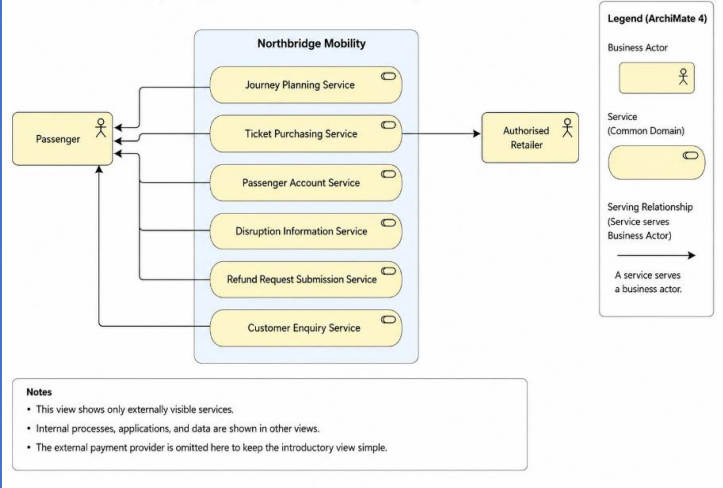
The first is broader and process-oriented: it follows refund handling from submission through review and case creation, explicitly showing the Customer-Service Agent's responsibilities, the two supporting applications, the information used, and the optional downstream decision step. It is therefore more useful for discussing the overall operational sequence and the division of work.

The second is narrower and more architectural: it focuses specifically on how the **Refund Request Submission Service** is realised, presenting only the elements necessary to explain that service and making the manual re-entry of refund information particularly visible. It is therefore cleaner for introducing ArchiMate concepts and relationships.

Neither view is more correct in isolation; each selects different elements from the same model because it serves a different modelling purpose.

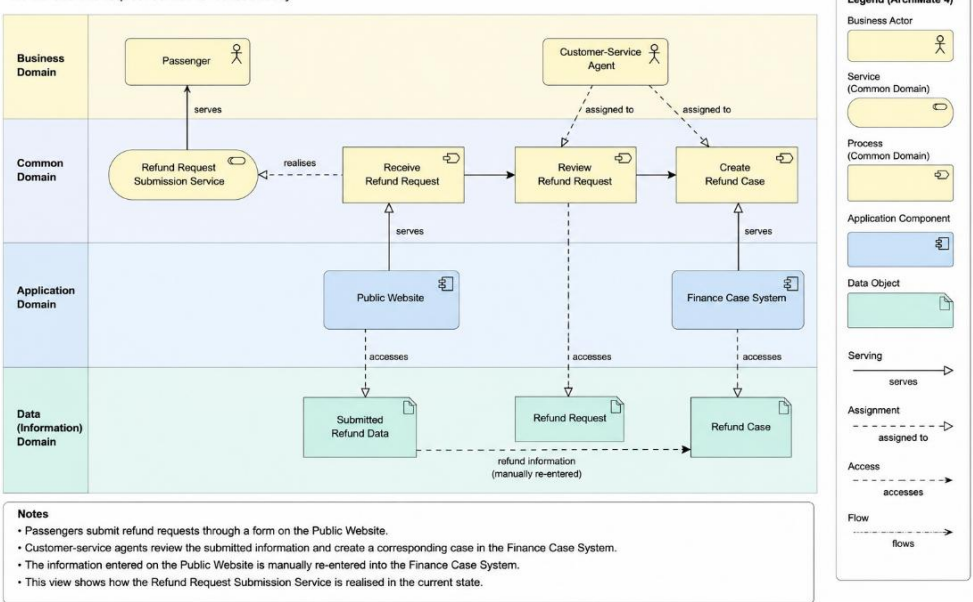
Northbridge Mobility – Current-State Context and Services View

Who interacts with Northbridge Mobility, and which services are visible externally



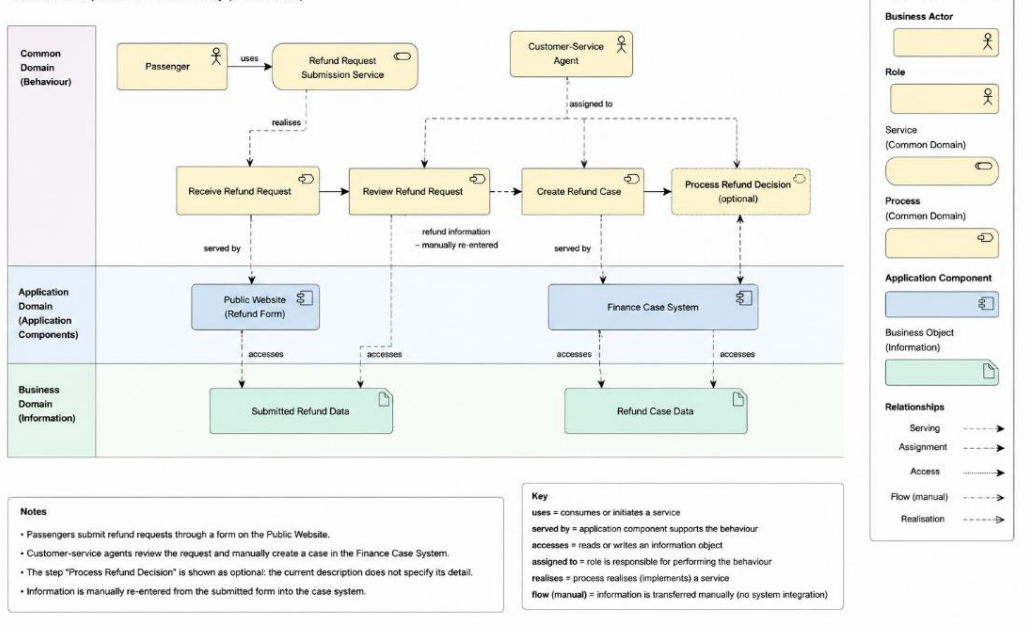
Northbridge Mobility – Current-State Refund Handling View

How the refund request service is realised today



Northbridge Mobility – Current-State Refund Handling View

How refund requests are handled today (current state)



5.4 <technique> The ArchiMate Language Structure

ArchiMate provides a structured vocabulary for describing different parts of an enterprise and the relationships between them.

5.4.1 The Domains of ArchiMate

Concepts are organised into **domains**, each concerned with a particular kind of architectural question. The value of these domains comes from connecting different concepts within one coherent description.

Four domains are defined as the core language:

- The **Common Domain** contains concepts that may be used across different architectural contexts. It includes generic concepts for behaviour, such as Service, Process, Function, and Event, as well as generic structural concepts such as Role, Collaboration, and Path. These concepts provide a shared grammar for describing what happens, who or what participates, and how behaviour is exposed or connected.
- The **Business Domain** describes organisational and operational aspects of an enterprise. It can represent business actors, interfaces, objects, products, and other concepts needed to understand how the enterprise delivers value and interacts with its environment.
- The **Application Domain** describes software-based structures and information handled by applications. It includes concepts such as Application Component, Application Interface, and Data Object. These concepts help explain how applications support or perform behaviour and how they manage or exchange data.
- The **Technology Domain** describes the technological and physical environment that supports applications and other enterprise behaviour. It includes nodes, devices, system software, technology interfaces, communication networks, artefacts, equipment, facilities, and materials.

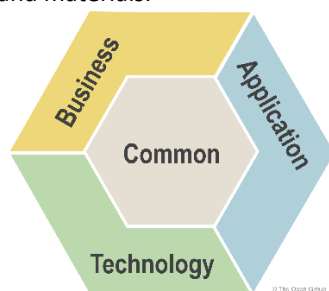


Figure 1: ArchiMate Core Language Domains

Other domains complement these core domains:

- The **Motivation Domain** describes why an architecture exists or must change. It represents stakeholders, drivers, assessments, goals, outcomes, principles, requirements, and values. It connects architectural choices to the concerns and intentions that justify them.
- The **Strategy Domain** describes how an enterprise intends to respond to its drivers and achieve its goals. It focuses on concepts such as capabilities, resources, and courses of action, helping to connect strategic direction to the architecture required to realise it.
- The **Implementation and Migration Domain** describes how architectural change is organised and introduced. It may represent work packages, deliverables, implementation events, and plateaux corresponding to relevant architectural states.

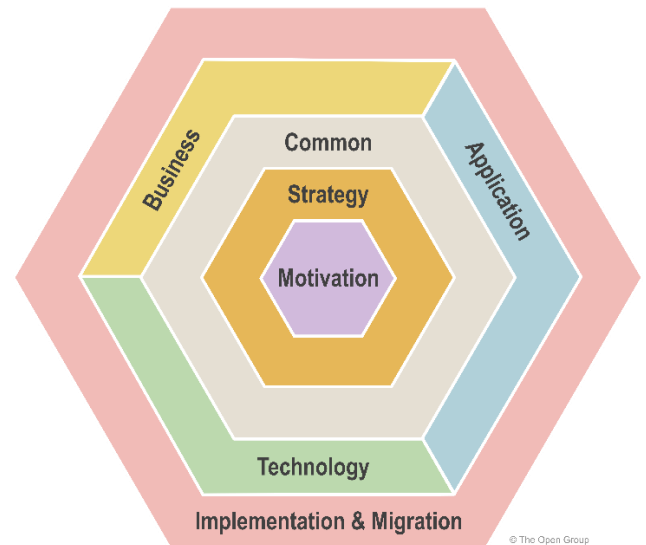


Figure 2: ArchiMate Full Language Domains

5.4.2 Aspects: Structure, Behaviour, and Objects

Across the language, many architectural concepts can be understood through three fundamental aspects:

- **Active structure aspect** represents entities capable of performing behaviour.
- **Behaviour aspect** represents what those entities do.
- **Passive structure aspect** represents things on which behaviour is performed, or which behaviour creates, uses, or changes.

For example, a role may be assigned to a process, and that process may access an object. The role is active structure, the process is behaviour, and the object is passive structure. This recurring pattern helps students interpret concepts across different domains without memorising each concept in isolation.

ArchiMate also distinguishes between internal and externally visible perspectives. Internal behaviour explains how something operates, while a service represents behaviour exposed to an environment. Similarly, an interface provides a point through which an active structure element makes services available.

5.4.3 A Connected Language

The domains should not be read as a fixed sequence or as separate organisational silos. A model may connect a goal to a capability, a capability to a process, a process to an application, and an application to supporting technology. Implementation concepts may then describe how the current architecture is transformed into a target architecture.

The language structure therefore supports a central purpose of Enterprise Architecture: tracing relationships from motivation and strategy through operational and technological reality to planned change. A useful ArchiMate model selects only the domains and concepts needed to address its declared concerns.

Images from:
<https://pubs.opengroup.org/architecture/archimate4-doc/chap03.html>

5.5 <concept> The Common Domain

Organisations, software tools, and technology infrastructures are very different kinds of systems. Nevertheless, architects often need to ask similar questions about each of them:

- What performs the behaviour?
- What behaviour takes place?
- What is offered to the environment?
- What information or other objects are used?
- How do different parts depend on one another?

ArchiMate addresses this similarity through a common conceptual structure, which can be reused in the other specialized domains. The Business, Application, and Technology Domains use specialised concepts, but those concepts follow the same underlying pattern. Learning that pattern first makes the language easier to understand: instead of memorising three unrelated vocabularies, we can recognise variations of a common architectural grammar.

5.5.1 Active Structure: Who or What Acts?

An **Active Structure** element represents something capable of performing behaviour. Depending on the domain, this may be an organisation, a person acting in a role, an application component, or a technology resource.

For example:

- a Customer-Service Agent may review a refund request;
- a Ticketing Platform may record a ticket purchase;
- a server may execute software.

These elements are not the activities themselves. They are the entities to which behaviour is assigned. A useful reading pattern is:

Active structure performs behaviour.

This distinction prevents a common modelling error: representing an organisational unit, application, or device as though it were an activity.

5.5.2 Behaviour: What Happens?

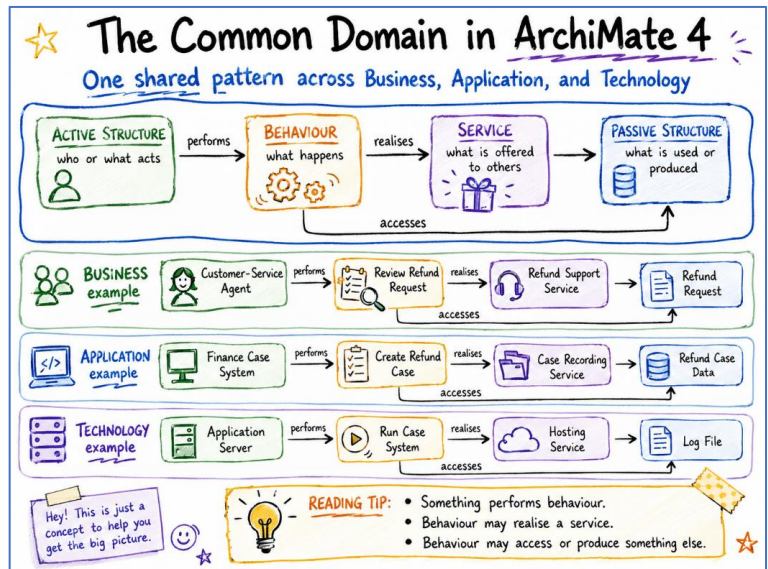
A **Behaviour** element represents something that is done. ArchiMate distinguishes several useful perspectives on behaviour:

- A **Service** presents behaviour as it is visible and valuable to its environment. It answers the outside-in question: *what does this system offer to others?* A service deliberately hides the internal work required to provide it, making it suitable for black-box views.
- A **Process** represents behaviour with a progression through time. It has a recognisable beginning and end and may contain an ordered sequence of activities.
- A **Function**, by contrast, groups behaviour according to a responsibility, competence, or required capability; it does not necessarily imply a temporal sequence.

A process or function may **realise** a service. This states that the internal behaviour provides the externally visible behaviour. For example, *Review Refund Request* and *Create Refund Case* may contribute to the realisation of a refund-handling service.

5.5.3 Passive Structure: What Is Used or Produced?

A **Passive Structure** element represents something upon which behaviour is performed, but which does not itself



perform behaviour. Examples include information, documents, messages, money, materials, and other objects. Behaviour may **access** passive structure by reading it, writing it, or both. A refund-handling process may therefore read a *Refund Request* and create or update a *Refund Case*. The relationship should express a meaningful fact, rather than merely connecting two nearby boxes.

5.5.4 One Pattern, Several Domains <AQUI>

ArchiMate 4 organises its core architectural vocabulary into four domains:

- the **Common Domain** provides concepts that can be used across the architecture, such as **Process, Function, Service, and Event**;
- the **Business Domain** describes organisations, people, responsibilities, and business information;
- the **Application Domain** describes software components and data;
- the **Technology Domain** describes technological and physical resources, infrastructure, and technological objects.

This differs fundamentally from earlier ArchiMate versions: Behaviour is no longer classified according to the domain of its performer. The same Process or Function may be performed by business, application, or technology active structure elements, or jointly by elements from several domains. The performer is identified through the **Assignment** relationship.

The recurring sentence therefore becomes:

Something performs behaviour, which may realise a service and access something that is acted upon.

This remains one of the most useful keys to reading ArchiMate diagrams. First identify the architectural role of each element: **active structure, behaviour, or passive structure**. Then determine which structural domains are involved. A Process is not inherently a business, application, or technology process; its architectural context follows from its performers, the objects it accesses, its relationships, and, where necessary, an explicit specialisation (**NOTE: this generic concept of "Process" aligns closely with BPMN**).

This separation allows a single behaviour element to represent, for example, a process performed jointly by a person and an application, without artificially duplicating it across domains.

5.6 <technique> Services, Processes, and Functions

ArchiMate offers several concepts for representing behaviour.

Three of the most important are **Service**, **Process**, and **Function**. They may initially appear similar because all describe something that happens.

However, they answer different architectural questions:

- **Service:** What behaviour is offered to others?
- **Process:** How does behaviour progress from beginning to end?
- **Function:** What continuing responsibility or category of behaviour exists?

Understanding these distinctions is more important than memorising their symbols.

5.6.1 Service: Behaviour Seen from Outside

A **Service** represents behaviour as it is made available to its environment. It describes what an organisation, application, or technology system offers to a user or another system, while hiding how that behaviour is internally performed.

A passenger may use a **Refund Request Submission Service**. From the passenger's perspective, the important fact is that a refund request can be submitted. The passenger does not need to know which employees, applications, or internal activities make this possible.

A service therefore supports a **black-box view**:

We describe what is offered without opening the provider to show its *internal operation*.

Services are particularly useful in context views, where the modeller wants to show how an enterprise or system interacts with its environment. In ArchiMate, a Service is external behaviour: behaviour considered from the perspective of those who use it.

When naming a service, use wording that describes something meaningful to its consumer: "Journey Planning Service", "Ticket Purchasing Service", "Passenger Account Service", "Refund Request Submission Service".

Avoid names that merely identify the internal department or application providing the service.

5.6.2 Process: Behaviour Through Time

A **Process** represents internal behaviour that progresses through time. It normally has a recognisable beginning and end and may contain an ordered sequence of more detailed behaviours.

For example, refund handling might include "Receive Refund Request", "Review Refund Request", "Create Refund Case". These activities describe how work develops. Their order matters: a request must be received before it can be reviewed, and it is reviewed before the corresponding case is created.

Processes are therefore useful when the concern involves:

- sequence;
- handovers;
- waiting or delay;
- responsibilities at different stages;
- information passing between activities;
- the path from an initiating event to a result.

ArchiMate can show a high-level process, but detailed operational workflows are normally better described later with BPMN.

5.6.3 Function: Behaviour Organised by a Criterion

A **Function** groups behaviour according to a set of criteria, such as responsibility, competence, purpose, or any other organisational criterion. Examples might include "Customer Support", "Ticket Management", "Passenger Information Management", "Payment Management", etc.

Customer Support, for example, continues to exist even when no particular enquiry is being processed. It groups related behaviour because that behaviour belongs to the same organisational responsibility.

Unlike a process, it does not necessarily describe a sequence with a beginning and end, but an enduring area of behaviour:

- A process asks: What happens next?
- A function asks: What kind of work is this, or who is responsible for it?

The distinction is useful, but not always perfectly sharp. A modeller should choose the concept that best communicates the intended perspective rather than searching for a supposedly unique classification for every activity.

5.6.4 Connecting the Concepts

A Process or Function may **realise** a Service. Realisation connects an externally visible promise to the internal behaviour that makes it possible.

For example:

The *Receive Refund Request* process realises the *Refund Request Submission Service*.

Read this as an architectural statement:

The service offered to the passenger exists because this internal process provides it.

The modeller should not automatically show all three concepts in every view. A context view may contain only services. A process-oriented view may open one service and show its internal realisation, or it even show only processes and no services. A functional view may organise the enterprise around stable areas of responsibility.

The choice depends on the concern being addressed:

- use a **Service** to describe externally available behaviour;
- use a **Process** for ordered behaviour through time;
- use a **Function** to describe behaviour grouped by criterion, such as responsibility or purpose.

5.7 <technique> The Business Domain

The **Business Domain** is used to describe the enterprise as a human and organisational system. It helps us model:

- the people, teams, and organisations involved;
- the behaviour performed by people;
- the services offered to customers or other parties;
- the information and objects that matter to the organisation.

In ArchiMate, the Business Domain follows the general Core pattern introduced earlier: an active structure element performs behaviour, that behaviour may provide a service, and the behaviour may access passive structure. The Business Domain gives this pattern a more precise organisational meaning.

For introductory modelling, three kinds of elements are especially important:

- **Business Actor**, representing the organisational system;
- **Process, Function, and Service**, representing behaviour (from core domain);
- **Business Object**, representing conceptual information or other objects used by the business.

5.7.1 Business Actor: The Organisational System

A **Business Actor** represents an organisational entity capable of performing behaviour. It may represent: one person; a team or department; an organisation; an external organisational party.

Examples in the Northbridge Mobility case include: Northbridge Mobility; Passenger; Authorised Retailer; External Payment Provider.

The same concept can therefore be used at different scales. A modeller might represent the whole transport operator in a context view and then represent particular organisational units in a more detailed view.

A Business Actor should answer the question:

Who or what organisational entity participates in this situation?

It should not be used for an activity, responsibility, application, or item of information. The Business Actor is the principal active structure element for modelling business systems and notes that it may represent an individual, a team, or a company.

5.7.2 Process: Behaviour Through Time

A **Process** represents behaviour organised as a progression through time. It normally has a recognisable starting point, develops through one or more steps, and contributes to an intended result.

For example: Receive Refund Request → Review Refund Request → Create Refund Case

We must use a Process when the order or progression of work matters, but it is not the same as a detailed workflow. ArchiMate is useful to provide an architectural overview, but when the detailed sequence of activities, decisions,

exceptions, and messages becomes the main concern, the BPMN language will usually be more suitable.

5.7.3 Function: Behaviour Grouped by Purpose

A **Function** represents behaviour grouped according to a continuing purpose, competence, or area of responsibility.

Examples in the Northbridge Mobility case include: Customer Support; Ticket Management; Passenger Information Management; Refund Management.

Unlike a Process, a Function does not primarily describe a sequence. It describes a stable category of work. A useful distinction is:

A Process emphasises progression.

A Function emphasises purpose or responsibility.

5.7.4 Service: Behaviour Offered to Others

A **Service** represents behaviour made available to another actor, whether inside or outside the enterprise. Examples in the Northbridge Mobility case include: Journey Planning Service; Ticket Purchasing Service; Refund Request Submission Service; Customer Enquiry Service.

A Service presents the enterprise from the consumer's perspective. It describes what is offered while hiding the internal work required to provide it.

Internal business behaviour may realise a Service. For example: The Receive Refund Request Process realises the Refund Request Submission Service.

This allows one view to show the service as a black box and another view to show how that service is internally provided.

5.7.5 Business Object: What the Work Concerns

A **Business Object** represents something meaningful from the organisation's perspective. Examples in the Northbridge Mobility case include: Ticket; Passenger Account; Refund Request; Refund Case; Disruption Notice.

A Business Object is conceptual. It represents something that people in the organisation recognise and discuss, not necessarily a particular file or database record. Business behaviour may access it by reading, creating, or changing it. The Open Group distinguishes this conceptual information from application-specific data structures, which are represented in the Application Domain.

5.7.6 Reading a Simple Business View

A basic Business Domain view can often be read as a small set of statements:

*A Business Actor performs a Process or Function.
That behaviour may realise a Service.
The service is made available to another Business Actor.
The behaviour accesses one or more Business Objects.*

For beginners, this is enough to construct useful views. The goal is not to reproduce the entire organisation, but to select the smallest number of elements needed to explain who participates, what the enterprise does, what it offers, and what its work concerns.

5.8 <technique> The Application Domain

The **Application Domain** adds the concepts needed to identify software structure and application data. It describes the software systems used by an enterprise, how they cooperate, which services they make available, and which data they process.

For introductory modelling, the most useful elements are:

- **Application Component**;
- **Application Interface**;
- **Data Object**;
- the Common Domain elements **Service**, **Process**, **Function**, and **Event**; a **Role** or **Collaboration** may also be used where responsibility or cooperation between application elements needs to be made explicit.

5.8.1 Application Component: Which Software System?

An **Application Component** represents a modular part of a software system. At the level used in this course, it will normally represent a recognisable application or a major independently managed part of one.

Examples from Northbridge Mobility include: Journey Planner; Ticketing Platform; Customer Management System; Finance Case System; Public Website.

An Application Component answers the question:

Which software system or software module performs this behaviour?

It should not be confused with the behaviour itself. The **Finance Case System** is an Application Component; **recording a refund case** is behaviour performed by that component.

A component may also be decomposed into smaller Application Components when that additional detail is useful. However, architecture views should not reproduce the complete internal design of an application. Detailed software design will normally be described later with UML or another specialised language.

5.8.2 Service: What Does the Application Offer?

A **Service** represents behaviour made available to its environment. In an Application Domain view, the provider of the service is normally an Application Component or a Collaboration of application elements.

Examples include: Journey Information Service; Ticket Purchase Recording Service; Passenger Account Management Service; Refund Case Recording Service.

The service describes what another actor, process, or application can obtain without showing how the provider performs it internally.

For example: The **Finance Case System** provides the **Refund Case Recording Service**.

Because **Service** belongs to the Common Domain, the element itself is not labelled "Application Service". Its application character follows from the architecture context and from the Application Component that provides or realises it. ArchiMate deliberately uses one common set of behaviour elements across the Core domains.

5.8.3 Process and Function: What Does the Software Do?

A **Process** represents behaviour that progresses through time. In an application context, it may describe an

automated sequence such as: **Receive Submitted Data** → **Validate Data** → **Record Refund Case**

A **Function** groups behaviour according to a stable application responsibility, such as: **Passenger Account Management**; **Ticket Recording**; **Refund Calculation**; **Journey Information Provision**.

The distinction remains the same across domains:

A Process emphasises progression.

A Function emphasises purpose or responsibility.

An Application Component may be assigned to a Process or Function. That internal behaviour may realise a Service.

For example: The **Finance Case System** performs the **Record Refund Case Process**, which realises the **Refund Case Recording Service**.

Not every view needs to expose this internal behaviour. A context view may show only Application Components and the Services they provide.

5.8.4 Data Object: Which Data Is Processed?

A **Data Object** represents data suitable for automated processing.

Examples include: **Timetable Data**; **Passenger Account Data**; **Ticket Purchase Data**; **Submitted Refund Data**; **Refund Case Data**.

A Process or Function may **access** a Data Object by reading, creating, or modifying it.

The distinction between a Business Object and a Data Object is important. A **Refund Request** may be meaningful to the organisation as a Business Object. Its electronic representation within a particular application may be modelled as **Submitted Refund Data** or **Refund Case Data**.

Several Data Objects may therefore represent the same organisational concept in different applications. This can reveal duplication, conversion, or inconsistencies between systems.

5.8.5 Application Interface: How Is the Service Accessed?

An **Application Interface** represents a point through which a Service is made available.

Examples might include: **passenger web interface**; **staff interface**; **mobile application interface**; **system integration API**.

An interface must not be added simply because every application must technically have one; it only must be expressed when the method of access is relevant to the concern of the view.

5.8.6 Reading a Simple Application View

A basic Application Domain view can often be read as:

An Application Component performs a Process or Function.

That behaviour realises a Service.

The Service serves business behaviour or another application.

The behaviour accesses Data Objects.

An Application Interface may expose the Service.

The aim is not to inventory every piece of software. It is to show how applications support the enterprise, what they provide, and which important dependencies or data responsibilities must be understood.

5.9 <technique> The Technology Domain

The **Technology Domain** describes the technological systems that support applications and other enterprise behaviour. It may represent computing devices; operating systems and middleware; communication networks; storage and infrastructure services; physical equipment; deployable software and data artefacts.

The domain should not be treated as a complete inventory of hardware and software. Its purpose is architectural: to reveal the technical structure, dependencies, deployment choices, and services that matter to the concern being studied.

For introductory modelling, the most useful concepts are **Node**; **Device**; **System Software**; **Communication Network**; **Service**; **Artifact**.

5.9.1 Node: A General Technological System

A **Node** represents a computational or physical resource that hosts, executes, or processes other elements. It is the most general active structure concept in the Technology Domain. A Node may represent a server environment; virtual machine; cloud execution environment; processing cluster; another identifiable technical resource.

For example, a **Customer Systems Hosting Environment** might be represented as a Node when its precise hardware and software structure is not relevant to the view.

A Node can also be decomposed into more specialised technological elements. This makes it useful when the architecture is known only at a high level or when the modeller deliberately wants to hide implementation detail.

5.9.2 Device: The Hardware System

A **Device** represents a physical or virtual hardware resource on which software can be deployed or data can be stored.

Examples include a physical server; virtual server; storage appliance; firewall; network switch; ticket-validation terminal.

A Device does not have to be an ordinary computer. It can represent any hardware system capable of supporting software execution, communication, or storage.

The level of detail should follow the purpose of the view. A Device named **Application Server** may be sufficient; showing processors, memory modules, and disks would normally be unnecessary in an enterprise architecture model.

5.9.3 System Software: The Supporting Software Environment

System Software represents software that provides or contributes to the environment in which applications execute.

Examples include an operating system; database-management system; message broker; application server; container platform; virtualisation platform.

This concept is distinct from an **Application Component**. An Application Component directly supports business activity or implements business rules. System Software provides the technical environment needed for applications to operate.

For example: The *Finance Case System* Application Component is deployed on an *Application Server Platform* represented as System Software.

5.9.4 Communication Network: How Are Systems Connected?

A **Communication Network** represents the technological structure through which devices, nodes, or other technical systems communicate.

Examples include an internal data-center network; public internet connection; secure organisational network; mobile communications network.

This element is useful when connectivity matters to the concern of the view. It is usually unnecessary to model every cable, port, protocol, or network route.

A Communication Network may connect Devices or Nodes, helping to show where technical dependencies, communication paths, or security boundaries exist.

5.9.5 Service: What Does the Infrastructure Provide?

A **Service** applied at the technology domain represents technological behaviour made available to applications or other consumers.

Examples include a Database Hosting Service; Application Execution Service; Message Delivery Service; Data Storage Service; Network Connectivity Service.

A (technology) service gives an external view of the infrastructure. It describes what the technical environment provides without exposing every internal mechanism.

For example: The *Database-Management System* provides a *Database Service* that supports the *Finance Case System*.

5.9.6 Artifact: What Is Deployed or Stored?

An **Artifact** represents a concrete item used or produced in a technological environment.

Examples include an executable software package; configuration file; database schema; deployment package; stored file; log file.

An Artifact is more concrete than a Business Object or Data Object. A Business Object represents an organisational concept; a Data Object represents application-structured data; an Artifact represents a deployable or physically stored form.

An Application Component may be realised by an Artifact, which may then be deployed on a Node.

5.9.7 Reading a Simple Technology View

A basic Technology Domain view may be read as:

A Device or Node hosts System Software.

The System Software provides a Service.

That service supports an Application Component.

An Artifact realises software and is deployed on a Node.

A Communication Network connects technological systems.

For beginners, the main objective is to connect applications to the infrastructure on which they depend. Include only technical elements that influence architectural decisions, risks, responsibilities, or constraints. A Technology Domain view should clarify the architecture, not become a detailed equipment catalogue.

5.10 <further> Motivation and Strategy

The Core Domain describes what the enterprise is made of and how it operates. The **Motivation Domain** and **Strategy Domain** address different questions:

*Why should the architecture change?
What direction should the enterprise take?*

The Motivation Domain represents the reasons that guide the design or change of an architecture. The Strategy Domain represents the enterprise's strategic direction, how it intends to create value, and the capabilities needed to do so. For introductory modelling, the most useful Motivation concepts are **Stakeholder**; **Driver**; **Assessment**; **Goal**; **Outcome**; **Requirement**. The most useful Strategy concepts are **Capability**; **Resource**; **Course of Action**.

5.10.1 Stakeholder: Who Has an Interest?

A **Stakeholder** represents a person, group, or organisation with an interest in the architecture and its effects. Examples include passengers; Northbridge Mobility management; customer-service staff; transport operations staff; regulators. A Stakeholder is not necessarily an Actor performing behaviour in the model. The concept emphasises interest, influence, expectations, or consequences. For example, passengers are concerned with convenient and reliable services; management is concerned with cost, performance, and strategic outcomes; customer-service staff are concerned with workable procedures and suitable application support.

5.10.2 Driver: What Creates Pressure for Change?

A **Driver** represents something that motivates the enterprise to define goals and consider change. Examples include fragmented digital services; changing passenger expectations; regulatory obligations; increasing operating costs; obsolete technology. A Driver does not yet state whether the situation is good or bad. It identifies a factor that matters. For example, **Changing Passenger Expectations** may influence Northbridge Mobility, but its architectural significance still needs to be interpreted.

5.10.3 Assessment: What Does the Driver Mean Here?

An **Assessment** represents the result of analysing a Driver in the context of the enterprise. Examples include the passenger experience is inconsistent; refund handling is slow and error-prone; passenger information is difficult to consolidate; existing integration may not support timely updates. The distinction is useful:

Driver: a relevant factor exists.

Assessment: this is what that factor means for this enterprise.

An assessment should be supported by available evidence. It must not present an analyst's unsupported assumption as an established fact.

5.10.4 Goal and Outcome: What Should Be Achieved?

A **Goal** represents a desired state or direction. It expresses what the enterprise intends to achieve. Examples include provide a coherent digital travel experience; reduce unnecessary manual work; improve access to passenger information.

An **Outcome** represents a result produced by the architecture or by a course of action. Examples include faster

refund handling; less duplication of passenger information; more timely disruption communication; improved passenger satisfaction.

Goals are generally broader and more directional. Outcomes make the expected result more concrete and may later be measured.

5.10.5 Requirement: What Must the Architecture Provide?

A **Requirement** represents something the architecture must satisfy. Examples include provide a common digital point of access; support online submission of refund information; avoid manual re-entry of submitted data; preserve continuity of existing passenger services.

A Requirement should be more precise than a Goal. The goal explains the intended direction; the requirement begins to constrain the architecture that will be designed.

A Requirement is not automatically an application, process, or technical component. For example, **Provide a Common Digital Point of Access** does not yet prove that a particular portal, product, or implementation has been selected.

5.10.6 Capability: What Must the Enterprise Be Able to Do?

A **Capability** represents an ability possessed by an enterprise, person, or system. Examples include Digital Passenger Interaction; Refund Management; Passenger Information Management; Service Disruption Communication. Capabilities describe abilities rather than organisational units, applications, or processes. A capability may be realised through a combination of people, behaviour, information, applications, and technology.

5.10.7 Course of Action: Which Strategic Approach Is Chosen?

A **Course of Action** represents an approach or direction adopted to achieve goals and intended outcomes. Examples include consolidate digital access channels; progressively integrate passenger information; automate the transfer of refund information; modernise integration between existing applications. A Course of Action should not be confused with a detailed implementation plan. It states the selected strategic approach, while projects, work packages, and transition stages are addressed later.

5.10.8 Reading a Motivation and Strategy View

A simple view can often be read as:

A Stakeholder is concerned with one or more Drivers.

Drivers lead to Assessments of the current situation.

Assessments motivate Goals and intended Outcomes.

Requirements express what the architecture must satisfy.

Courses of Action describe the chosen strategic response.

Capabilities identify what the enterprise must be able to do.

These elements connect purpose to architecture. They prevent a proposed system from appearing without explanation and help distinguish an identified problem, an intended result, a requirement, and an architectural solution. **ArchiMate 4 treats Motivation, Strategy, Core, and Implementation and Migration as complementary architecture domains**, each contributing a different part of the overall explanation.

5.11 <technique> Relationships as Architectural Statements

The meaning of an ArchiMate model comes largely from the **relationships** between those elements. A relationship should be read as a statement:

This element is connected to that element in this particular architectural way.

For example, placing a **Finance Case System** beside a **Record Refund Case** process does not explain their connection. Adding an appropriate relationship state that the system performs that behaviour. These relationships cover many common situations: **Composition**; **Assignment**; **Realisation**; **Serving**; **Access**; **Triggering**; **Flow**.

5.11.1 Composition: What Is Part of What?

A **Composition** relationship represents that one element consists of one or more other elements. Examples include a **Customer Support Function** composed of **Refund Management** and **Enquiry Handling**; a **Ticketing Platform** composed of several application components; a **refund process** composed of smaller refund processes.

Read the relationship as:

The whole is composed of the part.

This normally represents a strong whole-part relationship. The part belongs to that whole in the model and should not casually be presented as part of several unrelated wholes. Composition also can be shown through visual nesting in suitable situations.

5.11.2 Assignment: Who or What Performs the Behaviour?

An **Assignment** relationship connects active structure to behaviour or responsibility. Examples include a **Customer-Service Agent** assigned to **Review Refund Request**; a **Finance Case System** assigned to **Record Refund Case**; a **server environment** assigned to **application execution behaviour**.

Read the relationship as:

The active structure performs or is responsible for the behaviour.

Assignment is one of the most important relationships for distinguishing a system from what that system does. The Open Group's introductory material describes active structure as being assigned to its behaviour.

5.11.3 Realisation: What Makes the More Abstract Element Concrete?

A **Realisation** relationship connects a more concrete element to something more abstract that it implements or provides. A common use is connecting internal behaviour to an externally visible Service: **The Receive Refund Request Process** realises the **Refund Request Submission Service**.

Read this as:

The internal behaviour makes the service available.

Realisation is therefore useful when moving from a black-box view to a white-box explanation. The Service states what is offered; the Process or Function explains the behaviour through which it is provided.

5.11.4 Serving: Who Benefits from or Uses What Is Provided?

A **Serving** relationship represents that one element provides functionality to another. Examples include a **Refund Request Submission Service** serving a **Passenger**; a **Refund Case Recording Service** serving a **refund-handling Process**; an

infrastructure service serving an **Application Component**.

Read the relationship as:

The source provides something useful to the target.

Direction matters. The arrow runs from the provider towards the consumer. A useful test is to complete the sentence: "The source serves the target."

5.11.5 Access: Which Information or Object Is Used?

An **Access** relationship connects behaviour or active structure to passive structure. Examples include **Review Refund Request** reading a **Refund Request**; **Create Refund Case** creating a **Refund Case**; **Passenger Account Management** updating **Passenger Account Data**. Read it as:

The behaviour reads, creates, or changes the object.

Where the notation or modelling tool allows it, the modeller may distinguish read access, write access, or read-and-write access. This additional precision should be used only when it is known and relevant.

5.11.6 Triggering: What Causes Behaviour to Begin or Continue?

A **Triggering** relationship represents a causal or temporal dependency between behaviours or events. Examples include **Refund Request Received** triggering **Review Refund Request**; **Review Refund Request** triggering **Create Refund Case**; **Service Disruption Reported** triggering **Publish Disruption Information**. Read it as:

The source initiates or causes the target to occur.

Triggering is stronger than merely saying that two behaviours are related. It asserts an ordering or causal dependency.

5.11.7 Flow: What Is Transferred?

A **Flow** relationship represents the transfer of information, material, money, or another item from one element to another. Examples include **refund information** flowing from the **Public Website** to a **Customer-Service Agent**; **payment information** flowing to an **External Payment Provider**; **ticket information** flowing from the **Ticketing Platform** to **validation equipment**. A Flow should preferably be labelled with what is transferred: **refund information**; **payment authorisation request**; **ticket validation data**. Without a label, the reader may understand that something moves but not what it is.

5.11.8 Connect Only What You Can Explain

A relationship should never be added merely to make a diagram appear connected. Before drawing it, complete a sentence using its meaning:

*The agent is **assigned** to the review process.*

*The process **realises** the service.*

*The service **serves** the passenger.*

*The process **accesses** the refund request.*

*The event **triggers** the process.*

*Refund information **flows** to the next participant.*

When the sentence is unclear, the relationship is probably unclear as well.

The modeller must also avoid treating all relationships as generic arrows. Different relationships make different claims, and reversing a relationship may completely change its meaning. A good ArchiMate view should therefore be readable not only as a picture, but as a small collection of precise architectural statements.

5.12 <example> Constructing an End-to-End Architecture View

An **end-to-end architecture view** connects several parts of the enterprise around one meaningful concern. It may show why a change is needed, which business behaviour is affected, how applications support that behaviour, and which technology enables those applications. “End-to-end” does not mean “show everything”. A useful view follows one architectural thread across domains while omitting unrelated detail. ArchiMate supports this because its domains share a consistent structure and can be connected through relationships such as Serving, Realisation, Assignment, and Access.

Consider the problem of manually re-entering refund information at Northbridge Mobility.

5.12.1 Begin with the Concern

The concern is: **How is refund handling currently supported, and where does manual work occur?**

This concern determines what belongs in the view. Journey planning, ticket validation, disruption communication, and unrelated infrastructure should be omitted. The view may include three connected perspectives: business work; application support; relevant information.

5.12.2 Step 1: Identify the Business Behaviour

Begin with the work performed by the organisation. Relevant elements might include the Business Role **Customer-Service Agent**; the Business Processes **Receive Refund Request**, **Review Refund Request**, and **Create Refund Case**; the Business Objects **Refund Request** and **Refund Case**.

The Role is assigned to the review and case-creation behaviour. The processes access the relevant Business Objects.

These elements allow the view to state: A Customer-Service Agent reviews a Refund Request and creates a Refund Case. At this point, the model describes business work without yet showing its software support.

5.12.3 Step 2: Add the Applications That Support the Work

Add only the applications explicitly involved in this scenario:

- The Public Website supports the submission or receipt of refund information. The Finance Case System supports the creation and processing of the corresponding refund case.
- The Application Domain is intended to show applications that directly support business activity, while technical platforms such as operating systems and middleware belong in the Technology Domain.

The view can now express: The Public Website supports receipt of the refund request; the Finance Case System supports creation of the refund case.

Do not introduce an integration component unless there is evidence that one exists.

5.12.4 Step 3: Represent the Information

At the business level, the relevant concepts are **Refund Request** and **Refund Case**. At the application level, their automated representations might be shown as **Submitted Refund Data** and **Refund Case Data**.

The business and application representations should not be treated as identical. A Business Object represents something meaningful to the organisation; a Data Object represents data structured for automated processing.

The view may therefore distinguish: the request understood by the organisation; the data captured by the website; the case data recorded in the finance system.

This distinction helps reveal transformations, duplication, and gaps between systems.

5.12.5 Step 4: Make the Problem Visible

The important architectural fact is that information is manually transferred from the Public Website to the Finance Case System. A Flow relationship may be added between the relevant elements and labelled: **refund information - manually re-entered**.

The label matters. An unlabelled arrow would show that something moves but would not reveal what moves or why the relationship is significant. The model should not replace this fact with an invented direct application integration. The absence of an established automated transfer is precisely the problem being represented.

5.12.6 Step 5: Check the Architectural Story

Read the completed view as a sequence of statements:

*A Passenger submits a Refund Request.
The Public Website captures Submitted Refund Data.
A Customer-Service Agent reviews the request.
Refund information is manually re-entered.
The Finance Case System records Refund Case Data.
A Refund Case is created and processed.*

Every element and relationship should contribute to this story. Anything that cannot be explained in one of these statements should be reconsidered.

5.12.7 Stop at the Right Depth

The completed view should not automatically include servers, networks, databases, interfaces, security controls, or proposed future applications. These may be valid in other views, but they do not answer the present concern.

An end-to-end view is successful when it connects the necessary domains while remaining understandable to its intended audience. The modeller should follow one architectural thread, include only supported facts, and stop when the concern has been answered.

5.13 <heuristic> Choosing the Right Concept

Choosing an ArchiMate concept is an **analytical decision**. The modeller is not merely attaching a symbol to a word found in the case description. The modeller is deciding what architectural meaning that word should have in the view.

A useful first question is: **What kind of thing am I trying to represent?**

In the ArchiMate Core language, most elements can first be understood as **active structure**, **behaviour**, **service**, or **passive structure**. The generic metamodel helps explain these categories, although its generic elements are not themselves used in an actual ArchiMate model.

5.13.1 Is It Something That Acts?

Choose an **active structure** concept when the element represents something capable of performing behaviour.

Examples include a person or organisation represented as a **Business Actor**; a responsibility represented as a **Role**; a software system represented as an **Application Component**; a technological execution resource represented as a **Node, Device, or System Software**.

Ask: **Can this element reasonably be said to perform, host, or be responsible for behaviour?**

For example, the **Finance Case System** is an Application Component because it is a software system. **Record Refund Case** is not a component because it describes what the system does.

5.13.2 Actor or Role?

Use a **Business Actor** when the concern is the identifiable participant. Use a **Role** when the concern is the responsibility or capacity in which a participant acts.

Examples: **Maria Silva** or **Customer Services Department** may be Actors; **Customer-Service Agent** or **Refund Approver** may be Roles.

Use the test: **Could another business actor assume this responsibility?**

When the answer is yes, a Role is probably appropriate. One actor may assume several roles, and several actors may assume the same role.

5.13.3 Is It Behaviour?

Choose a behaviour concept when the element represents something that happens or is performed. Use a **Process** when progression through time matters; use a **Function** when behaviour is grouped by continuing purpose or responsibility; use an **Event** when representing an occurrence rather than work; use a **Service** when behaviour is presented as available to a consumer. Examples: **Review Refund Request** is a Process when it is part of an ordered handling sequence; **Refund Management** is a Function when it represents a stable area of responsibility; **Refund Request Received** is an Event; **Refund Request Submission Service** is behaviour offered to the passenger.

A quick test is:

- **Process:** What happens next?
- **Function:** What kind of work is this?
- **Event:** What occurred?
- **Service:** What is made available to someone else?

5.13.4 Service or Internal Behaviour?

A **Service** describes behaviour from the consumer's perspective. A Process or Function describes the internal behaviour that may provide it. For example, a passenger uses the **Refund Request Submission Service**. Internally, the organisation performs **Receive Refund Request**.

Do not use a Service merely because a name sounds useful or valuable. Identify a consumer and ask: **To whom is this behaviour made available?**

When no meaningful consumer can be identified, the element may be internal behaviour rather than a Service.

5.13.5 Is It Something Used or Produced?

Choose a **passive structure** concept when the element represents something that behaviour reads, creates, changes, transfers, or stores. Use a **Business Object** for something meaningful from the organisational perspective; use a **Data Object** for data structured for automated processing; use an **Artifact** for a concrete deployed, stored, or physically represented item. Examples: **Refund Request** may be a Business Object; **Submitted Refund Data** may be a Data Object; **refund-form.json** or a deployment package may be an Artifact. These elements can refer to related subject matter without being the same thing. The organisational concept, its application representation, and its deployed or stored form may require different elements.

5.13.6 Business, Application, or Technology?

When selecting an element, identify the perspective from which it is being described.

Ask: **Is this relevant because of organisational work, software responsibility, or technical execution?**

For example, a database containing passenger records may appear differently depending on the concern: **Passenger Account** as a Business Object; **Passenger Account Data** as a Data Object; **Passenger Database Schema** as an Artifact; **Database-Management System** as System Software.

5.13.7 Current Fact, Motivation, or Proposed Solution?

Not every concept belongs to the Core architecture. Use a **Driver**, **Assessment**, **Goal**, or **Requirement** when describing why change is considered; use a **Capability** or **Course of Action** when describing strategic ability or direction; use Core elements only when describing the organisational, application, or technological solution itself. For example, **Refund handling is slow and error-prone** is an Assessment; **Reduce refund-handling time** is a Goal; **Avoid manual re-entry of refund information** is a Requirement; **Automate refund information transfer** may be a Course of Action; a proposed integration component would be part of the solution architecture. Do not transform an intention into an implemented system merely by drawing it as an Application Component.

5.13.8 Prefer the Meaning Required by the View

The same real-world subject may be represented differently in different views. A passenger may appear as a Business Actor in a service context and as a Stakeholder in a motivation view. These are not competing descriptions: they express different architectural meanings.

The final test is therefore: **Which concept makes the intended architectural statement clear in this view?**

Choose the simplest concept that answers the concern; ensure that its name matches its type; verify that its relationships can be read as meaningful statements; avoid inventing detail merely to make the model appear complete. ArchiMate is intended to support coherent architectural communication, not to force every fact into one supposedly perfect classification.

5.14 <heuristic> Choosing the Right Relationship

Choosing the right elements is only half of modelling. The modeller must also state **how those elements are related**. An ArchiMate relationship is not a decorative line. It makes an architectural claim: **The source relates to the target in this specific way**.

The direction of the relationship, the types of its endpoints, and the meaning of the connector all matter. ArchiMate is intended to communicate relationships across an enterprise clearly and unambiguously.

5.14.1 Begin with a Sentence

Before drawing a connector, express the intended relationship as an ordinary sentence.

For example: the Customer-Service Agent performs Review Refund Request; Review Refund Request uses the Refund Request; Receive Refund Request provides the Refund Request Submission Service; the Refund Request Submission Service is available to the Passenger.

The verb in the sentence suggests the appropriate relationship. When no clear sentence can be formed, the relationship is probably not yet understood.

5.14.2 Assignment: Who or What Performs Behaviour?

Use **Assignment** when an active structure element performs behaviour or assumes responsibility.

Examples: Customer-Service Agent performs Review Refund Request; Finance Case System performs Record Refund Case; an organisational actor assumes a Role.

Ask: **Is the source responsible for performing the target behaviour?**

Assignment should not be used merely because an application supports a business process. Supporting another element is different from performing its behaviour.

5.14.3 Realisation: What Provides or Implements This?

Use **Realisation** when a more concrete element implements, produces, or provides a more abstract one.

Examples: Receive Refund Request realises the Refund Request Submission Service; a Data Object realises a Business Object; an Artifact realises an Application Component.

Ask: **Does the source make the target concrete or operational?**

Realisation often connects an internal explanation to an externally visible result. It is therefore useful when moving from black-box to white-box descriptions.

5.14.4 Serving: What Is Made Available to Whom?

Use **Serving** when one element provides functionality to another.

Examples: Refund Request Submission Service serves Passenger; Refund Case Recording Service serves Create Refund Case; Database Service serves Finance Case System.

Ask: **Does the source provide useful behaviour to the target?**

Read the connector in its direction: **The source serves the target**.

This prevents one of the most frequent errors: reversing the provider and consumer.

5.14.5 Access: What Is Read or Changed?

Use **Access** when behaviour or active structure interacts with passive structure.

Examples: Review Refund Request reads Refund Request; Create Refund Case creates Refund Case; Passenger Account Management updates Passenger Account Data.

Ask: **Does the source read, create, modify, or otherwise use the target object?**

Where the facts are known and the tool supports it, indicate read, write, or read-and-write access. Do not claim a specific kind of access when the source material establishes only that the object is used.

5.14.6 Triggering: What Causes or Precedes What?

Use **Triggering** for a temporal or causal dependency.

Examples: Refund Request Received triggers Review Refund Request; Review Refund Request triggers Create Refund Case; Service Disruption Reported triggers Publish Disruption Information.

Ask: **Does the source cause the target to begin, or establish a meaningful temporal precedence?**

Triggering makes a stronger claim than merely saying that two behaviours occur in some order. The relationship may represent immediate or delayed, intentional or unintentional effects.

5.14.7 Flow: What Is Transferred?

Use **Flow** when something moves from one element to another.

Examples: refund information flows from the Public Website to the Customer-Service Agent; payment authorisation data flows to the payment provider; ticket information flows to validation equipment.

Ask: **What exactly is transferred?**

Label the Flow whenever its content is not obvious: **refund information - manually re-entered; payment authorisation request; ticket validation data.**

A Flow represents transfer. It does not, by itself, state that the receiver provides a service, performs the sender's behaviour, or stores the transferred information.

5.14.8 Composition: What Belongs Inside What?

Use **Composition** for a strong whole-part relationship.

Examples: Refund Handling Process is composed of Receive Request, Review Request, and Create Case; Ticketing Platform is composed of several managed application components.

Ask: **Is the target genuinely a constituent part of the source?**

Do not use Composition merely to organise the page visually. Visual proximity and semantic ownership are different things.

5.14.9 Association: When Only a General Connection Is Known

Use **Association** when a relevant connection exists but the model does not justify a more specific ArchiMate relationship.

Examples: a Stakeholder is associated with a concern described informally; two elements are related in a way intentionally left outside the scope of the view.

Association can be useful, but it should not become a universal substitute for analysis. Where possible, label it with the meaning of the connection.

Ask: **Is the relationship genuinely general, or have I avoided choosing a more precise one?**

5.14.10 Support Is Not Always Assignment

The word **supports** is particularly dangerous because it can hide several different meanings.

"The Finance Case System supports refund handling" might mean that the system provides a Service to the refund process; performs an automated Process or Function; stores Data Objects used by the process; or is simply associated with the work at the level of detail shown. The modeller must replace vague wording with the precise architectural statement intended.

5.14.11 Check Both Meaning and Validity

A relationship may sound reasonable in ordinary language but still be invalid between the selected ArchiMate element types. Modelling tools can help by preventing or warning about invalid connectors, but tool acceptance is not enough: a valid relationship can still express an incorrect fact.

Use two checks:

- **Semantic check:** *Is the statement true and relevant?*
- **Language check:** *Is this relationship permitted between these element types?*

5.14.12 Prefer the Most Precise Supported Statement

Choose the most precise relationship that is justified by the available evidence, but no more precise.

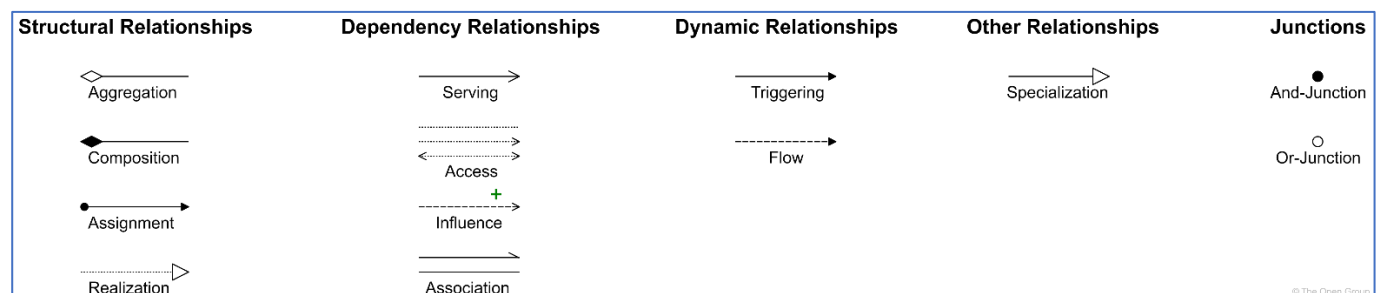
Do not replace an unknown relationship with an invented one; do not use Flow when nothing is transferred; do not use Triggering when only coexistence is known; do not use Assignment when an application merely assists human work; do not use Composition simply to group elements.

A reliable final test is to read every connector aloud:

- *The agent performs the process.*
- *The process accesses the request.*
- *The process realises the service.*
- *The service serves the passenger.*
- *Refund information flows to the next participant.*

A relationship belongs in the view only when its sentence is understandable, justified, and useful to the concern being addressed.

5.14.13 Relationships and Junctions



© The Open Group

5.15 <heuristic> Black-Box, White-Box, and Modelling Depth

An architecture element can often be described at different levels of detail. A modeller may show only what the element offers to its environment, or may open it and describe its internal structure and behaviour. These two perspectives are commonly called **black-box** and **white-box** modelling.

5.15.1 Black-Box: What Is Visible from Outside?

A **black-box view** describes an element through its externally relevant behaviour, services, interfaces, or relationships. Its internal operation remains hidden. For example, Northbridge Mobility may be shown as providing a **Refund Request Submission Service** to a Passenger. The view does not explain which people, applications, or processes provide that service.

The black-box perspective answers questions such as:

- *What does this element offer?*
- *Who uses it?*
- *On which other elements does it depend?*
- *What enters or leaves its boundary?*

Black-box modelling is useful when the internal design is unknown; unnecessary for the concern; managed by another party; still undecided; or deliberately treated as outside the scope of analysis. Hiding internal detail is not a modelling failure. It is often the correct decision.

5.15.2 White-Box: How Is It Internally Realised?

A **white-box view** opens an element and describes the internal structure or behaviour that explains how it operates. The Refund Request Submission Service might be opened to show **Receive Refund Request**; **Public Website**; **Submitted Refund Data**; the relationships among them.

The white-box perspective answers questions such as:

- *Which elements collaborate internally?*
- *Who performs the behaviour?*
- *Which applications support it?*
- *Which information is accessed?*
- *How is the external service realised?*

A white-box view should expose only the internal detail required by the concern. It does not require a complete decomposition.

5.15.3 The Same Element Can Appear in Both Perspectives

Black-box and white-box views are not competing descriptions. They may be complementary views of the same architecture. A context view may show: Northbridge Mobility provides the Refund Request Submission Service to the Passenger. A realisation view may then show: Receive Refund Request realises the Refund Request Submission Service; the Public Website supports the process; Submitted Refund Data is created. The Service provides continuity between the two views. One presents the external promise; the other explains its internal provision.

5.15.4 Modelling Depth Is Relative

An element that is white-box at one level may remain black-box at another. A view may open Northbridge Mobility and show the Public Website and Finance Case System. Those applications may still be represented as black boxes because their internal modules are not relevant. A later software architecture view might open the Public Website into several Application Components. Those components may themselves remain black boxes. There is therefore no single

absolute boundary between black-box and white-box modelling. Depth depends on the question being answered.

5.15.5 Do Not Invent Internal Detail

A common beginner error is to assume that a model is incomplete unless every element is decomposed. This encourages plausible but unsupported additions. Suppose the case description states that refund information is manually re-entered into the Finance Case System. It does not identify the system's modules, database, integration architecture, or hosting arrangement. The modeller should not invent these elements simply to create a more detailed diagram. The Finance Case System may remain a black box. A black box can represent: detail that is deliberately hidden; detail that is currently unknown; detail that has not yet been decided. The model should make these situations clear in its accompanying explanation when the distinction matters.

5.15.6 Do Not Hide the Relevant Problem

The opposite error is to remain at black-box level when internal detail is essential to the concern. A context view showing only a Refund Handling Service would hide the manual transfer of information between the Public Website and Finance Case System. To explain the delay and error risk, the modeller must open the relevant part of the architecture. The correct depth is therefore not "as simple as possible" in isolation. It is: **as simple as possible, but detailed enough to answer the concern.**

5.15.7 Avoid Mixing Incompatible Depths

A view becomes difficult to read when one part is shown at very high detail while the rest remains extremely abstract without a clear reason. For example, a diagram that shows individual server files beside a whole transport organisation, passenger services, and broad strategic goals mixes very different levels of abstraction. Different depths may coexist when they contribute to the same concern, but the modeller should be able to explain why each level is present. When detail becomes excessive, separate the model into related views: a context view; a service realisation view; an application cooperation view; a technology deployment view.

5.15.8 Use Viewpoint and Concern to Choose Depth

Before opening an element, ask:

- *Does the concern require its internal structure?*
- *Is the available information reliable enough?*
- *Will the additional detail clarify an architectural decision or dependency?*
- *Would a separate view communicate it more clearly?*

Remain at black-box level when the answer is no. Move towards white-box modelling when the answer is yes.

5.15.9 A Practical Rule

Begin with the outside perspective: Who consumes what service? Then open only the elements needed to explain an important responsibility, dependency, problem, risk, or design decision. Stop decomposing when additional detail no longer changes the architectural explanation. A good architecture model does not reveal everything that could be modelled. It controls modelling depth so that every visible element contributes to a clear, justified, and purposeful view.

5.16 <heuristic> Common Modelling Errors

Most beginner errors in ArchiMate are not caused by difficult notation. They arise because the modeller has not yet separated different kinds of architectural meaning.

The following mistakes are especially common:

- **Modelling Words Instead of Meanings:** A case description may use words such as *system*, *service*, *process*, *platform*, or *information* informally. These words do not automatically determine the ArchiMate concept. For example, **Customer Service** might refer to an organisational Function, an externally offered Service, a department, or simply an informal topic. Do not classify an element from its wording alone. Ask what the element means in the view.
- **Confusing Structure with Behaviour:** An Application Component is something that performs behaviour. A Process or Function is behaviour. Incorrect: *Finance Case System* represented as a Process. Better: *Finance Case System* represented as an Application Component; *Record Refund Case* represented as a Process or Function. Use nouns for structural elements and active verb phrases for processes.
- **Confusing Actors and Roles:** An Actor identifies a participant. A Role identifies a responsibility or capacity. Incorrect: *Customer-Service Agent* modelled as an identifiable employee when it actually represents a responsibility shared by many employees. Better: model the employee or department as an Actor; model *Customer-Service Agent* as a Role. Use the Role only when responsibility matters to the concern.
- **Treating Every Activity as a Process:** Not all behaviour requires a Process. Use a **Process** when progression through time matters; a **Function** when behaviour is grouped by purpose or responsibility; an **Event** when something occurs; a **Service** when behaviour is made available to a consumer. Incorrect: *Customer Support* represented as a Process although no sequence is intended. Better: represent it as a Function.
- **Calling Internal Work a Service:** A Service is behaviour made available to a consumer. Incorrect: *Validate Refund Data Service* when the behaviour is merely an internal step with no meaningful consumer. Better: model it as a Process or Function unless another element genuinely consumes it as a service. Before using Service, identify who or what it serves.
- **Confusing Business Objects, Data Objects, and Artifacts:** These concepts may refer to related subject matter but represent different architectural meanings. **Refund Request** may be a Business Object; **Submitted Refund Data** may be a Data Object; **refund-request.json** may be an Artifact. Incorrect: representing the organisational concept, application data, and stored file as one element merely because they concern the same refund request. Separate them only when the distinction is relevant to the view.
- **Using “Supports” Without Explaining How:** The statement “the system supports the process” is too vague for precise modelling. It might mean that the system provides a Service to the process; performs automated behaviour; gives access to Data Objects; or is only generally associated with the work.
- Replace *supports* with a clearer statement: provides a service to; performs; accesses; stores; transfers; realises. Then select the relationship that expresses that meaning.
- **Reversing Relationship Direction:** Direction changes the architectural statement. Incorrect: Passenger serves Refund Request Submission Service. Correct: Refund Request Submission Service serves Passenger. Incorrect: Refund Request realises Review Refund Request. Correct: Review Refund Request accesses Refund Request. Read each relationship aloud from source to target.
- **Using Flow for Every Interaction:** Flow means that something is transferred. It does not mean merely that two elements communicate or depend on one another. Incorrect: an

unlabelled Flow between two applications simply because they are related. Better: identify what is transferred, such as *refund information* or *payment authorisation request*, and label the Flow. When no transfer is established, use another justified relationship or omit the connector.

- **Using Triggering for Simple Ordering:** Triggering expresses a temporal or causal dependency. Incorrect: connecting two processes with Triggering merely because they appear beside one another in a list. Use Triggering only when the source meaningfully initiates, causes, or precedes the target in the architecture being described.
- **Using Composition as Page Layout:** Composition represents a strong whole–part relationship. It is not a visual grouping device. Incorrect: placing unrelated elements inside a larger box and treating them as components of that element. Use Composition only when the parts genuinely belong to the whole. Use a view boundary, grouping convention, or separate diagram when the objective is merely visual organisation.
- **Inventing Missing Architecture:** A plausible technical solution is not automatically a fact. Incorrect: adding an integration platform, shared database, API gateway, cloud environment, or organisational unit because such elements would be sensible. Better: represent only what is supported by the Universe of Discourse; state explicit assumptions separately; present proposed target elements as design decisions. Unknown information may remain unknown.
- **Mixing Current and Target States:** A diagram becomes misleading when existing elements and proposed elements appear together without distinction. Incorrect: showing a proposed Digital Travel Portal as though it already exists. Better: separate current-state and target-state views; or clearly mark proposed elements and decisions. A goal or requirement is not evidence that a solution has already been implemented.
- **Adding Too Much Detail:** A view can contain only correct elements and still be ineffective. Warning signs include many crossing connectors; long lists of infrastructure details; unrelated domains mixed together; small labels; elements that do not contribute to the concern. Remove anything that does not help answer the view’s question. When necessary, split one dense diagram into two or more related views.
- **Using Colour as Meaning:** Colour may improve readability, but it should not be the only way to communicate element type, state, ownership, or importance. Beginners may view diagrams in monochrome, with limited colour perception, or through different modelling tools. Use element notation, labels, boundaries, legends, and relationships to carry the meaning. Treat colour as support rather than semantics.
- **Correct but Unexplained Diagrams:** A technically valid diagram may still be difficult to understand. Every view should have a short explanation stating its purpose; the concern addressed; the scope and omissions; any assumptions; the most important architectural conclusion.
- The reader should not have to reverse-engineer the modeller’s intention.

A Final Review Routine: The objective is not a diagram without any possible criticism, but a view whose modelling decisions are clear, justified, and open to constructive review. Before submitting a view, check:

- Does every element have the correct architectural meaning?
- Can every relationship be read as a true sentence? Is its direction correct? Is every fact supported or explicitly proposed?
- Are current and target states distinguishable?
- Does every visible element contribute to the concern?
- Could the view be made smaller without losing its explanation?

5.17 <example> Reviewing and Correcting an ArchiMate View

Creating a diagram is only part of architecture modelling. A model should also be **reviewed, challenged, and corrected**.

Consider a beginner view intended to explain current refund handling at Northbridge Mobility. It contains the following elements: **Passenger; Refund Service; Customer-Service Agent; Finance Case System; Refund Request; Cloud Platform; Shared Customer Database**. The diagram also contains several unlabelled arrows and presents the Cloud Platform and Shared Customer Database as existing parts of the architecture. At first glance, the view may appear plausible.

A disciplined review, however, reveals several problems.

- **Step 1: Restate the Purpose:** Before inspecting individual elements, ask: **What concern is this view intended to address?** Suppose the stated purpose is: Explain how a passenger refund request is currently received, reviewed, and recorded. This purpose immediately establishes boundaries. The view should focus on the passenger; refund-related behaviour; the staff responsibility involved; the applications explicitly used; the relevant information. Any element unrelated to that explanation should be questioned.
- **Step 2: Check the Evidence:** The case states that the passenger submits a refund form through the Public Website; a customer-service agent reviews it; the agent manually creates a corresponding case in the Finance Case System. The case does **not** establish a Cloud Platform or Shared Customer Database. These elements must therefore be removed from a current-state view. They might later appear as explicit target-state proposals, but architectural plausibility is not evidence. A useful review question is: **Which sentence in the Universe of Discourse justifies this element?** When no supporting sentence or explicit design decision exists, the element should normally be omitted.
- **Step 3: Check Element Meanings:** The element **Refund Service** is vague. Does it represent behaviour offered to the passenger, the complete internal refund process, or an organisational unit? A more precise external element would be: **Refund Request Submission Service**. The name identifies what is made available to the passenger without claiming that the entire refund decision is completed through that service. The **Customer-Service Agent** represents a responsibility rather than an identified individual or department. It should therefore be modelled as a **Business Role**. The **Finance Case System** is software and should be represented as an **Application Component**. The **Refund Request** represents something meaningful to the organisation and may be represented as a **Business Object**. Its electronic capture in the Public Website may separately be represented as **Submitted Refund Data** when this distinction matters.
- **Step 4: Add the Missing Behaviour:** The original diagram connects the Customer-Service Agent directly to the Finance Case System but does not show what the agent actually does. The view should include relevant behaviour such as: **Receive Refund Request; Review**

Refund Request; Create Refund Case. This makes the organisational work visible and allows responsibilities and application support to be expressed precisely. Without behaviour, the diagram merely states that a person and a system are somehow connected.

- **Step 5: Replace Generic Arrows:** Every connector should become a clear architectural statement. Instead of an unlabelled arrow from Customer-Service Agent to Review Refund Request, use **Assignment**: The Customer-Service Agent performs Review Refund Request. Instead of a generic arrow between Review Refund Request and Refund Request, use **Access**: Review Refund Request reads the Refund Request. Instead of a generic connection from Refund Request Submission Service to Passenger, use **Serving**: The Refund Request Submission Service serves the Passenger. Instead of a vague line between Receive Refund Request and the Service, use **Realisation**: Receive Refund Request realises the Refund Request Submission Service.
- **Step 6: Make the Architectural Problem Visible:** The case identifies an important fact: refund information is manually re-entered into the Finance Case System. This should not be hidden behind a generic relationship. A labelled **Flow** can make the problem explicit: **refund information - manually re-entered**. The Flow may connect the relevant business or application elements, depending on the chosen level of abstraction. The essential point is that the view should reveal the transfer and its manual nature.
- **Step 7: Remove Unnecessary Detail:** The reviewed view does not need to show journey planning; ticket purchasing; passenger-account management; servers; networks; database-management software; future integration components. These elements may be valid elsewhere, but they do not contribute to the stated concern. Removing them is not a loss of completeness. It improves the view's explanatory focus.
- **The Corrected Architectural Story:** After correction, the view should support a concise reading: The Passenger uses the Refund Request Submission Service. Receive Refund Request realises that Service. The Customer-Service Agent reviews the Refund Request.
Refund information is manually re-entered. The Finance Case System supports the creation of a Refund Case. The exact diagram may vary. Different modellers may choose slightly different levels of detail while still producing defensible views.
- **Review the Reasoning, Not Only the Notation:** A modelling review should not be reduced to checking shapes and arrowheads. It should examine: whether the purpose is explicit; whether every element is supported; whether concepts express the intended meaning; whether relationships can be read as true statements; whether important problems are visible; whether unnecessary detail has been removed. A good correction does more than make the diagram formally valid. It makes the architectural explanation clearer, more honest, and more useful to its intended audience.

5.18 <exercise> From a Narrative to Consistent Architecture Views

This exercise develops the core skill of Architecture Description: transforming an informal narrative into a set of purposeful and mutually consistent views. The objective is not to reproduce every sentence in the case. It is to identify the architectural facts, select suitable ArchiMate concepts, and organise them according to distinct concerns.

5.18.1 The Narrative

Northbridge Mobility provides regional bus and light-rail services. Passengers can plan journeys, purchase tickets, manage passenger accounts, receive disruption information, submit refund requests, and contact customer support.

Refund requests are submitted through a form on the Public Website. A Customer-Service Agent reviews the submitted information and manually creates a corresponding case in the Finance Case System. The Finance Case System records and processes the refund decision. Management considers the current digital experience fragmented. Passenger information exists in several applications, refund handling involves manual re-entry, and customer-service staff must consult multiple systems. Northbridge Mobility intends to provide a more coherent digital travel experience, but it has not yet decided which applications will be retained, integrated, replaced, or retired.

5.18.2 Task 1: Establish the Universe of Discourse

Separate the narrative into three categories: **Established current-state facts**; **identified problems and motivations**; **undecided target-state questions**. Do not treat an intended improvement as an existing capability or implemented system. Examples: "Refund requests are submitted through the Public Website" is a current-state fact. "Refund handling involves manual re-entry" is an assessment of the current situation. "Which applications will be retained or replaced?" remains an open target-state decision. Record any assumptions separately. An assumption must never be presented as though it were directly stated in the narrative.

5.18.3 Task 2: Create a Context and Services View

Create a small black-box view answering: **Which external participants interact with Northbridge Mobility, and which services are available to them?**

Possible elements include **Northbridge Mobility**; **Passenger**; **Authorised Retailer**; **Journey Planning Service**; **Ticket Purchasing Service**; **Passenger Account Service**; **Disruption Information Service**; **Refund Request Submission Service**; **Customer Enquiry Service**. Use Serving relationships to connect services to their consumers. Do not include internal applications, processes, data, or technology in this view. Recommended size: eight to twelve elements.

5.18.4 Task 3: Create a Refund Handling View

Create a more detailed view answering: **How is a refund request currently received, reviewed, and recorded?** Possible elements include **Passenger**; **Customer-Service Agent**; **Refund Request Submission Service**; **Receive Refund Request**; **Review Refund Request**; **Create Refund Case**; **Refund Request**; **Refund Case**; **Public Website**; **Finance Case System**; **Submitted Refund Data**; **Refund Case Data**. Use appropriate relationships to express responsibility, service realisation, application support, information access, sequence, and transfer. The manual transfer must be visible and labelled: **refund information - manually re-entered**. Do not add an integration platform, shared database, API gateway, or automated transfer unless these are explicitly presented as target-state proposals.

5.18.5 Task 4: Create a Motivation for Change View

Create a view answering: **Why is architectural change being considered, and what results are expected?** Possible Stakeholders include **Passenger**; **Northbridge Mobility Management**; **Customer-Service Staff**. Possible Drivers include **Fragmented Digital Services**; **Duplicate Passenger Information**; **Manual Refund Processing**; **Changing Passenger Expectations**. Possible Assessments include **Passenger experience is inconsistent**; **Refund handling is slow and error-prone**; **Passenger information is difficult to consolidate**. Possible Goals and Outcomes include **Provide a Coherent Digital Travel Experience**; **Faster Refund Handling**; **Less Duplication of Passenger Information**; **More Timely Disruption Communication**; **Improved Passenger Satisfaction**. A Requirement or Course of Action such as **Provide a Common Digital Point of Access** may be represented, but it must not be modelled as an existing Application Component.

5.18.6 Task 5: Check Consistency Across Views

The views should be different, but they must not contradict one another. Check whether the same element is named consistently; whether the same service retains the same meaning; whether current and target states remain distinguishable; whether the detailed refund view explains part of the context view; whether the motivation view explains the reasons for change without inventing the solution. For example, the **Refund Request Submission Service** shown externally in the context view should correspond to the same service opened in the refund-handling view.

5.18.7 Task 6: Explain the Modelling Decisions

For each view, write a short paragraph stating: **Purpose** = which concern the view addresses; **scope** = what is included; **omissions** = what is deliberately hidden; **assumptions** = which statements were not explicit in the narrative; **main conclusion** = what the reader should learn from the view. The explanation is part of the Architecture Description. A diagram without its purpose and scope is easily misinterpreted.

5.18.8 Review Questions

Before submitting, ask: Does every element have a clear architectural meaning? Can every relationship be read as a defensible sentence? Is every current-state element supported by the narrative? Are proposed changes clearly distinguished from existing architecture? Does each view answer one concern? Could any view be made smaller without losing its explanation?

5.18.9 Closing

There is no single mechanically determined solution. Different analysts may produce different valid views because modelling requires interpretation and selection. A strong model will demonstrate that the analyst can: identify architectural facts without inventing missing detail; choose concepts according to meaning rather than wording; separate external services from internal realisation; distinguish motivation from solution design; maintain consistency across several views. The quality of the work depends less on the number of elements represented than on the clarity, justification, and coherence of the modelling decisions.

5.19 <further> How ArchiMate Evolved

ArchiMate was created because enterprise architects needed a shared language for describing relationships across organisations, information, applications, and technological infrastructure. It was originally developed in the Netherlands between 2002 and 2004 by a group led by the Telematica Instituut, working with partners from academia, government, and industry. The ownership and stewardship of the language were transferred to The Open Group in 2008, and the first ArchiMate specification published as an Open Group standard followed in 2009.

The central ambition has remained stable: provide a common visual language through which different stakeholders can describe, analyse, and communicate enterprise architectures.

This purpose distinguishes ArchiMate from modelling languages centred on the detailed design of one software system or the execution logic of one business process. ArchiMate is primarily concerned with architectural coherence: how organisational responsibilities, behaviour, information, applications, technology, motivation, strategy, and change fit together.

5.19.1 Growth Through Successive Specifications

The language has evolved through several major generations. Each generation extended or refined the language in response to practical modelling needs, experience from users, and changes in enterprise architecture practice.

The evolution included stronger alignment with enterprise architecture methods; broader support for strategic and motivational modelling; improved representation of implementation and migration; refinements to concepts, relationships, notation, and viewpoint guidance; mechanisms for exchanging models between compatible tools.

ArchiMate 2, for example, increased alignment with the TOGAF Architecture Development Method and widened the language beyond its earlier core architecture concerns. Later developments continued to expand the range of concerns that could be connected within one coherent architecture description.

5.19.2 Continuity Matters

A new language version does not imply that all previous architectural knowledge becomes useless. The language has evolved while retaining its fundamental purpose and many recurring modelling ideas.

Across its evolution, architects have continued to ask questions such as:

- Who or what performs behaviour?
- What services are made available?
- Which information is used?
- How do applications support organisational work?
- On which technology do applications depend?
- Why is change needed?
- How will the architecture move from one state to another?

The precise language structure and available concepts may change, but these architectural concerns remain recognisable.

5.19.3 ArchiMate 4

The Open Group announced the ArchiMate 4 Specification in April 2026. It remains an open and independent modelling language intended to help architects describe, analyse, and visualise complex enterprises and systems of systems.

For beginners, the important point is not to memorise a chronology of versions. **This course teaches ArchiMate 4 directly.** Its concepts, relationships, and modelling structure should be learned as a coherent current language, rather than reconstructed through comparisons with earlier specifications.

Historical information becomes useful only afterwards, particularly when students encounter older books, diagrams, modelling tools, organisational repositories, or online examples.

5.19.4 Why Older Material Still Appears

EA models can remain in use for many years. Organisations may also adopt new language versions gradually because their repositories, tools, methods, training materials, and governance rules cannot all be changed immediately.

Beginners may therefore encounter older terminology or diagrams even after learning ArchiMate 4. Such material should not automatically be rejected, but neither should its notation be copied uncritically into a current model.

The modeller should first determine:

- Which version does this material use?
- What architectural statement was its author trying to express?
- Is the model still factually current?
- How should that meaning be represented in ArchiMate 4?

5.19.5 Evolution Is Not the Same as Architecture Change

Two different forms of change must be distinguished:

- An organisation may change its architecture while continuing to use the same modelling language.
- Alternatively, a model may be updated to a new language version even when the represented enterprise has not changed.

Therefore: **A change in the model may reflect a change in the enterprise, a change in the modelling language, or both.** Keeping these possibilities separate is essential when reviewing or migrating existing architecture descriptions.

The history of ArchiMate should ultimately reinforce one lesson: modelling languages evolve, but architecture remains an analytical activity. The modeller must understand the enterprise, select an appropriate representation, and communicate architectural decisions clearly to the intended stakeholders.

5.20 <further> Reading Older ArchiMate Models

ArchiMate diagrams created with an earlier specification may appear in books, organisational repositories, research papers, online tutorials, or modelling tools that have not yet been updated. Such diagrams can still contain useful architectural knowledge. However, they should be treated as **historical models**, not as authoritative examples of ArchiMate 4 notation or terminology. The first objective is therefore not to redraw them immediately. It is to understand what architectural statements their authors intended to make.

5.20.1 Identify the Language Version

Before interpreting a diagram, look for evidence of the version used. Possible clues include the publication date; the version stated in the document or model metadata; terminology used in element names or legends; notation supported by the modelling tool; references cited by the author. The date alone is not conclusive. A recently published document may reproduce an older model, while an older repository may have been updated selectively. When the version cannot be determined, state the uncertainty rather than assuming that the model follows ArchiMate 4.

5.20.2 Begin with Meaning, Not Symbols

Do not start by translating shapes mechanically. First ask: What enterprise situation is being described? What concern does the view address? Which elements represent structure, behaviour, information, motivation, or change? What architectural claim does each relationship appear to make? A model can remain understandable even when its notation differs from the current language. Conversely, a visually familiar symbol does not guarantee that the intended meaning is the same.

Read each connector as a sentence. For example:

- This organisational participant performs this behaviour.
- This behaviour provides this service.
- This application supports this organisational activity.
- This information is used or produced.
- This technological environment hosts this application.

The precise ArchiMate 4 representation can be considered only after the intended statement is understood.

5.20.3 Separate Language Age from Factual Age

An older model may present two different problems.

Its **language representation** may be outdated, while the architecture it describes remains current. Alternatively, its notation may still be understandable, but the represented organisation, applications, responsibilities, or technology may no longer exist. These questions must be evaluated separately: Is the model expressed using an earlier language structure? Is the represented enterprise still factually accurate? Updating the notation does not automatically update the architecture. A model can be perfectly translated and still describe an obsolete enterprise.

5.20.4 Expect Differences in Vocabulary

Older material may classify concepts differently or use names that are not part of the ArchiMate 4 vocabulary taught in this course. Do not copy those names into a new model merely because they appear in a respected publication. Instead, determine their intended architectural meaning and select the ArchiMate 4 concept that expresses that meaning most clearly. For example, an older element may have been used to describe externally visible behaviour,

internal organisational work, software responsibility, or infrastructure behaviour. Its current representation should be chosen from its meaning in the view, not from superficial similarity between labels.

5.20.5 Preserve the Author's Intended Statement

When re-expressing an older model, avoid silently changing its meaning. Suppose an older diagram states only that an application supports a business activity. A new model should not automatically assert that the application performs the business behaviour, directly realises the service, or owns the relevant information. A more precise relationship may be desirable, but it must be justified by evidence. Translation should not become invention. Where the original relationship is ambiguous, record the ambiguity or use a deliberately general relationship until further information is available.

5.20.6 Do Not Translate the Diagram in Isolation

The diagram may depend on accompanying text, viewpoint definitions, legends, repository conventions, or organisation-specific modelling rules. Before interpreting it, examine the surrounding material: What purpose did the author give the view? Which stakeholders were expected to read it? Were colours or visual groupings assigned special meaning? Were some relationships intentionally omitted? Was the model simplified for communication? Without this context, a technically plausible translation may misrepresent the original analysis.

5.20.7 Reconstruct the Model, Not the Page Layout

An old diagram may contain valuable model content but weak visual organisation. When recreating it, do not preserve every position, colour, crossing connector, or decorative boundary. Preserve the architectural meaning, then design a new ArchiMate 4 view appropriate to its intended concern. This may require separating one crowded diagram into several views; removing duplicated elements; standardising names; adding explicit scope and assumptions; distinguishing current and historical states. The goal is not to create a visual replica. It is to produce a clearer Architecture Description.

5.20.8 Record Translation Decisions

When an older model is reused, document the decisions made. A brief migration note may identify the source model and its assumed language version; concepts whose meaning was reinterpreted; relationships that remained ambiguous; elements removed because the architecture was obsolete; facts that still require validation. This makes the new model auditable and prevents later readers from treating analytical decisions as facts supplied by the original author.

5.20.9 A Practical Reading Method

Use the following sequence: Identify the likely language version. Establish the view's purpose and concern. Read elements and relationships as architectural statements. Check whether the represented facts remain current. Re-express the supported meaning using ArchiMate 4. Record uncertainties and translation decisions. Redesign the view for clarity. Older models should therefore be approached with neither automatic trust nor automatic rejection. They are evidence created in a particular modelling context. The architect's task is to recover their useful meaning, verify the represented architecture, and express that meaning responsibly in the current language.

ArchiMate 4 was designed with substantial continuity for users of previous versions, but modelling tools may still implement only the ArchiMate 3.2 metamodel (as it is the case, for example, of the tool Archi).

However, such tools can still be useful for ArchiMate 4 diagram production, even if its palette and validation rules must not be treated as the definition of ArchiMate 4. The basic principle is:

Decide the intended ArchiMate 4 meaning first; represent it in the ArchiMate 3.2 tool second.

5.21.1 Begin with a Conceptual Model

Before using the tool, identify the elements and relationships required by the view using the ArchiMate 4 vocabulary taught in this course.

Record for each element: its ArchiMate 4 concept; its intended meaning; the evidence supporting it; the views in which it will appear.

The tool should record an analytical decision already made. It should not determine the concept merely because a convenient symbol is available.

5.21.2 Use Native Elements Only When Their Meaning Fits

Some ArchiMate 4 concepts can be represented through an element already available in the older tool without materially changing the intended statement.

In these cases, the existing element may be used as a practical representation. The modeller should still verify that its meaning and permitted relationships remain appropriate. Do not select an older element only because its name or appearance seems similar.

Superficial resemblance is not semantic equivalence.

5.21.3 Make Compatibility Decisions Explicit

When the tool lacks the required ArchiMate 4 concept or treats it differently, use an explicit compatibility convention. Possible conventions include assigning a specialisation; adding a property such as **Intended ArchiMate 4 Concept**; placing the ArchiMate 4 type in the documentation field; adding a small textual type marker to the diagram; maintaining a separate mapping table for the repository.

For example:

- **Element name:** Refund Request Submission
- **Intended ArchiMate 4 concept:** Service
- **Tool representation:** selected compatibility element
- **Reason:** closest available representation in the current tool

The exact convention matters less than consistency and transparency. Archi supports named specialisations and preserves specialisation names in its model exchange facilities, which can help document such mappings.

5.21.4 Do Not Misrepresent the Repository

A model created in an ArchiMate 3 tool using compatibility conventions is not automatically a native ArchiMate 4 repository.

This distinction should be stated in the model documentation: **This repository represents an ArchiMate 4 analysis using a tool whose native metamodel is ArchiMate 3. Compatibility mappings are documented explicitly.**

This warning is particularly important when models are exchanged, imported into other tools, or automatically analysed. Another tool may interpret only the stored ArchiMate 3 element types and ignore the intended ArchiMate 4 meaning recorded in labels or properties.

5.21.5 Treat Validation as Version-Specific

A tool may reject a relationship that is valid in the intended ArchiMate 4 model or accept one according to its older metamodel. Therefore, separate two checks:

- **Analytical check:** Is the architectural statement correct and supported?
- **Tool check:** Can the chosen representation be stored and drawn in this tool?

Never change the intended architectural meaning merely to remove a tool warning. Where a mismatch cannot be represented safely, document it and use another drawing method for that view.

5.21.6 Prefer Simple and Reversible Workarounds

Compatibility work should not make the repository difficult to migrate later.

Prefer clear names; reusable elements; documented properties; limited visual conventions; a small mapping table.

Avoid large numbers of invented stereotypes, unexplained colours, or tool-specific tricks that future users cannot interpret.

A suitable mapping record may contain: **ArchiMate 4 concept; tool element used; additional property or label; known semantic limitation; views affected.**

5.21.7 Use the Tool as a Repository, Not as the Authority

The Open Group defines ArchiMate as an independent language for describing, analysing, and visualising relationships across enterprise architecture. A particular modelling product is only one implementation of that language.

When using an older tool: ArchiMate 4 provides the intended semantics; the Universe of Discourse provides the evidence; the analyst makes the modelling decisions; the tool stores an explicitly documented approximation.

This approach allows to benefit from repositories, reusable elements, layouts, and exports without silently transforming an ArchiMate 4 analysis into an older model.

EA helps us understand an enterprise as a connected system rather than as a collection of isolated departments, applications, and technologies. to help stakeholders reason about coherence, dependencies, responsibilities, problems, and change. ArchiMate 4 supports that work by providing a shared language for Architecture Description. It allows different concerns to be represented through coordinated views while preserving a consistent underlying model.

5.22.1 Architecture Description Is Purposeful

A model should never begin with the question: Which symbols can I draw? It should begin with: Which concern must this view address? The modeller then selects only the elements and relationships needed for that purpose. A service context view, a process realisation view, a motivation view, and a technology view may all describe the same enterprise, but each answers a different question. No single view should be expected to explain everything.

5.22.2 Views Select; Models Connect

A **model** contains architectural elements and relationships. A **view** selects part of that model for a particular audience and concern. This distinction explains why the same element may appear in several diagrams. The **Refund Request Submission Service**, for example, may appear in a context view as something offered to a Passenger and in another view as behaviour realised by an internal Process. Reusing the same element across views supports coherence. Creating unrelated copies with the same name does not.

5.22.3 Read Elements by Their Architectural Meaning

The first modelling decision is to determine what kind of thing is being represented. Active structure represents something capable of performing or hosting behaviour; behaviour represents something that happens or is performed; a Service presents behaviour made available to a consumer; passive structure represents something used, produced, transferred, or stored. The architecture domains refine this distinction. The Business Domain describes organisational systems and meaningful business objects; the Application Domain describes software systems and automated data; the Technology Domain describes infrastructure, execution environments, communication, and deployed artefacts. Motivation and Strategy explain why change is considered, what results are intended, and which capabilities or courses of action may be required.

5.22.4 Relationships Are Architectural Statements

A diagram gains meaning through its relationships. Assignment identifies responsibility for behaviour; Realisation connects a concrete implementation or internal behaviour to something more abstract; Serving identifies a provider and consumer; Access shows that behaviour reads or changes an object; Triggering expresses temporal or causal dependency; Flow represents something transferred; Composition represents a strong whole-part structure. A relationship that cannot be explained clearly should be reconsidered, until it can be readable as a sentence:

- The Role performs the Process.
- The Process accesses the Business Object.
- The Process realises the Service.
- The Service serves the Actor.
- Information flows to another participant.

5.22.5 Select the Right Modelling Depth

Architecture can be described from a black-box or white-box perspective. A black-box view shows what an element offers to its environment while hiding its internal operation. A white-box view opens the element and explains the structure or behaviour through which it works. Neither perspective is inherently better. The right depth depends on the concern, available evidence, and intended audience. The guiding rule is: **Use the simplest view that still answers the concern.**

5.22.6 Model Evidence, Not Plausibility

Architecture modelling involves interpretation, but it must remain grounded. Represent current-state facts only when they are supported by the UoD. Record assumptions explicitly. Keep open questions open. Distinguish proposed target elements from existing architecture. A technically sensible application, database, integration platform, or hosting environment should not appear in a current-state view merely because it would be plausible. Unknown information is not a defect in the model. Concealing uncertainty through invention is.

5.22.7 Prefer Coherence Over Completeness

A good view is not the one containing the largest number of elements. It is the one whose purpose is clear, whose concepts are well chosen, whose relationships express justified statements, and whose level of detail is appropriate. Small views are easier to read, review, and correct. When a diagram becomes dense, separate it into related views.

5.22.8 Tools Assist; Analysts Decide

A modelling tool can store reusable elements, validate connectors, organise views, and produce diagrams. It cannot determine whether a model is factually correct, analytically justified, or useful to its stakeholders. When the available tool implements a different language version, compatibility decisions must be made explicit. The intended ArchiMate 4 meaning remains primary. The tool records the model. It does not own the analysis.

5.22.9 Modelling Is an Accountable Interpretation

Architecture models are not neutral copies of reality. They result from decisions about scope, abstraction, classification, relationships, omissions, and presentation. The client provides facts, concerns, constraints, and feedback. The analyst transforms that material into an Architecture Description and remains responsible for the modelling choices made. Client approval confirms that a view is accepted for its intended purpose. It does not remove the analyst's interpretative role.

5.22.10 What beginners Should Now Be Able to Do

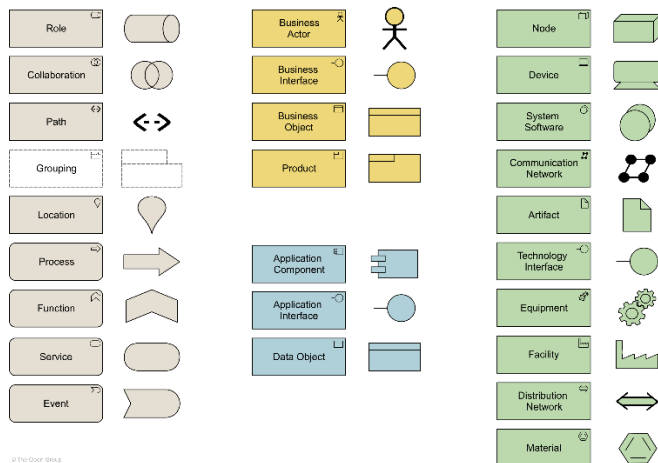
After this introduction, beginners should be able to: identify the concern addressed by a view; distinguish structure, behaviour, services, and passive structure; select suitable concepts from the core domains; choose relationships that form meaningful architectural statements; separate current facts, assessments, goals, requirements, and proposed solutions; control black-box and white-box depth; maintain consistency across several views; review a model for unsupported assumptions, unclear semantics, and unnecessary detail. The objective: **to use ArchiMate 4 as a disciplined way of thinking and communicating about enterprise architecture.**

5.23 <technique> ArchiMate Reference Cards

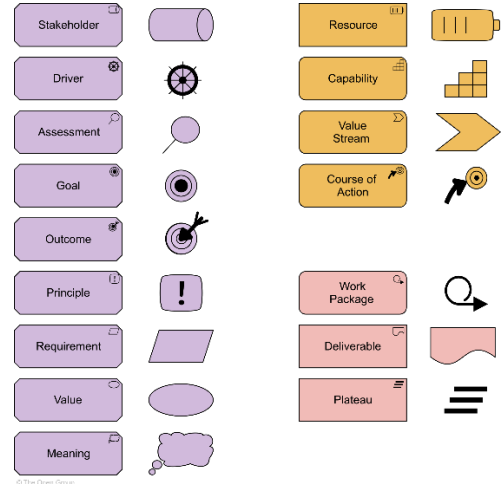
See:

Reference Cards: <https://pubs.opengroup.org/architecture/archimate4-doc/appendixa.html>

5.23.1 Core Elements

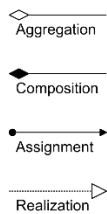


5.23.2 Motivation, Strategy, Implementation and Migration Elements



5.23.3 Relationships and Junctions

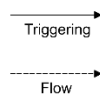
Structural Relationships



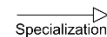
Dependency Relationships



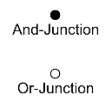
Dynamic Relationships



Other Relationships



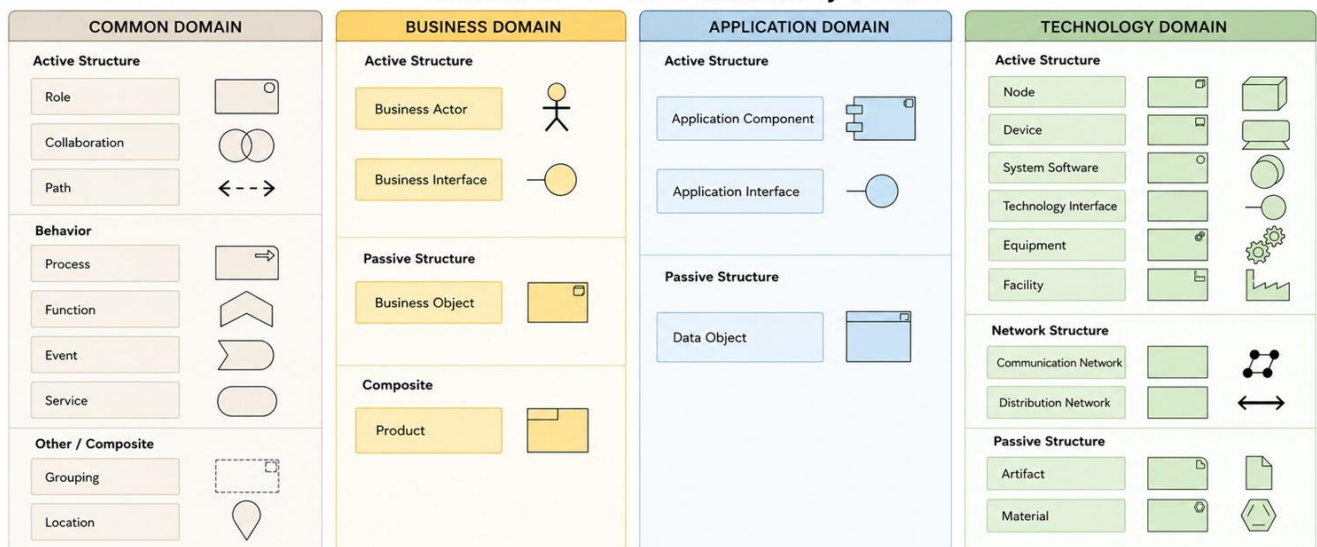
Junctions



© The Open Group

5.23.4 Overview

ArchiMate® 4 – Core Elements by Domain



Legend – Aspects

Active Structure	Elements that are capable of performing behavior
Behavior	Elements that describe behavior
Passive Structure	Elements on which behavior is performed
Network Structure	Elements used to connect active structure elements
Other / Composite	Elements that aggregate other elements

Notes

- Process, Function, Event and Service are in the Common Domain and can be used in the context of any of the other domains.
- Composite Elements (e.g., Product) combine or aggregate elements from the three aspects.

(...)

"If you please--draw me a sheep . . ."

When a mystery is too overpowering, one dare not disobey. Absurd as it might seem to me, a thousand miles from any human habitation and in danger of death, I took out of my pocket a sheet of paper and my fountain-pen. But then I remembered how my studies had been concentrated on geography, history, arithmetic and grammar, and I told the little chap (a little crossly, too) that I did not know how to draw. He answered me:

"That doesn't matter. Draw me a sheep . . ."

But I had never drawn a sheep. So I drew for him one of the two pictures I had drawn so often. It was that of the boa constrictor from the outside. And I was astounded to hear the little fellow greet it with,

"No, no, no! I do not want an elephant inside a boa constrictor. A boa constrictor is a very dangerous creature, and an elephant is very cumbersome. Where I live, everything is very small. What I need is a sheep. Draw me a sheep."

So then I made a drawing.



He looked at it carefully, then he said:

"No. This sheep is already very sickly. Make me another."

So I made another drawing.



My friend smiled gently and indulgently.

"You see yourself," he said, "that this is not a sheep. This is a ram. It has horns."

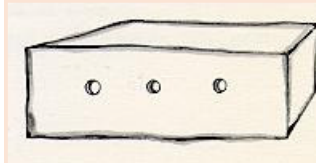
"So then I did my drawing over once more.



But it was rejected too, just like the others.

"This one is too old. I want a sheep that will live a long time."

By this time my patience was exhausted, because I was in a hurry to start taking my engine apart. So I tossed off this drawing.



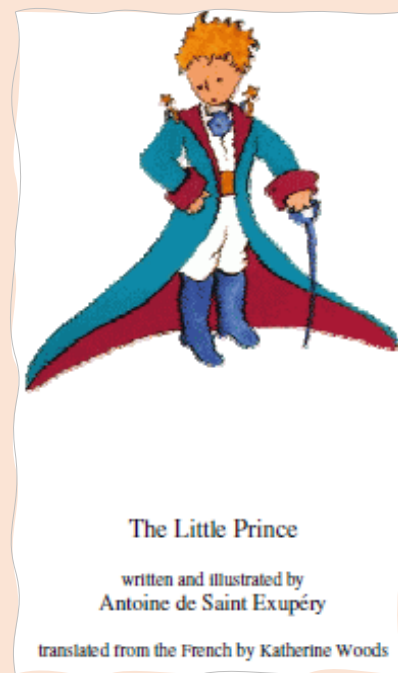
And I threw out an explanation with it.

"This is only his box. The sheep you asked for is inside."

I was very surprised to see a light break over the face of my young judge:

"That is exactly the way I wanted it! Do you think that this sheep will have to have a great deal of grass?"

(...)



5.25 <core> Wrap-up Enterprise Architecture and ArchiMate

In the world of Enterprise Architecture (EA), visualizing and communicating the structure of an organisation's systems, processes, and technology is critical. ArchiMate plays a key role in this by providing a framework to represent complex architectures in a clear, structured manner. It helps architects, analysts, and stakeholders alike to understand the relationships between business operations, software applications, and technological infrastructure, ensuring alignment with organisational goals.

ArchiMate is particularly effective because it addresses the challenge of managing the intricacies of EA across multiple layers (the core layers are business, application, and technology). By creating visual models that show how these layers interact, ArchiMate helps organisations understand how their various components fit together, which is essential for both current operations and future development.

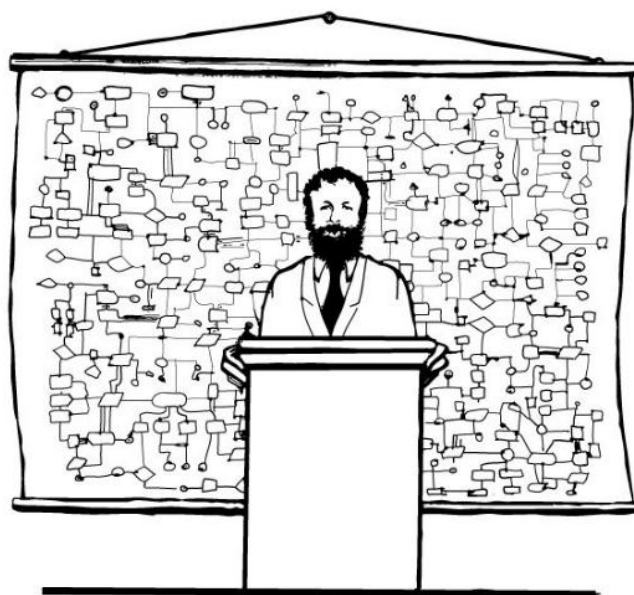
A crucial aspect of using ArchiMate effectively is defining the scope of what is being modelled. When modelling a system, it's important to avoid both oversimplification and unnecessary complexity. Proper scoping helps prevent the inclusion of irrelevant details, which can clutter the model, and ensures that essential elements are not omitted. The "white-box" and "black-box" approaches are central here: a white-box approach exposes the internal workings of a system or component when these are important to the model, while a black-box approach abstracts away these details, focusing only on inputs and outputs when internal structures are not relevant. ArchiMate is especially relevant for black-box views, while other languages might better address white-box concerns (see next sections for that).

ArchiMate therefore also supports the idea of balancing detail with simplicity in EA modelling. Overly complex models can be overwhelming and difficult for stakeholders to interpret, while models that are too simple might overlook critical details needed for decision-making. A pragmatic approach is recommended, where the model includes just enough detail to address the specific needs of the project without becoming unnecessarily complicated. This ensures that the model remains understandable, actionable, and aligned with the organisation's strategic objectives.

One of the weaknesses of ArchiMate is its ability to help resolve common conceptual dilemmas in EA modelling. For example, the distinction between "actors" and "roles" is often a source of confusion. ArchiMate offers clear guidelines: an actor represents an entity (like a person or organisation) that performs actions, while a role defines the behaviour and responsibilities of that actor in a particular context. Similarly, ArchiMate helps differentiate between "functions" and "services," ensuring that internal capabilities (functions) are separated from externally visible units of work (services) offered to users or other systems. However, the correct usage of either of those concepts always depends on the intended purpose of the model and the respective viewpoints under consideration. These are complex challenges that make the work of analysis at this level not easy even for experienced analysts...

Anyway, ArchiMate's structured approach not only makes complex architectures easier to visualize but also supports effective decision-making. By following best practices, architects can avoid ambiguities and create models that facilitate communication between technical teams and business stakeholders. This helps in bridging the gap between business strategy and IT implementation, ensuring that technology supports the broader organisational goals efficiently.

In conclusion, ArchiMate is a valuable tool for anyone involved in Enterprise Architecture. Its ability to represent complex architectures across multiple layers and aspects (while maintaining clarity and simplicity) makes it the most used present reference for organisations looking to align their business processes, applications, and technology. By using ArchiMate to model architectures effectively, organisations can not only improve their current operations but also better position themselves to adapt to future changes and challenges.



"Now that you have an overview of the system, we're ready for a little more detail"

6 <cover> Business Processes and BPMN

To complement this reading, see the detailed definitions and specifications of BPMN here:

- BPMN Specification: <https://www.omg.org/spec/BPMN>
 - Focus on these sections:
 - 7.1, 7.2.1, 7.3
 - 9.2, 9.3, 9.4
 - 10.1, 10.2, 10.3 (5, 6, 8), 10.4, 10.5, 10.6, 10.7, 10.8
 - 12.3.2
 - 13.3, 13.4, 13.5



A process refers to a sequence of tasks or activities carried out in a structured manner to achieve a specific outcome. In simple terms, it involves a set of inputs transformed through a series of steps into an output. Processes can be found everywhere, from manufacturing to service delivery, and they help ensure efficiency and consistency in achieving a goal. They are typically repeatable and can be optimised over time for better performance.

6.1.1 What is a Business Process?

A business process is a specific type of process that is essential for the operation of a business.

It is a collection of related, structured activities that produce a product or service, directly contributing to the fulfilment of business objectives. These processes typically span multiple departments or functions and may involve various stakeholders, such as customers, suppliers, and employees. Examples include order processing, payroll management, and customer service.

Business processes are often represented and modelled to allow organisations to analyse and improve them. One of the most commonly used modelling languages is BPMN (Business Process Model and Notation), which provides a standard way to depict the flow of tasks and decisions within a process.

6.1.2 The Concept of “Flow”

In process modelling, the term flow refers to the logical sequence in which tasks or activities are executed.

It shows the progression of tasks from one to another, indicating how information, decisions, or work items move through the process. In BPMN, flows are represented by arrows connecting different elements of the process, like tasks, decisions, and events.

Flows can be either sequential (where one task follows another) or parallel (where multiple tasks can happen simultaneously). Controlling the flow ensures that processes are executed in a structured and efficient manner, with dependencies between activities managed properly.

6.1.3 Case and Case Management

A case is a type of process that is less structured and more dynamic, involving work that cannot always be pre-defined in terms of a strict flow. Cases typically involve managing patients in hospitals or clinics, unexpected incidents in businesses, investigations, or escalated customer interactions, where tasks need to be handled based on evolving circumstances. This concept is especially relevant in domains like healthcare, legal services, or customer support, where each situation (case) may require a different approach.

Case management refers to the coordination and handling of cases from start to finish. Unlike standard processes, case management focuses on flexibility, allowing tasks and decisions to be made based on the specific context of each case. It prioritises outcomes over rigid workflows and enables workers to adapt as the case progresses.

6.1.4 Object-Centric Process Modelling

Traditional process models, like those in BPMN, are activity-centric, meaning they focus on the sequence of tasks. However, in some situations, it is more useful to focus on the entities or objects involved in the process. This is where object-centric process modelling comes into play.

An object-centric process views the business process from the perspective of the objects (e.g., documents, products, customers) that pass through it. Instead of focusing solely on the flow of activities, it emphasises the state changes of the objects involved. For instance, in a procurement process, the object would be the purchase order, and the model would track its status from creation to approval, shipment, and fulfilment.

This approach is useful in complex processes involving multiple interactions between different objects and actors, providing a holistic view of how these elements interact and change over time.

6.1.5 Conclusion

Processes, especially business processes, are essential for the smooth operation of any organisation. Understanding concepts like flow, case management, and object-centric processes is crucial for improving business efficiency and flexibility. By using process modelling languages and tools, businesses can visualise, analyse, and enhance their processes to meet ever-evolving demands.

6.2 <concept> Introduction to Business Processes and BPMN

Business Process Model and Notation (BPMN) is a standardized approach for visualizing, analysing, and automating business processes, making it a vital technique in systems analysis, modelling, and enterprise architecture.

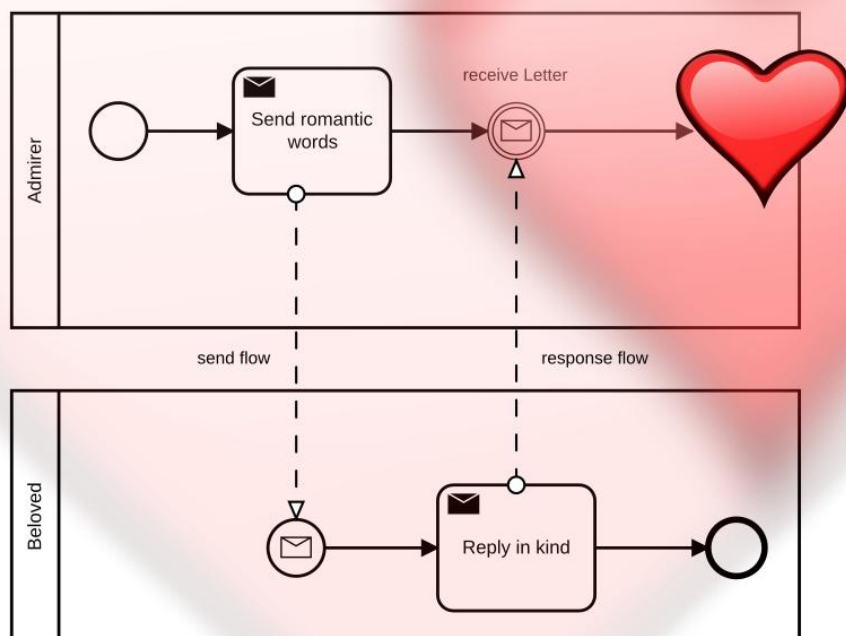
BPMN provides a clear and consistent way to document complex workflows, ensuring that business processes are well-understood and can be effectively communicated across different stakeholders.

However, organisations face multiple challenges when establishing a process architecture, such as defining process scope, maintaining consistency in terminology, and balancing standardization with flexibility. Business processes can be classified in types (core, support, and management). Also, compliance, auditing, and process maturity ensure that processes are effective and adhere to industry standards.

BPMN can be a foundation for all that. The language components include the concepts that are most common in process, such as flow objects (events, activities, gateways), connecting objects (sequence flows, associations), and artifacts (data objects, groups, annotations). Collaboration diagrams can model interactions between different participants, such as departments within an organisation or external partners. The concept of collaboration is tied to the need for effective coordination, highlighting how BPMN's use of pools and message flows makes these interactions explicit and manageable.

Determining the appropriate level of detail in process modelling is another challenge. The problem is to find right balance between the risks of overcomplicating models with too much detail versus the dangers of oversimplifying them and obscure critical elements. It is important to be aware of strategies for balancing detail and simplicity, including purpose-driven modelling, audience consideration, iterative refinement, and layered modelling.

Finally, the integration of BPMN with Model-Driven Engineering (MDE) can be an ultimate goal. Detailed BPMN diagrams can be deployed to process engines, enabling the automation of workflows directly from the models. Real-world examples of technology for how BPMN and MDE can be used in practice are Camunda, IBM Business Automation Workflow, or Bonita BPM. The benefits of this integration, such as improved efficiency, consistency, and scalability, demonstrate the powerful synergy that can be reached between BPMN and MDE in driving business process automation.



https://www.spiffworkflow.org/posts/deep_dives/messages/

6.3 <concept> Processes in a Business System

Business processes form the backbone of any organisation, orchestrating the sequence of activities that enable the delivery of products or services to customers. A business process is essentially a set of related tasks or activities, performed by people or systems, that together achieve a specific outcome.

6.3.1 The Value of Business Processes

The primary value of business processes lies in their ability to provide clarity and structure to organisational activities. By defining clear processes, organisations can ensure consistency, efficiency, and quality in their operations. Making processes explicit help eliminate redundancies, streamline workflows, and reduce the likelihood of errors, leading to improved productivity, cost savings, and higher customer satisfaction. Moreover, having documented and standardized processes is critical for training, compliance, and the scalability of operations.

Process-oriented organisations are those that make their processes explicit, making them better equipped to adapt to changes, whether due to market dynamics, technological advancements, or regulatory requirements. That is because making processes explicit facilitates communication and collaboration within and across stakeholders. When everyone in an organisation understands the processes, it is easier to coordinate activities and achieve goals.

6.3.2 Challenges in Establishing Process Architecture

Introducing a process-oriented approach in an organisation that previously did not consider the concept of processes presents several challenges. One of the most significant challenges is overcoming resistance to change. Employees and managers alike may be accustomed to informal or ad-hoc ways of working, thus introducing formal processes can be perceived as unnecessary or restrictive. This also is often compounded by the fact that different stakeholders may refer to the same concepts using different terminologies, leading to confusion and miscommunication. Another challenge is aligning the various processes within the organisation. In a business where processes have not been formally defined, different departments or teams might have developed their own ways of working, leading to inconsistencies. For example, what one department refers to as "customer service" might be what another department refers to "support service." Harmonizing processes requires careful analysis and a willingness to standardize practices across the organisation.

The complexity of mapping out processes also cannot be underestimated. It involves understanding the interdependencies between different activities, identifying bottlenecks, and ensuring that the processes align with the organisation's strategic objectives. Moreover, as organisations grow and evolve, processes must be continually reviewed and updated, adding another layer of complexity to the task of process management.

6.3.3 Types of Business Processes

Roughly speaking, business processes can be broadly categorized into three main types:

- **Core (or Production) Processes:** These are the processes that directly contribute to the creation of the organisation's primary products or services. In a manufacturing company, this might include processes like production, assembly, and quality control. These processes are critical to the organisation's value chain and have a direct impact on customer satisfaction. It is common to refer that these are the real Business Processes that matter, being all the remaining others "processes of the business", but really not "business processes."
- **Support Processes:** These processes do not directly produce the organisation's primary output but are essential for ensuring that core processes run smoothly. Examples include human resources, IT services, and procurement. While these processes might not be visible to the customer, they play a crucial role in maintaining the overall efficiency and effectiveness of the organisation.
- **Management Processes:** These processes involve the planning, monitoring, and controlling of the core and support processes. They include activities like strategic planning, performance management, and compliance monitoring. Management processes are essential for ensuring that the organisation is operating in alignment with its goals and regulatory requirements.

6.3.4 Compliance, Auditing, and Maturity

Compliance ensures that processes adhere to legal and regulatory requirements, while auditing involves regularly reviewing processes to ensure they are being followed correctly and remain effective. In process-oriented organisations, compliance and auditing are closely tied to the concept of process maturity. Process maturity refers to the extent to which processes are defined, managed, and optimized. At the lowest level, processes might be informal and inconsistently applied, while at the highest level, they are fully integrated into the organisation's operations, continuously monitored, and improved. High process maturity is often associated with certifications (like ISO 9001, ISO 27001, CMMI, etc.), which signify that the organisation has met rigorous standards for process management and quality.

6.3.5 Conclusion

The management of business processes is vital to the success of any organisation, providing the structure and clarity needed to operate efficiently and effectively. However, introducing that practice in organisations not previously oriented around them, comes with significant challenges

6.4 <further> Reference Process and Process Framework

In the context of Business Process Management (BPM), the terms **reference process** and **process framework** play a crucial role in helping organisations model, improve, and standardise their processes. Both concepts are widely used to ensure best practices, efficiency, and alignment across different business units or industries.

6.4.1 Reference Process

A reference process refers to a predefined, generic model of a business process that took in consideration the industry best practices.

It serves as a template or a benchmark that organisations can adopt or customise according to their specific needs. Reference processes are typically valuable for common business functions such as order-to-cash, procure-to-pay, or customer service, and provide a structured approach to executing tasks efficiently.

These processes are often developed by industry experts or software vendors who have a deep understanding of the common challenges and goals within specific sectors. For example, SAP offers reference processes as part of its ERP systems, which organisations can use to quickly implement business functions in areas such as finance, logistics, and human resources.

The key benefits of reference processes include:

1. **Standardisation:** Organisations can adopt standardised approaches to common tasks, reducing variability and ensuring consistency across operations.
2. **Efficiency:** Reference processes are typically conceived to be efficient and avoid unnecessary steps or redundancies, helping organisations save time and resources.
3. **Best Practices:** These processes incorporate industry best practices, enabling organisations to benefit from proven methods and strategies.
4. **Customisation:** While a reference process provides a starting point, organisations can tailor it to their specific context, adding or removing steps as necessary to fit their unique requirements.

Using a reference process is particularly beneficial when implementing new systems or entering new markets, as it allows organisations to start with a process model that is already optimised for success.

6.4.2 Process Framework

A process framework is a structured collection of related processes, arranged in a hierarchical manner, which provides a high-level overview of all business processes within an organisation or industry.

It serves as a blueprint that describes the relationships and dependencies between various processes, often across different departments or functions. A process framework offers a strategic view of how an organisation's processes are interconnected, ensuring alignment with overall business objectives.

Process frameworks are often industry-specific, but some generic frameworks are widely used across various sectors. Examples include:

- **APQC Process Classification Framework (PCF):** This framework offers a common language and structure for process management, helping organisations compare performance and share best practices across industries²².
- **ITIL (Information Technology Infrastructure Library):** ITIL is a widely adopted process framework for managing IT services, ensuring that IT aligns with business needs²³.

The key benefits of process frameworks include:

1. **Holistic View:** A process framework provides a comprehensive view of all processes within an organisation, making it easier to manage complex operations that span multiple departments or functions.
2. **Alignment:** It helps ensure that processes are aligned with business strategy, ensuring that the organisation's activities are contributing to its overall goals.
3. **Consistency:** A framework ensures that all processes follow a consistent approach, reducing fragmentation and improving coordination across the organisation.
4. **Process Improvement:** By mapping out all processes, a framework enables organisations to identify inefficiencies or bottlenecks and focus on areas where improvements can be made.

6.4.3 Relationship Between Reference Process and Process Framework

While a reference process is a model for a specific business function, a process framework provides the overarching structure that ties together multiple processes within an organisation. The framework offers the strategic context, while the reference processes deliver the operational details. Together, they help organisations standardise and optimise their processes at both the strategic and operational levels.

6.4.4 Conclusion

In summary, a reference process is a predefined model of a business function that incorporates industry best practices, while a process framework provides the larger structure that organises and connects various processes across an organisation. Both concepts are essential for businesses seeking to improve efficiency, ensure alignment, and maintain consistency in their operations.

²² <https://www.apqc.org/>

²³ <https://www.axelos.com/certifications/itil-service-management/what-is-itil/>

6.5 <concept> Process Architecture

Process architecture refers to the structured design and organisation of an organisation's business processes.

It provides a high-level overview of how various processes are interconnected, how they contribute to the organisation's goals, and how they interact with resources such as technology, people, and data. The concept is essential in aligning business activities with the organisation's strategy, ensuring efficiency, and supporting continuous improvement efforts.

6.5.1 Key Elements of Process Architecture

Process architecture consists of several key components that work together to provide a comprehensive understanding of how an organisation operates:

1. **Processes:** At the core of process architecture are the business processes themselves—sequences of activities or tasks that deliver value by transforming inputs into outputs. These processes can range from high-level, end-to-end processes (such as order-to-cash) to more granular sub-processes and tasks.
2. **Process Hierarchy:** Processes within an organisation are typically organised in a hierarchy, which is one of the essential features of process architecture. At the top of this hierarchy are core processes—those that directly contribute to delivering products or services to customers. Supporting processes, such as HR or IT services, sit beneath these, providing essential services that enable the core processes to function.
3. **Relationships Between Processes:** A key function of process architecture is to define the relationships and dependencies between various processes. This includes how inputs and outputs move between processes, the sequence of activities, and the decision points where processes intersect. These relationships help ensure that processes are not isolated but contribute to a cohesive organisational system.
4. **Process Roles and Responsibilities:** For each process in the architecture, roles and responsibilities are assigned to specific teams or individuals. This ensures accountability and clarity about who is responsible for each part of the process, contributing to better governance and operational efficiency.
5. **Process Interfaces:** Process architecture also defines interfaces between processes and other organisational elements, such as technology systems, organisational units, and external stakeholders. These interfaces ensure smooth coordination between various parts of the organisation and its external environment.

6.5.2 The Importance of Process Architecture

1. **Alignment with Organisational Strategy:** Process architecture is essential for ensuring that business processes align with the organisation's overall strategy. By mapping out processes and understanding their relationships, organisations can identify gaps, overlaps, or inefficiencies that may hinder their ability to achieve strategic goals.
2. **Operational Efficiency:** With a well-structured process architecture, organisations can streamline operations by eliminating redundant processes, improving process flow, and ensuring that resources are allocated effectively. This leads to better use of time, technology, and human resources.
3. **Process Standardisation:** Process architecture provides a foundation for standardising processes across the organisation. This is particularly important for large, multi-departmental organisations, where ensuring consistency and coherence in how processes are executed can drive quality and efficiency.
4. **Continuous Improvement:** By providing a clear overview of how processes operate and interact, process architecture facilitates continuous improvement initiatives. It allows organisations to regularly review and optimise their processes in response to changing market conditions, technology advancements, or internal business needs.

6.5.3 Conclusion

Process architecture serves as the blueprint for an organisation's business processes. It helps align operations with strategy, optimise efficiency, and support continuous improvement efforts. By providing a structured and holistic view of processes, roles, and interactions, process architecture enables organisations to better understand and manage their business activities, ensuring that they remain competitive and capable of adapting to changing circumstances.

6.6 <further> Languages to Model Business Processes

We can find several modelling languages in the industry to represent business processes. These languages provide frameworks for designing, analysing, and optimising processes across different industries.

6.6.1 Business Process Model and Notation (BPMN)

BPMN is one of the most widely adopted languages for business process modelling. Developed by the Object Management Group (OMG), BPMN uses a flowchart-like notation to describe the steps involved in a business process, making it accessible to both technical and non-technical stakeholders.

- Strengths: Visually intuitive, supports a wide range of process types, and bridges the gap between business and IT teams.
- Use Cases: Process documentation, workflow automation, and process improvement initiatives.
- Website: <https://www.omg.org/bpmn/>

6.6.2 Unified Modelling Language (UML)

UML, originally developed for software engineering, also supports business process modelling through its Activity Diagrams. These diagrams are useful for modelling workflows and dynamic behaviour within systems.

- Strengths: Versatile, suitable for both business and IT modelling, and ideal for projects requiring detailed interaction modelling.
- Use Cases: Modelling business processes that closely interact with software systems.
- Website: <https://www.uml.org/>

6.6.3 Event-Driven Process Chain (EPC)

EPC is a modelling language to depict workflows, particularly in conjunction with enterprise resource planning (ERP) systems like SAP. It focuses on the flow of events and functions, showing how events trigger activities.

- Strengths: Effective for organisations relying on SAP or similar ERP systems, with strong support for event-driven processes.
- Use Cases: Business process modelling in industries with complex ERP-driven workflows.
- Website: https://en.wikipedia.org/wiki/Event-driven_process_chain (Note: EPC does not have an official standalone website; information is available through many online resources)

6.6.4 Integrated Definition for Function Modelling (IDEF)

IDEF is a family of modelling languages, including IDEF0 (for function modelling) and IDEF3 (for process description). IDEF0 focuses on functions and activities, while IDEF3 captures the behaviour of processes.

- Strengths: Highly structured, ideal for detailed process and system analysis.
- Use Cases: Process improvement and systems analysis in industries such as defence, aerospace, and manufacturing.
- Website: <http://www.idef.com/>

6.6.5 Business Process Execution Language (BPEL)

BPEL is a language used for specifying business process behaviours that interact with web services, particularly in service-oriented architectures (SOA). It is focused on process automation and orchestration.

- Strengths: Excels in environments where web services are critical, enabling the automation of complex workflows.
- Use Cases: Business process automation in web service-heavy industries like e-commerce or telecommunications.
- Website: <https://www.oasis-open.org/committees/wsbpel/> (Note: BPEL is maintained by OASIS)

6.6.6 Petri Nets

Petri Nets are a mathematical modelling language that also provide a graphical technique for modelling dynamic processes with conditions and events. They are particularly well-suited to processes involving concurrency or complex synchronisation.

- Strengths: Powerful for analysing process performance, particularly in systems with parallel activities.
- Use Cases: Process optimisation in technical and engineering domains like manufacturing or logistics.
- Website: <http://www.informatik.uni-hamburg.de/TGI/PetriNets/> (Petri Nets does not have an official standalone website; information is available through many online resources)

6.6.7 Conclusion

The choice of business process modelling language depends on the specific requirements of the problem to be solved and the context of the organisation. BPMN is the most versatile and widely adopted, while UML, EPC, IDEF, BPEL, and Petri Nets offer distinct advantages for particular use cases and industries.

6.7 <technique> BPMN

BPMN (Business Process Model and Notation) is a widely accepted standard for modelling business processes. It offers a graphical representation that allows organisations to document and analyse their processes in a structured manner. BPMN diagrams are used to depict how work is done, how tasks are sequenced, and how different participants interact with one another. The primary goal is to bridge the gap between technical and business users, providing a common language for process analysis and improvement.

6.7.1 Types of Processes in BPMN

In BPMN, processes are classified into three main types, depending on the scope and the participants involved:

- **Private (Internal) Processes:** These are processes that occur entirely within a single organisation and do not involve external entities. They represent the internal workflows of an organisation, detailing tasks, decision points, and events that take place within its boundaries. Private processes are typically the most detailed in BPMN and are used for optimising internal efficiency.
- **Abstract (Public) Processes:** Abstract processes depict the interactions between an organisation and external participants. While they show the activities involved in exchanging information or services, they do not reveal the internal details of the organisation's workflow. Instead, they represent only the interfaces where the organisation interacts with external parties, making them ideal for modelling customer-facing processes or partner interactions.
- **Collaborative Processes:** Collaborative processes involve two or more participants (usually different organisations, departments, or entities) that interact with each other. These processes are modelled in collaboration diagrams, which show how messages and tasks flow between participants and how each entity contributes to the overall process, using pools to represent each participant and lanes to depict different roles or departments within a pool.

6.7.2 Types of BPMN Diagrams

To capture the various aspects of business processes, BPMN provides several types of diagrams.

Each type of diagram serves a specific purpose and focuses on different facets of process management:

- **Process Diagrams:** The most commonly used BPMN diagram, a process diagram illustrates the internal flow of activities within an organisation. These diagrams are typically used to model private processes, detailing tasks, events, gateways, and the sequence in which activities occur. The primary aim is to visualise the workflow in a way that enables optimisation and automation.
 - **Tasks:** Represent individual units of work or actions within the process.
 - **Events:** Indicate something that happens, such as the start or end of a process, or intermediate actions like waiting for a timer.
 - **Gateways:** Show decision points or divergence in the process, guiding the flow depending on conditions.
- **Collaboration Diagrams:** Collaboration diagrams represent interactions between different participants or entities. These diagrams use pools to represent organisations or departments and lanes to break down roles or subunits within the pool. The focus is on how participants communicate with each other, using message flows to show the exchange of information. Collaboration diagrams are ideal for processes where cross-functional or cross-organisational interactions are key, such as supply chains or joint ventures.
- **Choreography Diagrams:** Choreography diagrams focus on the interactions between participants rather than the individual activities within each participant's workflow. These diagrams model the sequence of messages exchanged between participants and the rules that govern these interactions. Choreography diagrams are useful when the primary concern is how communication between participants is orchestrated, rather than the internal workings of each participant.
- **Conversation Diagrams:** Conversation diagrams offer a high-level view of the interactions between participants, abstracting away the detailed flow of activities. Instead, they focus on the conversations—collections of message exchanges—between participants. This type of diagram is used to model how different parties collaborate at a higher level without diving into the intricacies of task management.

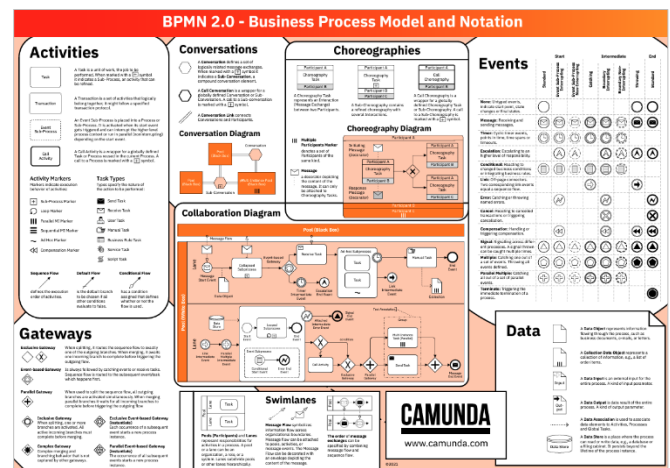
6.7.3 Conclusion

BPMN provides a versatile toolkit for modelling business processes, ensuring that both business and technical stakeholders can communicate effectively.

The classification of processes into private, abstract, and collaborative allows organisations to model workflows at various levels, whether for internal optimisation or cross-organisational cooperation.

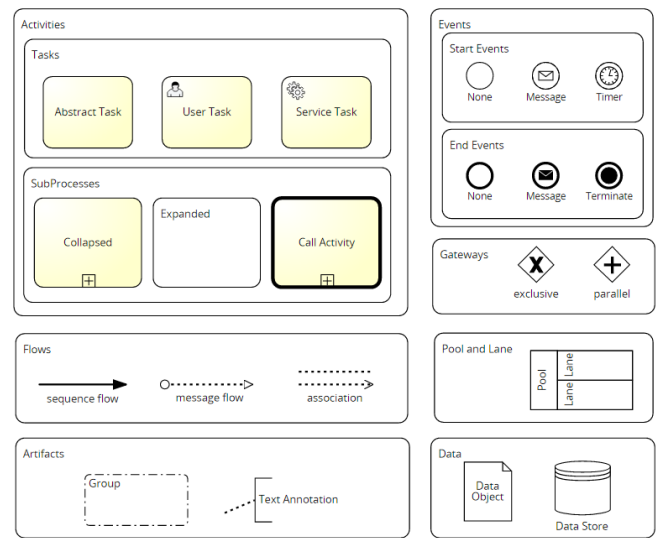
Meanwhile, the different types of BPMN diagrams (process, collaboration, choreography, and conversation) provide tailored views depending on the focus of the analysis.

This flexibility ensures that BPMN can be adapted to fit the needs of virtually any business context, from routine internal operations to complex, multi-participant collaborations.



6.8 <technique> The Core BPMN Elements

BPMN is a language for modelling business processes in a way that is both detailed and accessible to various stakeholders. By using a standardised set of symbols, BPMN enables clear visualisation and understanding of how business processes operate. Following, we refer to the core BPMN elements that are essential for modelling individual processes, excluding those specifically relevant to collaboration processes. These foundational elements are crucial for creating effective and comprehensible process models.



6.8.1 Flow Objects

Flow Objects are the primary elements that define the behaviour and progression of a business process. These are the essential building blocks of any BPMN diagram and include three main types:

- **Events:** Events are occurrences that happen during a process and typically trigger or influence the flow of the process. They are represented by circles and can occur at the start, during, or at the end of a process. Events are further categorised as Start Events (which indicate where the process begins), Intermediate Events (which indicate events that happen between the start and end of a process), and End Events (which indicate where the process ends). Example: A Start Event might be triggered by receiving an order, an Intermediate Event could represent a delay to wait, and an End Event could signify the completion of order processing.
- **Activities:** Activities represent the work or tasks that are performed in a process and are depicted as rounded rectangles. The most common type of Activity is a Task, which represents a single unit of work. There are also Sub-processes, which encapsulate a set of tasks that can be broken down further if needed. Example: In an order fulfilment process, tasks could include "Check Inventory," "Package Item," and "Ship Order."
- **Gateways:** Gateways control the flow of the process by dictating the branching and merging of paths. They are depicted as diamonds and represent decision points in the process. Common types of Gateways include Exclusive Gateways (which allow only one path to be taken) or Parallel Gateways (which allow multiple paths to be taken simultaneously). Example: An Exclusive Gateway might be used to determine if an order is in stock, directing the flow either to "Process Order" or "Notify Customer of Delay," depending on availability.

6.8.2 Connecting Objects

Connecting Objects are elements that link Flow Objects together, showing the flow and relationships between them. The primary Connecting Objects are:

- **Sequence Flows:** These are arrows that connect activities, events, and gateways, indicating the order in which tasks and events occur in the process. Sequence Flows model the logical progression of the process. Example: A Sequence Flow might connect a "Start Event" (Order Received) to a "Task" (Check Inventory), and then to an "End Event" (Order Completed).
- **Association:** Associations are used to link Artefacts (like data or documents) to Flow Objects, clarifying what information is used or produced by specific tasks or events. Example: An Association might link a "Data Object" representing an "Invoice" to a "Task" like "Generate Invoice."

6.8.3 Artifacts

Artifacts provide additional context and detail to the process being modelled. The most common are:

- **Data Objects:** These represent information that is used or produced during the process. Data Objects are depicted as rectangles with a folded corner and can be associated with specific tasks or events to show the flow of data. Example: A Data Object might represent a "Purchase Order" that is created during the process.
- **Data Stores:** A data store represents a persistent storage element where data is stored. It allows activities to read or write information, maintaining data beyond the duration of individual process instances.
- **Groups:** Groups are used to visually organise and categorise elements within a BPMN diagram without affecting the flow of the process. They help to clarify and simplify complex diagrams. Example: A Group might be used to highlight all tasks related to "Order Processing" within a larger process that addresses a wider goal.
- **Annotations:** Annotations provide additional textual information to the elements in the diagram. They add explanations or context to specific parts of the process. Example: An Annotation could be used to note that a particular Task should only be performed under certain conditions, such as "Only if order value exceeds 100€."

6.8.4 Conclusion

The core elements of BPMN (Events, Activities, Gateways, Connecting Objects, and Artifacts) are essential for modelling business processes in a clear and structured manner. By effectively using these elements, we can create BPMN diagrams that accurately represent the flow and interactions within a process.

6.9 <technique> BPMN Events in Short

Events represent things that "happen" during the lifecycle of a process. They are depicted by circles and come in different forms. In BPMN, there are three main types of events: Start Events, Intermediate Events, and End Events.

6.9.1 Start Events

A Start Event marks the initiation of a process. It acts as the trigger that kicks off a series of activities within the business model. In BPMN diagrams, Start Events are represented by a simple, thin-bordered circle. The trigger for a Start Event can vary, which is often indicated by an internal symbol. For instance, a clock symbol represents a Timer Start Event, suggesting that the process starts at a specific time or interval. Similarly, a message icon indicates a Message Start Event, triggered when a message is received.

6.9.2 Intermediate Events

Intermediate Events occur between the Start and End Events, reflecting points where something might happen to impact the flow of the process. They can either catch (receive) or throw (generate) something during the process execution. Intermediate Events are depicted by a double-bordered circle.

- **Catch Events:** These are events that wait for a trigger before continuing the process. For example, a Message Catch Event waits for a message to be received before proceeding. Similarly, a Timer Catch Event pauses the process until a specific time or date condition is met.
- **Throw Events:** These are events that actively trigger something to occur. A Message Throw Event, for example, sends a message to another participant, indicating that something has happened.

Intermediate Events can come in various forms:

- **Message Events:** These can either catch or throw messages between participants. They are crucial in modelling communication within business processes.
- **Timer Events:** Representing delays or waiting periods, Timer Intermediate Events might indicate that the process should wait for a specific duration or until a particular date.
- **Error Events:** These are used to model exceptions that might disrupt the usual flow, allowing the process to manage such errors effectively.
- **Signal Events:** Unlike message events, which involve specific recipients, Signal Events broadcast information across the entire process without targeting a particular receiver.

Intermediate Events can also appear on the boundary of an activity, representing boundary events. These can either be interrupting or non-interrupting:

- **Interrupting Boundary Events:** These events stop the ongoing activity when they are triggered. For example, an Error Boundary Event attached to an activity will halt that activity if an error occurs, diverting the process flow.
- **Non-Interrupting Boundary Events:** These allow the activity to continue while also triggering an alternative flow. For example, a Timer Non-Interrupting Boundary Event might trigger an alert or parallel action without stopping the ongoing activity.

6.9.3 End Events

An End Event indicates where a process or a particular path within a process completes. It is depicted as a circle with a thick border. The type of End Event helps to explain what happens when the process ends. For example:

- **Message End Events** are used to show that a message is sent when the process concludes.
- **Terminate End Events** indicate that all ongoing activities in the process must immediately cease.
- **Error End Events** suggest that the process ends due to an error, which might require further attention or action.

6.9.4 Special Event Types

In addition to these standard events, BPMN also has Conditional Events, which are triggered by the fulfilment of a condition, and Link Events, which can be used to connect different parts of the diagram for simplicity. Escalation Events indicate the need for additional attention or resources, while Compensation Events manage activities that need to be undone.

Events	Start		Intermediate				End
	Standard	Event Sub-Process Interrupting	Event Sub-Process Non-Interrupting	Catching	Boundary Interrupting	Boundary Non-Interrupting	Throwing
None: Untyped events, indicate start point, state changes or final states.							
Message: Receiving and sending messages.							
Timer: Cyclic timer events, points in time, time spans or timeouts.							
Escalation: Escalating to an higher level of responsibility.							
Conditional: Reacting to changed business conditions or integrating business rules.							
Link: Off-page connectors. Two corresponding link events equal a sequence flow.							
Error: Catching or throwing named errors.							
Cancel: Reacting to cancelled transactions or triggering cancellation.							
Compensation: Handling or triggering compensation.							
Signal: Signalling across different processes. A signal thrown can be caught multiple times.							
Multiple: Catching one out of a set of events. Throwing all events defined.							
Parallel Multiple: Catching all out of a set of parallel events.							
Terminate: Triggering the immediate termination of a process.							

6.10 <technique> BPMN Activities in Short

Activities represent the core tasks or units of work that are executed within a business process. They capture the actions necessary to achieve a process's objectives, ranging from simple tasks to more complex operations. BPMN categorises activities into different types to support a wide range of scenarios.

6.10.1 Task

A task is the most basic type of activity in BPMN. It represents an atomic unit of work, which cannot be broken down further within the process model. Tasks are used to illustrate straightforward actions, such as approving an invoice or sending an email. There are various types of tasks, each serving a specific role:

- **User Task:** Performed by a human actor, typically involving direct interaction, such as reviewing a document, filling out a form, or approving a request.
- **Service Task:** Executed by a system or software application without human intervention. Examples include automatically sending a notification or processing data through an API.
- **Manual Task:** Completed manually without system support. These tasks are often physical in nature, such as filing a paper document or assembling parts in a manufacturing process.
- **Script Task:** Automated tasks that execute a script (e.g., JavaScript, Python). These scripts are run internally to perform functions like data transformation, typically without user involvement.
- **Business Rule Task:** Evaluates business rules to make automated decisions based on predefined criteria. It is typically linked to a decision management system and is useful for tasks like validating a loan application or calculating discounts.
- **Send Task:** Represents sending a message or notification to an external participant or system, such as dispatching an email or sending a message to a partner organisation.
- **Receive Task:** Waits to receive a message from an external participant or system. This type of task is used to model events such as waiting for an order confirmation or receiving customer feedback.

6.10.2 Sub-Process

A sub-process is a collection of tasks grouped to form a larger, meaningful unit of work. Sub-processes provide a way to manage complexity by breaking down processes into smaller, more manageable components. They can be expanded to reveal the underlying sequence of tasks or kept collapsed to maintain simplicity. Sub-processes allow for greater modularity and reusability in process models.

6.10.3 Call Activity

A call activity is used to invoke another process or sub-process, promoting reusability across multiple workflows. This mechanism allows modellers to reference a predefined set of activities from different locations within a model, which simplifies the creation of consistent, modular, and easy-to-maintain process diagrams. For example, a "Customer Onboarding" process might be reused in various contexts by calling it through a call activity.

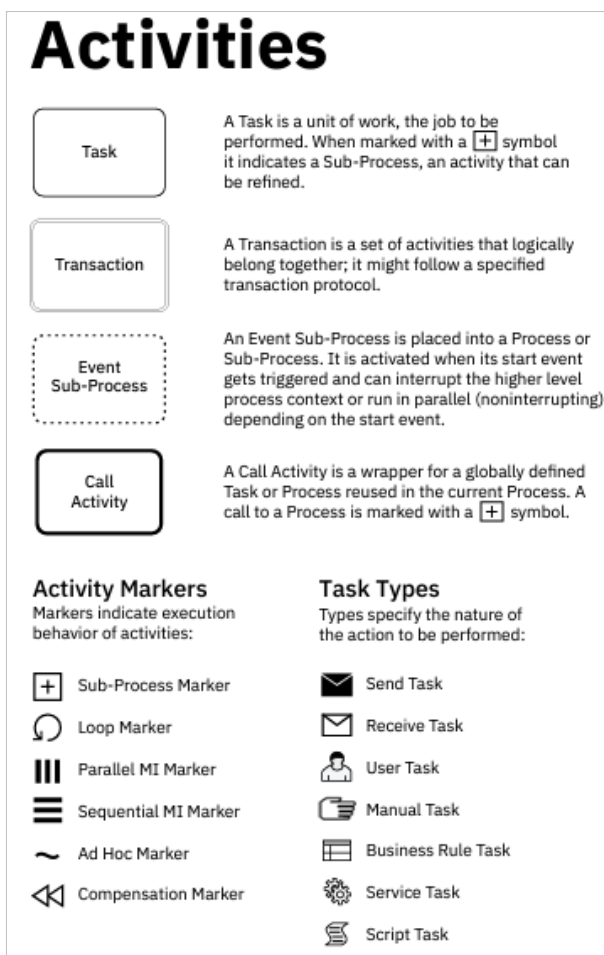
6.10.4 Loop and Multi-Instance Activities

BPMN supports activities that need to be repeated or performed multiple times to address dynamic requirements.

- **Loop Activities:** These activities repeat until a specified condition is satisfied. They are used in cases where the number of repetitions depends on a certain criterion, such as iterating until all data entries are verified.
- **Multi-Instance Activities:** These activities are repeated for multiple instances simultaneously. They can be either sequential (executed one after another) or parallel (executed concurrently). Multi-instance activities are ideal for processes like collecting survey responses from multiple participants or processing multiple orders concurrently.

6.10.5 Compensation and Ad-Hoc Activities

- **Compensation Activities:** These are used to define actions that should be performed to undo or compensate for work completed earlier, often employed in processes where rollback mechanisms are required. For example, reversing a financial transaction if an order is cancelled.
- **Ad-Hoc Activities:** Represent tasks that are not performed in a fixed sequence but rather when certain conditions or circumstances arise. They are typically used to model flexible, dynamic processes where the flow is not entirely predictable, such as handling exception cases during customer service.



6.11 <technique> BPMN Gateways in Short

Gateways are a fundamental concept in Business Process Model and Notation (BPMN), a widely used visual modelling language for representing business processes.

Gateways are used to control the flow of a process, determining how paths diverge and converge as the process progresses. In essence, gateways are the decision points that define the route a workflow will take, helping to manage the complexity of a business process by representing splits, merges, and synchronisation points.

There are different types of gateways in BPMN, each with specific behaviour for handling the flow of processes. Understanding these different types is essential for creating clear, precise process diagrams. Below are some of the key gateways in BPMN:

- **Exclusive Gateway (XOR):** The exclusive gateway, often represented by a diamond shape with an "X" marker, is used to define a point in the process where only one of several potential paths can be taken. It acts like a simple decision point: based on a condition, the process flow follows only one of the available paths. For example, an exclusive gateway might determine whether a customer order qualifies for a discount based on predefined criteria, sending the flow along one path if it qualifies and along a different path if it doesn't.
- **Parallel Gateway (AND):** A parallel gateway, represented by a diamond with a plus sign (+) marker, is used to split the process into multiple parallel paths or to synchronise parallel flows back into one. Unlike the exclusive gateway, all outgoing paths are taken simultaneously, meaning each parallel branch will run independently. This type of gateway is useful when multiple activities need to be completed simultaneously, such as when different departments are tasked with distinct activities that do not depend on each other.
- **Inclusive Gateway (OR):** An inclusive gateway, depicted by a diamond with a circle marker, allows one or more paths to be taken based on conditions. This type of gateway can enable multiple outgoing flows to execute in parallel if certain conditions are met, making it more flexible than the exclusive gateway. It is also capable of merging those paths back together. For example, if a customer order can qualify for multiple types of discounts, the inclusive gateway can determine which discounts apply, potentially taking multiple paths simultaneously.
- **Event-Based Gateway:** The event-based gateway, represented by a diamond with a small pentagon marker, directs the flow based on events rather than conditions. When reaching an event-based gateway, the process must wait for an external event to occur, such as receiving a customer response or a timer expiring, before proceeding along the appropriate path. This type of gateway is particularly useful in processes where actions are driven by customer interactions or external triggers.
- **Complex Gateway:** The complex gateway, symbolised by a diamond with an asterisk (*) marker, is used in situations where more advanced conditions control the process flow. It allows for complex combinations of conditions, events, and paths, which may involve multiple merging or diverging scenarios that cannot be represented by simpler gateways. This is often used when specific, detailed control of a process is needed.
- **Exclusive Event-Based Gateway:** A variation of the event-based gateway, the exclusive event-based gateway waits for exactly one of several possible events to occur, determining the path based on whichever event happens first. This gateway helps ensure that only one event-driven path is taken at a time.

In summary, gateways are essential elements in BPMN that allow business processes to be represented in a structured and understandable manner. Each type of gateway provides a different method for controlling the flow, enabling the process modeller to define sequences, decisions, and parallel activities effectively. By using the appropriate gateways, BPMN diagrams can accurately reflect the decision-making and concurrency requirements of real-world business processes.

Gateways

Exclusive Gateway



When splitting, it routes the sequence flow to exactly one of the outgoing branches. When merging, it awaits one incoming branch to complete before triggering the outgoing flow.

Event-based Gateway



Is always followed by catching events or receive tasks. Sequence flow is routed to the subsequent event/task which happens first.

Parallel Gateway



When used to split the sequence flow, all outgoing branches are activated simultaneously. When merging parallel branches it waits for all incoming branches to complete before triggering the outgoing flow.



Inclusive Gateway

When splitting, one or more branches are activated. All active incoming branches must complete before merging.



Exclusive Event-based Gateway (Instantiate)

Each occurrence of a subsequent event starts a new process instance.



Complex Gateway

Complex merging and branching behavior that is not captured by other gateways.



Parallel Event-based Gateway (Instantiate)

The occurrence of all subsequent events starts a new process instance.

In BPMN, collaboration processes are used to model interactions between different participants, known as "pools", which typically represent individuals, departments, or systems. These processes are essential for illustrating how various parts of an organisation, or even different organisations, work together to achieve a common goal. Collaboration processes capture the exchange of information and the flow of tasks between these participants, making them invaluable for understanding and optimizing inter-organisational or cross-departmental workflows.

6.12.1 The Concept of Collaboration

At its core, a collaboration process is about depicting how different entities (participants) interact with each other through the exchange of messages. Each participant in a collaboration is represented by a pool, and the interactions between these pools are shown using message flows. This contrasts with a simple process diagram, which typically focuses on the flow of tasks within a single participant or organisational unit.

6.12.2 Key Elements of Collaboration Processes

In addition to the core BPMN elements, collaboration processes introduce several other elements:

- **Pools:** A pool represents a participant in the process as an "execution environment." It can be an entire organisation, a department, or even an external entity. Each pool acts as a container for the process flow related to that participant. In a collaboration diagram, multiple pools are used to depict the different participants involved in the collaboration. Example: In a supply chain process, one pool might represent the supplier, while another pool represents the buyer. The processes within each pool show the steps taken by each participant.
- **Lanes:** Lanes are subdivisions within a pool that help to organize and categorize activities according to roles, teams, or departments. Lanes do not necessarily represent different participants but rather different functions or roles within the same participant's process. Example: Within a pool representing a "Sales Department", lanes could be used to differentiate between the "Marketing Department" and the "Support Department," among others, each with its own set of activities within a "Selling process."
- **Message Flows:** are the connectors used to show the communication between different pools (participants) in a collaboration process. They are represented by dashed arrows with a small envelope icon, indicating the flow of information or messages between entities. Unlike Sequence Flows, which depict the order of tasks within a single participant, Message Flows are used exclusively for interactions between separate pools. Example: In the same supply chain process, a Message Flow might represent the sending of a purchase order from the buyer's pool to the supplier's pool. Another Message Flow might show the supplier sending an invoice back to the buyer.

6.12.3 Collaboration Process Scenarios

Collaboration processes are especially useful in scenarios where different entities must work together yet operate independently. Some common scenarios include:

- **Inter-Organisational Processes:** These involve collaboration between different companies, such as in supply chain management, where manufacturers, suppliers, and distributors coordinate their activities.
- **Cross-Departmental Processes:** Within a large organisation, different departments may need to collaborate in a coordinated way on a shared project, such as a new product development that involves research, marketing, and production departments.
- **Client-Vendor Relationships:** Businesses often collaborate with external vendors or clients, requiring clear communication of requirements, deliverables, and feedback.

6.12.4 Benefits of Modelling Collaboration Processes

The concept of coordination is closely tied to collaboration processes. When a process is depicted as a simple flow, even if it models a single pool including multiple lanes, the simple handovers between lanes can obscure the underlying need for coordination among different entities or roles. These lanes, while useful for organizing tasks within a single participant, do not fully expose the complexities of coordination required between different functional areas. In contrast, when each entity is modelled in a separate pool, the interactions between them are explicitly depicted through message flows. This approach makes the need for coordination more apparent, as the exchanges between pools highlight the dependencies and communication required to synchronize activities across different parts of the organisation. By using pools instead of lanes, the BPMN diagram shifts focus from internal task flow to the external coordination necessary to achieve the overall process objectives, thus providing a clearer understanding of how collaboration truly functions within the process.

6.12.5 Conclusion

Collaboration processes in BPMN are essential for modelling and understanding how different participants interact within a business process. By using Pools, Lanes, and Message Flows, BPMN provides a robust framework for visualizing these interactions. Whether dealing with inter-organisational processes, cross-departmental workflows, or client-vendor relationships, collaboration processes help ensure that all participants are effectively coordinated, leading to more efficient and successful outcomes.

6.13 <heuristic> Common Issues in BPMN Models

When modelling business processes with BPMN, it is essential to avoid common errors that can undermine clarity, efficiency, and accuracy. These issues often stem from improper use of BPMN elements, leading to confusion and misinterpretation. Understanding these pitfalls can help modellers create clear and effective BPMN diagrams. Below are some of the most frequent issues encountered and examples to illustrate them.

6.13.1 Uncontrolled Flow

One of the most basic yet frequent mistakes in BPMN modelling is the omission of clear start and end events. Even if the formal specification of the BPMN language does not mandate that, every real-life process flow must have a defined beginning and at least one end, to avoid ambiguity.

- *Example:* Consider a BPMN diagram representing a customer support process that includes activities such as "Receive Request," "Assess Issue," and "Resolve Issue." If this model lacks a start event that indicates how the process begins or an end event to show when it finishes, the process flow becomes unclear. Those interpreting the diagram may struggle to understand when the process should be initiated or when it is complete, leading to potential confusion.

Heuristic: Assure there is always only one start event and as few as possible end events (two end events always must mean different outcomes; using two events for the same outcome is a bad practice).

6.13.2 Gateway Divergences without Convergences

Gateways in BPMN are crucial for managing process flow, often representing decision points or parallel paths. However, overusing diverging gateways without corresponding converging gateways can lead to loose ends in the process.

- *Example:* Imagine a process that splits into several paths using decision gateways but never converges them back into a unified flow. This can suggest that the process has multiple possible endpoints, potentially leaving tasks unfinished or paths unresolved. If that is really the case, bad luck, we are facing a potentially "wicked" process. However, usually that is not the case... Therefore, to ensure a well-managed flow, each diverging gateway should have a corresponding converging gateway, where the alternative process paths come back together once the reasons for those alternatives have been addressed.

Heuristic: This risk can be assessed with the same heuristic for uncontrolled flow, meaning of too many end events might imply divergences where not consolidated by the corresponding convergences...

6.13.3 Improper Use of Sequence Flows

Sequence flows are used to show the logical and temporal relationships between elements in a process. A common issue arises when sequence flows are used to connect activities that should be independent or occur in a different sequence, which can distort the process logic.

- *Example:* In a model for order processing, "Verify Payment" might be connected directly to "Ship Order" without including necessary intermediate steps like "Prepare Invoice." This misrepresents the sequence, leading to confusion or implementation errors. Ensuring that sequence flows reflect the actual process order is essential.

Heuristic: If two activities can occur simultaneously, the model must use parallel gateways; independent tasks should either be placed on different paths or represented in separate pools.

6.13.4 Misuse of Task Types

In BPMN, different task types (such as user tasks, service tasks, and manual tasks) have specific meanings and should be used appropriately to avoid confusion. Using the wrong task type can mislead those interpreting the diagram about who or what is responsible for a task.

- *Example:* If a model uses a service task to represent manual data entry performed by a human, this would be misleading, as a service task typically indicates an automated process. The appropriate task type for manual tasks or user interactions should be used to accurately reflect the nature of the task and its executor.

Heuristic: In early process analysis stages, it may be beneficial to avoid specifying task types until absolutely necessary.

6.13.5 Incorrect Use of Message Flows

Message flows in BPMN are intended to represent communication between participants in different pools (e.g., different departments or organisations). A common mistake is using message flows to connect activities within the same pool.

- *Example:* A diagram might incorrectly use a message flow to connect two activities, "Review Application" and "Approve Application," within the same department. In this case, a sequence flow should be used instead. This misuse can lead to confusion, as message flows imply communication between separate contexts, which is not the case here.

Heuristic: Experienced modelers really do not do these mistakes... Novice modelers must use proper tools to model, as tools that recognize BPMN usually do not allow to model these flows 😊

6.13.6 Conclusion

Avoiding these common BPMN modelling issues is vital for producing clear, accurate, and useful process diagrams. Ensuring that processes have defined start and end points, that gateways are properly managed, that sequence flows reflect actual process logic, and that task types are correctly used will result in more effective BPMN models. By following these guidelines, modellers can create BPMN diagrams that accurately represent business processes and support successful implementation.

6.14 <technique> BPMN – Execution in Private Diagrams

In **private BPMN** diagrams (e.g., those that are presented without pools and therefore where the entity executing it is not explicit, or those that are represented inside one pool, where the entity is explicit), **only Activities** (Tasks or Sub-Processes) are executed and consume resources of the entity performing the process.

In other words:

- **Activities** represent concrete actions carried out by people, systems, or devices.
- **Sequence flows** (arrows) and **decision elements** (gateways) **do not consume the entity's resources**: they are interpreted and executed by the **process execution infrastructure** (e.g. BPM engine, orchestration rules).

6.14.1 Practical Implication

- When modelling a private process, never assume that “evaluating a condition” or “following an arrow” is something done by the worker.
- The worker or system only performs activities. Path selection and transitions between activities are automatically managed by the process infrastructure (orchestration engine, administrative workers that simply must support the work with no need of specific skills for the purpose of the process, etc.).

6.14.2 Common Mistakes to Avoid

- **Mistake 1:** assuming gateways imply human work (“the manager decides whether to go to A or B”) → In the model, the manager’s decision is not an explicit **activity** (e.g. *Review Request*), but, for example, a gateway.
- **Mistake 2:** assigning “costs” or “duration” to events or sequence flows → These are merely triggers or logical connectors, with no resource consumption by themselves (use of resources only can be measured in relation to activities).

6.14.3 Simple Examples

Expense Reimbursement Process

1. In a diagram we see:
 1. Activity: *Complete Reimbursement Form*.
 2. Gateway: *Is the amount over £100?*
 3. Next Activities: *Process Reimbursement* and *Analyse for Approval*
2. Reading → The employee only **completes** the form and only **analyses for approval** depending of the gateway condition (> £100) which is a business rule evaluated automatically by the infrastructure.

Internal Purchasing Process

1. In a diagram we see:
 1. Activity: *Submit Purchase Request*.
 2. Catch Event: *Supplier Data*
 3. Next Activity: *Register New Supplier* (if not registered).
2. Reading → The event is not executed by anyone: the system moves the flow based on the event that produces supplier data (an event that was produced by some external entity: an email that was sent, a form that was upload, etc.).

Medical Appointment Scheduling

1. In a diagram we see:
 - Activity: *Enter Appointment Request*.
 - Gateway: *Is the doctor available?*
 - Next Activities: *Confirm Appointment* or *Suggest New Date*.
2. Reading → The gateway represents only the process logic. Resources are consumed in the activities of entering, confirming, or rescheduling.

6.14.4 Golden Rule

Whenever we ask, “who performs this?”, the answer should never be “the flow”, “the event” or “the gateway”. Only **Activities** are executed by process agents; control elements are handled by the **execution infrastructure**.

Model-Driven Engineering (MDE) and Model-Based Systems Engineering are approaches that focus on creating and exploiting models, which are the primary artifacts of the development process, meaning that the intended outcomes of the engineering result directly from the models.

In the context of business process management, MBSE & MDE leverages detailed BPMN models to automate the execution of business processes using process engines. By bridging the gap between high-level process design and actual system implementation, BPMN models can be transformed into executable workflows, significantly improving efficiency and consistency across organisational operations.

6.15.1 BPMN in the Context of MBSE & MDE

What makes BPMN particularly valuable in MBSE & MDE is its ability to serve not only as a design technique but also as a blueprint for automation. If enough detailed, BPMN models can be interpreted by process engines, which automate the execution of the processes defined in these models. This capability turns BPMN from a simple modelling technique into a central component of a fully automated workflow system.

6.15.2 How BPMN Models Drive Automation

When a BPMN model is created with sufficient detail, including all tasks, decision points, events, and flows, it can be used directly by a process engine to automate the process. This is achieved through a series of steps in MBSE & MDE:

- 1 Designing the BPMN Model: The process begins with designing a detailed BPMN model that accurately represents the process. This model includes all necessary components, such as tasks (automated or manual), gateways (for decisions), events (start, end, or intermediate), and the sequence flow that connects them.
- 2 Deploying the Model to a Process Engine: Once the model is complete, it is deployed to a process engine. This is a software system that interprets the BPMN model and manages the execution of the defined process. The process engine handles task execution, monitors progress and ensures that the workflow follows the rules defined in the model.
- 3 Execution of the Process: The process engine automates the execution of the process by triggering tasks, sending notifications, handling exceptions, and interacting with other systems as defined in the model. For example, when a task is completed, the engine automatically moves to the next step in the process, whether it's triggering another task, taking a decision, or waiting for a specific event.
- 4 Monitoring and Optimization: As the process is executed, the process engine can monitor performance, track metrics, and identify bottlenecks. This data can be used to refine the model, making continuous improvements to the process.

Several technologies and products leverage BPMN models for process automation in the context of MBSE & MDE. Here are just a few examples:

- Camunda: Camunda is an open-source platform that provides a process engine capable of executing BPMN models. Organisations can design their workflows in BPMN, deploy them to the Camunda engine, and automate their execution.
- IBM Business Automation Workflow: IBM's platform combines BPMN-based process modelling with other automation capabilities.
- Bonita BPM: Bonita BPM is another platform that enables BPMN-based process automation.

6.15.3 Benefits of Using BPMN in MBSE & MDE

The integration of BPMN and MDE offers several significant benefits:

- Consistency and Accuracy: Models as the basis for automation, organisations ensure that the process is executed exactly as designed, reducing the risk of human error and ensuring consistency.
- Efficiency: Automated processes run faster and more efficiently than manual ones, freeing up human resources for more strategic tasks.
- Flexibility and Scalability: Models can be easily modified to adapt to changing business needs, and these changes can be quickly implemented through the process engine, allowing the business to scale effectively.
- Transparency and Monitoring: Process engines provide real-time monitoring and reporting, giving organisations visibility into process performance and enabling data-driven decision-making.

6.15.4 Conclusion

BPMN, when combined with MBSE & MDE, transforms business process models from mere diagrams into powerful artifacts for process automation. By using detailed models and deploying them to process engines, organisations can automate complex workflows, ensuring that their processes are efficient, consistent, and adaptable to changing needs. Products like Camunda, IBM Business Automation Workflow, and Bonita BPM are real-world examples for that.

6.16 <further> Tools for Business Process

There is a wide offer of tools that provide comprehensive platforms for managing business processes, in some cases comprising feature like process mining (a technique that uses data from event logs to analyse, monitor, and improve real business processes by discovering patterns, checking compliance, and identifying inefficiencies):

- **ARIS by Software AG**
 - ARIS (Architecture of Integrated Information Systems) is a leading tool for enterprise business process analysis and management. It offers extensive support for BPMN, UML, and other modelling standards, providing a robust platform for designing, analysing, and optimising business processes.
 - Website: https://www.softwareag.com/en_corporate/platform/aris.html
- **IBM Blueworks Live**
 - IBM Blueworks Live is a cloud-based business process management tool that enables teams to collaborate on process modelling. It is known for its simplicity and ease of use, making it ideal for organisations looking to streamline their business processes without deep technical expertise.
 - Website: <https://www.ibm.com/products/blueworks-live>
- **Signavio by SAP**
 - Signavio is a business process management tool that integrates business process modelling, decision management, and workflow automation. Now part of SAP, Signavio offers extensive capabilities for process optimisation and transformation, making it a powerful tool for large enterprises.
 - Website: <https://www.signavio.com/>
- **Bizagi**
 - Bizagi is a digital business platform that combines business process management, automation, and workflow execution. It provides tools for modelling and automating processes using BPMN, with a focus on improving operational efficiency.
 - Website: <https://www.bizagi.com/>
- **Lucidchart**
 - Lucidchart is an intuitive, cloud-based diagramming tool that supports business process modelling and many other types of diagrams. While not as feature-rich as other enterprise-level tools, it is highly accessible and widely used for creating quick and clear process diagrams.
 - Website: <https://www.lucidchart.com/>
- **Oracle BPM Suite**
 - Oracle BPM Suite is a comprehensive business process management platform that integrates with Oracle's broader suite of enterprise solutions. It offers robust capabilities for modelling, executing, and monitoring business processes, with strong support for BPMN.
 - Website: <https://www.oracle.com/middleware/technologies/bpm.html>
- **Pega**
 - Pega is a platform focused on business process management and customer engagement. It offers powerful tools for modelling, automating, and optimising processes, particularly in environments requiring high adaptability and integration with customer-facing operations.
 - Website: <https://www.pegacorp.com/products/platform/bpm>
- **Appian**
 - Appian is a low-code automation platform that combines business process management with AI and automation. It allows businesses to quickly model, execute, and monitor complex workflows and processes.
 - Website: <https://www.appian.com/>
- **TIBCO Business Studio**
 - TIBCO Business Studio is a comprehensive BPM tool that supports process modelling, simulation, and execution. It integrates with TIBCO's suite of analytics and integration tools, making it a strong choice for data-driven process management.
 - Website: <https://www.tibco.com/products/tibco-business-studio>
- **Enterprise Architect by Sparx Systems**
 - Enterprise Architect is a comprehensive modelling tool that supports UML, BPMN, and many other languages and standards. It is widely used for enterprise architecture, software development, and business process modelling.
 - Website: <https://sparxsystems.com/products/ea/>



6.17 <core> Wrap-up Business Processes and BPMN

Business processes are the fundamental building blocks of how organisations function and deliver value. They are the mechanisms through which organisations transform inputs into outputs, delivering value to customers and achieving business objectives. Business processes provide the **structure and clarity** essential for organisational success. Well-defined business processes are crucial for:

- **Efficiency:** By eliminating redundancies and streamlining flows, organisations can optimise resource and reduce costs.
- **Consistency:** Standardised processes ensure that tasks are performed uniformly, regardless of personnel, technology or location, leading to predictable and reliable outcomes.
- **Quality:** Well-defined processes incorporating quality control measures help minimise errors and ensure that outputs consistently meet predefined standards.
- **Scalability:** Documented (modelled) processes provide a blueprint for replicating successful practices and onboarding new employees or teams efficiently.
- **Compliance:** In regulated industries, documented processes demonstrate adherence to legal and industry standards.
- **Agility:** Managed processes enable to adapt more readily to changing markets, customers, or technology.

We can categorise business processes into three main types: **Core Processes** (these that directly contribute to the primary value proposition of the business, such as product development, manufacturing, or service delivery); **Support Processes** (these that underpin the core processes with essential functions like human resources, IT support, or accounting); and **Management Processes** (these processes govern the operation of the other two types, encompassing strategic management, governance, and process improvement itself).

However, establishing a **process-oriented culture** can be challenging. Organisations might encounter resistance to change, difficulties in achieving consistent terminology, and the need to balance standardisation with the flexibility required in dynamic environments. Overcoming these hurdles necessitates: **Clear Communication** (articulate benefits of process management to stakeholders, addressing concerns transparently); **Gradual Implementation** (starting with pilots and incrementally expanding process initiatives across the organisation); **Employee Training** (equipping employees with the skills to understand, follow, and contribute to process improvement); and **Appropriate Tools** (implementing tools can significantly aid in visualising, analysing, and managing business processes).

6.17.1 BPMN

BPMN as a vital tool for visualising, analysing, and communicating business processes. BPMN offers a standardised set of symbols and constructs, enabling clear and unambiguous representation of process workflows. Its key advantages include: **Accessibility** (the graphical notation is intuitive and readily understandable by both technical and non-technical stakeholders); **Standardisation** (as widely adopted, it ensures consistency in process modelling across different departments, organisations, and industries); **Flexibility** (it can represent a wide range of process complexities, from simple, linear workflows to highly dynamic, collaborative processes); and **Automation** (models can be directly executed by process engines, enabling the automation of workflows and further enhancing efficiency). Essential elements of BPMN are:

- **Flow Objects:** The building blocks of a BPMN diagram, representing events, activities, and gateways that guide the process flow.
- **Connecting Objects:** Depict the relationships and flow of control between flow objects, using sequence flows, message flows, and associations.
- **Swimlanes:** Organise activities based on the roles or departments responsible, enhancing clarity and accountability.
- **Artefacts:** Provide additional information and context to the diagram, such as data objects, groups, and annotations.

6.17.2 Collaboration Processes in BPMN

BPMN excels at modelling **collaboration processes**, which involve interactions between different participants. By using pools and lanes to represent participants and message flows to illustrate communication, BPMN provides a clear way to model and analyse complex interactions.

6.17.3 Common BPMN Pitfalls to Avoid

Common BPMN modelling errors can lead to confusion and misinterpretations. These include, for example:

- **Uncontrolled Flow:** Lack of clear start and end events, resulting in ambiguous process boundaries.
- **Gateway Divergences Without Convergences:** Leading to incomplete or illogical process paths.
- **Improper Use of Sequence Flows:** Connecting activities within the same pool with message flows.
- **Misuse of Task Types:** Using incorrect task types (e.g., service task for a manual task).

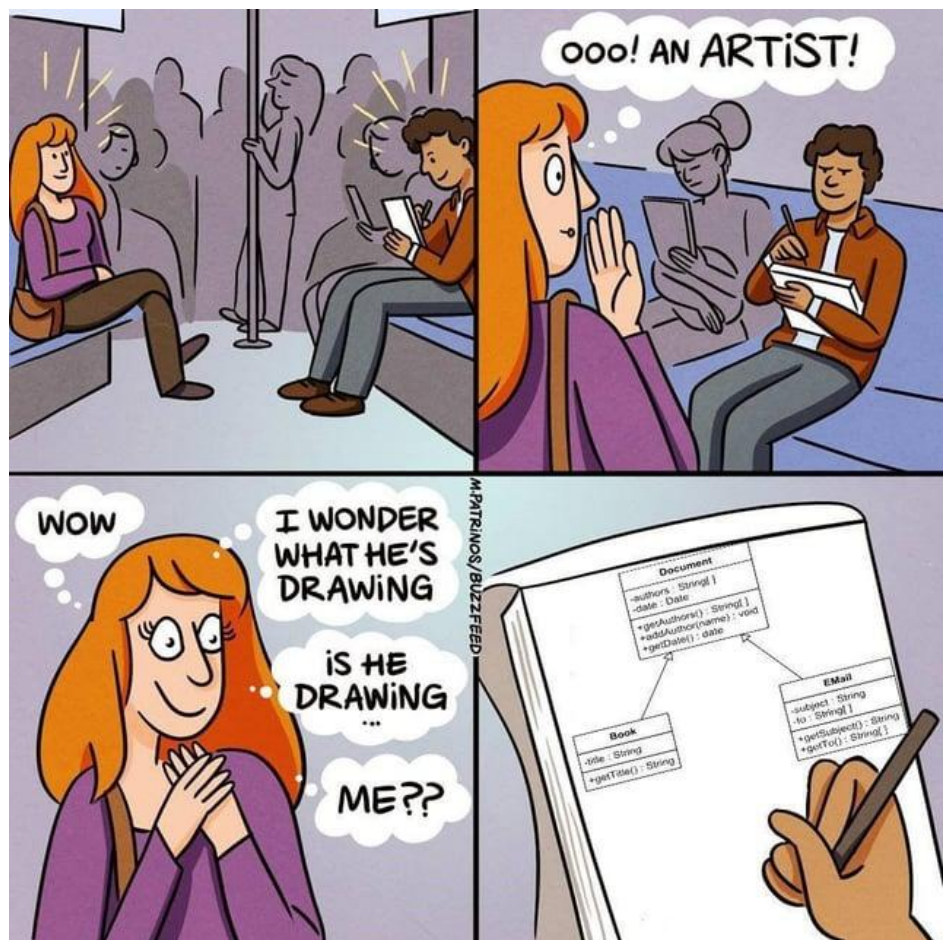
6.17.4 Conclusion

Effectively managing business processes is crucial for any organisation aiming to thrive in today's dynamic environment. By embracing a process-centric approach, leveraging BPMN for modelling and analysis, and exploring the potential of process automation, organisations can streamline operations, improve customer satisfaction, and achieve a significant competitive edge. Also, BPMN, when combined with **Model-Driven Engineering**, can move beyond mere process visualisation to drive automation. By creating sufficiently detailed BPMN models, organisations can directly execute them using **process engines**.

7 <cover> Software Systems and UML

To complement this reading, see the detailed definitions and specifications of BPMN here:

- UML Specification: <https://www.omg.org/spec/UML>
 - Focus on these sections:
 - 10.1, 10.2, 10.4
 - 11.1., 11.2, 11.4, 11.5
 - 13.1, 13.2, 13.3
 - 14.1, 14.2, 14.3, 14.4
 - 15.1, 15.2, 15.3, 15.6
 - 17.1, 17.2, 17.3, 17.4, 17.5, 17.6, 17.7, 17.8, 17.9, 17.10, 17.11
 - 18.1



7.1 <core> Introduction to Software Systems and UML

In the rapidly evolving landscape of software management, the complexity of systems and the methodologies used to design and implement them have significantly expanded. The Unified Modelling Language (UML) as a pivotal technique for the analysis of problems in this domain.

Software systems today are integral to the operation of almost every modern organisation, serving as the backbone for processes ranging from simple data management to complex, mission-critical applications. The scale and complexity of these systems have grown exponentially, necessitating a more structured approach to their design and development. Traditional methods of software engineering, while still relevant, are often insufficient when dealing with the intricate architectures and extensive requirements of contemporary systems. This is where Systems Analysis and Modelling, underpinned by UML, plays a critical role.

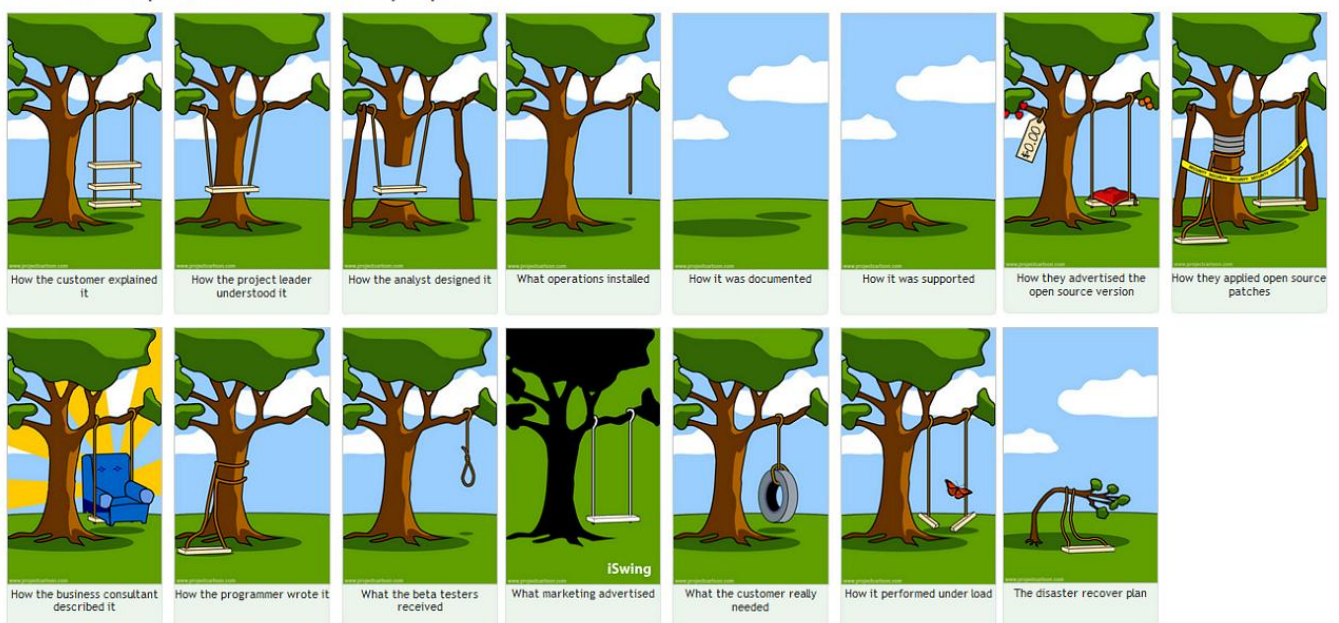
The Unified Modelling Language (UML) has emerged as a standardised, general-purpose modelling language that offers a comprehensive set of techniques for analysing, visualising, specifying, constructing, and documenting the artefacts of software systems. UML provides a rich visual framework that facilitates communication among stakeholders, ensuring that complex systems are understood from multiple perspectives, including structural and behavioural viewpoints and views.

This document delves into various facets of software systems analysis, beginning with an exploration of their key components and further challenges. It highlights how these components (ranging from application software to databases and ultimately user interfaces) must be carefully orchestrated to create systems that are not only functional but also scalable, resilient, and secure.

Object-Oriented (OO) concepts form the foundation of many modern software systems. The principles of OO design, such as encapsulation, inheritance, and polymorphism, bring benefits but also potential downsides. This exploration is critical for understanding how OO concepts align with the practicalities of system analysis, particularly when using UML.

The document outlines the application of UML in various stages of the software development lifecycle, but the focus is on problem analysis and requirements engineering. Specific UML techniques are relevant that purpose, including use case diagrams, class diagrams, state machine diagrams, and sequence diagrams. Those are essential instruments for capturing and communicating the intricate details of software systems.

Product development from an IT failures perspective



<https://i.pinimg.com/originals/2b/e7/a4/2be7a4ef07b7fa83e85119491548303c.png>

7.2 <technique> The Unified Modelling Language

The Unified Modelling Language (UML) is a standardized general-purpose object-oriented modelling language that has become a cornerstone in software engineering. UML consists of various diagram types, which are grouped into two main categories: structural diagrams and behavioural diagrams.

7.2.1 Structural Diagrams

Structural diagrams focus on the static aspects of a system, describing the system's components, their relationships, and how they interact:

- **Class Diagrams:** These are fundamental to object-oriented design, depicting classes, their attributes, methods, and the relationships between them (e.g., inheritance, association).
- **Component Diagrams:** These diagrams show the organisation and dependencies among different software components, providing a high-level view of the system's architecture. This is particularly useful in large systems.
- **Composite Structure Diagrams:** Show the internal structure of a structured classifier, such as a Class or Component, in a particular context. They represent its Parts, Ports, and Connectors, making explicit how the internal parts collaborate and how the whole interacts with its environment. They are particularly useful for moving from a black-box description of an element to a white-box description of its internal composition.
- **Deployment Diagrams:** Illustrate the physical deployment of software artifacts on nodes, such as servers or devices.
- **Object Diagrams:** Object diagrams represent instances of classes at a specific moment in time, showing objects and their relationships. They provide insight into the system's state during execution.
- **Package Diagrams:** Package diagrams help organize large systems by grouping related classes, components, or other elements into packages. This modular approach makes the system easier to manage and understand.

7.2.2 Behavioural Diagrams

Behavioural diagrams capture the dynamic aspects of a system, showing how it behaves and interacts over time:

- **Sequence Diagrams:** Sequence diagrams model the interaction between objects over time, showing the sequence of messages exchanged. They are crucial for understanding the flow of control within the system.
- **State Machine Diagrams:** These diagrams depict the states an object can be in and the transitions of these states in response to events. They can model the lifecycle of objects in systems where state changes are significant.
- **Activity Diagrams:** Activity diagrams are like flowcharts and model the workflow of a system. They show the high-level sequences of activities and decision points, making them ideal for representing complex processes.
- **Use Case Diagrams:** Use case diagrams provide a high-level view of the system's functionality by illustrating the interactions between external actors (users or other systems) and the system itself. They are essential for gathering and understanding system requirements.
- **Communication Diagrams:** Communication diagrams, also known as collaboration diagrams, focus on the interactions between objects, emphasizing their relationships rather than the time sequence of interactions.
- **Timing Diagrams:** Timing diagrams are specialized sequence diagrams that focus on the timing constraints of interactions. They are particularly important in real-time systems where timing is critical.
- **Interaction Overview Diagrams:** These diagrams combine elements of activity and sequence diagrams, providing simplest overviews of how complex interactions within a system occur.

7.2.3 The Role of UML in the Software Development Lifecycle

UML can be used in various stages of the software development lifecycle, enhancing efficiency and effectiveness:

- **Requirements Gathering:** UML diagrams, such as use case diagrams, are used to visualize system functionality and gather requirements. They help ensure that all stakeholders have a shared understanding of the system's capabilities and constraints.
- **System Design:** During the design phase, UML diagrams like class, component, and deployment help define the system's architecture. These diagrams serve as blueprints that guide developers in implementing the system's structure.
- **Implementation:** Although UML is not used directly for coding, the models created during the design phase provide detailed guidance for implementation. Developers can refer to UML diagrams to ensure that the system is built according to the specified design.
- **Testing and Validation:** UML supports testing and validation by providing models that can be used to develop test cases. For example, sequence and state machine diagrams are particularly valuable in identifying scenarios and conditions that need to be tested.
- **Documentation:** UML diagrams serve as an important part of system documentation. They offer a visual representation of the system that can be easily understood by developers, maintenance teams, and new team members. This documentation is crucial for ongoing maintenance and future enhancements.

7.2.4 Conclusion

UML comprehensive set of diagrams allows for the clear, standardized modelling of both the static structure and dynamic behaviour of software systems. In the software development lifecycle, it can cover requirements gathering and system design to implementation, testing, and documentation. By providing a visual framework that supports communication, clarity, and consistency, UML helps to create robust, scalable, and maintainable software systems.

UML offers a suite of diagrams that can be used as techniques to help capture, analyse, and communicate complex requirements clearly and effectively. UML is particularly useful for problem analysis and requirements engineering because it provides standardised, visual ways to represent system functionalities and behaviours.

When a system will be built with object-oriented technology, all the UML diagrams might be useful. However, for a phase of early problem analysis, when we want to abstract of the technology, a small number of those diagrams are especially valuable (even if we already know that the system will be not built on OO technology).

7.3.1 Use Case Diagrams to Capture Functional Requirements

Use case diagrams can play a vital role in the early stages of requirements engineering, offering a high-level overview of the system's functionality from the user's perspective. They help identify interactions between the system and external actors (users or other systems), ensuring all necessary functionalities are captured.

Benefits of Use Case Diagrams:

- **Identifying Stakeholders:** They clarify which users (humans or systems) will interact with the software.
- **Defining System Boundaries:** These diagrams delineate the context of the system and what the system will do and what lies outside its scope, helping to prevent scope creep.
- **Ensuring Comprehensive Coverage:** By listing all use cases, these diagrams act as a checklist, ensuring no critical functions are overlooked.
- **Facilitating Communication:** Use case diagrams are easily understood by non-technical stakeholders.

7.3.2 Class Diagrams for Domain Modelling

Class diagrams, when used for domain modelling, model structural aspects of the problem domain. Rather than focusing on software implementation, these diagrams illustrate the key entities, their attributes, and relationships within the problem space.

Benefits of Class Diagrams in Domain Modelling:

- **Abstracting the Problem Domain:** They focus on essential concepts, without technical details.
- **Clarifying Relationships:** They help in understanding how entities relate to one another, highlighting dependencies and potential bottlenecks.
- **Consistency Across Models:** They provide a consistent reference point across various views, ensuring alignment.
- **Smooth Transition to Design:** They can bridge the gap between problem analysis and detailed design.

7.3.3 State Machine Diagrams to Model Dynamic Behaviour

State machine diagrams model the dynamic behaviour of objects within the system, focusing on their states and transitions triggered by events, showing their lifecycle.

Benefits of State Machine Diagrams:

- **Capturing Object Lifecycles:** They detail how objects transition between states, ensuring all dynamic behaviours are documented.
- **Ensuring Behavioural Consistency:** State machine diagrams verify that objects behave consistently with the requirements, which is essential for systems where state changes impact overall functionality.
- **Identifying Edge Cases:** These diagrams force the exploration of all possible states, helping to identify and address potential edge cases early.
- **Improving Developer Understanding:** By clearly illustrating state changes, these diagrams reduce ambiguity during implementation, ensuring that the developed system aligns with the intended design.

7.3.4 Sequence Diagrams for Detailed Interaction Analysis

Sequence diagrams are focused on the time-ordered flow of messages between objects or components with or within the system. They provide a detailed view of how actors interact with the system or different parts of the system interact during specific use cases, offering insights into behaviour.

Benefits of Sequence Diagrams:

- **Visualising Interaction Flow:** Sequence diagrams clearly represent the sequence of interactions, making it easier to understand the flow of control in the system.
- **Clarifying Responsibilities:** They show which objects or components handle specific operations, helping to clarify roles and improve system design.
- **Ensuring Comprehensive Coverage:** By detailing how use cases are realised, sequence diagrams ensure that all necessary interactions are accounted for.
- **Aiding in Test Case Development:** These diagrams provide a basis for developing detailed test cases, ensuring all interaction paths are tested and validated.

7.3.5 Conclusion

UML diagrams like use case, class (for domain modelling), state machine, and sequence, are powerful techniques in problem analysis and requirements engineering. Each diagram can provide unique insights that ensure system requirements are thoroughly understood, clearly documented, and accurately communicated.

By effectively using these UML techniques, we can align technical designs with business needs, anticipate potential issues, and provide a solid foundation for subsequent analysis and development phases.

7.4 <technique> Use Case Diagrams in UML

Use Case Diagrams are a fundamental tool within the UML, designed to capture the functional requirements of a system by illustrating the interactions between external actors and the system itself. These diagrams are particularly valuable during the early stages of system development, as they provide a high-level overview of what the system should do from the perspective (viewpoint) of end-users and other interacting systems.

7.4.1 Core Concepts

- **Actors:** An actor represents a role that interacts with the system. This can be a human user, an external system, or even a hardware device. Each actor must represent a specific role that defines the interaction, making it a crucial element for understanding the system's boundaries and interfaces.
- **Use Cases:** A use case is a specific functionality or process that the system performs in response to an interaction with an actor. It is represented by an oval in the diagram. Use cases capture the essential requirements of the system by focusing on what the system needs to do rather than how it will do it. Each use case should have a clear purpose and outcome, often documented in more detail elsewhere.
- **System Boundary:** The system boundary defines the scope of the system being modelled, typically shown as a rectangle encompassing all the use cases. This boundary helps to distinguish between what is inside the system (what the system is responsible for) and what is external to it.
- **Relationships:**
 - **Association:** **A relation between an actor to a use case**, in the form of a line, indicating that the actor participates in the use case. The nature of the interaction (e.g., whether the actor initiates the use case or simply receives some output from it) is often elaborated in the accompanying documentation.
 - **Include:** **This relationship is used between two use cases**, when a use case is always executed as part of another use case. It allows for the reuse of common functionality across multiple use cases.
 - **Extend:** **This relationship is used between two use cases**, to indicate that a use case can be extended by another under certain conditions, adding optional or conditional behaviour to the base use case.
 - **Generalization:** **Both actors and use cases can be generalized**, indicating inheritance of behaviour or characteristics. For example, a general "User" actor might have specialized actors like "Admin" or "Regular User," each inheriting the general behaviours of "User" but with specific interactions unique to them.

7.4.2 Purpose and Benefits

Use case diagrams serve multiple purposes:

- **Requirement Gathering:** They help in capturing functional requirements by focusing on what the system must do, making them an essential technique during the requirements gathering phase. This ensures that all stakeholders, including non-technical ones, have a clear understanding of the system's capabilities.
- **Defining System Scope:** By clearly delineating the system boundary, use case diagrams help prevent scope creep, ensuring that the project remains focused on the agreed-upon functionality.
- **Facilitating Communication:** Use case diagrams are easily understood by all stakeholders, from developers to end-users, making them an excellent technique for facilitating communication and ensuring that everyone has a shared understanding of the system's functions.
- **Verification and Validation:** They can be used to verify that the system meets the user's needs by checking that all necessary interactions and outcomes are accounted for. During testing, use cases can be mapped to test cases, ensuring comprehensive coverage.

7.4.3 Best Practices

When creating use case diagrams:

- **Keep it Simple:** Focus on clarity and simplicity. Avoid cluttering the diagram with too many details; instead, use complementary documentation to elaborate on each use case.
- **Consistent Naming:** Use consistent and descriptive names for actors and use cases to avoid confusion.
- **Iterative Refinement:** Develop the use case diagram iteratively. Start with broad strokes and refine it as more information becomes available.

7.4.4 Conclusion

Use Case Diagrams are an indispensable technique in software development, providing a clear, high-level view of the system's functionality. Their ability to bridge the gap between stakeholders' needs and the technical design makes them a cornerstone of effective requirements engineering.

7.5 <technique> Scenarios in Use Cases

A use case is a powerful tool for modelling requirements in systems engineering. It represents a functional requirement, describing a behaviour of the system from an external perspective of the actors. A use case focused on how actors interact with the system to achieve a specific goal, meaning when executed, it changes the state of the system. Use cases are particularly effective for capturing requirements in user-oriented, goal-driven ways, providing a bridge between stakeholders and developers.

A key aspect to consider is that each use case often has more than one scenario. A scenario is a sequence of steps that describe a particular path through the use case, showing how the system and actors interact under certain conditions. The main or "happy path" scenario typically describes the most common or ideal sequence of events that leads to achieving the user's goal. However, other scenarios, known as alternative or exception paths, may also occur. These could include variations where different conditions are met, errors arise, or alternative decisions are made by the user. Including multiple scenarios within a use case ensures a more comprehensive understanding of system behaviour, accommodating different circumstances that the system may face in real-world usage.

When documenting use cases, it is helpful to use a standardised template. This ensures consistency and completeness across all use cases and makes it easier for stakeholders to understand and validate requirements. The following elements are typically included in a use case template (each analysis team must define the best template that fits in the purpose of the modelling activity underway):

- **Use Case Name:** A descriptive name that succinctly identifies the purpose of the use case.
- **Actors:** The primary and secondary actors involved in the use case. Actors are entities (users, external systems, etc.) that interact with the system. Identifying all relevant actors helps clarify who will interact with the system and ensures that all perspectives (viewpoints) are considered.
- **Goal:** The specific goal that the primary actor wishes to achieve by interacting with the system. This is the primary purpose of the use case and helps define what value the system provides.
- **Assumptions:** Any assumptions that are being made about the use case, actors, or environment. Including assumptions provides clarity and context, helping stakeholders understand the scope of the use case.
- **Preconditions:** The conditions that must be true before the use case begins. Preconditions define the system's state or requirements that need to be met for the use case to start successfully. They help ensure that all the necessary conditions are in place, thereby reducing ambiguity.
- **Triggers:** The event that initiates the use case, often representing an actor's action or an external condition that causes the system to react.
- **Basic Flow (Main Scenario):** The standard sequence of steps describing how the actor and the system interact to achieve the use case goal. This flow is typically the most common and successful path, also known as the "happy path".
- **Alternative Flows:** Variations from the main scenario that lead to different outcomes. These may occur due to alternative choices made by the actor or due to certain conditions being met. Including alternative flows helps describe the system's behaviour comprehensively and ensures all possible interactions are accounted for.
- **Exception Flows:** Specific scenarios where errors or issues occur, preventing the actor from achieving the intended goal. These flows highlight how the system handles failures or unexpected conditions, ensuring robustness in system design.
- **Postconditions:** The conditions that are true after the use case has been successfully completed (or failed). Postconditions describe the resulting state of the system and help clarify the expected outcomes, both in successful and unsuccessful scenarios.

USE CASE 5		Buy Goods
Description	Buyer issues request directly to our company, expects goods shipped and to be billed.	
Used by	Manage customer relationship (use case 2)	
Preconditions	We know Buyer, their address, and needed buyer information.	
Success End Condition	Buyer has goods, we have money for the goods.	
Failed End Condition	We have not sent the goods, Buyer has not spent the money.	
Actors	Buyer, any agent (or computer) acting for the customer. Credit card company, bank, shipping service	
Trigger	purchase request comes in.	
DESCRIPTION	Step	Action
	1	Buyer calls in with a purchase request
	2	Company captures buyer's name, address, requested goods, etc.
	3	Company gives buyer information on goods, prices, delivery dates, etc.
	4	Buyer signs for order.
	5	Company creates order, ships order to buyer.
	6	Company ships invoice to buyer.
	7	Buyers pays invoice.
EXTENSIONS	Step	Branching Action
	3a	Company is out of one of the ordered items: 3a1. Renegotiate order.
	4a	Buyer pays directly with credit card: 4a1. Take payment by credit card (use case 44)
	7a	Buyer returns goods: 7a. Handle returned goods (use case 105)
VARIATIONS		Branching Action
	1	Buyer may use phone in, fax in, use web order form, electronic interchange
	7	Buyer may pay by cash or money order check credit card

By adhering to a standard template, we ensure that each use case is well-structured, complete, and easy to understand. This consistency allows stakeholders, developers, and testers to communicate effectively, leading to better alignment between requirements and system implementation.

Documenting multiple scenarios within use cases also provides insight into how the system behaves under varying circumstances, helping identify potential issues, define system boundaries, and develop robust solutions. Ultimately, use cases, supported by a detailed and consistent template, are a fundamental technique for effective requirements engineering, ensuring that system behaviour is fully understood, documented, and validated.

7.6 <example> A Use Case with Scenarios

Let us model a University Course Management System (UCMS), focusing on how students, professors, and administrators interact with the system. The UCMS allows students to enrol in courses, professors to manage course content, and administrators to manage users. In this UCMS, several actors interact with the system to perform various tasks, such as enrolling in courses, submitting assignments, and managing course content.

A single use case can have multiple scenarios that illustrate different ways a user might achieve the same goal or handle unexpected events. Let's explore how a use case, "Enrol in Course," might have multiple scenarios, including both the main scenario and some alternative scenarios, to demonstrate the versatility of the system.

7.6.1 Main Scenario: Enrol in Course

The primary scenario of the "Enrol in Course" use case describes the most straightforward path where a student successfully enrolls in a course that is available and meets the system's requirements:

1. Student Logs In: The student logs into the UCMS using their credentials.
2. UCMS Displays Courses: The system displays a list of available courses.
3. Student Selects Course: The student selects the course they wish to enrol in.
4. UCMS Verifies Eligibility: The system checks if the student meets the prerequisites for the course and if there are available seats.
5. UCMS Updates Enrolment: Upon successful verification, the system adds the student to the course, updates the enrolment status, and sends a confirmation email to the student and notifies the professor of the new enrolment.

In this scenario, everything proceeds smoothly, and the student successfully enrolls in the course with no issues.

7.6.2 Alternative Scenario 1: Course Full

In this alternative scenario, the course the student wishes to enrol in is full, and they are unable to complete the enrolment.

- 5.1. Course Full: The system determines that the course is fully booked and displays a message to the student indicating that no seats are available.
- 5.1.1. Alternative Options: The student is given the option to either join a waiting list for the course or choose a different course from the available list.

In this scenario, the enrolment process is interrupted because the course is full, but the system provides the student with alternative options to manage this situation.

7.6.3 Alternative Scenario 2: Student Does Not Meet Prerequisites

In this scenario, the student attempts to enrol in a course but does not meet the required prerequisites.

- 5.2. Prerequisite Not Met: The system determines that the student does not meet the necessary prerequisites for the course and displays an error message explaining the situation.
- 5.2.1. Alternative Options: The system suggests alternative courses that match the student's qualifications or informs the student that they need to complete the required prerequisites before enrolling in the selected course.

In this scenario, the student cannot proceed with the enrolment due to unmet prerequisites, but the system provides guidance on what steps to take next.

7.6.4 Alternative Scenario 3: System Error During Enrolment

Occasionally, unexpected system errors may occur, preventing the student from completing their enrolment.

- 5.3. System Error: Due to an unexpected technical issue, the system is unable to complete the enrolment process and displays a system error message to the student.
- 5.3.1. Retry Option: The student is prompted to either retry the enrolment process or contact system support for assistance.

This scenario addresses a viewpoint concerning the "quality of the system"²⁴. The enrolment fails due to a system error, but the student is provided with options for how to proceed.

7.6.5 Conclusion

As illustrated in these examples, a single use case can have multiple scenarios that depict different paths depending on specific conditions. By modelling both the main and alternative scenarios, developers and stakeholders can better understand how the system behaves under various circumstances. Each scenario adds depth to the overall use case, ensuring that all possible interactions are accounted for, and the system is designed to handle real-world complexities.

²⁴ The term "quality of the system", which in business domains also can be referred to as "excellence", extends beyond minimal functional requirements to include user experience, safety, and well-being. It encompasses usability, ensuring ease of use; emotional satisfaction in interactions (UX); and safety, protecting against physical, mental risks or other risks. Accessibility promotes inclusivity, while trust and reliability build user confidence. By reducing stress and confusion, design choices enhance overall user satisfaction, aligning with human-centred design and striving for excellence in delivering a superior user experience.

Domain models are essential for representing the structural aspects of a problem space, abstracting the real-world entities, their attributes, and the relationships between them. Modelling that with class diagrams, therefore, provide a blueprint for understanding the key components of a system and how they interact, without delving into implementation details.

7.7.1 Core Concepts

In UML, a class represents a blueprint for objects within the system. It encapsulates attributes (properties or data) and methods (functions or behaviours) that define the state and behaviour of the objects. Classes are depicted as rectangles divided into three compartments: the top for the class name, the middle for attributes, and the bottom for methods.

Attributes represent the data or properties of a class. For example, a "Customer" class might have attributes like "Name", "Address", and "PhoneNumber." Attributes are typically defined with their data types, such as string, integer, etc., providing clarity on what kind of data the class handles.

Methods define the behaviour or functionality of a class, representing the operations that objects of the class can perform, thereby defining the behaviour of the class in the system. When doing an early problem analysis, it is very important to consider if methods really should be modelled when making a class diagrams for domain modelling, as that might not only complicate the present model but also constraint the future design (or be totally useless for that if the system will be developed using a technology not compatible with that object-oriented approach).

Diagrams are made of classes associated by relationships

- **Association:** An association represents a relationship between two or more classes. It is depicted as a line connecting the classes. For example, a "Customer" class might be associated with an "Order" class, indicating that customers place orders.
- **Multiplicity:** Multiplicity defines the number of instances of one class that can be associated with a single instance of another class. For example, a customer might place multiple orders, while each order is linked to one customer.
- **Aggregation and Composition:** These are special types of association that represent whole-part relationships. Aggregation (a hollow diamond) implies a relationship where the part can exist independently of the whole, while composition (a filled diamond) indicates a stronger relationship where the part cannot exist independently.
- **Inheritance (Generalization):** Inheritance depicts a hierarchical relationship where one class (subclass) inherits attributes and methods from another class (superclass). For example, a "PreferredCustomer" class might inherit from the "Customer" class, gaining all its attributes and methods but also adding new ones specific to preferred customers.

7.7.2 Purpose and Benefits

Class diagrams serve several critical purposes in domain modelling:

- **Abstraction:** They abstract complex real-world systems into manageable components, focusing on essential aspects without unnecessary details. This simplification is crucial for understanding and communicating the structure of the system.
- **Clarification of Relationships:** Class diagrams clarify how different entities within the system interact with each other. By modelling associations, dependencies, and hierarchies, these diagrams help in identifying the roles and relationships that exist within the domain.
- **Consistency Across Models:** Class diagrams provide a consistent reference that can be used across various stages of system development, ensuring that all team members have a shared understanding of the system's structure.
- **Transition to Design:** Although primarily used for domain modelling, class diagrams also facilitate the transition to system design by providing a clear outline of the objects and their interactions, which can then be translated into detailed design models and eventually into code (this, only if an object-oriented language will be used for coding).

7.7.3 Best Practices

When constructing class diagrams for domain modelling:

- **Focus:** Focus on the problem domain, avoiding implementation details like data structures or algorithms.
- **Use Meaningful Names:** Ensure that class and attribute names are meaningful and reflective within the domain.
- **Simplify Relationships:** Avoid overly complex relationships; use aggregation and composition appropriately to represent whole-part relationships and keep the model understandable.
- **Iterate and Refine:** Diagrams should evolve with the analysis. Start with a broad overview and refine the diagram as more details become clear during the process.

7.7.4 Conclusion

Class diagrams are a powerful technique in domain modelling, providing a visual representation of the structure and relationships within a system. Their use ensures that the complexities of the domain are captured and understood, facilitating better communication, design, and implementation.

7.8 <heuristic> Normalization Principles

When creating a domain model using UML class diagrams during problem analysis, applying principles akin to database normalization can significantly enhance the clarity, consistency, and maintainability of our model. Although these principles originate in the context of database design, where they help reduce redundancy and ensure data integrity, their application in UML class diagrams is equally beneficial in structuring domain models effectively.

7.8.1 Eliminating Redundancy

Redundancy in a domain model can lead to confusion and errors. In UML, redundancy might occur if the same attributes or methods are unnecessarily repeated across different classes. To eliminate this, shared attributes or behaviours should be abstracted into a common superclass.

Example: Suppose we are modelling a system that handles various types of vehicles, such as cars, trucks, and motorcycles. If each of these classes individually defines attributes like `passengerCapacity` and `fuelCapacity`, we have redundancy. Instead, we can create a `Vehicle` superclass that holds these common attributes, with `Car`, `Truck`, and `Motorcycle` inheriting from it.

7.8.2 Ensuring Consistency

Consistency in relationships and dependencies is crucial for a coherent domain model. In UML, this means clearly defining the associations, aggregations, and compositions between classes, ensuring that each relationship accurately reflects the domain's real-world logic.

Example: In a library management system that can manage multiple libraries, a `Library` class might be associated with multiple `Book` classes. The relationship should clearly define that a library can hold multiple books, but each book belongs to only one library at a time. Specifying this with proper multiplicities (e.g., one-to-many) helps maintain consistency and accurately represents the domain.

7.8.3 Managing Dependencies

Dependencies between classes should be logical and manageable. In UML class diagrams, this often involves ensuring that each class has a single, well-defined responsibility, avoiding complex or indirect dependencies that could complicate the model.

Example: If we have the classes `Customer` and `Order`, the `Order` class might have a direct relationship with `Product`, representing the items being ordered. However, the `Customer` should not have direct dependencies on `Product`; instead, it interacts with `Product` through `Order`. This keeps the model clean and focused.

7.8.4 Modular Design

A modular design, akin to breaking down tables in database normalization, helps manage complexity in UML class diagrams. Each class should encapsulate a specific aspect of the domain, allowing for better organisation and easier updates.

Example: In an e-commerce system, instead of a monolithic `Order` class that includes customer details, payment information, and product lists, we can decompose it into smaller, focused classes like `CustomerInfo`, `PaymentDetails`, and `OrderItem`. These classes can then be composed into the `Order` class, providing a modular and maintainable structure.

7.8.5 Conclusion

Applying these principles when creating UML class diagrams ensures that the domain model is well-structured, avoiding common pitfalls such as redundancy, inconsistency, and overly complex relationships. This approach leads to a clearer, more accurate representation of the system, facilitating better communication among stakeholders and providing a solid foundation for further development phases. By focusing on clarity and logical structure, we create a model that is easier to maintain, extend, and understand, ultimately leading to more robust and successful systems.

7.9 <example> A University Course Management System

Let us return to the University Course Management System (UCMS), focusing on how students, professors, and administrators interact with the system. The UCMS allows students to enrol in courses, professors to manage course content, and administrators to manage users.

7.9.1 Use Case Diagram

We begin by identifying the main actors and their interactions with the system.

The actors in this example are:

- Student: Enrols in courses, views course materials, and submits assignments.
- Professor: Creates courses, uploads materials, and grades assignments.
- Administrator: Manages user accounts and monitors system usage.

The main use cases are:

- [1] Enrol in Course: The student can enrol in a course that is available in the system.
- [2] Manage Course Content: The professor can add materials to a course, such as lecture notes and assignments.
- [3] Get Lecture Notes: The student can access, in the system, to lecture notes of an enrolled course.
- [4] Submit Assignment: The student submits an assignment through the system to an enrolled course.
- [5] Grade Assignment: The professor grades student submissions to a course.
- [6] Manage User Accounts: The administrator manages user roles and permissions.

In a UML Use Case Diagram, we represent these actors and their interactions with the UCMS. The student and professor interact with "Enrol in Course," "Submit Assignment," and "Manage Course Content" use cases, while the administrator interacts with "Manage User Accounts."

7.9.2 Domain Model

The static structure of the system using a Class Diagram, focusing on the domain, would contain:

- Course: Represents a course in the system, with attributes like courseID, courseName and courseDescription.
- Student: Has attributes like studentID, name, and email.
- Professor: Includes attributes such as professorID, name, and department.
- Lecture Notes: Has attributes like notesID, notesContents.
- Assignment: Contains attributes like assignmentID, dueDate, and maxMarks.
- Submission: A student's assignment submission, with attributes like submissionID, submissionDate, and grade.

Relationships:

- A Student can enrol in multiple Courses, and each Course can have multiple Students. This is a many-to-many association in the two ends of the association.
- A Professor can manage multiple Courses, and each Course is managed by one Professor.
- A Course can have multiple Assignments, and a Student can submit multiple Submissions for an Assignment of a Course it is Enrolled.

This diagram represents the core entities and relationships without going into implementation details like methods or internal logic.

7.9.3 Sequence Diagram

Finally, Sequence Diagrams can model the dynamic behaviour. For example, for the interaction when a student enrolls in a course (Use case "Enrol in Course").

1. Actor: The student initiates the enrolment process.
2. UCMS: The system presents the list of available courses to the student.
3. Student: The student requests to enrol in one course
4. UCMS: The system updates the enrolment status, the student is added to the course, and an email confirmation is sent to the student and the course's professor about the new enrolment.

This Sequence Diagram illustrates the flow of messages between the student and the UCMS for the main scenario of the use case "Enrol in Course". This exercise could continue with the modelling of the main scenarios for the other uses cases, as also with the modelling of the alternative scenarios when relevant...

7.9.4 Conclusion

In this very simple example, we have used three UML diagrams, Use Case, Class, and Sequence Diagrams, to model the University Course Management System. The Use Case Diagram captures the high-level interactions between actors and the system, the Class Diagram represents the static structure of the system, and the Sequence Diagram shows the dynamic interaction during a student's enrolment in a course. This multi-view approach, supported by UML, helps ensure a clear and structured understanding of both the system's behaviour and its underlying architecture.

7.10 <example> Normalization Principles in the UCMS

To demonstrate the application of normalization principles in UML class diagrams, we will extend the University Course Management System (UCMS) example, focusing on how each principle can be effectively applied.

7.10.1 Modular Design

A modular approach can be applied to the Course class, which might initially include details such as course materials, assignments, and student enrolments.

Instead of creating a large, complex Course class, we break it down into smaller, more focused classes, such as CourseMaterial, and Assignment.

These modular components are then aggregated into the Course class. This decomposition simplifies the system, making it easier to update individual components without affecting the entire course structure.

7.10.2 Eliminating Redundancy

In the UCMS, both Student and Professor classes might initially define attributes such as name, email, and userID, resulting in redundancy.

By abstracting these common attributes into a User superclass, we eliminate duplication. Student and Professor can then inherit these attributes from User, making the model more concise and consistent.

This abstraction ensures that any changes to user-related data need only be made in one place, improving maintainability.

7.10.3 Ensuring Consistency

Consider the many-to-many relationship between Student and Course.

A student can enrol in multiple courses, and each course can have multiple students.

This relationship is correctly modelled by introducing an intermediary class, Enrolment, which maintains consistency by explicitly managing the association between students and courses.

By clearly defining the multiplicities, we ensure that the relationship aligns with the real-world logic, avoiding potential confusion or misinterpretation of the model.

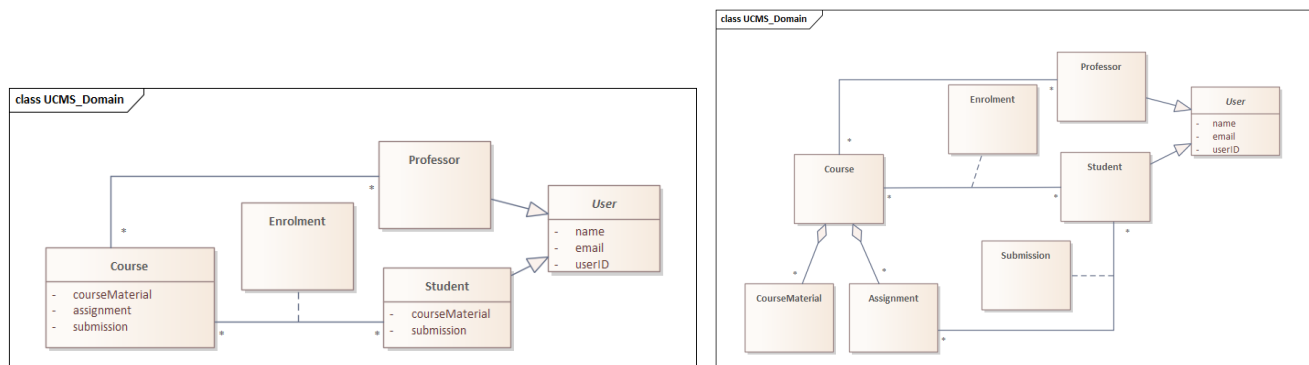
7.10.4 Managing Dependencies

In the UCMS, the Assignment class represents tasks assigned to students, and the Submission class represents the student's work for an assignment.

Courses are taught by Professors, and each Professor may also be involved in several courses. By ensuring that Student is related to Assignment through Submission, rather than directly to Assignment, we maintain a logical separation. This avoids unnecessary direct dependencies between Student and Assignment, making the model cleaner and easier to manage. Each class retains a single responsibility, reducing the risk of over-complication.

7.10.5 Conclusion

By applying these normalization principles to the UCMS, we create a more structured, maintainable, and scalable model. Eliminating redundancy, ensuring clear relationships, managing dependencies logically, and adopting a modular approach result in a domain model that accurately represents the system while remaining easy to understand and extend.



State Machine Diagrams, also known as State Diagrams or Statechart Diagrams, are a type of behavioural diagram in the Unified Modelling Language (UML). These diagrams are used to represent the dynamic behaviour of an individual object or an interaction within a system. They are particularly effective for modelling the life cycle of an object, focusing on how it transitions between different states in response to various events.

7.11.1 Core Concepts

In a State Machine Diagram, a state represents a condition or situation in the life of an object during which it satisfies some condition, performs some activity, or waits for an event. States are depicted as rounded rectangles.

A transition is a relationship between two states, indicating that an object in the first state will enter the second state in response to an event. Transitions are depicted as arrows pointing from the source state to the target state. The event that triggers the transition, as well as any guard conditions or actions associated with the transition, can be annotated on the arrow. For instance, a transition from "Active" to "Suspended" might be triggered by an event like "Failed Payment."

Events are occurrences that trigger transitions between states. These could be signals received, conditions met, or time elapsed. In UML, events are often labelled on the transitions. For example, in a security system, an event could be "Motion Detected," which might trigger a transition from "Idle" to "Alert" state.

Actions are activities that occur because of a transition. They are often associated with the execution of a transition or may occur upon entering or exiting a state. For example, in a device driver using a state machine to control a printer, an action might be "Start Printing," which occurs when transitioning from a state "Ready" to a state "Printing".

The initial state is the starting point of the state machine, depicted by a filled black circle. The final state represents the completion of the state machine's activity, shown as a circle with a dot inside. For example, a "Document Review" process might begin in an "Under Review" state and conclude in a "Approved" or "Rejected" state.

Composite states allow for the nesting of states within a parent state, enabling more complex behaviours to be modelled. They can contain sub-states and transitions, which help to simplify the representation of a state machine by grouping related states. For instance, a "Processing" state in an order management system could be decomposed into "Payment Processing," "Packaging," and "Shipping" sub-states.

These states enable the modelling of concurrent behaviours within the same object. Parallel states, also known as orthogonal regions, allow an object to be in multiple states simultaneously, reflecting that different aspects of the object can change independently.

7.11.2 Purpose and Benefits

State Machine Diagrams serve several critical purposes:

- **Modelling Object Lifecycles:** They effectively model the life cycle of objects, showing how an object moves through various states from creation to deletion, which is particularly useful in systems where objects undergo significant state changes.
- **Clarifying Complex Behaviours:** By visualising state transitions and actions, these diagrams help clarify complex object behaviours, especially in real-time and embedded systems.
- **Ensuring Consistency:** They ensure that all possible states and transitions of an object are considered, reducing the risk of overlooked scenarios in the system design.
- **Facilitating Testing and Validation:** These diagrams are valuable for testing and validation by providing a clear roadmap of all possible states and transitions, aiding in the creation of comprehensive test cases.

7.11.3 A Very Simple Example

Consider a turnstile in a subway system. The turnstile has states such as "Locked" and "Unlocked." When a user inserts a coin (event), the turnstile transitions from "Locked" to "Unlocked." After the user passes through, it automatically returns to the "Locked" state. If an invalid ticket is used, the turnstile remains in the "Locked" state, possibly with an action to display an error message.

7.11.4 Best Practices

When constructing State Machine Diagrams, clearly label states, transitions, events, and actions to ensure the diagram is easy to understand. Ensuring the diagram accurately reflects the real-life behaviour of the object being modelled, focusing on the significant states and transitions that affect system functionality.

7.11.5 Conclusion

State Machine Diagrams are powerful techniques in UML, providing a structured way to model the dynamic behaviour of objects. They are essential in systems where understanding the sequence of states and transitions is critical for ensuring accurate and reliable behaviour.

7.12 <example> What is the State of the Student?

Building upon the previous model of the University Course Management System (UCMS), we now extend the example by incorporating a State Machine Diagram to represent the dynamic behaviour of a Student within the system. This diagram focuses on the various states a student may experience throughout their interaction with the UCMS, particularly in the process of enrolling in, managing, and completing courses.

7.12.1 Recapping

As previously outlined, the actors and key use cases in the system include:

- Student: Enrols in courses, views course materials, and submits assignments.
- Professor: Manages course content and grades student submissions.

The primary use cases for a student include:

- a) Enrol in a course.
- b) Submit an assignment for the course.
- c) Is informed of grades for completed assignments.

The domain model, represented using a Class Diagram, includes entities such as Course, Student, Professor, Assignment, and Submission. The relationships between these classes capture the many-to-many relationship between students and courses, as well as the submission of assignments within a course.

7.12.2 State Machine Diagram for a Student

A State Machine Diagram models the lifecycle of a student within the UCMS, capturing the various states they transition through based on their interactions with the system.

These states reflect the student's progress through course enrolment, participation, and completion.

- States:
 - Idle: The student has an active account in the system but is not enrolled in any course.
 - Enrolled: The student has successfully enrolled in at least one course.
 - Participating: The student is actively engaging with course materials, submitting assignments, and interacting with the course content.
 - Assignment Submitted: The student has submitted an assignment for grading.
 - Graded: The student's assignment has been graded and feedback is available.
 - Completed: The student has completed all required assignments and received final grades for the course.
- Transitions:
 - Idle → Enrolled: Triggered when the student enrolls in a course.
 - Enrolled → Participating: Occurs when the student accesses the course materials and begins engaging with the content (e.g., downloading lecture notes, watching videos, etc.).
 - Participating → Assignment Submitted: Triggered when the student submits an expected assignment.
 - Assignment Submitted → Graded: The professor grades the assignment and posts the results.
 - Graded → Participating: If the course requires additional assignments or tasks, the student continues participation.
 - Graded → Completed: Once all assignments are fulfilled, the student transitions to the "Completed" state.

Example Scenario: Lifecycle of a Student Enrolling and Completing a Course²⁵

- [1] Idle: The student logs into the system but has not yet enrolled in any courses => Transition: The student selects a course and completes the enrolment process.
- [2] Enrolled: The student is now enrolled in a course => Transition: The student begins to interact with the course materials, moving to the "Participating" state.
- [3] Participating: The student attends lectures, downloads lecture notes, and works on assignments => Transition: The student completes an assignment and submits it to the system.
- [4] Assignment Submitted: The student awaits the professor's grading => Transition: The professor grades the assignment and provides feedback.
- [5] Graded: The student is informed of the grade. If there are additional assignments, the student continues participation in the course => Transition: The student submits all required work and completes the course.
- [6] Completed: The student has finished all the requirements for the course and received final grades.

7.12.3 Conclusion

The inclusion of a State Machine Diagram complements the previous Use Case, Class, and Sequence Diagrams, providing a comprehensive view of how a student interacts with the UCMS over time. The State Machine Diagram captures the transitions a student undergoes, from initial enrolment to course completion, while highlighting the system's response at each step. This multi-perspective (viewpoint) approach ensures a robust understanding of both the static structure and dynamic behaviour within the UCMS.

²⁵ This is still an intentionally very simple analysis and modelling, considering only a small set of core requirements; it can be interesting to consider a discussion on how more requirements might have implications in this overall architectural representation...

7.13 <technique> Sequence Diagrams

Sequence Diagrams are a key type of interaction diagram in the Unified Modelling Language (UML). They model the dynamic behaviour of a system by illustrating how objects interact with each other through a sequence of messages over time. These diagrams are essential for understanding the flow of control within a system, particularly in scenarios involving complex interactions or communication between components.

7.13.1 Core Concepts

A lifeline represents an individual participant in the interaction. This can be an object, an actor, or a system component. Lifelines are depicted as vertical dashed lines descending from a rectangle that names the participant. The lifeline symbolises the participant's existence during the interaction.

Messages are the interactions between lifelines, represented by arrows pointing from the sender to the receiver. Synchronous Messages are depicted with a solid arrowhead, these indicate that the sender waits for a response before continuing. Asynchronous Messages are depicted with an open arrowhead, these indicate that the sender continues without waiting for a response. Reply Messages are dashed arrows used to show the return of control or data to the sender, typically following a synchronous message.

Fragments are portions of the sequence diagram that encapsulate interactions, often used to indicate modular sections of the interaction, such as optional scenarios or alternative paths.

Combined fragments are used to represent complex control structures such as loops, conditionals, and parallel processing. These are enclosed in a frame with an operator in the top-left corner that specifies the type of control.

Gates are interaction points used to represent the connection between different sequence diagrams or between a sequence diagram and the outside world. They serve as an entry or exit point for messages in complex diagrams.

7.13.2 Purpose and Benefits

Sequence Diagrams offer several important benefits in system design and analysis:

- **Clarification of Processes:** By visually representing the sequence of interactions, these diagrams help clarify how processes are carried out, making them easier to understand and communicate to stakeholders.
- **Identification of Responsibilities:** Sequence Diagrams help identify which objects or components are responsible for specific actions, thereby clarifying roles within the system and ensuring that the system's architecture aligns with its intended design.
- **Detection of Bottlenecks and Optimisation:** By detailing the interaction flow, sequence diagrams can help detect potential bottlenecks or inefficiencies in the communication between objects, facilitating optimisation in system design.
- **Support for Test Case Development:** These diagrams provide a clear basis for developing test cases, ensuring that all interactions and paths are tested.

7.13.3 A Very Simple Example

Consider a simple online booking system. The primary participants could include the "User," "Booking System," "Payment Gateway," and "Notification Service."

A sequence diagram might depict the interaction starting with the user selecting a date and ending with the system sending a booking confirmation. Messages would include "select date," "process payment," and "send confirmation." A loop fragment could represent the user's option to modify their selection, while an alt fragment might handle scenarios where payment fails.

7.13.4 Best Practices

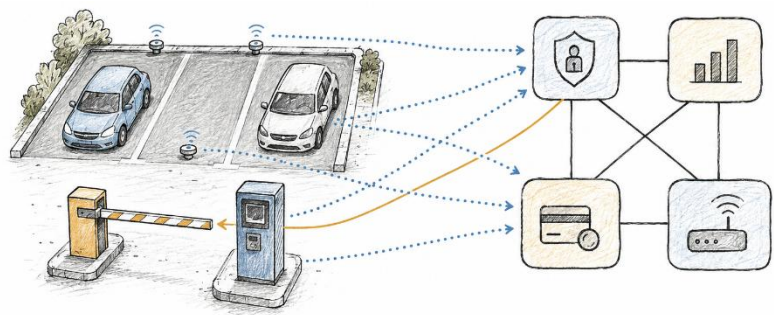
When constructing Sequence Diagrams clearly name lifelines, messages, and fragments to ensure that the diagram is understandable immediately. Use combined fragments to manage complexity, ensuring that the diagram remains easy to follow even as the interactions grow more complicated.

7.13.5 Conclusion

Sequence Diagrams are powerful techniques in UML, providing a clear and detailed view of how different parts of a system interact over time. They are invaluable for both design and testing phases, ensuring that complex interactions are thoroughly understood and correctly implemented.

7.14 <concept> Logical Architectures and Physical Referents

Software-intensive systems frequently interact with entities and processes that exist in the physical world. Sensors observe physical conditions, controllers issue commands to devices, and information systems maintain data about vehicles, machines, rooms, products, or people.



A UML model can represent the software-relevant understanding of such a situation, but the representation must not be confused with the physical entities it concerns.

7.14.1 Physical Referents

A **physical referent** is an entity or phenomenon in the UoD to which a model element refers. Examples include a parking space, barrier, temperature sensor, vehicle, or payment terminal.

A physical referent may appear in a UML domain model as a concept. For example, the class `ParkingSpace` may represent the relevant properties and relationships of physical parking spaces. This does not imply that the parking space is a software object or UML Component. The class formalises the information required about that kind of entity.

7.14.2 Logical Architecture

A **logical architecture** describes how software responsibilities are distributed among logical elements and how those elements interact. It abstracts from particular servers, devices, programming languages, and executable files.

Logical architecture may include:

- components with defined responsibilities;
- interfaces that components provide or require;
- internal parts and their collaborations;
- information maintained about physical referents;
- behaviour triggered by observations or commands.

A logical element that represents the identity, state, or behaviour of a physical referent may be described informally as its **logical counterpart**. For example, a `ParkingSpaceRegistry` may maintain the known occupancy state of physical parking spaces.

The logical counterpart is not the physical entity itself. Its recorded state may be incomplete, delayed, or incorrect. Requirements may therefore be needed to define how observations are obtained, how often information is updated, and how inconsistencies are detected.

7.14.3 No Necessary One-to-one Correspondence

Logical and physical decompositions address different concerns and must not be expected to correspond one-to-one.

One logical component may represent or coordinate many physical entities. Conversely, several logical components

may process information about the same physical entity. A physical sensor might involve a device adapter, an event-processing component, a state repository, and a monitoring service.

The logical decomposition should therefore be based on software responsibilities and interactions, not obtained by mechanically translating every physical part into a UML Component.

7.14.4 Logical Models and Digital Twins

A digital twin combines a digital representation with an operational relationship that keeps it synchronised with a real-world referent at the frequency and fidelity required for its purpose.

In a digital-twin solution, UML can specify the logical and software architecture through which observations are acquired, the logical counterpart is updated, and analyses or interventions are supported. When these specifications are realised through sensing, communication, data-management, and synchronisation mechanisms, the resulting operational digital representation can function as a digital twin.

7.14.5 Scope of UML

UML is suitable for modelling the logical and software aspects of software-intensive and cyber-physical systems. It can describe domain concepts, software components, interfaces, internal structures, interactions, states, and deployment onto computational infrastructure.

UML should not be used as though it provided dedicated semantics for mechanical composition, continuous physical behaviour, units of measurement, or energy and material flows. UML can be used for those purposes, and we can find in the literature old examples of profiles for that, but when nowadays those concerns are central, SysML or specialised engineering languages may be more appropriate.

7.14.6 Conclusion

A logical architecture is a concern-driven representation of software responsibilities and interactions. Physical entities remain referents in the UoD, while UML represents the concepts, software elements, and behaviour through which those entities become relevant to the system. The relationship between physical and logical structures must be justified rather than assumed.

7.15 <technique> Components, Parts, Ports, Interfaces, and Connectors

UML provides several related concepts for describing the logical architecture of a software-intensive system.

Together, Components, Parts, Ports, Interfaces, and Connectors support both black-box and white-box representations.

7.15.1 Components

A **Component** represents a modular and replaceable unit that encapsulates implementation and exposes defined behaviour to its environment.

A Component should have a coherent responsibility. Examples include `AccessController`, `PaymentManager`, or `OccupancyManager`.

A Component is not an executable file, library, server, or physical device. Those concerns are represented through Artifacts and Nodes in deployment models.

7.15.2 Interfaces

An **Interface** defines a contract that a Component or other Classifier provides or requires. It specifies externally visible features without describing how they are implemented.

A **provided Interface** identifies behaviour offered to other elements. It may be shown using the “lollipop” notation: a small circle connected to the provider.

A **required Interface** identifies behaviour needed from another element. It may be shown using the “socket” notation: an open semicircle connected to the consumer.

Provided and required Interfaces make dependencies explicit and support substitution: a Component may be replaced when the replacement respects the relevant contracts.

7.15.3 Ports

A **Port** is an identified interaction point on the boundary of a Component or another encapsulated Classifier. It is normally represented by a small square placed on that boundary.

A Port is different from an Interface. The Port identifies the interaction point; provided and required Interfaces specify the contracts available through it. Ports are especially useful when a Component has several distinct interaction roles.

For example, an `AccessController` might have separate ports for vehicle-access requests, payment authorisation, and barrier commands.

7.15.4 Parts and Internal Structure

A Component may be described internally as a collaboration of **Parts**. A Part is a property representing a constituent role within the context of a structured element.

A Part is commonly labelled as “`partName : Type`”.

For example “`payment : PaymentManager`” means that payment is a role within the enclosing structure and is typed by `PaymentManager`.

7.15.5 Connectors

A **Connector** represents an interaction path between Parts or Ports within a structured context.

An **assembly connector** connects elements that provide and require compatible Interfaces. A **delegation connector** links an external Port of the enclosing Component to an internal Part or Port that handles the interaction.

Connectors describe possible structural communication. They do not, by themselves, specify the temporal order of messages; Sequence Diagrams can do that when it matters.

7.15.6 Component and Composite Structure Diagrams

A **Component Diagram** normally provides an architectural overview showing Components, their Interfaces, and their dependencies.

A **Composite Structure Diagram** exposes the internal structure of a Component or another Structured Classifier, showing Parts, Ports, and Connectors in a particular context.

These diagrams are complementary: the **Component Diagram** commonly emphasises architectural modules, while the **Composite Structure Diagram** explains how an element is internally composed.

7.15.7 Modelling Guidance

In simple terms:

1. Assign one coherent responsibility to each Component.
2. Begin with its external Interfaces before exposing internal structure.
3. Introduce Ports only when distinct interaction points matter.
4. Use Parts to represent contextual constituents, not merely to place elements inside containers.
5. Connect required Interfaces only to compatible provided Interfaces.
6. Use delegation to show how external interactions reach internal Parts.
7. Use Sequence or State Machine Diagrams to complement structural models with behaviour.

7.15.8 Conclusion

Components encapsulate logical responsibilities; Interfaces define contracts; Ports identify interaction points; Parts describe contextual internal constituents; and Connectors establish interaction paths. Used together, these concepts provide a disciplined transition from black-box responsibilities to white-box logical architecture.

A logical architecture describes software responsibilities without deciding exactly where or how they will execute. A **Deployment Diagram** complements that architecture by representing the computational environments in which software Artifacts are deployed in Nodes.

7.16.1 Nodes

A **Node** is a computational resource upon which Artifacts may be deployed. UML distinguishes two particularly useful kinds of Node:

- A **Device** represents a physical computational resource, such as a server, edge controller, mobile device, or embedded computer.
- An **Execution Environment** represents a software environment that provides services for executing deployed Artifacts, such as an operating system process, virtual machine, application server, container runtime, or database management environment.

Nodes may be nested. For example, a Java Virtual Machine may be shown inside a physical server.

7.16.2 Artifacts

An **Artifact** represents a physical piece of information used or produced during development, deployment, or operation. Examples include executable files, software libraries, configuration files, database schemas, and deployment packages.

An Artifact may **manifest** a logical model element such as a Component. The Component describes a modular logical responsibility, while the Artifact represents a deployable implementation of that responsibility.

This distinction prevents a common modelling error: Components are not themselves executable files. Deployment Diagrams should normally show the Artifacts that implement Components and the Nodes on which those Artifacts are deployed.

7.16.3 Deployment and Communication Paths

A **Deployment** relationship indicates that an Artifact is deployed on a Node. An Artifact may also be drawn inside a Node as a compact representation of the same placement.

A **Communication Path** between Nodes indicates that their deployed elements may communicate. The path may be

labelled when the relevant communication mechanism is known, such as a local network, secure connection, or mobile network.

A Communication Path should not be used to model every cable or network route. It should be included only when connectivity is relevant to the concern of the diagram.

7.16.4 From Logical to Physical Decisions

Deployment establishes a relationship between logical software architecture and computational infrastructure. However, it does not model the complete physical composition of a cyber-physical system.

Passive mechanical parts, material connections, and non-computational devices should not be forced into the UML Node concept. A physical element should be represented as a Device only when its computational role is relevant and consistent with the intended model.

7.16.5 Modelling Guidance

A practical deployment analysis may follow these steps:

1. Identify the logical Components whose implementation must be deployed.
2. Identify the Artifacts that manifest those Components.
3. Identify the Devices and Execution Environments required to execute them.
4. Place each Artifact on the appropriate Node.
5. Add Communication Paths required by interactions between deployed Artifacts.
6. Check security, availability, performance, and connectivity concerns.
7. Keep external infrastructure outside the system boundary clearly identified.

7.16.6 Conclusion

Deployment Diagrams relate software architecture to computational infrastructure. Components express logical responsibilities; Artifacts provide deployable implementations; Nodes provide execution resources; and Deployment relationships specify placement.

This separation helps distinguish logical design from physical execution decisions while maintaining traceability between them.

7.17 <example> Automated Parking System: From Physical Referents to Logical Architecture

Consider an Automated Parking System that manages entry, occupancy, payment, and exit from a car park. The purpose of this example is to distinguish physical referents, logical Components, internal structure, and deployment.

7.17.1 Physical Referents and Domain Concepts

The UoD includes physical parking spaces, vehicles, entry and exit barriers, occupancy sensors, payment terminals, and drivers.

A UML domain model may include concepts such as ParkingSpace, Vehicle, ParkingSession, and Payment. These Classes represent information relevant to the problem domain. They are not software Components and do not imply that the corresponding physical entities are software objects.

7.17.2 Black-box Logical Architecture

As a black box, the AutomatedParkingSystem provides behaviour for:

- requesting and authorising vehicle entry;
- maintaining information about space availability;
- registering parking sessions;
- collecting or validating payment;
- authorising vehicle exit.

It also requires interactions with external payment services and with the devices through which physical events are observed or commands are executed.

7.17.3 Internal Components

A possible white-box logical structure contains these Parts:

- access : AccessController (coordinates entry and exit authorisation);
- occupancy : OccupancyManager (maintains the known state of parking spaces);
- payment : PaymentManager (manages payment requests and results);
- devices : DeviceGateway (isolates the remaining software from device-specific communication).

This is one defensible, logical decomposition, not the only possible one. Details for that must be taken from the UoD and the interactions with the stakeholders.

7.17.4 Ports, Interfaces, and Connectors

The DeviceGateway provides an OccupancyObservation Interface through which sensor observations are communicated. The OccupancyManager requires this Interface and updates its logical representation of parking-space occupancy.

The OccupancyManager provides a SpaceAvailability Interface required by the AccessController.

The PaymentManager provides a PaymentValidation Interface. It may itself require an external PaymentAuthorisation Interface from a payment provider.

The DeviceGateway also provides a BarrierControl Interface required by the AccessController.

Assembly connectors link compatible required and provided Interfaces. Delegation connectors may link Ports on the boundary of AutomatedParkingSystem to the internal Parts responsible for handling each interaction.

7.17.5 Behaviour Across the Structure

A simplified entry scenario may proceed as follows:

1. A vehicle requests entry.
2. The AccessController asks the OccupancyManager whether a space is available.
3. The OccupancyManager returns the currently known availability.
4. If entry is authorised, the AccessController requests the DeviceGateway to open the barrier.
5. Sensor observations later reach the OccupancyManager through the DeviceGateway.

A Sequence Diagram can represent the order of these messages.

A State Machine Diagram can represent the states of a ParkingSession or the logical occupancy state maintained for a parking space.

7.17.6 Deployment

The logical Components may be manifested by two Artifacts:

- parking-services.jar, implementing access, occupancy, and payment responsibilities;
- device-gateway.jar, implementing communication with local devices.

A “parking-services.jar” may be deployed in an application Execution Environment on a central server and a “device-gateway.jar” may be deployed on an edge-controller Device located at the car park. A Communication Path connects the two Nodes.

The physical barriers and sensors remain physical referents. They should be represented as UML Devices only when they provide relevant computational resources; otherwise, their relationship with the software may remain part of the UoD description or another engineering model.

7.17.7 What This Example Shows

The logical architecture does not reproduce the physical structure. One DeviceGateway may interact with many sensors and barriers, while several logical Components may use information originating from the same physical sensor.

The OccupancyManager maintains a logical counterpart of parking-space occupancy, but this does not automatically constitute a digital twin. Synchronization frequency, observation accuracy, failure detection, and reconciliation would need to be explicitly required and implemented.

7.17.8 Conclusion

The example shows how UML can represent software responsibilities, internal composition, interfaces, interaction paths, behaviour, and deployment in a cyber-physical situation. The resulting model is a selective logical view grounded in physical referents, not a literal copy of the physical system.

7.18 <technique> Object Constraint Language

The Object Constraint Language (OCL) is a formal language used in conjunction with UML to specify detailed constraints and conditions that cannot easily be depicted in graphical models. It adds precision to models by allowing rules about attributes and relationships to be expressed clearly. OCL defines primitive types and operators.

7.18.1 Primitive Types in OCL

In OCL, several primitive types and functions are defined to enable precise constraints, allowing to define specific rules about attributes and their values: Boolean, Integer, Real, String, Collection Types (like Set, Bag, Sequence, and OrderedSet)

7.18.2 Operators in OCL

OCL also defines functions to operate on those types:

- Logical Operators: and, or, not, implies, xor
- Relational Operators: =, <>, <, <=, >, >=
- Arithmetic Operators: +, -, *, /, mod
- Collection Operators: size(), isEmpty(), notEmpty(), includes(), excludes(), forAll(), exists(), select(), collect()
- String Operators: size(), concat(), toUpper(), toLower(), substring(start, end)
- Type-Related Operators: oclIsTypeOf(), oclIsKindOf(), oclAsType(), oclIsUndefined()
- Null Handling Operators: isUndefined(), notEmpty()

7.18.3 Use of OCL

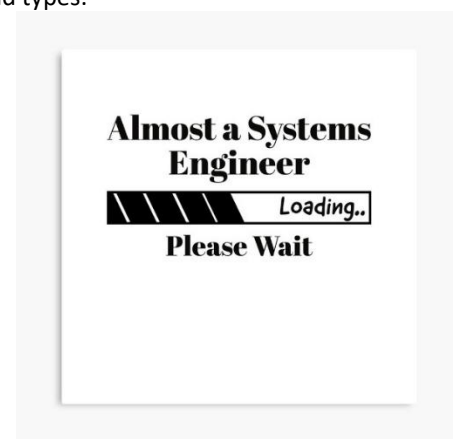
In UML, OCL is often used to define constraints on attributes of classes, ensuring that the values of these attributes always meet specific conditions. For example, if a model includes a Person class with an age attribute, we can use OCL to ensure that the age is always a positive number. In this case, the constraint would specify that the age attribute must always be greater than zero.

Another example in a UML class diagram could involve a Product class with a price attribute. A constraint might be placed to ensure that the price is always greater than or equal to zero, ensuring that no product can have a negative price. This type of constraint ensures the logical consistency of the model, preventing invalid values from being used.

In a more complex case, consider a Company class with a numberOfEmployees attribute. OCL could specify that this number should always be equal to or greater than one, ensuring that the company cannot exist without at least one employee. This type of rule ensures that certain business logic is maintained directly in the model, adding a layer of validation.

7.18.4 Conclusion

OCL types and operators enable rich expressions for constraints, from logical and arithmetic conditions to sophisticated collection manipulations and string operations. OCL can play a critical role in ensuring the correctness of UML models by allowing for detailed validation and checking of attribute values, relationships, and types.



Let's illustrate the use of OCL (Object Constraint Language) in the context of a library system's Universe of Discourse (UoD), using constraints applied to a class diagram (as a domain model), a sequence diagram, and a state machine diagram. This example demonstrates how enforce consistency and correctness in the system through formal rules, following OCL syntax.

7.19.1 UoD Description:

We are modelling a library system where books, members, and loans are managed. Each book has a title, an author, and a stock count (indicating how many copies are available). Members can borrow books, but they are limited to borrowing no more than five books at any given time. Furthermore, a book can only be borrowed if it is available in stock. Loans track the date a book is borrowed and its due date, ensuring that books are returned before members can borrow them again.

7.19.2 Class Diagram (Domain Model):

In the class diagram, the key classes are:

- Book: Has attributes title, author, and stock (representing the number of available copies).
- Member: Includes attributes name and a collection of borrowedBooks that tracks the books a member currently has.
- Loan: Contains the loanDate and dueDate, and is associated with a Member and a Book.

Several constraints are necessary to ensure the logical consistency of the system:

- Book Stock: A book can only be borrowed if it is available in stock, meaning the stock cannot fall below zero. The OCL constraint could ensure that `"stock >= 0"`, to guarantee that the stock of any book is always zero or higher, preventing any book from being borrowed if no copies are available.
- Member Borrowing Limit: A member is restricted to borrowing no more than five books at a time. The OCL constraint for this could be `"borrowedBooks->size() <= 5"`, to ensure that the borrowedBooks collection of each member does not contain more than five books, enforcing the borrowing limit.
- Loan Due Date: The dueDate of a loan must always be after the loanDate, ensuring that the book is returned in the future. The OCL expression could be `"dueDate > loanDate"`, ensuring the logical consistency of the loan record, preventing any invalid dueDate from occurring before the book was borrowed.

7.19.3 Sequence Diagram:

A sequence diagram models the process of a member borrowing a book. The interactions include:

1. A member requests to borrow a book.
2. The system checks if the member has reached their borrowing limit.
3. The system checks if the book is available in stock.
4. If both checks pass, a loan is created, and the book's stock is updated.

During the borrowing process, certain conditions must be met. For example, before a loan is created, the system must check that the member has not borrowed more than five books, and the book must be available in stock. The combined OCL constraint for this could be `"borrowedBooks->size() <= 5 and borrowedBooks->forAll(b | b.stock > 0)"`

7.19.4 State Machine Diagram:

A state machine diagram might represent the lifecycle of a Book, with states such as:

- Available: The book is available to be borrowed.
- On Loan: The book has been borrowed by a member.
- Unavailable: The book is out of stock and cannot be borrowed until returned.

We can use OCL to ensure that books transition correctly between states.

1. Available state: A book can only be in the Available state if there are copies available in stock. The OCL constraint for this can be `"stock > 0 implies self.state = 'Available'"`, to ensure that a book cannot be borrowed unless there is at least one copy available, allowing it to remain in the Available state, otherwise, `"stock = 0 implies self.state = 'Unavailable'"`
2. On Loan state: A book is in the On Loan state when a copy of it is taken by a member, therefore decreasing the number of copies in the library, so `"self.state = 'On Loan' implies self.stock < originalStock"` (note: be aware that in this exercise we are not considering a state for "Not On Loan"...)

7.19.5 Conclusion:

In this example, OCL adds precision to the model by enforcing critical constraints across class diagrams, sequence diagrams, and state machine diagrams. These constraints, which are difficult to represent graphically, play a key role in maintaining the logical integrity of the system, preventing errors such as negative stock levels or members exceeding their borrowing limits. OCL helps define these rules directly within the model, enhancing its accuracy and reliability.

UML, as a standardised modelling language, can play a pivotal role in MBSE & MDE by providing a rich set of diagrams and notations that can model both the structure and behaviour of complex software systems. MBSE & MDE leverages these models to generate, analyse, and validate the system, ensuring consistency and reducing the manual effort involved in development.

7.20.1 Core Concepts of MBSE & MDE with UML

- **Models as Central Artefacts:** In MDE, models are not merely documentation artifacts but are central to the engineering process. These models encapsulate various aspects of the system, from high-level architecture to detailed design, and are used to generate code, perform analysis, and ensure alignment with requirements.
- **Meta-Models:** A meta-model defines the rules and constructs for creating models within a specific domain. In the context of UML, the UML meta-model specifies the elements (such as classes, relationships, and behaviours) and their permissible configurations. This meta-model ensures that UML diagrams are consistent and adhere to the standards, making them interpretable by tools that support MBSE & MDE.
- **Model Transformations:** One of the key features of MDE is the ability to transform models from one level of abstraction to another. For example, a high-level UML class diagram can be transformed into a more detailed design model, and eventually into code. This transformation process is guided by rules and mappings that ensure the integrity and correctness of the system as it evolves from concept to implementation.
- **Code Generation:** UML models can be used to automatically generate code, reducing the need for manual programming and minimising human errors. This is particularly valuable in environments where consistency, reliability, and speed are critical. The generated code is often aligned with the architecture and design specified in the UML diagrams, ensuring that the implementation is faithful to the original design.
- **Model Validation and Verification:** In MBSE & MDE, UML models are used to verify and validate system behaviour before implementation. For instance, sequence diagrams can be used to simulate interactions between objects, ensuring that the system will behave as expected under various conditions. State machine diagrams can be analysed to verify that all possible states and transitions have been accounted for, helping to prevent errors in system logic.

7.20.2 Integration of UML in MBSE & MDE Processes

- **Requirements Engineering:** UML use case diagrams are instrumental in capturing functional requirements and providing a high-level view of the system's interactions with external actors. These diagrams serve as the foundation for creating more detailed models that will be used throughout the development process.
- **System Design:** UML class diagrams, component diagrams, and deployment diagrams provide the necessary views for designing the architecture of the system. These diagrams ensure that all components are correctly specified and that their interactions are well-defined. In MBSE & MDE, these design models can be directly transformed into implementation models or code, streamlining the development process.
- **Implementation:** UML sequence diagrams and activity diagrams guide the implementation phase by detailing the dynamic aspects of the system. These diagrams help developers understand the flow of control and data, ensuring that the implemented system adheres to the design specifications. The use of automated tools in MBSE & MDE can directly translate these models into executable code.
- **Testing and Validation:** UML models are crucial for testing and validation in MBSE & MDE. Sequence diagrams, state machine diagrams, and other behavioural models can be used to generate test cases, simulate system behaviour, and verify that the system meets its requirements. This ensures that issues are identified early in the development process, reducing the risk of costly errors later.
- **Documentation and Maintenance:** UML models serve as comprehensive documentation of the system, which is essential for maintenance and future development. In MBSE & MDE, these models are kept up to date throughout the lifecycle of the system, ensuring that they accurately reflect the current state of the system and provide a reliable reference for ongoing work.

7.20.3 Conclusion

UML, as part of the MBSE & MDE approaches, transforms traditional software development by making models the central artefacts throughout the lifecycle of a system. This shift ensures greater consistency, accuracy, and efficiency in system development, particularly in complex environments where precision is paramount. By leveraging UML's extensive set of diagrams and notations, MBSE & MDE facilitates the automatic generation of code, the validation of system behaviour, and the seamless integration of design and implementation, making it an indispensable approach for modern software and systems engineering.

7.21 <core> Wrap-up Software Systems and UML

From managing vast datasets to powering mission-critical applications, software systems demand a robust and disciplined approach to their creation. Traditional software engineering practices, while still relevant, often prove insufficient when dealing with the intricate architectures and demanding requirements of today's software.

At the core of many modern software systems lie **object-oriented (OO)** principles: **Encapsulation** (bundling data and the operations that manipulate it within "objects" enhances modularity and information hiding, making systems easier to maintain and evolve); **Inheritance** (creating new objects from existing ones, inheriting their properties and behaviours, promotes code reuse and reduces redundancy); and **Polymorphism** (allowing objects of different classes to respond to the same method call in different ways enhances flexibility and extensibility).

While OO design brings significant advantages, the sources caution against prematurely introducing certain OO concepts, such as polymorphism and detailed method modelling, during the early analysis phase. Analysis should focus on understanding the problem domain and capturing requirements rather than dictating implementation specifics.

UML takes centre stage as a **standardised, general-purpose modelling language** specifically designed for OO approaches. It offers a rich set of diagrams that provide multiple views of a software system, catering to different stakeholders and purposes.

7.21.1 UML Diagrams

UML defines 13 types of diagrams: Class Diagram; Object Diagram; Use Case Diagram; Sequence Diagram; Activity Diagram; State Machine Diagram; Component Diagram; Deployment Diagram; Package Diagram; Interaction Overview Diagram; Timing Diagram; Communication Diagram, and Composite Structure Diagram.

UML's **structural diagrams** focus on the static aspects of a system, representing its elements and their relationships. The most commonly used are:

- **Class Diagrams:** Cornerstones of OO design, illustrating classes (blueprints for objects), their attributes, methods (operations), and the relationships between them. These diagrams are crucial for domain modelling and defining the system's core structure.
- **Component Diagrams:** Depicting the high-level view of the system's architecture, in the form of components (e.g., executables, libraries) and their dependencies.
- **Deployment Diagrams:** Illustrating how software components are deployed onto hardware nodes, these diagrams are essential for understanding the system's physical distribution and communication.
- **Object Diagrams:** An object diagram is a static diagram that shows instances of classes (objects), their relationships, and the state of these objects at a particular point in time, representing a snapshot of the system's behaviour.

Complementing the structural perspective, **behavioural diagrams** model the dynamic aspects of a system, showing how its elements interact over time. The most commonly used are:

- **Use Case Diagrams:** Illustrating how actors (users or external systems) interact with the system to achieve specific goals, these diagrams capture functional requirements from a user-centric perspective.
- **Sequence Diagrams:** Visualising the time-ordered exchange of messages between objects, these diagrams are essential for understanding workflows, collaborations, and the sequence of operations within a use case.
- **State Machine Diagrams:** Depicting the different states an object can be in and the transitions between them, these diagrams are crucial for modelling systems with complex state-dependent behaviour.

7.21.2 UML for Problem Analysis and Requirements Engineering

UML can be employed throughout the **software development lifecycle (SDLC)**, from requirements gathering and analysis to design, implementation, testing, and documentation.

Nowadays, it is mainly used for the early stages of software development: **problem analysis** and **requirements engineering**, where these diagrams can be of value:

- **Use Case Diagrams:** Provide a high-level understanding of system functionality by illustrating user interactions and system responses, helping to define system boundaries and ensure comprehensive coverage of user needs.
- **Class Diagrams (for Domain Modelling):** Instead of focusing on implementation specifics, class diagrams can represent the key entities, attributes, and relationships within the problem domain, providing a clear, implementation-independent model of the subject matter.
- **State Machine Diagrams:** When capturing requirements, these diagrams can model the different states a system or component might go through during its operation, ensuring that all possible scenarios and transitions are considered.
- **Sequence Diagrams:** By illustrating the flow of interactions between objects or actors in a specific scenario, these diagrams help to refine use cases and uncover detailed system behaviour, ensuring that all necessary interactions and responses are captured.

7.21.3 Object Constraint Language

Graphical representations alone might lack precision. Object Constraint Language (OCL) is a formal language that allows developers to add constraints and rules to UML models, ensuring that they are unambiguous and can be rigorously checked for consistency and completeness.

ASPIRINA

da conversa que ontem tive
com o senhor do universo
resultou o seguinte:

dentro de cada cabeça
há uma árvore do bem e do mal
com o respectivo jardineiro

dentro de cuja cabeça naturalmente
há também uma árvore do bem e do mal
com o respectivo jardineiro

o qual dentro da cabeça
tem também uma árvore do bem e do mal
e o respectivo jardineiro

e assim por diante
tudo já se vê em dimensões microscópicas

mas não sei se entendi bem
o que ele quis dizer.



Alberto Pimenta

8.1 The Big Picture

Systems analysis begins with a situation that stakeholders want to understand, improve, or change. Different stakeholders may perceive the same situation differently because they have different responsibilities, knowledge, goals, and concerns.

Reality does not come already divided into systems.

Analysts define a system of interest (SoI) for a particular purpose and according to relevant stakeholder concerns.

This choice is not arbitrary: the proposed boundary, concepts, and relationships must be justified by evidence from the Universe of Discourse (UoD).

A model is therefore neither a complete reproduction of reality nor a neutral drawing. It is a selective and accountable interpretation constructed to support understanding, communication, analysis, and decision-making.

8.1.1 A disciplined modelling path

A practical analysis normally follows an iterative path:

1. **Identify the problem and purpose** – clarify why the analysis is needed and which decisions it should support.
2. **Identify stakeholders and concerns** – determine whose interests must be considered and what matters to them.
3. **Establish the UoD** – collect evidence, terminology, rules, existing artefacts, and relevant knowledge.
4. **Define the system of interest** – state its boundary, environment, responsibilities, and assumptions.
5. **Elicit and analyse requirements** – distinguish needs, goals, constraints, behaviours, qualities, and proposed solutions.
6. **Explore informally** – use mind maps, concept maps, examples, or sketches when the problem is still being understood.
7. **Select viewpoints and languages** – choose representations appropriate to the concern being addressed.
8. **Construct coherent views** – model only the information required for the intended purpose.
9. **Review and validate** – check evidence, semantics, consistency, traceability, readability, and stakeholder understanding.
10. **Revise when knowledge changes** – models evolve as assumptions are tested and the UoD becomes better understood.

This must be not a waterfall sequence. Analysts frequently return to earlier decisions when modelling exposes missing information, contradictions, or an unsuitable system boundary.

8.1.2 The modelling languages used in this course

The course introduces three complementary formal modelling languages:

- **ArchiMate 4** – for describing enterprise architecture and change across motivation, strategy, business,

application, and technology concerns. When a tool supports only ArchiMate 3.2, compatibility decisions must be explicit.

- **BPMN 2.0.2** – for describing business processes, responsibilities, events, decisions, flows, and collaborations between participants.
- **UML 2.5.1** – for describing software-intensive systems through complementary structural and behavioural views, including use case, class, sequence, state machine, component, and deployment diagrams.

SysML is introduced only for awareness. It is an important language for Model-Based Systems Engineering and multidisciplinary systems, but its notation is not taught or assessed in this course.

No language is universally best. The correct language and viewpoint depend on the concern, intended audience, required precision, and part of the system being examined.

8.1.3 What makes a model useful?

A useful model:

- has an explicit purpose and audience;
- addresses identified stakeholder concerns;
- uses concepts according to the semantics of its language;
- is supported by evidence from the UoD;
- makes assumptions, uncertainties, and compatibility decisions visible;
- remains consistent with related views;
- contains enough detail for its purpose, but no unnecessary detail;
- can be explained and challenged by its author.

Tools can store elements, reuse model content, check some formal rules, organise views, and support traceability. But they cannot decide whether a model is relevant, factually justified, or useful; the analyst must remain responsible for those decisions!

8.1.4 Looking forward

The value of conceptual modelling lies in the ability to move between stakeholder concerns, requirements, architectural structures, processes, software behaviour, and implementation constraints without losing meaning or traceability.

These capabilities provide a foundation for further work in software engineering, systems engineering, enterprise architecture, business analysis, requirements engineering, project governance, and digital transformation.

Key takeaway

Conceptual modelling is not the production of diagrams for their own sake. It is the disciplined construction of selective, evidence-based representations that help people understand systems, question assumptions, coordinate decisions, and manage change.

8.2 <further> AI at the Front Edge of Systems Analysis and Modelling

Artificial Intelligence (AI), and Generative AI (GenAI) in particular, may become one of the most valuable toolsets ever available to systems analysts and modellers. The activities covered in these notes are an unusually good match for its capabilities: they involve interpreting language, connecting information from multiple sources, identifying patterns, generating alternative representations, and moving between text, diagrams, rules, examples, and scenarios.

Given appropriate UoD material, GenAI can help extract terminology, stakeholders, concerns, rules, and candidate requirements; summarise interviews and documents; formulate elicitation questions; identify possible omissions and inconsistencies; propose alternative system boundaries; generate initial ArchiMate, BPMN, or UML models; translate between textual and graphical representations; suggest traceability links; and explain models to different audiences. Integrated with modelling repositories, AI may eventually make large collections of models conversationally accessible, maintain links between requirements and views, detect the effects of changes, and generate stakeholder-specific representations from shared model content.

These capabilities can reduce routine effort and allow analysts to investigate more alternatives, perform more systematic reviews, and maintain richer and more coherent sets of artefacts. Nevertheless, the most important expected impact is not cheaper or faster diagram production, but in the increased value of analysis and modelling at the leading edge of business design.

8.2.1 Analysis at the Leading Edge of Business Design

Systems analysis is performed at the boundary between an existing situation and a situation that stakeholders may want to create. Sometimes the objective is incremental improvement: reducing delays, correcting inconsistencies, replacing obsolete technology, or improving an existing process. In other cases, innovation or disruption is itself part of the purpose.

An organisation may seek to create a new service, introduce a new form of collaboration, redistribute responsibilities, exploit a previously unavailable capability, or challenge the assumptions on which an existing market operates. In such situations, analysis is not limited to discovering facts and requirements already present in the UoD.

The analyst must help stakeholders:

- reconsider how the problem has been framed;
- question established boundaries and responsibilities;
- recognise possibilities not represented in current practice;
- explore alternative value propositions and operating models;
- anticipate consequences for different stakeholders;
- decide which future is desirable, feasible, and sufficiently responsible to pursue.

The as-is situation can be investigated through evidence. The to-be situation, however, does not yet exist as a set of observable facts. It contains hypotheses, inventions, preferences, commitments, and choices under uncertainty.

GenAI can make an important contribution by proposing analogies, combining ideas from different domains, generating scenarios, exposing assumptions, and exploring alternative designs. It can sometimes produce combinations that humans would not readily consider. However, its proposals are generated from patterns found in its training and supplied context. This makes it particularly effective at generating plausible continuations and recombinations of what is already known.

However, innovation may require something different: rejecting the statistically typical response, introducing knowledge that has not yet been documented, recognising an emerging possibility for which little evidence exists, or deliberately departing from established practice. GenAI does not participate in the organisation as a situated and accountable stakeholder. It does not possess organisational purpose, accept business risk, negotiate legitimate interests, or commit people and resources to a course of action.

Humans therefore need to contribute more than supervision. They need to contribute with domain insight, tacit knowledge, imagination, entrepreneurial intent, ethical and political judgement, willingness to accept risk, and commitment to action. AI can enlarge the space of possible futures; people must decide which future they are prepared to create.

As routine model production becomes easier, the valuable part of the activity moves increasingly towards problem framing, evidence management, alternative generation, critical evaluation, negotiation, and accountable decision-making. Understanding modelling languages remains essential: someone who does not understand their semantics cannot reliably evaluate or challenge an AI-generated model.

8.2.2 Practices for Human–AI Co-Modelling

A disciplined use of GenAI should follow several principles:

- **Frame the task before using the tool.** State the analytical purpose, intended decision, Sol, relevant stakeholders, and applicable viewpoint.
- **Ground the work in controlled UoD evidence.** Provide relevant sources while respecting confidentiality, privacy, intellectual property, and organisational restrictions.
- **Request alternatives, not a single authoritative answer.** Ask for different interpretations, counterexamples, missing information, and conditions under which each proposal would fail.
- **Preserve epistemic status.** Distinguish what is evidenced, inferred, assumed, proposed, decided, or still unresolved.
- **Maintain traceability.** Material requirements and model elements should be connected to primary UoD evidence or to an explicit human decision—not merely to an AI-generated summary.
- **Separate generation from validation.** Review outputs against the original evidence, stakeholder concerns, related views, and the semantics of the modelling language.
- **Make consequential decisions explicitly human.** System boundaries, requirements, priorities, acceptable risks, and unresolved stakeholder conflicts require approval by people with appropriate knowledge and authority.
- **Retain relevant configuration information.** When reproducibility or auditability matters, record the tool, model version, prompt, supplied context, and subsequent human changes.
- **Approve only what can be explained.** Analysts remain responsible for every element they incorporate into an accepted model.

8.2.3 Ambiguity and Silent Decisions

A particularly serious risk arises when UoD evidence is incomplete or ambiguous. A competent human analyst who notices an ambiguity should seek clarification, retain alternative interpretations, or document an assumption. A GenAI tool is normally expected to produce a complete and plausible response. It may therefore resolve the ambiguity silently and proceed as if its interpretation were an established fact.

An ambiguous description may leave unspecified who performs an action, whether exceptions exist, which event occurs first, where the system boundary lies, or whether a relationship is mandatory. GenAI may insert a statistically typical actor, gateway, multiplicity, component, or relationship. The result may be precise, conventional, and internally coherent, while containing a decision unsupported by the UoD—a decision that an informed human might never make once the ambiguity was recognised.

GenAI operates over representations of the world, not through situated participation in the actual organisation and its consequences. Its statistical plausibility does not reliably distinguish between an established fact, a reasonable inference, a common pattern from another context, and a question that must remain unresolved.

The hidden decision may then propagate from a requirement into BPMN, ArchiMate, UML, tests, and implementation. Because all these artefacts inherit the same unsupported assumption, they may be mutually consistent. Neither additional generation nor automated consistency checking necessarily exposes the original mistake. The detail may remain unnoticed for a long time (or forever).

Humans can make the same error, but GenAI amplifies the risk through the speed, scale, fluency, and apparent precision with which unsupported decisions can be generated and propagated.

8.2.4 AI-Enabled Systems as Objects of Analysis

AI will also make analysis and modelling more important in AI-enabled systems because that technology changes the systems being analysed. AI-enabled behaviour may depend on training and retrieval data, prompts, model versions, operating context, provider changes, and feedback after deployment.

Requirements and models must consequently address uncertainty, error distributions, robustness, fairness, explainability, security, privacy, human oversight, fallback behaviour, monitoring, drift, and incident response. The Sol may need to include data pipelines, external model providers, affected stakeholders, human reviewers, evaluation mechanisms, and governance processes.

AI is therefore both a powerful instrument for analysis and a source of new analytical complexity.

Key takeaway: *GenAI can propose, transform, criticise, and explain models, and can greatly expand the futures we explore. But it may also convert ambiguity silently into apparently precise decisions. Plausibility is not evidence, consistency is not correctness, and AI cannot decide which future an organisation ought to create or assume responsibility for that choice.*

1. Acceptance Criterion - A condition used by an authorised party to decide whether a defined work item, capability, increment, or deliverable is acceptable. An acceptance criterion may clarify applicable requirements but is not automatically a requirement, a test case, or a complete requirements specification.
2. Activity Diagram (in UML) - A behaviour diagram that models the flow of activities within a system, showing the sequence and conditions for coordinating lower-level behaviours, similar to a flowchart.
3. Active Structure (in ArchiMate) - An architectural element capable of performing behaviour. Depending on the domain, it may represent an organisational entity, an application component, or a technological resource.
4. Actor - See "Actor (in UML)" and "Business Actor (in ArchiMate)".
5. Actor (in UML) - A role played by an entity external to the subject when interacting with it. One entity may play several actor roles, and the same actor role may be played by several entities.
6. Analysis (in Requirements Engineering) - The systematic examination of evidence about the Universe of Discourse, including goals, needs, concerns, rules, practices, assumptions, proposed solutions, and candidate requirements. Its purpose is to establish a coherent understanding from which justified requirements can be formulated, negotiated, and evaluated.
7. Actor (in ArchiMate) - An entity that can perform behaviour within an organisation, which can be a person, group of people, or a system.
8. Actor (in UML) - An external entity that interacts with a system, usually representing a user or another system that plays a role in one or more interactions with the system.
9. Agile (in Systems Engineering) - An iterative and flexible approach to systems engineering that emphasizes collaboration, adaptability to changes, and rapid delivery of working systems.
10. Aggregation - Aggregation expresses a "whole-part" relationship between elements, where one element (the whole) is composed of others (the parts) but the parts may exist independently. In UML diagrams, it is represented by a hollow diamond at the "whole" end. It denotes a structural association less strict than composition, which implies shared existence and lifecycle dependency. See "Composition".
11. Analysis (in Requirements Engineering) - The process of refining raw requirements to ensure they are clear, consistent, complete, and feasible, involving techniques such as requirements classification, modelling, and prioritization.
12. Application (in Software Engineering) - A software program designed to perform a specific set of tasks for the user, such as word processing, web browsing, or database management.
13. ArchiMate - A standardised visual modelling language for describing enterprise architectures. ArchiMate 4 organises its concepts into the Common, Business, Application, Technology, Motivation, Strategy, and Implementation and Migration domains. ArchiMate defines a language, not a method for conducting Enterprise Architecture.
14. Architecture - The organisation of the relevant elements and relationships of an Entity of Interest, together with the principles guiding its evolution. Architecture should not be confused with an Architecture Description, which represents selected aspects of that architecture.
15. Architecture Description - A work product that represents an architecture for a stated purpose. It addresses stakeholder concerns through one or more views constructed according to defined viewpoints.
16. Artefact - Any tangible or information-bearing item produced, modified, or used during an engineering activity, such as a document, model, prototype, program, or test report. See also the language-specific concept "Artifact".
17. Artifact - See "Artifact (in ArchiMate)" and "Artifact (in UML)".
18. Artifact (in ArchiMate) - A concrete item used or produced in a technological environment, such as an executable package, configuration file, database schema, deployment package, stored file, or log file. An Artifact may realise an Application Component and may be deployed on a Node.
19. Artifact (in UML) - A physical piece of information used or produced during system development, deployment, or operation, such as an executable file, software library, configuration file, database schema, or deployment package. An Artifact may manifest a logical model element and may be deployed on a Node.
20. Association - An association defines a relationship between two or more elements, typically representing how objects or blocks in a model interact or reference each other. In UML, associations can include multiplicities, roles, and navigability. They serve as the basis for understanding structural dependencies and can be refined into specific relationship types such as aggregation, composition, or generalisation.
21. Assumption - A proposition treated as true for the purposes of analysis despite not being guaranteed by the system. Important assumptions should have an identified source, justification, dependencies, consequences if false, and means of checking. An assumption is not a weaker form of requirement.
22. Behaviour - Behaviour refers to the set of actions, reactions, and interactions that a system or component exhibits over time in response to internal or external

stimuli. Behavioural modelling focuses on describing what the system does rather than how it is built. In UML, behaviours are expressed through activity, state machine, or sequence diagrams.

23. Black-box - An approach in systems engineering or modelling where the internal workings of a system or component are not a concern, which focus on the system's functionality against specified requirements or user expectations. A *black-box* representation focuses on the system's externally observable behaviour without revealing its internal structure or implementation. It is defined by its inputs, outputs, and externally visible interactions. Black-box modelling supports abstraction, enabling analysts to reason about system functionality and interfaces while deferring internal design decisions. See "White-box"
24. Boundary - A conceptual distinction between what is considered part of a system and what is treated as its environment. A boundary establishes which responsibilities may be assigned to the system and need not coincide with ownership, organisational control, or physical enclosure.
25. BPMN – The Business Process Model and Notation (BPMN) is a graphical representation for specifying business processes in a business process model, maintained by the Object Management Group (OMG). It uses a flow-based notation with events, activities, and gateways to depict the logical sequence of process steps. BPMN promotes understanding between technical and non-technical stakeholders and supports process automation by bridging analysis models and execution engines.
26. Business Actor (in ArchiMate) - An organisational entity capable of performing behaviour. A Business Actor may represent an individual, team, organisational unit, organisation, or external organisational party. It should not be used to represent an application, activity, responsibility, or item of information.
27. Business Process Model and Notation – See "BPMN"

28. Class Diagram (in UML) - A UML diagram that represents the static structure of a system by showing its classes, attributes, operations, and relationships among objects.
29. Component (in UML) - A modular and replaceable unit that encapsulates implementation and exposes defined behaviour through provided and required Interfaces. A Component represents a logical responsibility and should not be confused with an executable file, server, or physical device.
30. Component Diagram (in UML) - A UML diagram that shows the organisation and dependencies of components in a system, used to model the aspects of software architecture.
31. Composite Structure Diagram (in UML) - A structural diagram that exposes the contextual internal structure of a Component, Class, or other structured classifier through Parts, Ports, Interfaces, and Connectors.

32. Composition - See "Composition (in ArchiMate)" and "Composition (in UML)".
33. Composition (in ArchiMate) - A structural relationship indicating that an element consists of one or more other concepts. It expresses a whole-part relationship but does not inherit the ownership and lifecycle semantics of UML composition.
34. Composition (in UML) - A strong whole-part relationship in which a composite has responsibility for its parts. A part may belong to at most one composite at a time, and destruction of the composite normally entails destruction of its parts.
35. Conceptual Model - An abstract representation of a system focusing on the key elements and their relationships without implementation details, used in the early stages of systems engineering.
36. Concern - An interest or issue relevant to one or more stakeholders, influencing the system's requirements and architecture.
37. Configuration Management - The discipline ensuring that artefacts and their versions are properly controlled and recorded.
38. Connector (in UML) - An interaction path between Parts or Ports within a structured context. Assembly Connectors link compatible provided and required Interfaces, while Delegation Connectors link an external Port to the internal Part or Port responsible for handling the interaction.
39. Constraint - A system requirement that restricts the permitted solution space beyond what is inherently necessary to satisfy the functional and quality requirements. A freely selected solution is a Design Decision, not a Constraint. A Constraint should have an explicit source and rationale.
40. Context - *Context* refers to the environment and conditions within which a system exists and operates. It includes all external entities, processes, and constraints that influence or interact with the system-of-interest. Defining the context is essential to establish system boundaries, interfaces, and assumptions. In modelling, context captures both operational surroundings (actors, systems, regulations) and conceptual factors (purpose, scope, dependencies) that frame system understanding. The context defines the system boundary and the scope of analysis; it is therefore a selected view of the broader environment, focused on what matters for the system-of-interest. See "Context Diagram". See "Environment".
41. Context Diagram - A context diagram illustrates the system as a single entity (the "system-of-interest") and shows its context. Context diagrams are useful for clarifying system scope and identifying interfaces at an early stage of analysis. It is rare to have modelling languages that have explicit context diagrams, as usually this is a concept that can be addressed by creating views with diagrams already defined for other specific purposes. See "Context".
42. Coupling - Coupling refers to the degree of interdependence between components or subsystems. Low coupling is desirable because it improves

modularity, maintainability, and reusability. High coupling indicates strong dependency, meaning changes in one component are likely to affect others. In modelling, coupling analysis supports architectural decisions that balance integration with independence.

43. Coverage . Coverage expresses the extent to which elements of one model (e.g. requirements) are represented or satisfied by elements of another (e.g. components or test cases). Coverage analysis is essential in traceability, ensuring completeness and verifying that all stakeholder needs are addressed by design or implementation artefacts.

44. Data - Data consists of raw, unorganised facts or figures without context or meaning. See “DIKW.”

45. Data Flow - A data flow represents the movement of information between processes, components, or systems. It defines what data is exchanged, by whom, and under what conditions. Data-flow modelling supports understanding of system interfaces and communication patterns, and complements structural models by showing how information is transformed and propagated throughout the system.

46. Data Object (in BPMN) - Represents data required or produced by activities within a process, providing information about the process's data requirements and outputs.

47. Decomposition - Decomposition is the process of breaking down a system or problem into smaller, more manageable parts. In systems engineering, it helps identify subsystems, components, and functions that collectively achieve system goals. Decomposition supports analysis, modularity, and traceability, and can be applied to both structure (e.g. blocks, components) and behaviour (e.g. activities, processes).

48. Dependency - A dependency represents a relationship in which one element relies on another for definition, implementation, or execution. In modelling languages such as UML, it indicates that changes to the supplier element may affect the client element. Dependency management is crucial for maintaining model consistency and understanding the propagation of change across system artefacts.

49. Deployment Diagram (in UML) - A UML diagram that models the physical deployment of artifacts on nodes such as servers or devices, showing how software components are distributed across hardware.

50. Design Decision - A justified selection of a particular way of satisfying one or more requirements. A Design Decision should remain traceable to the requirements, risks, evidence, and alternatives that motivated it.

51. Diagram - A visual arrangement of selected model content. A diagram may present part of a model, but it is not necessarily a complete Model or a complete View.

52. DIKW - The DIKW hierarchy stands for “Data, Information, Knowledge, and Wisdom”, which guides the transformation of raw facts into actionable insights

for designing new systems. See “Data,” “Information”, “Knowledge”, “Wisdom”.

53. Domain – A domain is a distinct area of knowledge, activity, or responsibility that defines the scope of a model or analysis. In conceptual modelling, it refers to the part of the real world that is relevant to the problem being studied. For example, the Universe of Discourse (UoD) represents the domain of interest from which requirements and system boundaries are derived.

54. Domain of a System - The domain of a system is the portion of the real world (physical, organisational, conceptual, etc.) that is relevant to understanding, designing, or analysing that system. It includes the entities, activities, and constraints that the system concerns itself with or that influence its behaviour. Clearly identifying the domain of the system helps delimit what belongs to the system-of-interest and what remains external to it, ensuring that requirements and models reflect the intended operational reality.

55. Dynamic Behaviour - Dynamic behaviour describes how a system evolves over time in response to internal events or external stimuli. It captures state changes, message exchanges, and activity flows. For example, UML expresses dynamic behaviour through state machine, sequence, and activity diagrams. Understanding dynamic behaviour is essential for analysing concurrency, timing, and event-driven interactions within a system.

56. Element - An element is any identifiable construct within a model, such as a block, requirement, actor, or connector. In UML, all model elements share a common meta-structure that allows them to be related, classified, and traced. Elements represent the building blocks of a model and are organised to capture structure, behaviour, or relationships across multiple views.

57. Elicitation (in Requirements Engineering) - The systematic discovery of information relevant to understanding the Universe of Discourse, defining the System of Interest, and formulating its requirements. Information may be elicited from people, documents, observation, existing systems, operational data, physical environments, and applicable rules. Elicitation does not merely collect pre-existing requirements.

58. Emergent Behaviour - Emergent behaviour arises when the interactions among system components produce outcomes that cannot be fully predicted from the behaviour of individual parts. **It is a key notion in systems thinking and complexity science.** Understanding emergence helps engineers anticipate non-linear effects, unexpected dependencies, and global properties that result from local interactions in complex systems.

59. Engineering-process Requirement - An obligation governing how a system is to be analysed, designed, implemented, verified, validated, or managed. Evidence of satisfaction concerns whether the prescribed engineering activity was performed, not

whether the resulting system possesses a particular quality.

60. Entity - An entity represents a real-world object, concept, or phenomenon that has a distinct existence within a domain. Entities may be physical (e.g., sensor, vehicle) or abstract (e.g., order, contract). In conceptual modelling, entities are characterised by attributes and relationships, forming the foundation of information and data models such as Entity-Relationship Diagrams (ERDs).
61. Entity of Interest (Eoi) - The entity whose architecture is being considered or described. It may be a system, enterprise, service, process, collection of systems, or another entity relevant to the stakeholders and concerns of an Architecture Description.
62. Environment (of a system) - The environment of a system is the part of reality that surrounds it and can influence or be influenced by its operation, even if not directly modelled. It includes physical conditions, social settings, organisational structures, and external systems. While the context defines the specific elements and interactions considered relevant for a given model or analysis, the environment is broader: it encompasses everything external that may affect the system but lies outside its defined scope. See "Context".
63. Eoi - See "Entity of Interest."
64. Event - An event is an occurrence that triggers a change of state or the execution of behaviour in a system. Events can be external (e.g., user command, sensor reading) or internal (e.g., timeout, signal reception). In UML, events are central to behavioural models such as state machines and sequence diagrams, capturing the system's temporal dynamics.
65. Evaluation - Evaluation is the process of assessing a model, design, or system against defined criteria such as correctness, completeness, performance, or stakeholder satisfaction. It may involve qualitative review, quantitative measurement, or simulation. Evaluation supports decision-making throughout the system life cycle, ensuring that results remain aligned with objectives and constraints.
66. Execution Environment (in UML) - A software Node that provides an environment in which deployed Artifacts execute, such as a virtual machine, application server, container runtime, operating system process, or database-management environment.
67. External System - An external system is any system outside the boundary of the system-of-interest that exchanges information, energy, or material with it. External systems are represented as actors or interfaces in models and help define the system's operational context. They are not under the designer's control but must be understood for integration and interoperability.
68. Explicit Knowledge - Refers to codified, documented, and can be easily communicated. See "Knowledge," "Tacit Knowledge"

69. Feasibility Study (in Requirements Engineering) - An assessment of whether the requirements can be realistically achieved within the project's constraints, such as time, budget, and technology.
 70. Feedback Loop - A feedback loop occurs when the output of a system or process is routed back as input, influencing future behaviour. Feedback can be negative (stabilising, maintaining equilibrium) or positive (amplifying deviations). Understanding feedback loops is central to systems thinking and control engineering, as they explain how systems adapt, self-regulate, or drift towards instability.
 71. Framework - A framework is a structured set of concepts, principles, or conventions that provides a common basis for organising and reasoning about a domain. In systems and enterprise architecture, frameworks such as TOGAF, Zachman, or NAF help ensure consistency across viewpoints and support traceability between requirements, design, and implementation artefacts.
 72. Function - A function defines what a system or component does (its intended transformation of inputs into outputs). Functions describe purpose or capability, independently of physical realisation. Functional analysis decomposes system behaviour into functions and sub-functions, supporting design alternatives and ensuring that each function contributes to achieving stakeholder goals.
 73. Functional Requirement - A System Requirement specifying behaviour or an outcome that the system or one of its constituents must provide under stated conditions. Functional does not mean implemented by software.
-
74. General Systems Theory - A multidisciplinary framework for understanding and analysing complex systems across various domains, which emphasizes concepts like hierarchy, feedback, and the interdependence of system components.
 75. Generalisation - Generalisation defines an inheritance relationship in which a more specific element (the specialisation) adopts the properties and behaviour of a more general one. In UML, it represents a "kind of" relationship (for example, a Truck is a kind of Vehicle). Generalisation supports abstraction, reuse, and taxonomy creation within structural and conceptual models.
 76. Goal - A high-level statement of intent that provides strategic direction for a system or organisation. A goal is a desired state or outcome that a system, project, or stakeholder aims to achieve. Goals express purpose and direction, providing a rationale for requirements and design decisions. In modelling, goals can be organised hierarchically and traced to functions or requirements that contribute to their satisfaction. They are essential in aligning system behaviour with stakeholder intentions.
 77. Governance - Governance refers to the set of rules, roles, and decision-making mechanisms that guide and control how a system or organisation is managed. In

systems engineering, governance ensures accountability, compliance, and coherence across design, operation, and evolution. Architectural governance frameworks define how changes are proposed, approved, and monitored throughout the system's life cycle.

78. Granularity - Granularity indicates the level of detail or resolution used in describing or decomposing a system. A coarse-grained model shows major components and interactions, while a fine-grained model reveals internal details and dependencies. Choosing the right granularity depends on the purpose of analysis — higher abstraction supports understanding, while finer granularity aids implementation and verification.
79. Graphical Notation - A graphical notation is a formal visual language used to represent models through standard symbols and rules. Examples include UML, BPMN, and ArchiMate. Graphical notations support shared understanding among stakeholders by making structures and behaviours explicit, reducing ambiguity, and allowing automated consistency checks when supported by modelling tools.
80. Heuristic - A heuristic is a practical rule or method used to guide reasoning or problem solving when formal analysis is difficult or infeasible. Heuristics provide insight based on experience, helping modellers make sound judgements about abstraction, decomposition, or trade-offs. In systems analysis, they support early design exploration and the identification of “good enough” solutions within complex domains.
81. Hierarchy - A hierarchy is an organisational structure in which elements are arranged at different levels of abstraction or authority. In systems engineering, hierarchies can describe component decomposition (system → subsystem → component) or functional refinement (function → sub-function). Hierarchical modelling supports traceability, modularity, and progressive elaboration from high-level concepts to detailed design.
82. Human–System Interaction (HSI) - Human–System Interaction refers to the way users or operators engage with a system to achieve goals. It includes user interfaces, control mechanisms, feedback, and ergonomic or cognitive considerations. In systems engineering, HSI ensures that system functions, automation, and human roles are designed coherently to maximise safety, efficiency, and usability.
83. Hybrid System - A hybrid system combines discrete and continuous behaviours within the same model. Discrete elements respond to events or state changes (as in digital logic), while continuous elements evolve over time according to physical laws (as in mechanical or electrical systems). Hybrid modelling is essential in domains such as robotics, automotive control, and cyber-physical systems.
84. Hypothesis - A hypothesis is a provisional assumption or statement proposed to explain an observation or guide investigation. In systems analysis and modelling, hypotheses help capture design assumptions, predict system behaviour, or test alternative solutions.

Validating or refuting hypotheses through simulation, experimentation, or stakeholder feedback supports evidence-based decision-making during system development.

85. Implicit Knowledge - See “Tacit Knowledge.”
86. Incremental Development - An approach in systems engineering that involves developing a system through repeated cycles (iterations) and in small portions at a time (increments), allowing for continuous feedback and adaptation. See “Iteration”
87. Information - Information emerges when data is processed, organised, and contextualised to provide meaning and answers to questions like “who,” “what,” “where,” and “when”, which involves inference to identify relationships and drawn patterns from the raw data. Seek “DIKW”.
88. Interface - An interface defines the boundary through which a system or component interacts with its environment or with other components. It specifies what is exchanged (data, signals, materials, energy, etc.) and under what conditions. Interfaces can be logical (information-based) or physical (hardware connections), and their clarity is essential for ensuring system integration and interoperability.
89. Interaction - An interaction represents the exchange of information, energy, or control between elements of a system. In UML, interactions are described using sequence diagrams or communication diagrams, which capture the order and timing of messages. Understanding interactions helps ensure that system behaviour aligns with functional requirements and stakeholder expectations.
90. ISO/IEC/IEEE 29148:2018 - The international standard that defines processes and requirements for Requirements Engineering in systems and software engineering. It specifies how to elicit, analyse, verify, and manage requirements, distinguishing between functional, non-functional, and constraint types. The standard promotes traceability, consistency, and verifiability across the system life cycle.
91. ISO/IEC/IEEE 42010:2022 - The international standard defining concepts and requirements for Architecture Descriptions. Its principal concepts include Entity of Interest, Architecture Description, Stakeholder, Concern, View, Viewpoint, Model Kind, Architecture Description Language, and Architecture Description Framework.
92. Iteration - Iteration is the repeated application of a process or activity to progressively refine results. In systems engineering, iteration supports learning, validation, and risk reduction by allowing partial solutions to be tested and improved. Iterative development underlies modern engineering methodologies such as Agile and Model-Based Systems Engineering (MBSE), promoting adaptability and continuous improvement. See “Incremental Development”.

93. Joint Development - Refers to a collaborative approach in which multiple stakeholders or organisations participate in defining, designing, or building a system. It emphasises shared ownership of requirements and architecture, promoting coherence across organisational boundaries. In systems engineering, joint development is common in large projects where interoperability, shared infrastructure, or co-funded research are required. 94. Junction (in Diagrams) - A junction is a graphical element used in certain modelling notations, such as state machine or activity diagrams, to connect multiple flows or transitions. It allows several incoming and outgoing paths to converge or diverge, supporting compact representation of complex control logic. Junctions simplify models without altering the underlying semantics of flow or transition. 95. Justification - A justification provides the reasoning or evidence supporting a decision, requirement, or design choice. Documenting justifications enhances transparency and traceability, allowing reviewers to understand why certain options were adopted. In formal models, justifications may link to assumptions, constraints, or stakeholder needs, helping maintain the integrity of the decision-making process throughout the system life cycle.	systems and AI-driven diagnostic tools. In systems engineering, such systems are used to automate analysis, verification, and configuration tasks.
96. Key Performance Indicator (KPI) - A measurable value that reflects how effectively a system, process, or organisation achieves defined objectives. KPIs are used to monitor performance, guide decision-making, and assess progress towards goals. In systems engineering, KPIs may relate to quality, reliability, response time, or resource efficiency, supporting continuous evaluation throughout the lifecycle. 97. Knowledge - Knowledge adds interpretation and deeper understanding to information, which involves applying experience, technical expertise, and analysis to transform information into actionable insights. See "DIKW." 98. Knowledge Representation - Refers to the formal way in which information, concepts, and relationships are structured so they can be understood, processed, or reasoned about by humans and machines. In modelling, it underlies the use of ontologies, taxonomies, and metamodels to ensure that meaning (semantics) is explicit, consistent, and machine-interpretable. 99. Knowledge Management - The discipline concerned with capturing, organising, sharing, and maintaining the information and experience generated during system development and operation. Effective knowledge management supports traceability, reuse, and learning across projects. In model-based approaches, the model repository itself serves as a structured knowledge base for the engineering process. 100. Knowledge-Based System - A computer system that applies explicit knowledge (often expressed as rules, facts, or models) to support reasoning, problem solving, or decision-making. Examples include expert	101. Level of Abstraction - The level of abstraction defines how detailed or general a model or description is. Higher abstraction emphasises essential structure and behaviour while hiding implementation details; lower abstraction exposes specific mechanisms and configurations. Managing levels of abstraction allows engineers to reason about systems progressively (from conceptual understanding to detailed design) maintaining coherence across models. 102. Lifecycle - A lifecycle represents the complete sequence of stages a system undergoes from conception through development, operation, maintenance, and disposal. The concept underpins standards such as ISO/IEC/IEEE 15288. Lifecycle thinking ensures that requirements, design, and evaluation consider not only initial functionality but also long-term sustainability, evolution, and decommissioning. 103. Link - A relationship or connection between two model elements. Depending on the modelling language, links can represent associations, dependencies, or traceability connections. In UML, links connect instances (objects) at runtime, realising the structural relationships defined at the model level. Links help visualise how parts of a system communicate or depend on one another. 104. Logical Architecture - Logical architecture is an abstract representation of a system that identifies its main functions and their relationships, independent of any physical implementation. It defines what must exist to realise system behaviour before design choices about technologies or components are made. Logical architecture supports traceability between stakeholder needs and the eventual physical architecture. 105. Loop - A repeated sequence of actions, events, or computations controlled by a condition. In behavioural diagrams such as activity or sequence diagrams, loops represent iteration in response to feedback, data availability, or control signals. Loops are fundamental for modelling recurring processes, cyclic operations, and feedback mechanisms within dynamic systems. 106. M0 - Meta-Object Facility, Layer 0, is used by MOF to refer to the objects of the real world (the book W, the dog X, the person Y, the machine Z, ...). 107. M1 - Meta-Object Facility, Layer 1, is used by MOF to refer to the models created according to what is defined by the Layer M2. 108. M2 - Meta-Object Facility, Layer 2, is used by MOF to refer to the metamodels. The concept of "Class", "Attribute", and "Instance", for example, are defined as "instanceOf" the concept of "Class" as defined by the Layer M3.

109. M3 - Meta-Object Facility, Layer 3, is the language used by MOF to build metamodels, called M2-models. The upper concept of “Class” as a “generic thing”, is defined at this Layer 3.

110. Mapping - The correspondence or transformation between elements of different models or abstraction levels. It ensures consistency and traceability between views — for instance, between requirements, logical functions, and physical components. Mappings are fundamental in model-based engineering, supporting integration, verification, and automated generation of design artefacts.

111. MBSE (Model-Based Systems Engineering) - An approach that uses formal models as the primary means for system definition, analysis, and communication throughout the life cycle. It replaces document-centric practices with interconnected models that describe requirements, architecture, behaviour, and verification. MBSE enhances traceability, consistency, and collaboration across multidisciplinary teams.

112. MDE (Software Engineering) – The Model Driven Engineering (MDE) is a broader software engineering paradigm that focuses on the use of models to automate and streamline the software development process.

113. MDSE – The Model-Driven Systems Engineering (MDSE) is an approach that uses models as the primary means for capturing, analysing, designing, and validating systems throughout their lifecycle.

114. Metamodel - Defines the language for expressing a model, providing the structure and rules for creating models, essential in defining modelling languages like UML and ArchiMate. In the modelling hierarchy, a metamodel operates one level above the models it governs.

115. Meta-Object Facility – See “MOF.”

116. Model - A purposeful and selective representation of a subject, situation, or Entity of Interest, created to support understanding, analysis, communication, prediction, or action. A Model may be descriptive, exploratory, or normative and should not be confused with one of its diagrams.

117. Model Driven Engineering – See “MDE.”

118. Model-Driven Systems Engineering - See “MDSE.”

119. Model Transformation - The process of automatically converting one model into another according to defined rules. Transformations can occur between abstraction levels (e.g., logical to physical) or between modelling languages. They are essential in MDE for maintaining consistency and enabling partial automation of system development.

120. Modelling Language - A modelling language is a formal system of notation and semantics used to express models. It defines symbols, syntax, and interpretation rules that ensure models are understandable and processable by humans and tools. Examples include UML, SysML, BPMN, ArchiMate, and IDEF0. A good

modelling language balances expressiveness with clarity and analytical rigour.

121. Modularity - Modularity is the design principle of dividing a system into distinct components (modules) that can be developed, tested, and maintained independently while functioning coherently as a whole. It improves comprehensibility, scalability, and reuse. Modularity is achieved through well-defined interfaces and minimal coupling between modules.

122. MOF - The Meta-Object Facility (MOF) is a standard developed by OMG that provides a framework for defining metamodels, ensuring consistency across different modelling languages.

123. Need - Something that a stakeholder considers necessary in relation to a concern, problem, or goal. A Need is a source for analysis but is not automatically a System Requirement.

124. Network - A set of interconnected elements or nodes linked by relationships or flows. In systems engineering, networks can represent physical infrastructures (such as communication or transport systems) or logical structures (such as dependency or data-flow graphs). Network models help analyse connectivity, resilience, and performance within complex systems.

125. Node - See “Node (in ArchiMate)” and “Node (in UML)”.

126. Node (in ArchiMate) - A computational or physical technological resource that may host, execute, store, or process other elements. A Node provides a general representation when a more specialised technological element is unnecessary or unavailable.

127. Node (in UML) - A computational resource on which Artifacts may be deployed. UML distinguishes Devices, representing physical computational resources, and Execution Environments, representing software environments that execute deployed Artifacts.

128. Non-Functional Requirement (NFR) - A widely used but ambiguous external term that may refer to a Quality Requirement, a Constraint, or another kind of obligation. This course does not use NFR as a primary requirement category. When the term is encountered, determine the actual nature and target of the obligation.

129. Notation - A notation is the set of symbols, graphical conventions, and syntactic rules used in a modelling language. It provides the visible form of concepts defined by the language’s metamodel. Notation ensures that models can be read and interpreted consistently across stakeholders and tools. Examples include the graphical notation of ArchiMate, BPMN, or UML diagrams.

130. Normalization - The process of organising or refining models to eliminate redundancy and ambiguity while preserving meaning. In data modelling, it ensures consistent representation of information through well-structured entities and relationships. In broader systems modelling, normalization refers to making

conceptual distinctions explicit, improving clarity and analytical precision.

131. Object - An object is an identifiable instance of a class or block that encapsulates data (attributes) and behaviour (operations). In object-oriented modelling, objects interact through message exchanges, representing concrete manifestations of abstract model elements. Objects help connect conceptual models to executable or simulated representations of system behaviour.
132. Object Diagram (in UML) - A UML diagram that shows a snapshot of the system at a specific time, depicting objects and their relationships, useful for representing the static aspects of a system.
133. Ontology - a branch of philosophy that deals with the nature of being, existence, and reality, exploring fundamental questions about what entities exist and how they relate to one another, and that in was adopted in engineering to refer to the structured representation of specific systems' domains in a machine-readable way. In that context, an ontology is a formal specification of concepts and their relationships within a domain. It defines shared vocabulary and structure for meaning, enabling consistent interpretation by humans and machines. Ontologies support interoperability, reasoning, and semantic traceability across models and systems, and underpin techniques used in knowledge representation and model integration
134. Ontology engineering - creating ontologies.
135. Open System - An open system is one that exchanges matter, energy, or information with its environment. In systems engineering, openness implies that system boundaries are permeable to external influences, making interactions with other systems or actors essential for operation. Understanding openness helps model interfaces, dependencies, and environmental variability.
136. Operation - A defined behaviour or service that an element of a system can perform. In structural models, operations are associated with blocks or classes and represent actions triggered by events or interactions. Operations specify what a component does and under what conditions, supporting behavioural definition and interface design.
137. Output - Any result, product, or signal produced by a system, process, or function. Outputs may be physical artefacts, data, or states. In modelling, outputs represent the effect or consequence of system behaviour, forming part of the transformation of inputs into desired outcomes. Defining outputs is essential for specifying functional completeness.

138. Package Diagram (in UML) - A UML diagram that shows how distinct parts of the model are grouped and related, helping organize large systems into manageable sections.

139. Parameter - A parameter is a variable used to define or constrain a system property, function, or model element. Parameters quantify attributes such as dimensions, capacity, or performance and can be adjusted to explore design alternatives. In modelling, parameters help simulate system behaviour and perform sensitivity or optimisation analyses.
140. Part (in UML) - A property representing a constituent role within the context of a Component, Class, or other structured element. A Part is commonly identified by a name and a Type and should not be interpreted merely as an element drawn inside another element.
141. Passive Structure (in ArchiMate) - Something upon which behaviour is performed or which behaviour creates, uses, or changes. Examples include information, documents, messages, money, materials, and other relevant objects.
142. Physical Architecture - Physical architecture represents the concrete implementation of a system in terms of its physical components, hardware, and interconnections. It realises the logical architecture by mapping functions onto physical structures. Physical architecture is essential for integration, verification, and maintainability, ensuring that the designed system can be built and operated as intended.
143. Physical Referent - An entity or phenomenon in the Universe of Discourse to which a Model element refers. A Physical Referent must not be confused with the logical or software representation through which information about it is maintained.
144. Port (in UML) - An identified interaction point on the boundary of a Component or another encapsulated classifier. A Port identifies where interaction occurs, while provided and required Interfaces specify the contracts available through that interaction point.
145. Project-delivery Requirement - An obligation governing the execution or outputs of the project that creates or changes a system. It may concern delivery dates, budgets, contractual deliverables, installation, migration, training, or other transition activities.
146. Process - A process is a set of activities that transform inputs into outputs according to defined rules and constraints. In systems engineering, processes may refer to organisational procedures, physical transformations, or computational operations. Process modelling languages such as BPMN or IDEF0 depict how processes flow, who performs them, and how they contribute to system objectives.
147. Property - A property is a characteristic or attribute that defines an element's structure, behaviour, or constraints. Properties give quantitative or qualitative meaning to model elements, allowing them to be measured, compared, or verified.
148. Protocol - A protocol is a defined set of rules governing how components or systems communicate and interact. It specifies message formats, timing, and sequencing constraints. Protocols ensure interoperability and predictability in distributed or networked systems, whether for digital

communication, service interaction, or physical control exchanges.

149. Purpose - The purpose of a system expresses the fundamental reason for its existence, what need or goal it addresses. It provides orientation for defining requirements, evaluating design options, and assessing success. In modelling, purpose helps justify scope and abstraction choices, ensuring that models remain aligned with stakeholder intent and mission objectives.

150. Quality - Quality is the degree to which a system satisfies stated and implied needs under specified conditions. It encompasses both functional correctness and non-functional characteristics such as reliability, usability, and performance. Standards such as ISO/IEC 25010 define quality models that provide structured criteria for evaluating and comparing systems.

151. Quality Attribute - A characteristic that contributes to the quality of a system, constituent, function, or service, such as availability, reliability, usability, accessibility, security, safety, or maintainability. A Quality Attribute becomes the subject of a Quality Requirement when a required degree and relevant evaluation conditions are specified.

152. Quality Requirement - A System Requirement specifying a required degree of a quality exhibited by the system, one of its constituents, or a function it performs. A useful Quality Requirement identifies the relevant subject, operating conditions, required level, and means of evaluation.

153. Quantitative Analysis - Quantitative analysis is the study of system properties using numerical or mathematical techniques. It includes simulation, optimisation, and statistical evaluation of models to predict performance, cost, or reliability. In model-based engineering, quantitative analysis complements qualitative reasoning by providing evidence to support design decisions and risk assessment.

154. Query - A query is a formal request for information extracted from a model or data repository according to defined conditions. Queries allow engineers to inspect, validate, or trace model elements automatically. In model repositories and MBSE tools, query languages such as OCL (Object Constraint Language) or SQL-like syntaxes support consistency checking and automated reporting.

155. Refinement - The process of elaborating a model element or requirement into a more detailed or specific form. It ensures that high-level abstractions are progressively translated into implementable specifications while maintaining traceability.

156. Repository (in ReqEng and Modelling) - A repository is a structured storage environment for managing models, artefacts, and related data. It enables collaboration, version control, and consistency checking across teams. In MBSE, repositories serve as central sources of truth, integrating information from multiple modelling languages and ensuring that system

knowledge remains traceable and reusable throughout the lifecycle.

157. ReqEng – See “Requirement Engineering”

158. Requirement - An explicit obligation concerning a System of Interest or one of its constituents. A Requirement may prescribe behaviour or an outcome, a required degree of quality, or a restriction that the system must respect. Unless otherwise qualified, “Requirement” is used as a short form of “System Requirement”.

159. Requirement Traceability – See “Traceability”.

160. Requirements Engineering - The iterative and disciplined practice of framing an analysis, eliciting relevant information, analysing and negotiating responsibilities, specifying requirements, verifying and validating them, and managing traceability and change throughout the relevant lifecycle.

161. Requirements Set - A managed collection of related requirements evaluated as a whole. Its quality depends on justified coverage, consistency, collective feasibility, coherence, comprehensibility, traceability, modifiability, and the ability to be validated and verified.

162. Requirements Source - The origin of information, evidence, authority, or justification used to formulate or support a requirement. Sources may include stakeholder goals, needs and concerns, laws, regulations, standards, contracts, existing practices and systems, operational data, risks, and authorised decisions.

163. Requirements Specification - A controlled work product that organises and communicates the requirements for a defined system, scope, purpose, and audience. It may combine textual requirements, models, definitions, assumptions, sources, relationships, traceability, verification information, and change history and need not be maintained as a single document.

164. Risk - A risk is a potential event or condition that could negatively affect the achievement of system objectives. Risk management involves identifying, analysing, mitigating, and monitoring such uncertainties. In systems engineering, risks may stem from technical complexity, integration challenges, or external dependencies, and are tracked throughout the lifecycle to maintain system reliability and project stability.

165. Role - A role defines the responsibilities or behaviour expected of an actor, component, or stakeholder in a system or process. Roles clarify who or what performs each activity and under what conditions. In modelling, roles may appear as swimlanes in activity diagrams or as part roles in structural diagrams, representing interaction responsibilities within a system.

166. ROME (Return on Modelling Effort) - A principle used in systems and requirements engineering to evaluate the value gained from modelling activities, ensuring the benefits justify the resources and effort invested.

167. Rule - A prescription governing a domain, organisation, process, or relevant participant. A Rule may provide an authoritative source for requirements, but it becomes

a System Requirement only after it has been interpreted in context and an obligation has been assigned to the system or one of its constituents.

168. Scenario - A scenario is a descriptive sequence of events showing how a system interacts with actors or other systems under specific conditions. Scenarios illustrate use cases, operational contexts, or failure situations, helping stakeholders visualise behaviour over time. In modelling, scenarios are expressed through textual descriptions, sequence diagrams, or activity diagrams.
169. Scope - Scope defines the extent or limits of what a project, model, or analysis covers. It clarifies which aspects of the environment, functions, or requirements are included and which are excluded. A well-defined scope ensures focus, avoids ambiguity, and supports alignment between stakeholder expectations and system boundaries. See also: "Context", "Domain".
170. Sequence Diagram (in UML) - A behavioural diagram representing time-ordered interactions between Lifelines. Lifelines may represent actors, objects, Parts, Components, or other relevant participants in a particular scenario.
171. Service (in ArchiMate) - Behaviour made available to an environment while hiding its internal realisation. A Service provides a black-box perspective of what an organisation, application, or technological system offers to another actor or system.
172. Simulation - Simulation is the execution of a model to study system behaviour under specific conditions. It allows quantitative evaluation of performance, reliability, or response without needing a physical prototype. Simulations are used to test design hypotheses, explore alternatives, and validate requirements, often integrating parametric and behavioural models in MBSE environments.
173. Sociotechnical System - A system whose relevant constituents may include people, organisational roles, processes, rules, responsibilities, information, software applications, services, devices, and other physical resources.
174. Sol - See "System of Interest".
175. SoS - See "System of Systems"
176. Specification (in Requirements Engineering) - The process of documenting requirements in a structured, clear, and unambiguous format, serving as the official reference for development and stakeholders.
- 177. Stakeholder - A stakeholder is any individual, group, or organisation that defines, uses, influences, or is affected by a system. Stakeholders give the system its identity by recognising it as a coherent entity created to satisfy their needs or interests; without stakeholders, a "system" has no defined purpose or boundaries. Stakeholders include users, operators, developers, maintainers, managers, regulators, and any party with legitimate concerns regarding the system's existence or behaviour.**
178. State - A state represents a condition or situation of a system or component at a specific point in time, determined by its properties and past events. State-based models describe how systems respond to events through transitions. State Machine Diagrams in UML capture these transitions to define dynamic system behaviour.
179. State Machine Diagram (in UML) - A UML diagram that describes the states an object or interaction can be in and how it transitions from one state to another based on events.
180. State of Affairs - a specific arrangement of entities, objects, or events; a specific configuration of a system components, conditions, and relationships at a given point in time, representing a system's state.
181. Structure -Structure refers to the organisation of system elements and their relationships. Structural modelling defines how components are arranged, composed, and interconnected to realise functions.
182. SysEng - See "Systems Engineering"
183. SysML (Systems Modelling Language) - SysML is an open standard modelling language tailored for systems engineering, initially derived from UML but independent of it from SysML 2.0. It supports representation of structure, behaviour, requirements, and parametric relationships in a coherent framework. SysML facilitates model-based systems engineering (MBSE) by allowing consistent integration of multidisciplinary perspectives across the system lifecycle.
- 184. System - A set of interrelated or interacting elements recognised as a coherent whole for one or more purposes. A System may be natural, technical, social, or sociotechnical. Its boundary and relevant composition depend on the concerns and purpose of the analysis.**
185. System Requirement - An obligation concerning the System of Interest or one of its constituents. It may govern behaviour, outcomes, qualities, interfaces, or permitted solutions and is evaluated against the resulting system.
186. Systems Engineering - An interdisciplinary practice concerned with understanding, defining, developing, integrating, operating, changing, and retiring systems throughout their lifecycles. It applies both to the creation of new systems and to existing systems whose operation, structure, documentation, or shared knowledge must be reconstructed or updated to support decisions about their use and evolution.
187. SysML (Systems Modelling Language) - A standard modelling language for systems engineering that supports the representation of requirements, structure, behaviour, and quantitative relationships. In this course, SysML is presented for awareness, while the taught and assessed system-modelling work uses UML with selected extensions and conventions.
- 188. System-of-Interest (Sol) - The system-of-interest is the specific system that is the focus of analysis, design, or operation within a given context. It is distinguished from its environment and from enabling or supporting**

systems. Defining the Sol is a key step in systems engineering, ensuring that models, requirements, and interfaces all refer to the same boundary of concern.

189. Systems Modelling Language - See "SysML"

190. System of Systems - a collection of independent, interoperable systems that function together to achieve a common goal, where each constituent system keeps its own context and objectives, but their collaboration creates added value for a stakeholder beyond what any system can accomplish alone.

191. Systems Thinking - An approach to problem-solving and understanding that views complex systems as interconnected and interdependent components, rather than as isolated parts.

192. Tacit knowledge - Also often named implicit knowledge, refers to the unwritten, unspoken knowledge individuals possess, gained through personal experience, which is often difficult to formalise or share directly. See "Knowledge", "Explicit Knowledge."

193. Test - A test is an activity performed to verify that a system, component, or function meets specified requirements. Tests can be analytical, simulated, or physical, and are planned according to verification criteria. In systems engineering, tests are traceable to requirements and integrated into the lifecycle to ensure correctness and compliance. See also "Verification".

194. Test Case - A controlled specification of initial conditions, inputs, execution steps, expected results, and pass-or-fail criteria used to evaluate a defined condition or Requirement. A Scenario or Acceptance Criterion may motivate one or more Test Cases but is not itself necessarily a Test Case.

195. Traceability - The ability to establish and maintain logical links among related artefacts (such as requirements, design elements, and tests) across the system lifecycle. It ensures that every element has a justified origin and that changes can be assessed for impact. In MBSE, traceability is managed through repositories and supported by modelling toolchains. See also "Requirement Traceability".

196. Toolchain - A toolchain is an integrated set of software tools used together to support system development activities such as modelling, simulation, analysis, and configuration management. In model-based engineering, toolchains enable the exchange of models across notations (e.g. UML, BPMN, ArchiMate) and can automate parts of the design process, ensuring consistency and traceability across lifecycle stages.

197. Topology - Topology describes the arrangement and interconnection pattern of components within a system. It defines how elements are linked rather than what they are. In systems engineering, topological models help analyse communication networks, mechanical linkages, or architectural dependencies, supporting reasoning about robustness, redundancy, and spatial configuration.

198. Trade-off - A trade-off is a decision that balances conflicting objectives, such as performance versus cost or reliability versus complexity. Trade-off analysis evaluates alternatives quantitatively and qualitatively to identify acceptable compromises. Effective trade-off management is central to systems engineering, ensuring that design solutions remain feasible and aligned with stakeholder priorities.

199. Transformation – See "Model Transformation".

200. Transition - A transition represents a change of state within a system or component, triggered by an event or condition. In state machine diagrams, transitions are shown as directed connectors between states, often labelled with triggering events and associated actions. Understanding transitions helps describe dynamic system behaviour and responses to stimuli.

201. Type - A type defines a classification or category that constrains the form or behaviour of model elements. In UML, types determine the allowable values, operations, or connections of objects, blocks, or properties. Typing supports consistency, reuse, and validation, ensuring that elements conform to defined semantic structures.

202. UML (Unified Modelling Language) - UML is a general-purpose modelling language maintained by the Object Management Group (OMG). It provides a standard set of diagrams to describe system structure and behaviour, including class, activity, state machine, and sequence diagrams. UML is widely used in software and systems engineering to support design, documentation, and communication.

203. Unified Modelling Language - See "UML"

204. Universe of Discourse - The portion of reality and available knowledge that is relevant to a particular analysis and about which the participants communicate. It may include people, organisations, activities, information, technologies, physical objects, existing or proposed systems, rules, events, evidence, assumptions, and environmental conditions. The Universe of Discourse is a source for analysis, not a system specification.

205. UoD – See "Universe of Discourse".

206. Use Case - A goal-oriented description of interaction across an explicit system boundary. A Use Case normally organises a family of related Scenarios, including a main success Scenario and relevant alternatives or exceptions. It can help discover and organise required behaviour but is not automatically an atomic Requirement or a complete Requirements Specification.

207. Use Case Diagram (in UML) - A diagram showing Actors, Use Cases, their relationships, and the boundary of the subject being considered. It supports the discovery, organisation, and communication of externally meaningful behaviour but does not by itself constitute a complete set of Functional Requirements.

208. User - A user is a stakeholder who interacts directly or indirectly with a system to achieve a purpose. Users can

include operators, administrators, customers, or beneficiaries of system outcomes. Modelling user goals and interactions helps define functional requirements and ensures that system design supports intended usability and operational needs. See also “Stakeholder”, “Use Case”.

209. User Story - A short statement expressing a desired capability from a stakeholder perspective and intended to initiate analysis and conversation. A User Story is not automatically a well-formed System Requirement and may conceal several obligations, assumptions, or Design Decisions.
210. Utility - Utility refers to the degree of usefulness, benefit, or value a system provides to its stakeholders. It expresses how well the system’s outcomes satisfy stakeholder needs and expectations. In decision analysis, utility functions quantify preferences, supporting trade-off evaluation and optimisation among competing design alternatives.
211. Usability - Usability is the quality attribute that measures how easily and effectively users can learn, operate, and obtain results from a system. It encompasses efficiency, satisfaction, and error tolerance. In systems engineering, usability is considered a non-functional requirement and is often assessed through human–system interaction testing and design heuristics.

212. Validation - The evaluation of whether a Requirement, Requirements Set, Model, or System is suitable for its intended purpose and justified stakeholder concerns. Requirements Validation asks whether the right obligations have been specified; System Validation asks whether the resulting system is fit for its intended use in its operational context.
213. Value - Value represents the benefit, utility, or satisfaction derived by stakeholders from the system’s existence or operation. It is defined relative to stakeholder goals, costs, and trade-offs. In systems engineering, the concept of value supports decision-making by linking design choices to measurable outcomes, helping justify investments and prioritise requirements.
214. Verification - The evaluation, using objective evidence, of whether defined criteria have been satisfied. Requirements Verification examines whether requirements are well formed, consistent, feasible, and verifiable; System Verification examines whether the resulting system satisfies its specified requirements.
215. Version - A version identifies a specific state or revision of a model, document, or system configuration. Version control enables traceability of changes over time and supports collaboration among multiple contributors. Managing versions is essential in model-based engineering, where consistent evolution of models and artefacts must be maintained across lifecycle stages.

216. View - A view is a representation of a system from the perspective of a specific set of concerns. Each view shows selected elements and relationships relevant to

particular stakeholders or engineering activities. In ISO/IEC/IEEE 42010, views are organised according to defined viewpoints, ensuring consistent communication across disciplines and abstraction levels. See also “Viewpoint”.

217. Viewpoint - A viewpoint defines the conventions, languages, and rules for constructing a view that addresses particular stakeholder concerns. It specifies what information should be represented and how it should be interpreted. Viewpoints promote clarity and separation of concerns (for example, structural, behavioural, or operational perspectives) ensuring that different models of the same system remain coherent. Defined formally in: ISO/IEC/IEEE 42010.

218. V-Model - An extension of the Waterfall Model that emphasizes verification and validation at each stage of development.

219. Waterfall Model - A linear sequential approach to systems engineering where each phase of the project must be completed before the next begins.

220. Weighting - Weighting is the assignment of relative importance or influence to criteria, factors, or requirements in decision-making or evaluation. In systems engineering, weighting supports trade-off analysis, allowing quantitative comparison among alternatives based on stakeholder priorities. Weighting schemes must be transparent and justified to ensure fair and reproducible design choices.

221. Well-Formed Model - A well-formed model complies with the syntactic and semantic rules of its modelling language. It ensures that all elements, relationships, and constraints are valid and interpretable within the language’s metamodel. Maintaining well-formedness allows models to be processed by tools, verified automatically, and shared consistently across different engineering domains.

222. Well-Formed Requirement - A Requirement that is necessary and justified, appropriately scoped, unambiguous, sufficiently complete, singular, feasible, verifiable, correct, and conforming to the adopted terminology and writing conventions. SMART may support some purposes but is not a sufficient general definition of requirement quality.

223. White-box - An approach in systems engineering and modelling where the internal structure, workings, and implementation details of a system or component are fully visible and considered, allowing for thorough examination of logic, control flow, and data structures to ensure correctness and efficiency. A white-box representation exposes the internal structure and behaviour of a system or component, showing how it performs its functions. It contrasts with the black-box view, which focuses only on external interactions. White-box modelling supports design, debugging, and performance analysis by revealing how internal elements collaborate to produce system outputs. See also “Black-box”

224. Wisdom - Wisdom is the ability to apply knowledge with foresight, anticipating future, considering long-

term implications of alternatives, and making strategic decisions involving weighing trade-offs. See “DIKW”.

225. Workflow - A workflow defines the ordered sequence of tasks, responsibilities, and information flows required to complete a process. Workflows can describe organisational, procedural, or computational activities and are often modelled using BPMN. In systems engineering, workflow models help optimise coordination, traceability, and automation of engineering processes.

226. Work Product - A work product is any tangible outcome produced during a process, such as a document, model, prototype, or test report. Work products provide evidence of progress and compliance with process requirements. In system development, they are managed under configuration control and serve as key artefacts for review, validation, and knowledge sharing.

227. XMI (XML Metadata Interchange) - XMI is an open standard defined by the Object Management Group (OMG) for exchanging metadata, particularly models, between different software tools. It provides a common XML-based format for representing UML and MOF (Meta-Object Facility) models, ensuring interoperability across modelling environments. In practice, XMI allows models created in one tool to be

imported, merged, or analysed in another while preserving their structure and semantics.

228. XML (eXtensible Markup Language) - XML is a structured markup language used to encode information in a format that is both human- and machine-readable.

229. Y-Chart - The Y-Chart is a conceptual framework for system design and analysis that separates function, structure, and behaviour into three complementary domains. Originating in hardware–software co-design, it helps engineers to reason about how functionality is mapped onto structure and how behaviour emerges from their interaction²⁶. The Y-Chart illustrates the iterative nature of design refinement.

230. Z Notation - Z notation is a formal specification language based on set theory and first-order predicate logic. It is used to describe and reason rigorously about system properties, particularly in safety or mission-critical domains. Z enables precise definition of system states and operations, supporting formal verification and validation of complex designs²⁷.

²⁶ https://en.wikipedia.org/wiki/Gajski%E2%80%93Kuhn_chart?utm_source=chatgpt.com

²⁷ https://en.wikipedia.org/wiki/Z_notation?utm_source=chatgpt.com

8.4 <core> Key Terms in European Portuguese

The following list gives the preferred translations into European Portuguese of key terms used in these lecture notes. It is intended to support the correct use of systems engineering terminology when speaking or writing in Portuguese, rather than to provide word-for-word translations. Notes identify frequent mistakes, false friends and cases in which the usual Brazilian Portuguese term differs. The abbreviation pt-BR identifies a form commonly used in Portuguese as spoken and written in Brazil.

1. Acceptance Criterion - critério de aceitação
2. Access Relationship - relação de acesso
3. Active Structure - estrutura ativa
4. Actor - ator
5. Activity Diagram - diagrama de atividades
6. Aggregation - agregação
7. Analysis - análise
8. Application - aplicação (pt-BR: aplicativo, particularly for an end-user software application)
9. Application Component - componente aplicacional (pt-BR: componente de aplicação)
10. Application Domain - domínio aplicacional (pt-BR: domínio de aplicação)
11. Application Interface - interface aplicacional (pt-BR: interface de aplicação)
12. Architecture - arquitetura
13. Architecture Description - descrição de arquitetura
14. ArchiMate - ArchiMate (the name of the language is not translated)
15. Artefact / Artifact - artefacto (pt-BR: artefato)
16. As-is - estado atual or situação atual (do not translate literally)
17. Assignment Relationship - relação de atribuição
18. Association - associação
19. Assumption - pressuposto (avoid the calque "assunção"; pt-BR also frequently uses "premissa")
20. Baseline - referência de base or linha de base
21. Behaviour - comportamento
22. Black Box – “caixa-preta” or “perspetiva de caixa-preta” (pt-BR: perspectiva de caixa-preta)
23. Boundary - fronteira or limite (in "system boundary", prefer "fronteira do sistema"; this is a conceptual boundary and need not be physical)
24. BPMN - Modelo e Notação de Processos de Negócio (the acronym BPMN is normally retained)
25. Business Actor - ator de negócio (an organisational entity capable of performing behaviour; the term does not imply a commercial organisation)
26. Business Domain - domínio de negócio (the term also applies to public and non-commercial organisations)
27. Business Object - objeto de negócio
28. Business Process - processo de negócio
29. Business Requirement - requisito de negócio (in these lecture notes, this is an awareness term and not a primary category in the adopted requirements classification)
30. Can – “pode” or “é capaz de” (expresses ability; it should not be used to express permission when the distinction from "may" is relevant)
31. Candidate Requirement Statement - enunciado de candidato a requisito
32. Change Management - gestão de alterações (use "gestão da mudança" for organisational change, not for the control of changes to requirements or models)
33. Class Diagram - diagrama de classes
34. Collaboration - colaboração
35. Common Domain - domínio comum
36. Communication Network - rede de comunicações
37. Communication Path – “caminho de comunicação” or “via de comunicação”
38. Component - componente
39. Component Diagram - diagrama de componentes
40. Composite Structure Diagram - diagrama de estrutura composta
41. Composition - composição
42. Concern – “preocupação” or “questão de interesse” ("preocupação" does not necessarily imply anxiety; it denotes something relevant to a stakeholder)
43. Connector - conector
44. Constraint - restrição (prefer this term for a requirement category; "constrangimento" has a broader meaning in ordinary Portuguese)

45. Context - contexto	68. Function - função
46. Context Diagram - diagrama de contexto	69. Functional Requirement - requisito funcional
47. Current State - estado atual	70. Generalisation - generalização
48. Data Object - objeto de dados	71. Goal - objetivo ("meta", frequent in pt-BR, is better reserved for a specific or measurable target when that distinction matters)
49. Deliverable - resultado a entregar or produto a entregar ("entregável" is common project-management jargon, especially in pt-BR, but is often avoidable)	72. Interface - interface
50. Deployment – “instalação” or “disponibilização”, depending on the context (pt-BR: implantação)	73. Lifecycle - ciclo de vida
51. Deployment Diagram - diagrama de instalação (pt-BR: diagrama de implantação)	74. Logical Architecture - arquitetura lógica
52. Design - concepção (pt-BR: projeto; avoid "desenho" unless referring specifically to a drawing or using an established technical expression)	75. Mapping - correspondência or mapeamento (pt-BR normally prefers "mapeamento")
53. Design Decision - decisão de concepção (pt-BR: decisão de projeto)	76. May - pode (expresses permission or an allowed possibility; it does not express an obligation)
54. Device - dispositivo	77. Model - modelo
55. Diagram - diagrama	78. Model-Based Systems Engineering (MBSE) - Engenharia de Sistemas Baseada em Modelos
56. Digital Twin - gêmeo digital (pt-BR: gêmeo digital)	79. Model-Driven Engineering (MDE) - Engenharia Orientada por Modelos (pt-BR: Engenharia Dirigida por Modelos)
57. Elicitation - levantamento de informação para requisitos (the traditional expression "levantamento de requisitos" is also used, but may misleadingly suggest that requirements already exist and merely need to be collected; avoid "elicitação" in European Portuguese; pt-BR: "elicitação de requisitos" is common in academic and professional usage)	80. Modelling - modelação (pt-BR: modelagem)
58. Emergent Behaviour - comportamento emergente	81. Modelling Language - linguagem de modelação (pt-BR: linguagem de modelagem)
59. Engineering-process Requirement - requisito do processo de engenharia	82. Must - tem de or deve (expresses a binding obligation; some standards distinguish "must", as an externally imposed obligation, from "shall", as a requirement stated by the specification)
60. Enterprise Architecture – arquitetura empresarial	83. Need - necessidade (a need is not yet necessarily a requirement)
61. Entity of Interest - entidade de interesse	84. Node - nó
62. Event - evento	85. Non-Functional Requirement - requisito não funcional (prefer the more precise categories "quality requirement" and "constraint" whenever possible)
63. Evidence - evidência or elementos de prova (choose according to whether the intended meaning is observable support, an indication or formal proof)	86. Obligation - obrigação
64. Execution Environment - ambiente de execução	87. Outcome - resultado or efeito alcançado (distinguish from "output", which is something produced)
65. External System - sistema externo	88. Output - saída or resultado produzido (distinguish from "outcome", which concerns the effect achieved)
66. Flow - fluxo	89. Part - parte
67. Framework - quadro de referência (pt-BR: arcabouço; the English term "framework" is also frequently retained)	90. Passive Structure - estrutura passiva
	91. Physical Referent - referente físico
	92. Port - porta (not "porto")

93. Process - processo	118.Stakeholder Concern - preocupação or questão de interesse de uma parte interessada
94. Project-delivery Requirement - requisito de entrega do projeto	119.Stakeholder Need - necessidade de uma parte interessada
95. Prototype - protótipo	120.Stakeholder Requirement - requisito de uma parte interessada (in these lecture notes, this is an awareness term rather than a primary category in the adopted requirements classification)
96. Purpose - finalidade or propósito	121.State - estado
97. Quality Attribute - atributo de qualidade	122.State Machine Diagram - diagrama de máquina de estados
98. Quality Requirement - requisito de qualidade	123.State of Affairs – “estado de coisas” or “situação” or “situação atual”
99. Realisation Relationship - relação de realização	124.System - sistema
100.Refinement - refinamento	125.System Boundary - fronteira do sistema
101.Requirement - requisito (do not use "requerimento", which denotes a formal request, petition or application submitted to an authority)	126.System Context - contexto do sistema
102.Requirements Engineering - Engenharia de Requisitos	127.System Domain - domínio do sistema
103.Requirements Set - conjunto de requisitos	128.System of Interest - sistema de interesse
104.Requirements Source - fonte de requisitos	129.System Requirement - requisito do sistema
105.Requirements Specification - especificação de requisitos	130.Systems Engineering - Engenharia de Sistemas
106.Responsibility - responsabilidade	131.Target State - estado-alvo or estado futuro pretendido
107.Role - papel	132.Technology Domain - domínio tecnológico
108.Rule - regra	133.Test - teste or ensaio ("teste" is usual for software; "ensaio" may be more appropriate for physical systems, materials or experimental procedures)
109.Scenario - cenário	134.Test Case - caso de teste
110.Scope - âmbito (pt-BR: escopo)	135.To-be - estado futuro pretendido or estado-alvo (do not translate literally)
111.Sequence Diagram - diagrama de sequência	136.Traceability - rastreabilidade
112.Service - serviço	137.Trade-off - solução de compromisso or ponderação entre objetivos conflitantes (avoid using only "compromisso", which can also mean a commitment or obligation; pt-BR frequently retains the English term)
113.Serving Relationship - relação de serviço (in ArchiMate, it indicates that one element provides functionality to another)	138.Transition - transição
114.Shall - deve (in a requirement, use the present form "deve"; avoid "deverá" or "irá", which can be interpreted as statements about the future rather than binding obligations)	139.Trigger - evento desencadeador (pt-BR: gatilho)
115.Should - convém que or deveria (expresses a recommendation, not a binding requirement)	140.Triggering Relationship - relação de desencadeamento (pt-BR also uses "relação de disparo")
116.Sociotechnical System - sistema sociotécnico	141.UML - Linguagem de Modelação Unificada (pt-BR: Linguagem de Modelagem Unificada)
117.Stakeholder - parte interessada ("interveniente" can be used for someone who actively participates, but is narrower than "stakeholder"; pt-BR also uses "parte interessada", although the English term is often retained)	142.Universe of Discourse - Universo de Discurso

143. Use Case - caso de utilização (pt-BR: caso de uso)
144. Use Case Diagram - diagrama de casos de utilização (pt-BR: diagrama de casos de uso)
145. User - utilizador (pt-BR: usuário)
146. User Story - história de utilizador (pt-BR: história de usuário)
147. Validation - validação (determines whether the artefact, model or requirement is appropriate for its intended purpose and context)
148. Verification - verificação (determines whether the artefact, model or requirement satisfies specified rules, criteria or properties)
149. View - vista (pt-BR: visão; in architecture descriptions, distinguish a "view" from the "viewpoint" according to which it is constructed)
150. Viewpoint - ponto de vista
151. Well-formed Model - modelo bem formado
152. Well-formed Requirement - requisito bem formado
153. White Box - caixa-branca or perspectiva de caixa-branca (pt-BR: perspectiva de caixa-branca)
154. Will - irá or será (expresses a future event, intention or declaration; it should not be used to state a binding requirement)
155. Wireframe - esboço estrutural de interface or wireframe
156. Workflow - fluxo de trabalho
157. Work Product - produto de trabalho
-

